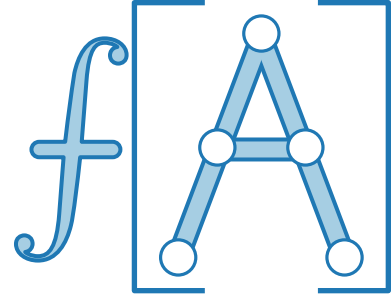
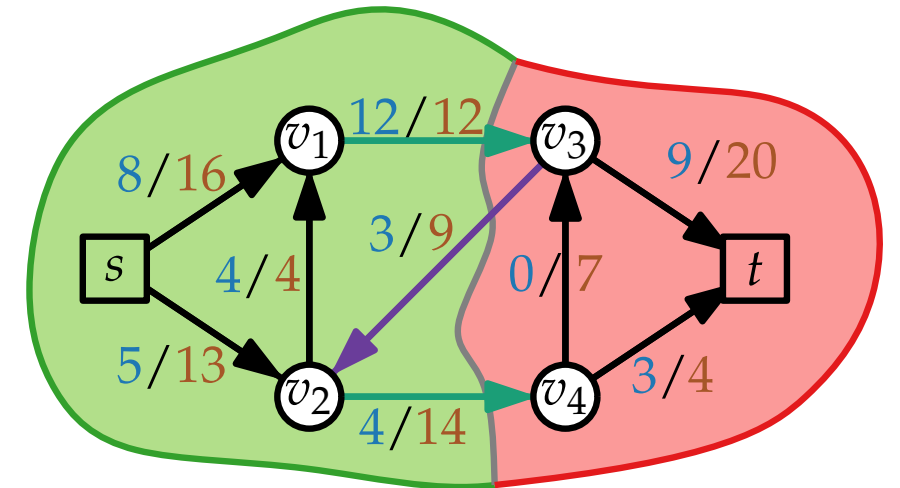
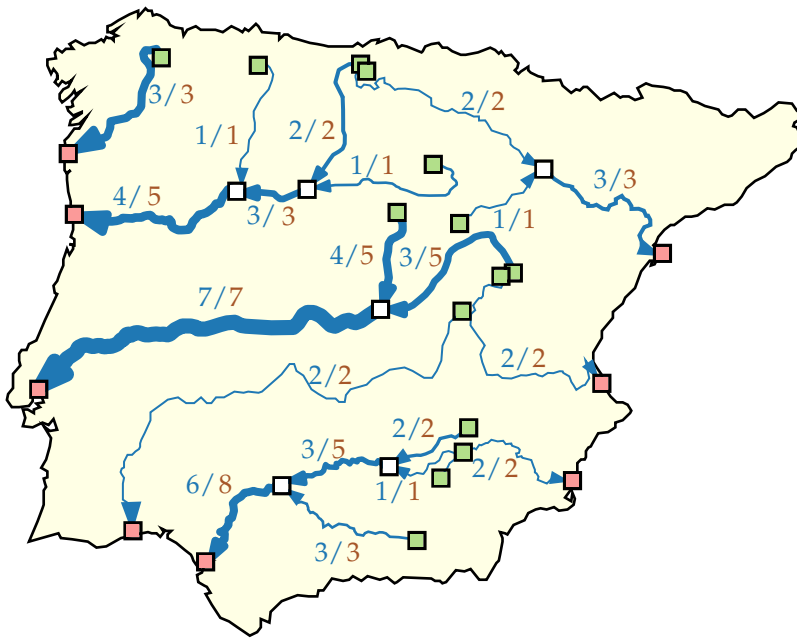




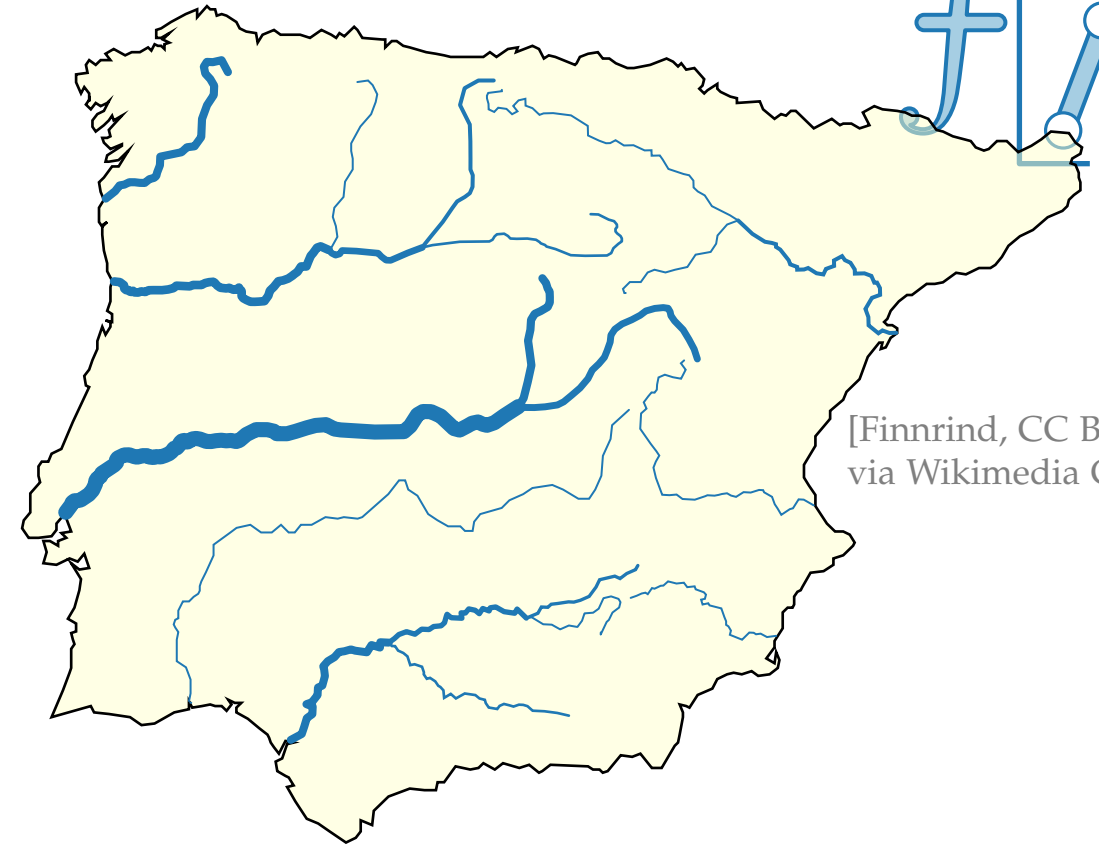
Fortgeschrittene Algorithmen

8. Vorlesung Graphalgorithmen II: Flussnetzwerke

Teil I: Problemstellung



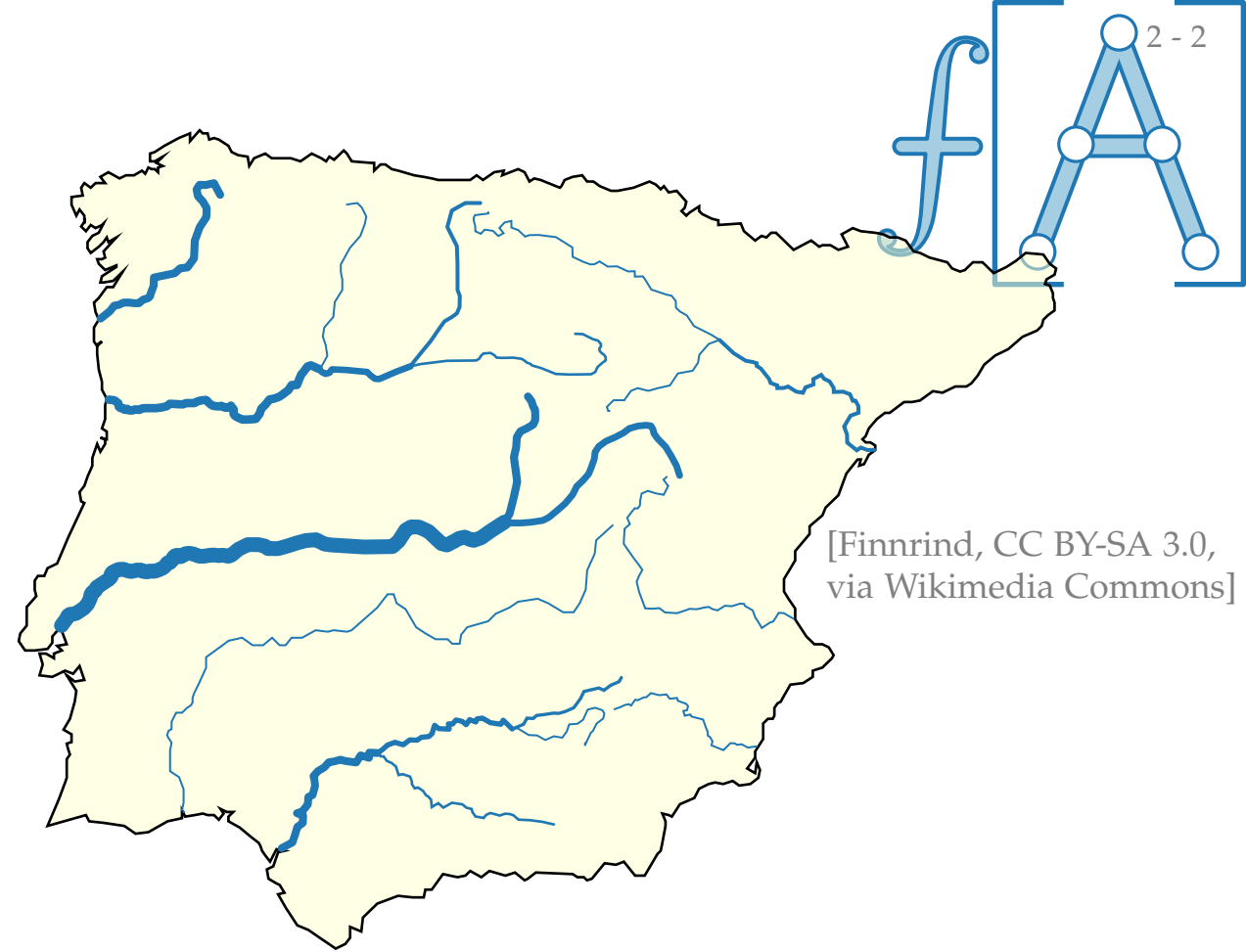
Flussnetzwerke



[Finnrind, CC BY-SA 3.0,
via Wikimedia Commons]

Flussnetzwerke

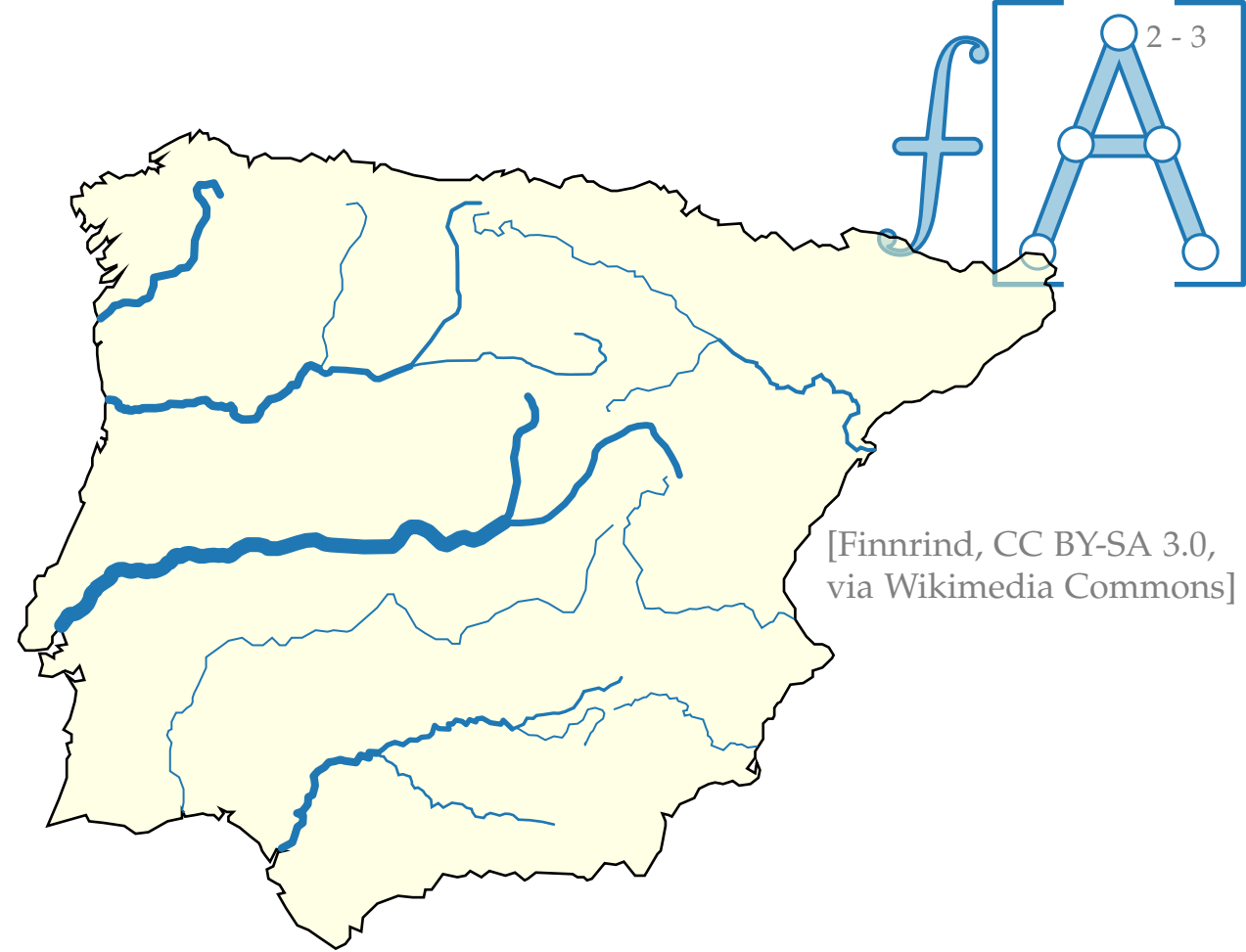
Flussnetzwerk $(G = (V, E); S; T; c)$ mit



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

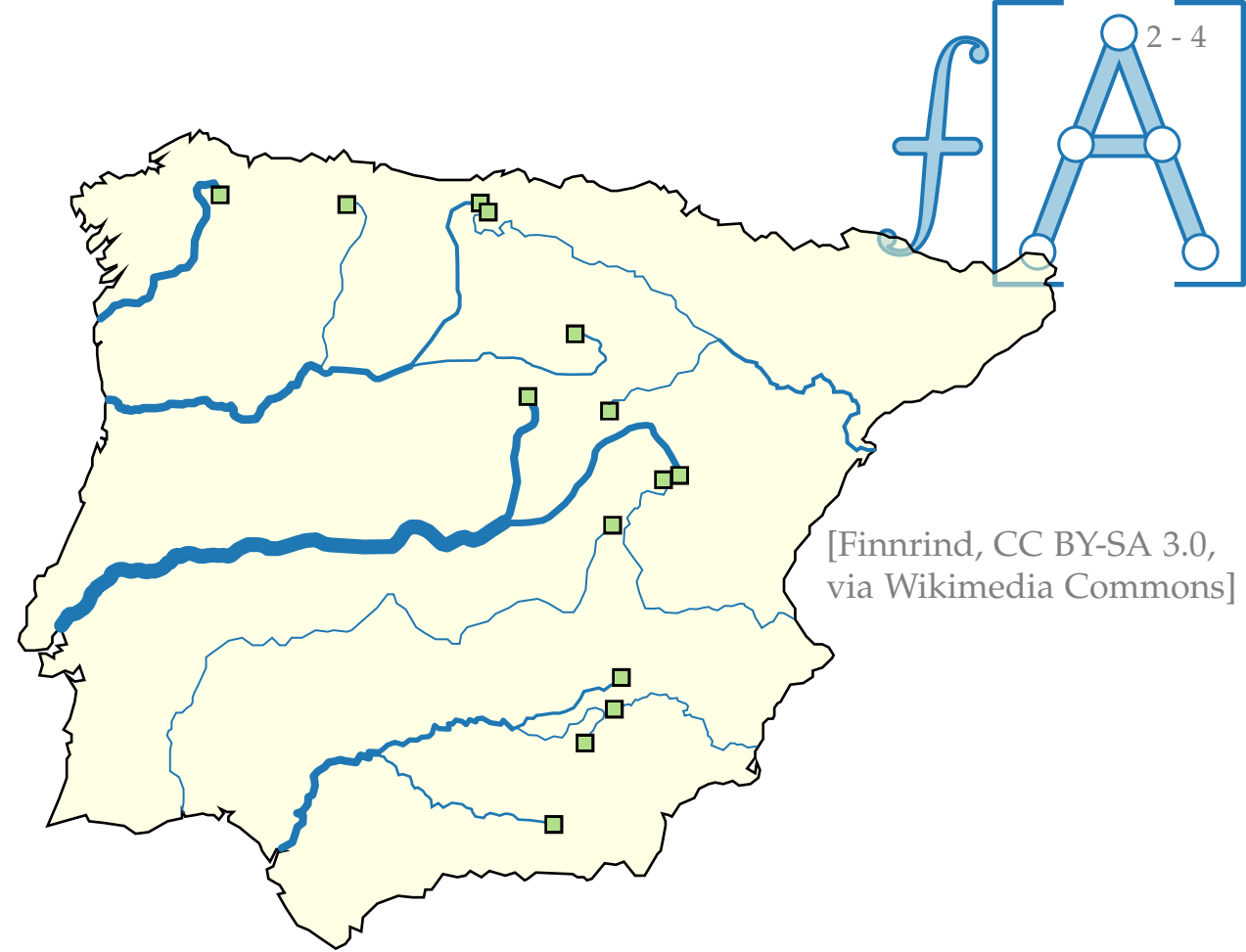
- gerichteter Graph $G = (V, E)$



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

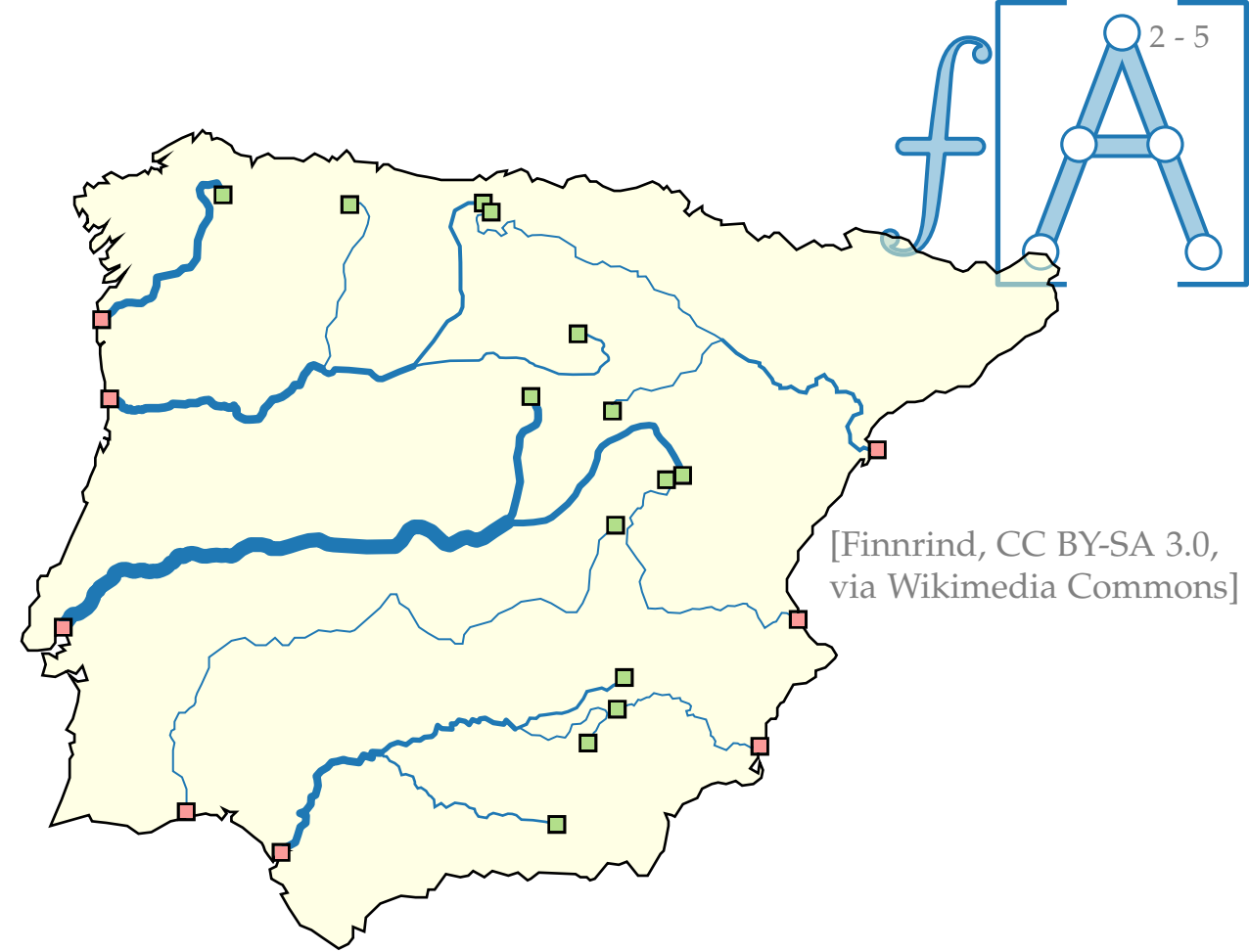
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

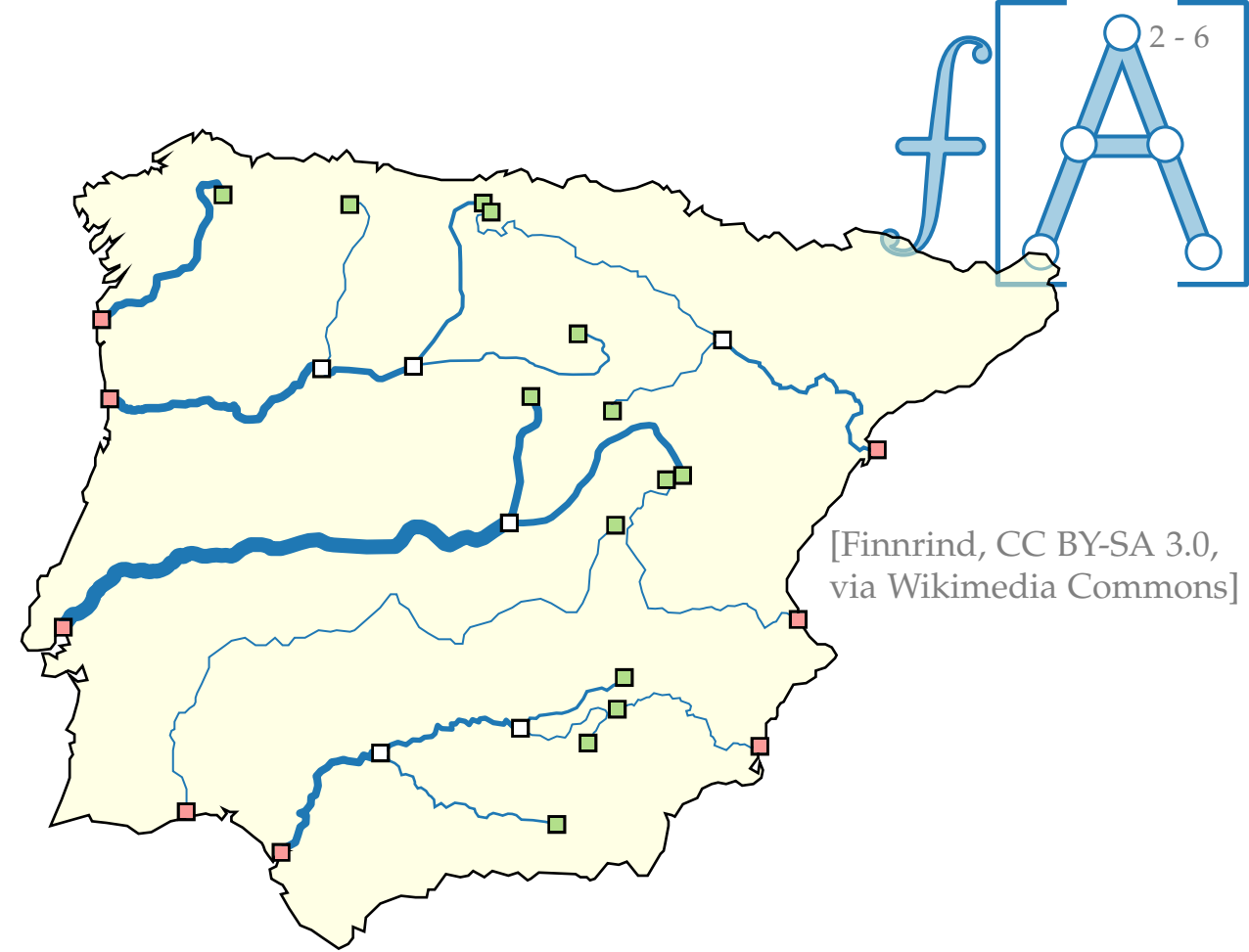
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

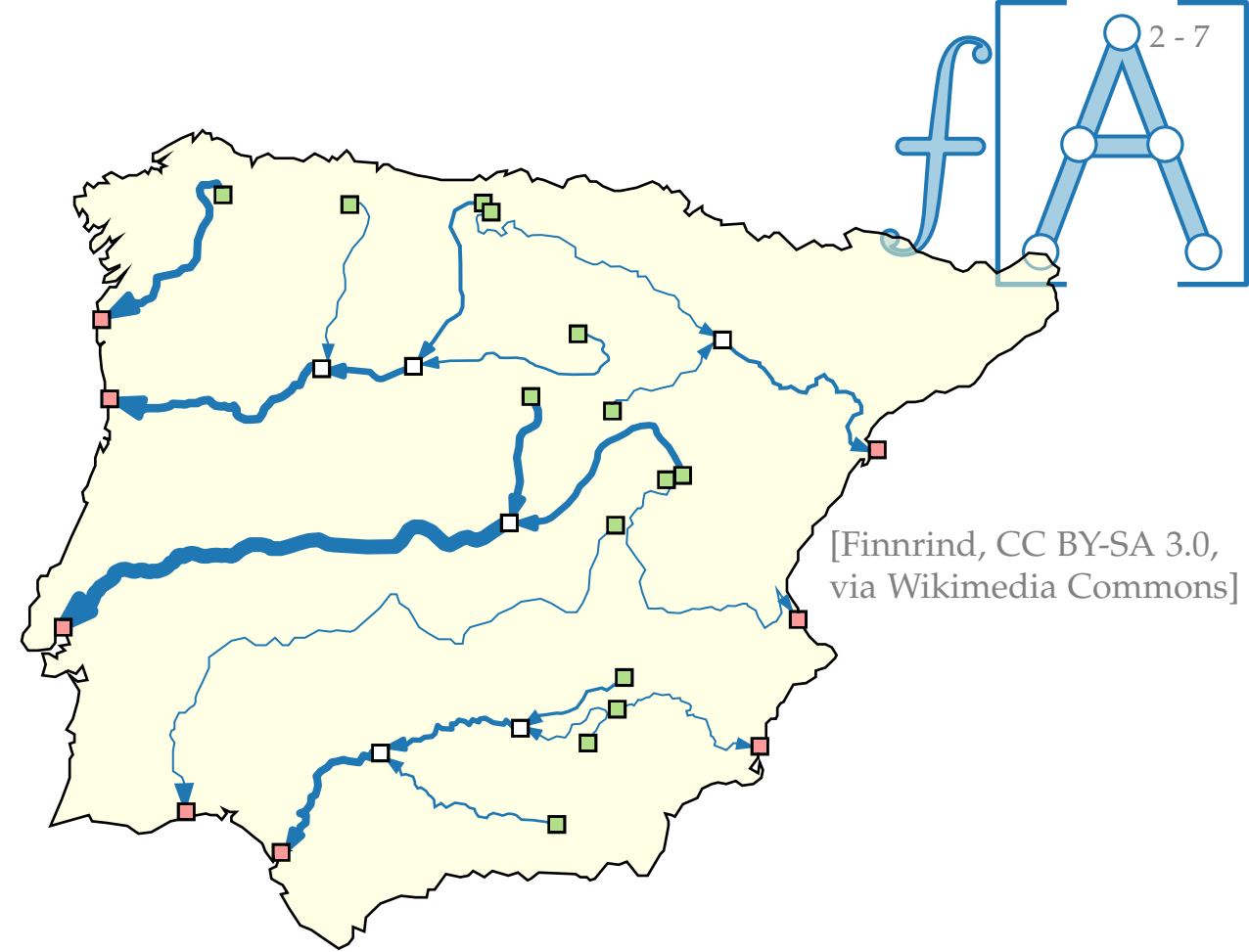
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

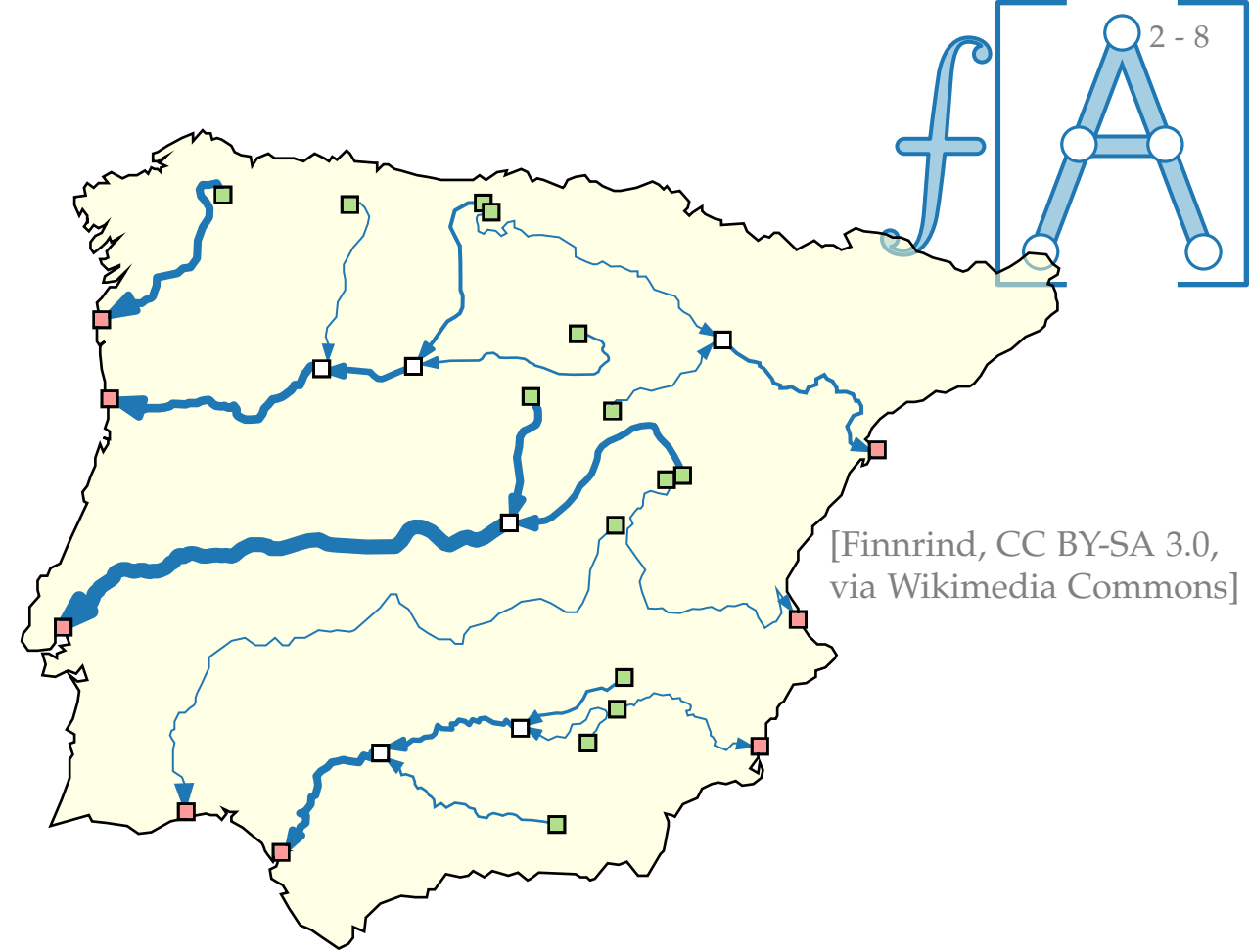
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

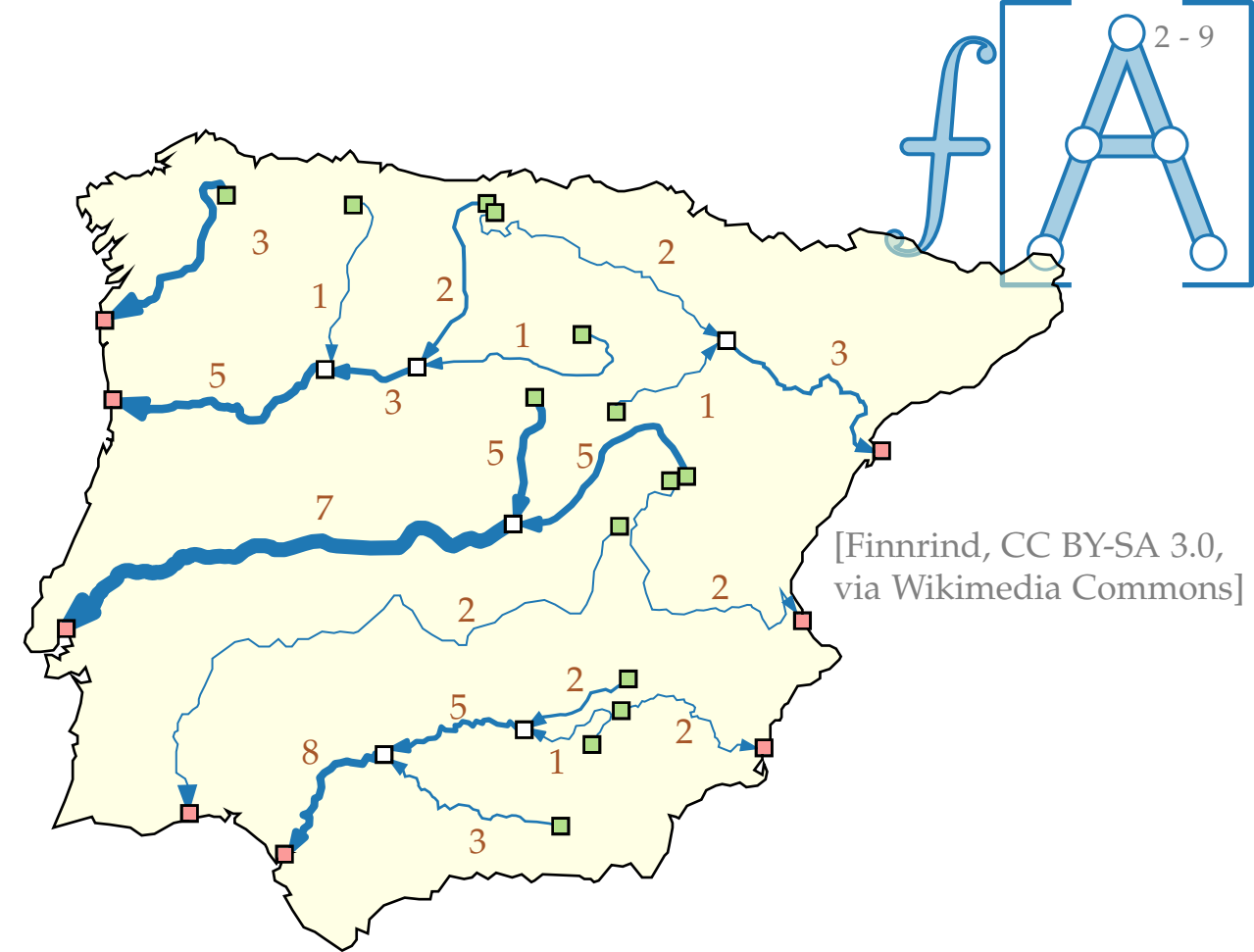
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

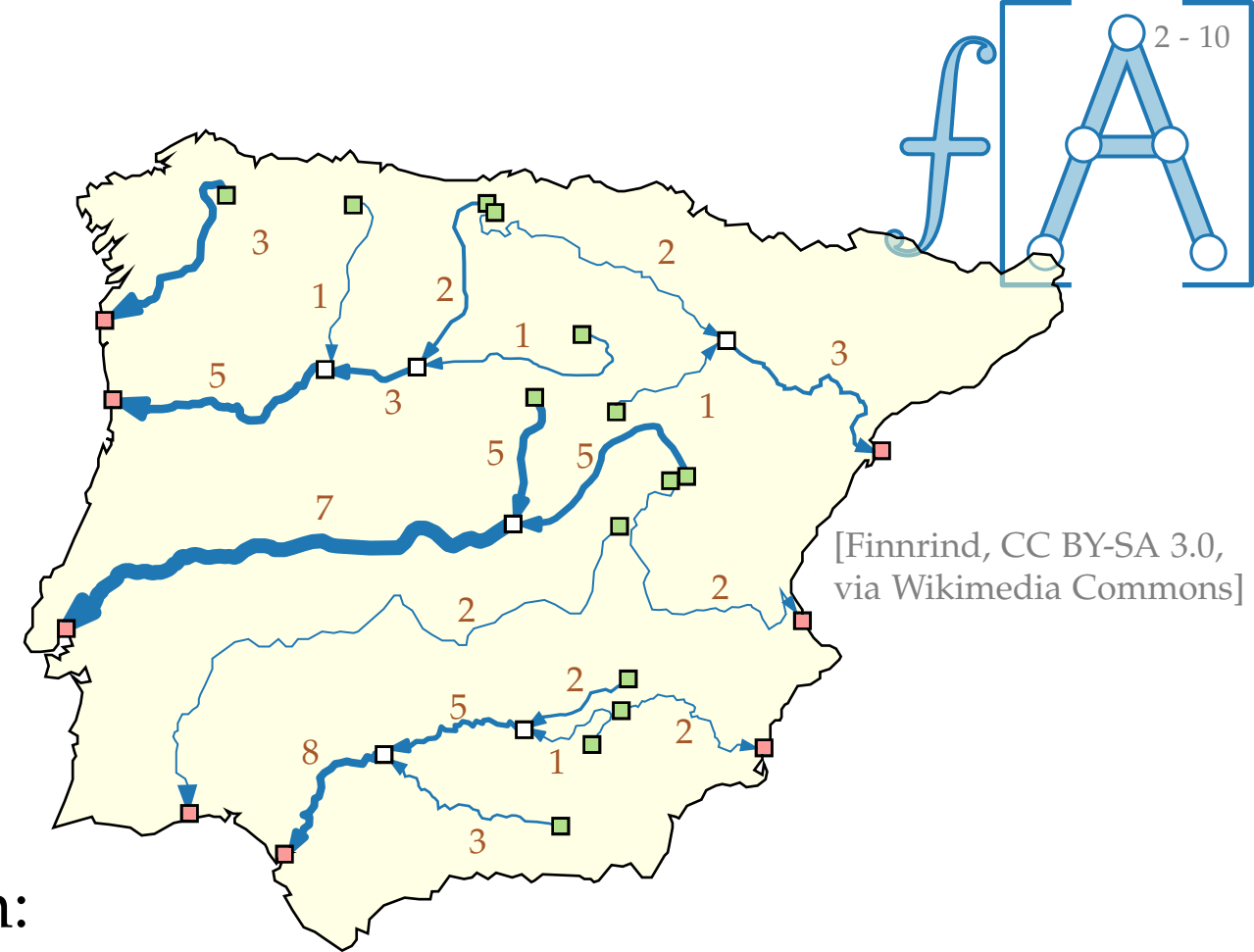


Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:



Flussnetzwerke

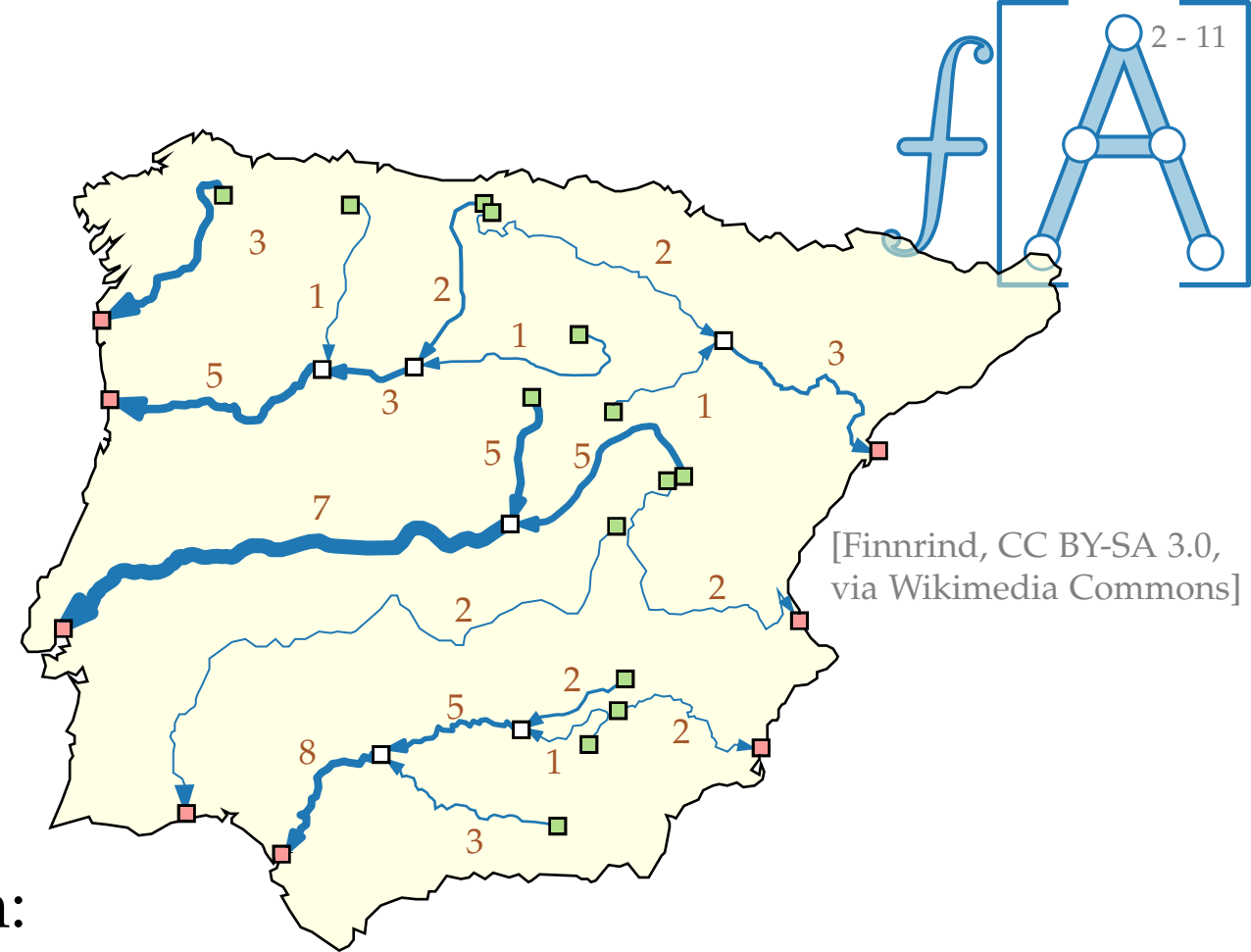
Flussnetzwerk $(G = (V, E); S; T; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S-T-Fluss** wenn:

$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

Kapazitäten einhalten



Flussnetzwerke

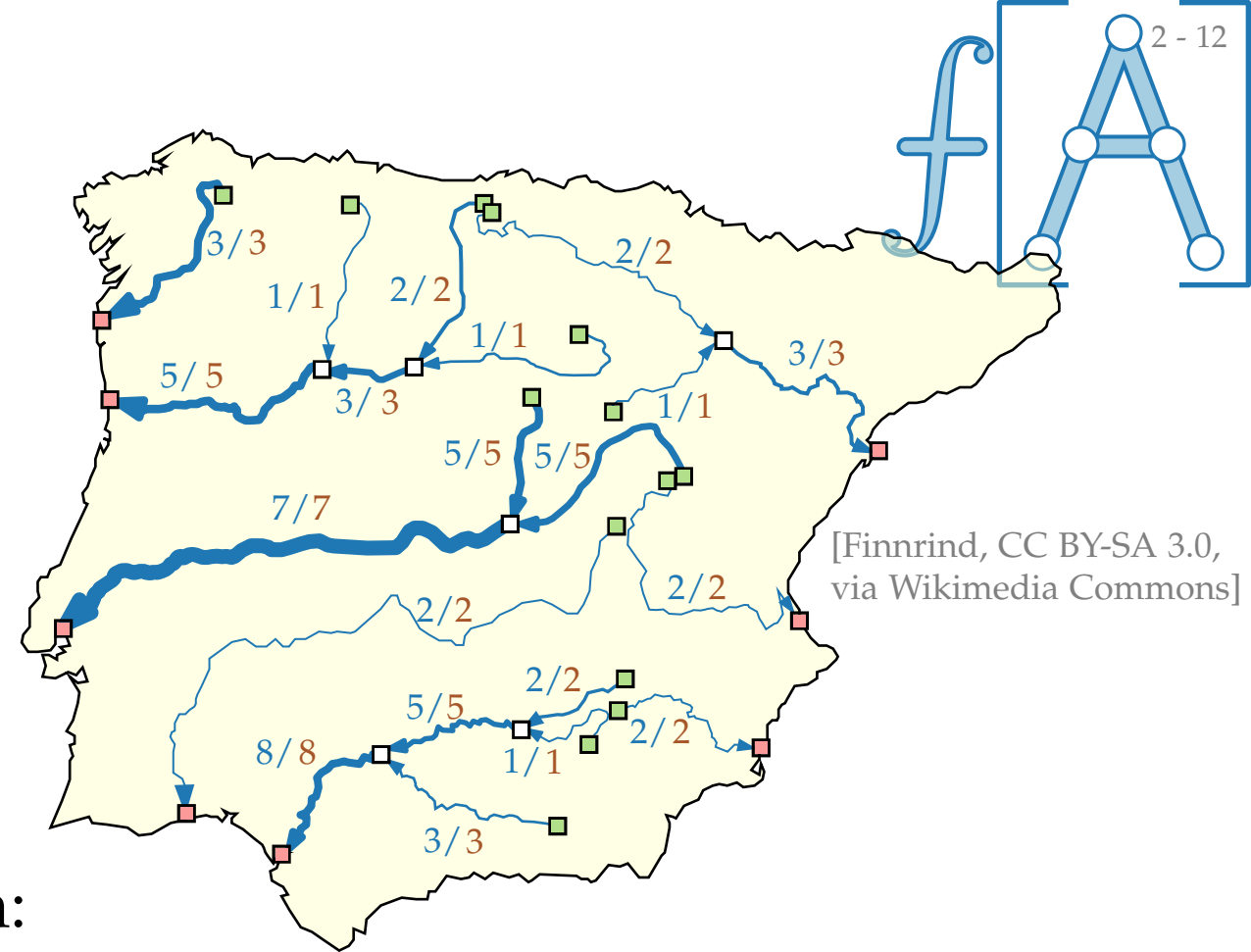
Flussnetzwerk $(G = (V, E); S; T; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S-T-Fluss** wenn:

$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

Kapazitäten einhalten



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

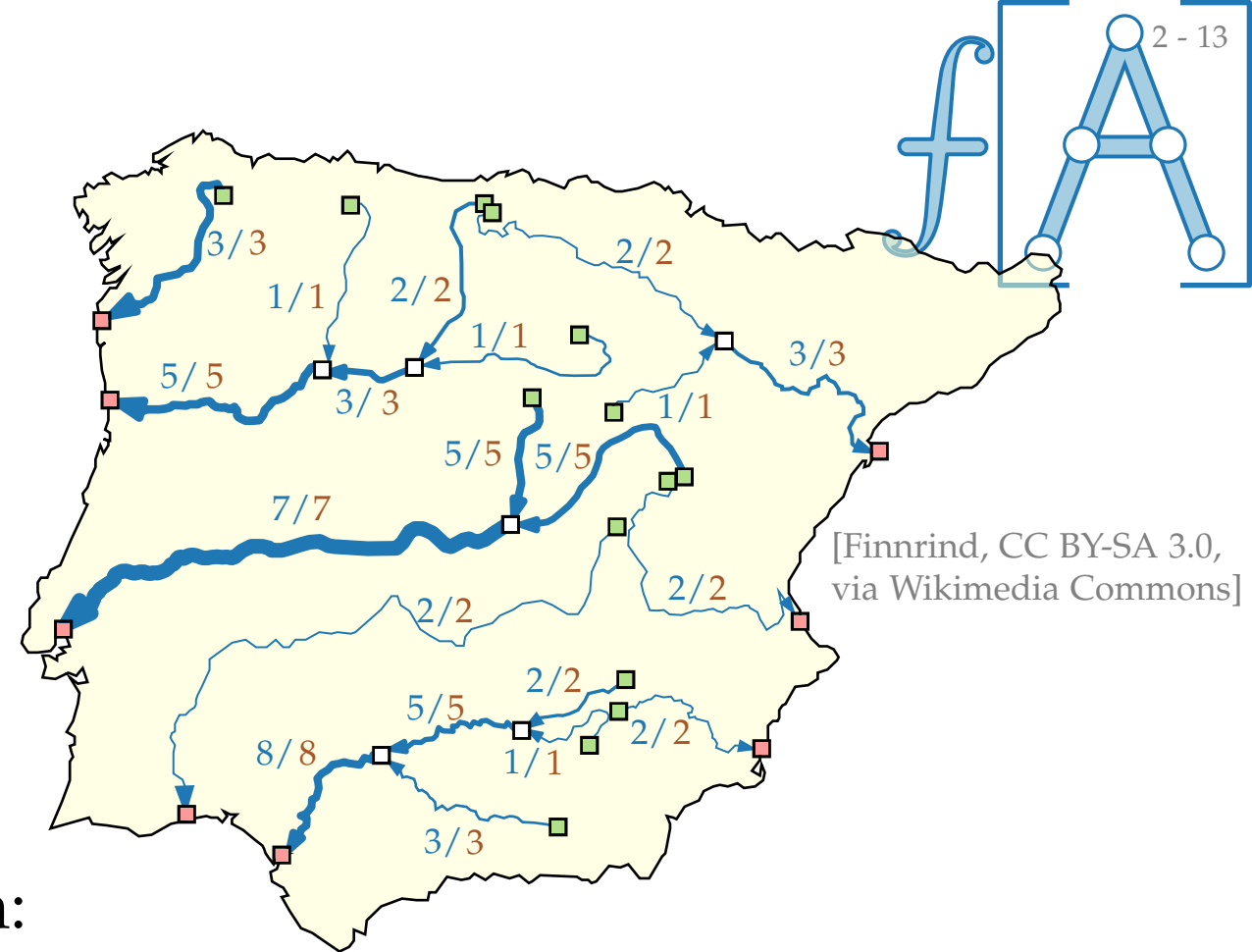
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S-T-Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

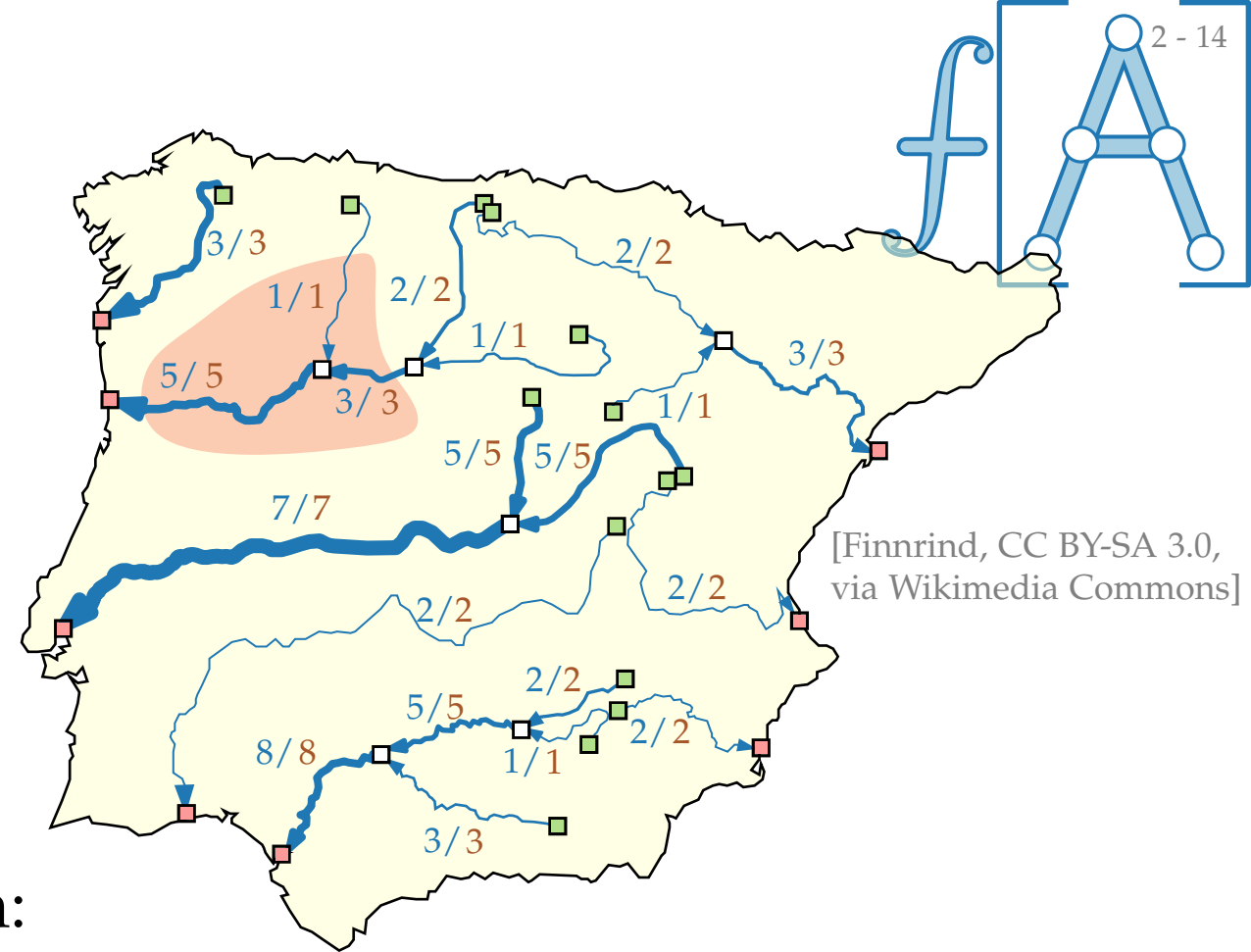
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

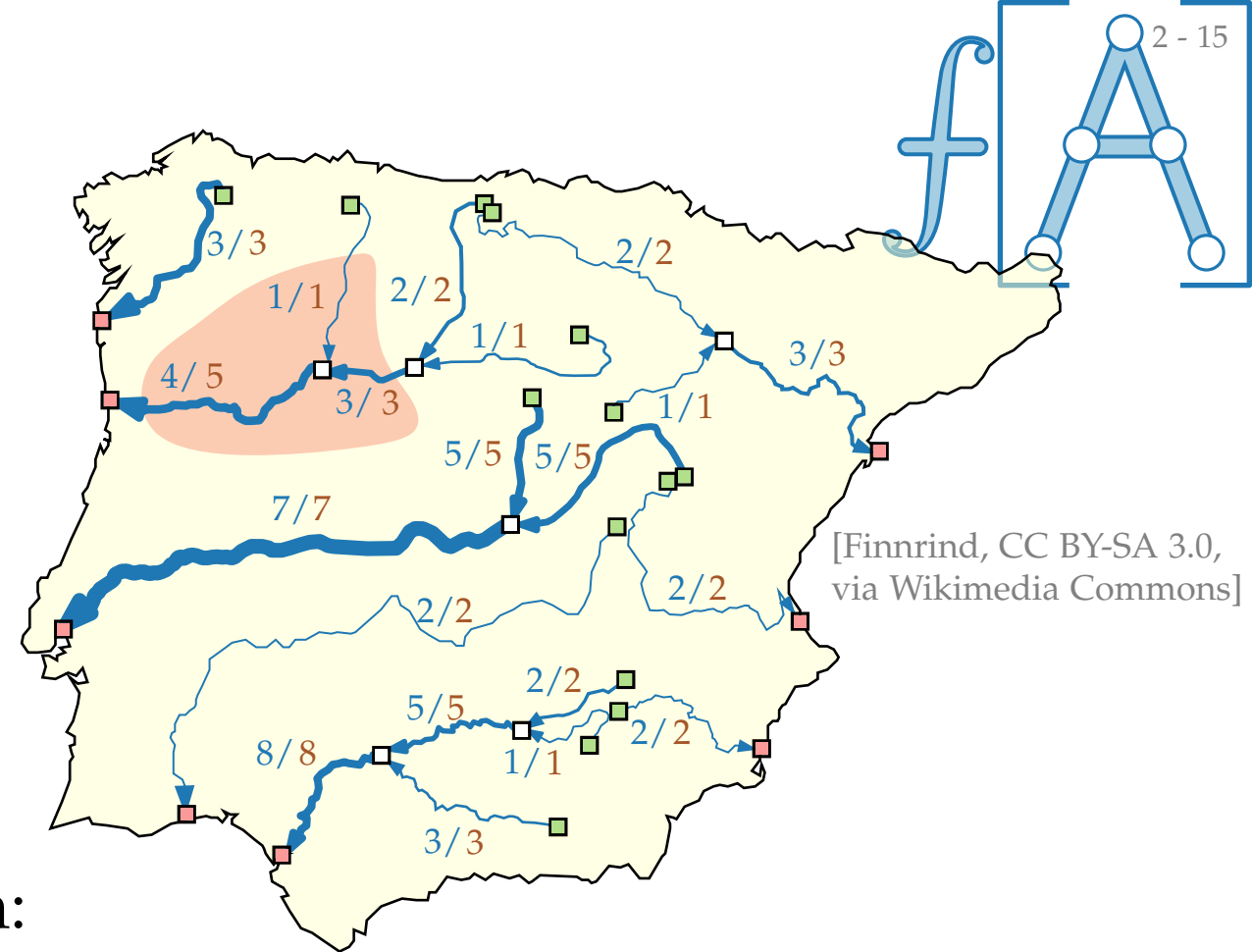
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

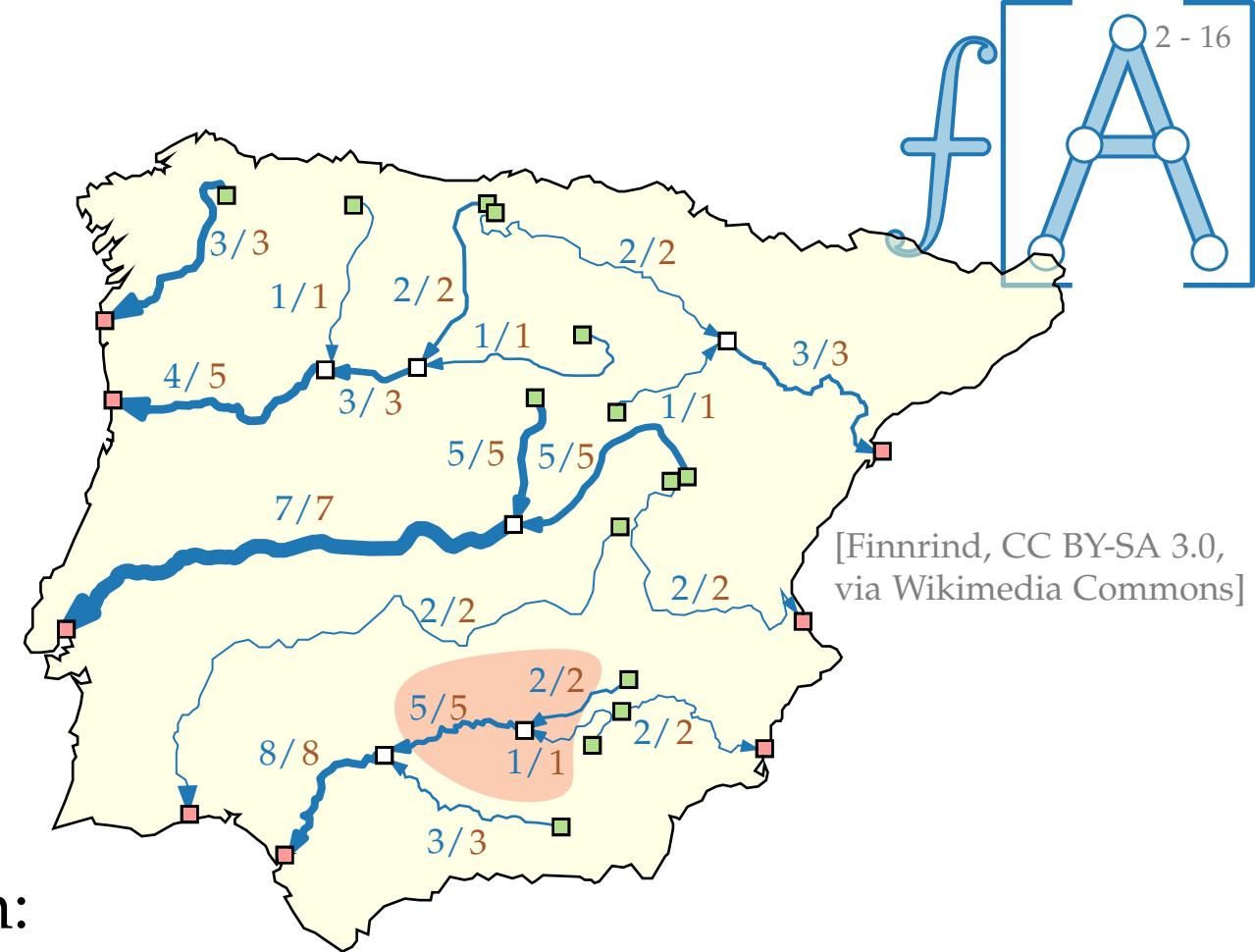
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

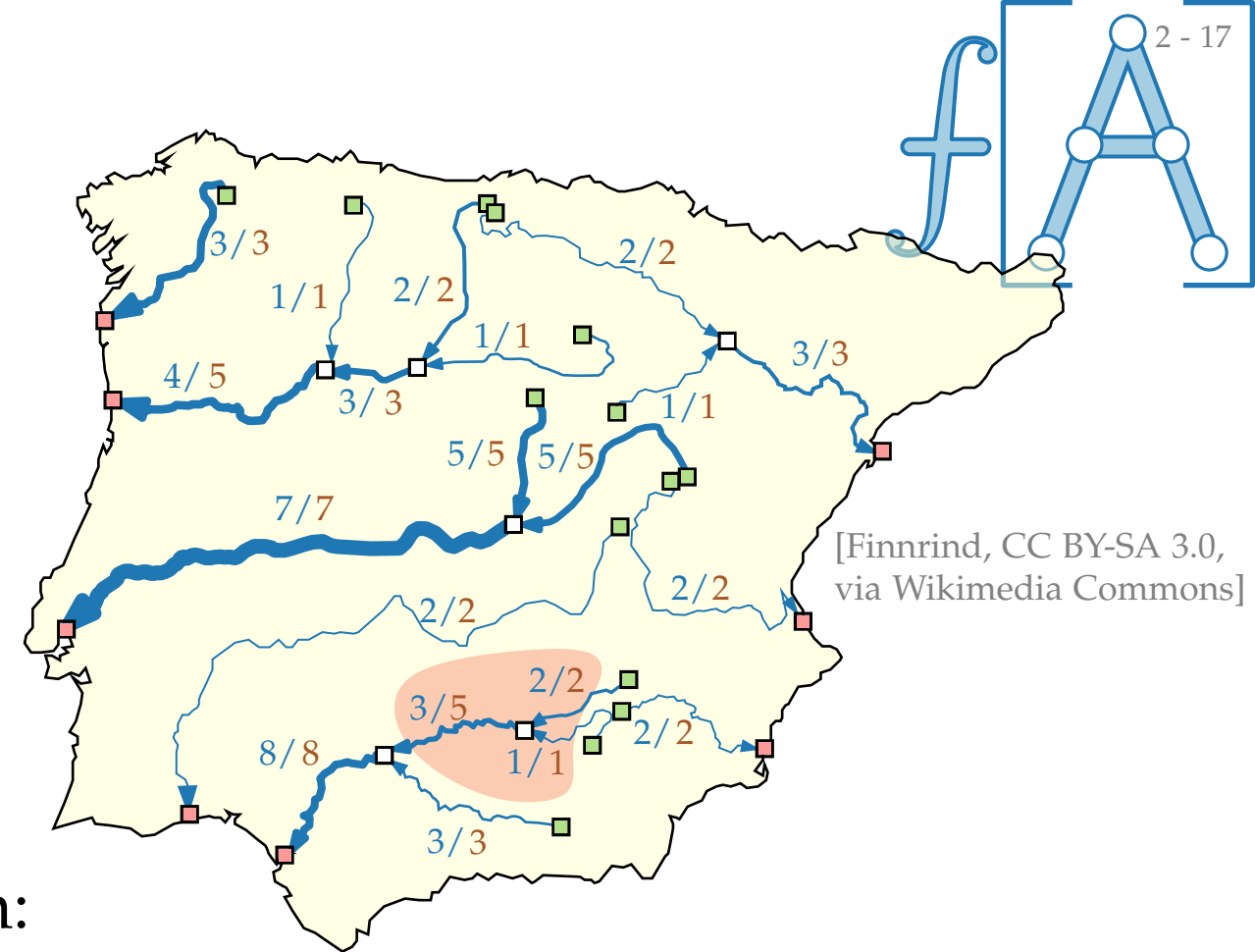
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



[Finnrind, CC BY-SA 3.0,
via Wikimedia Commons]

Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

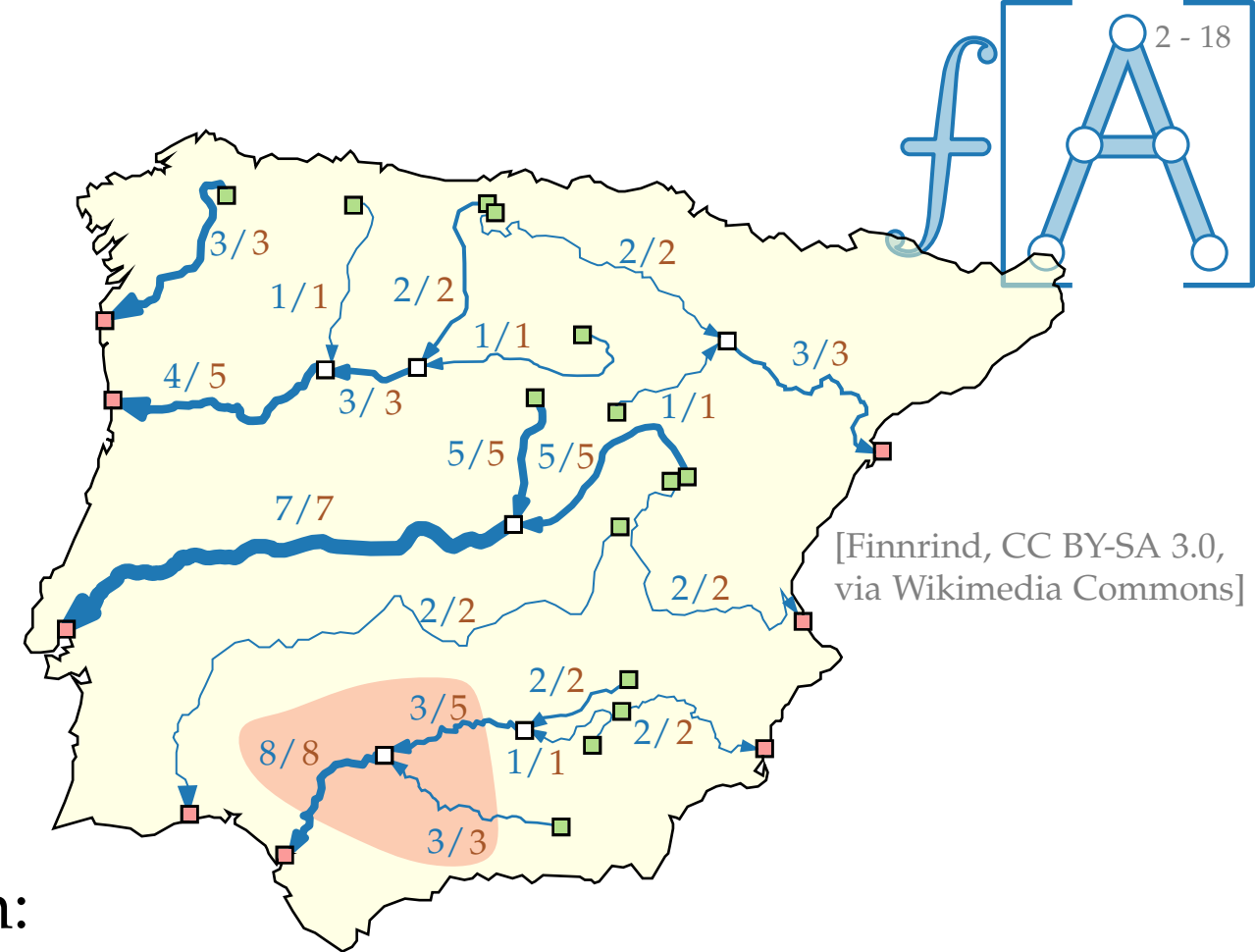
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S-T-Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

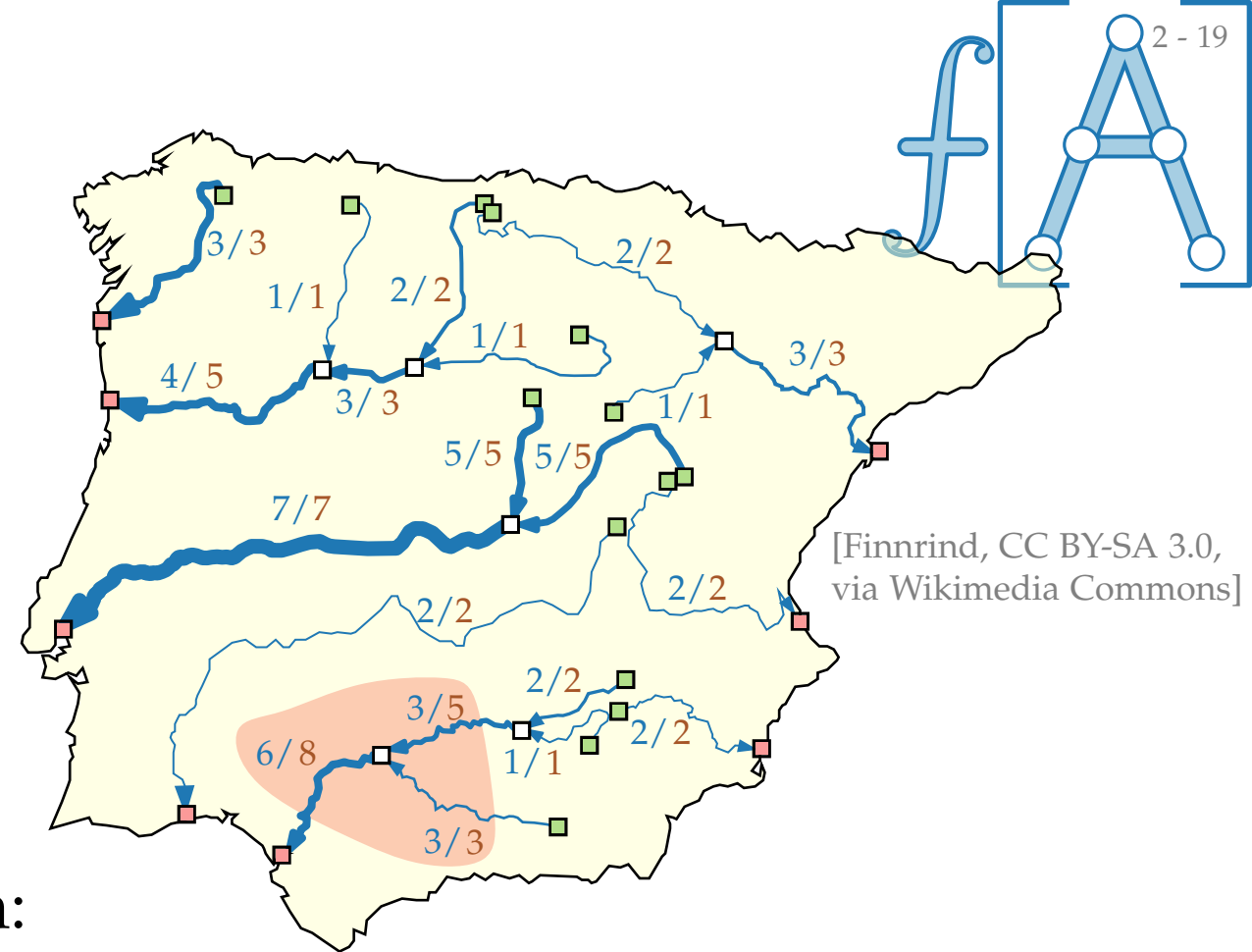
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S-T-Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

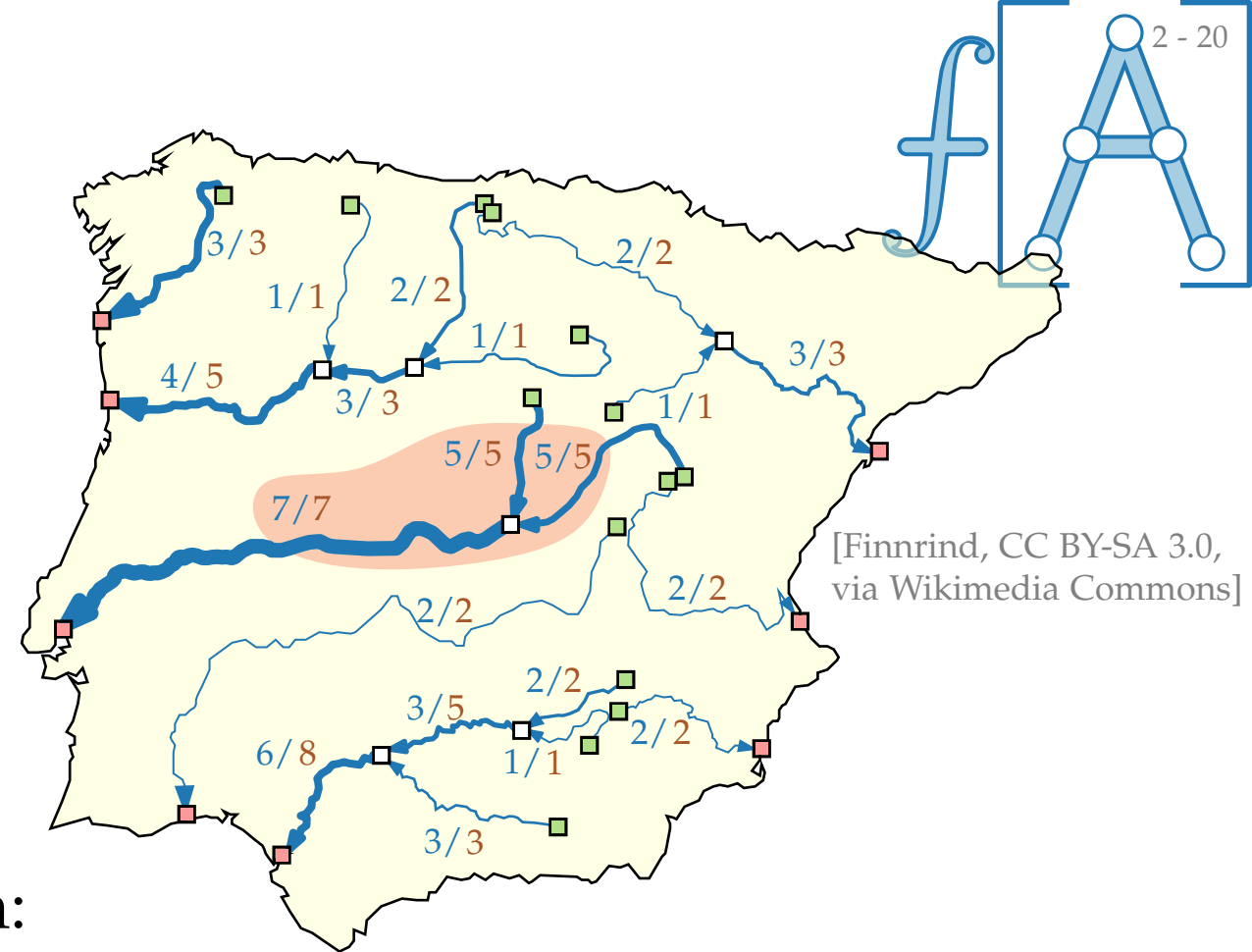
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

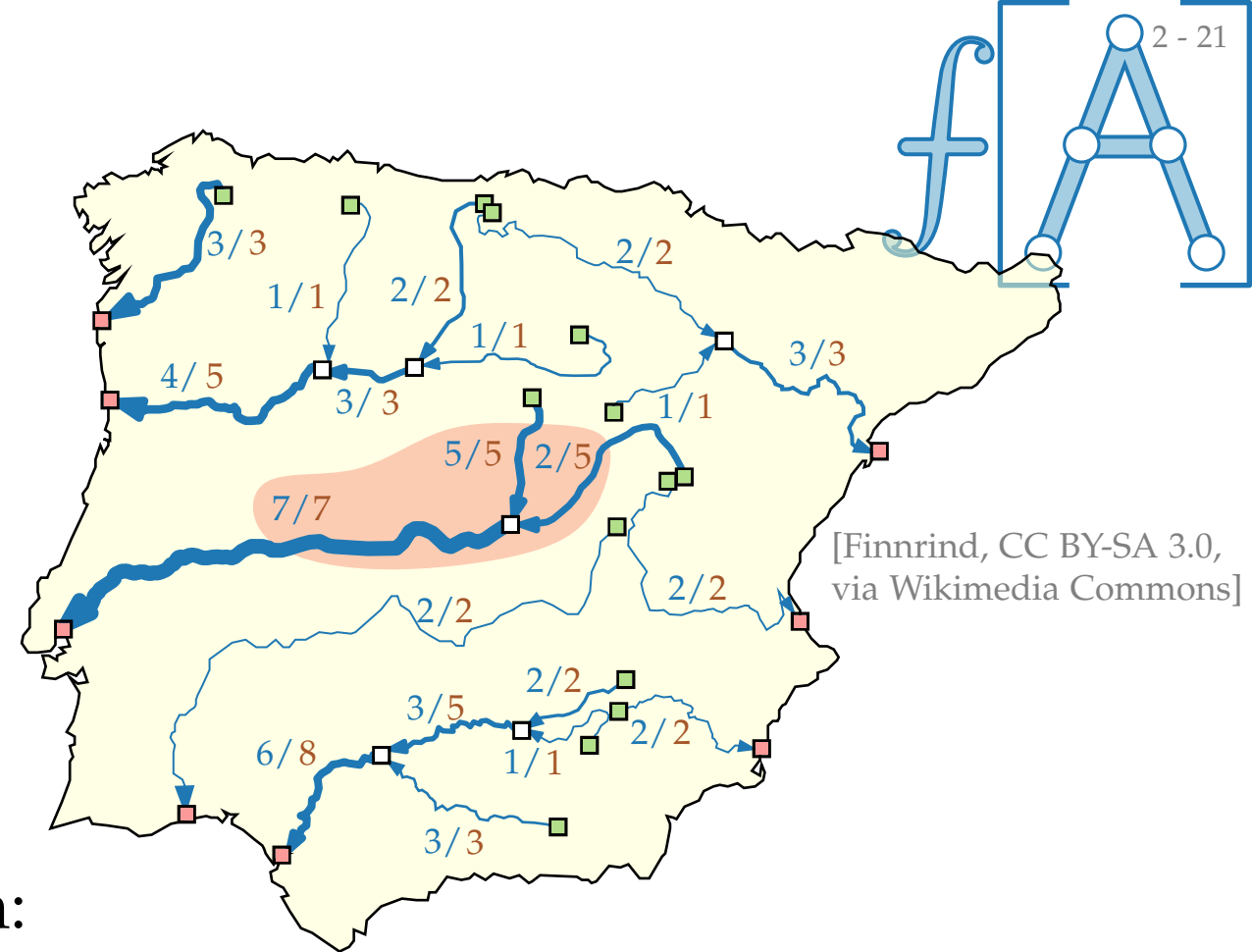
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S-T-Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

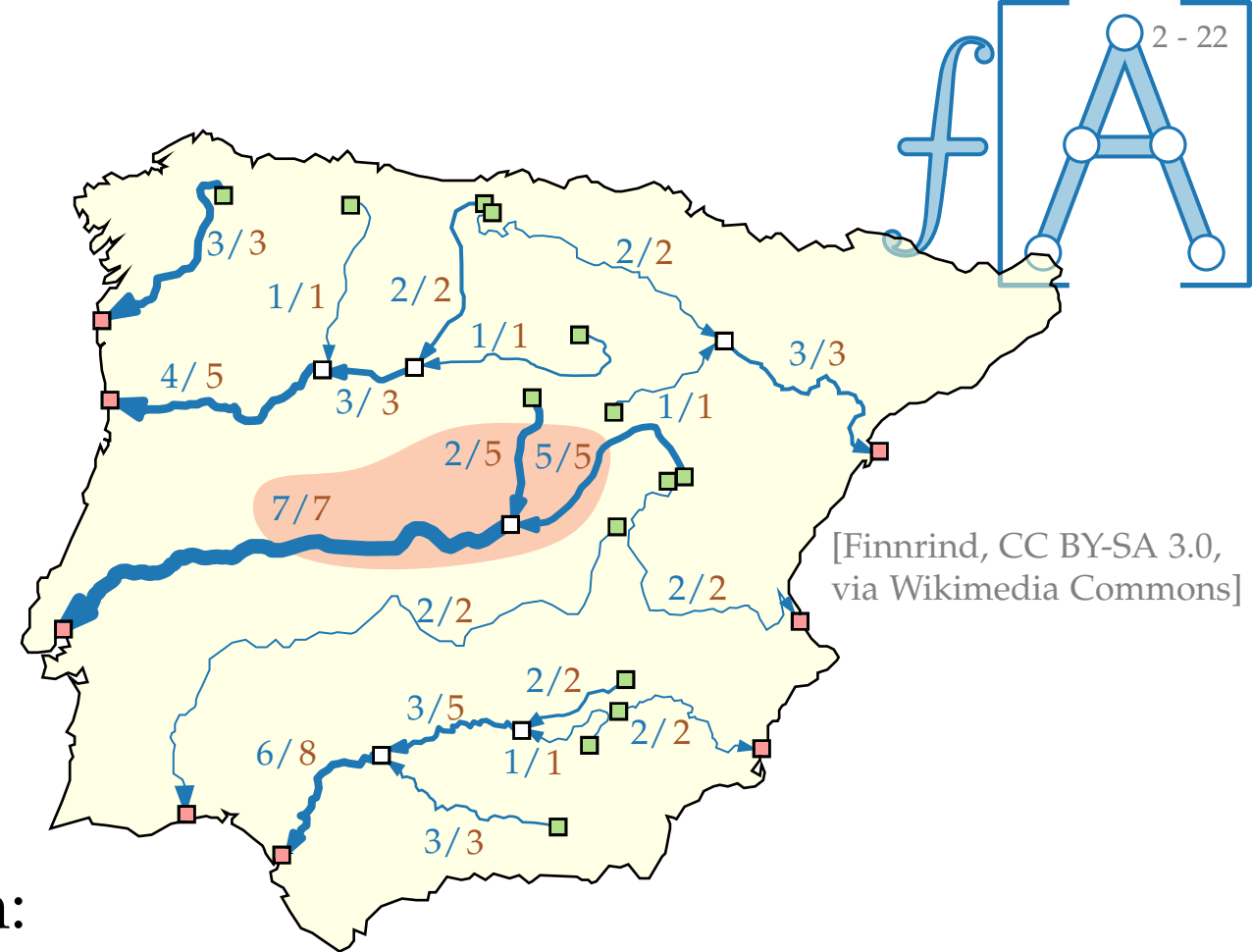
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

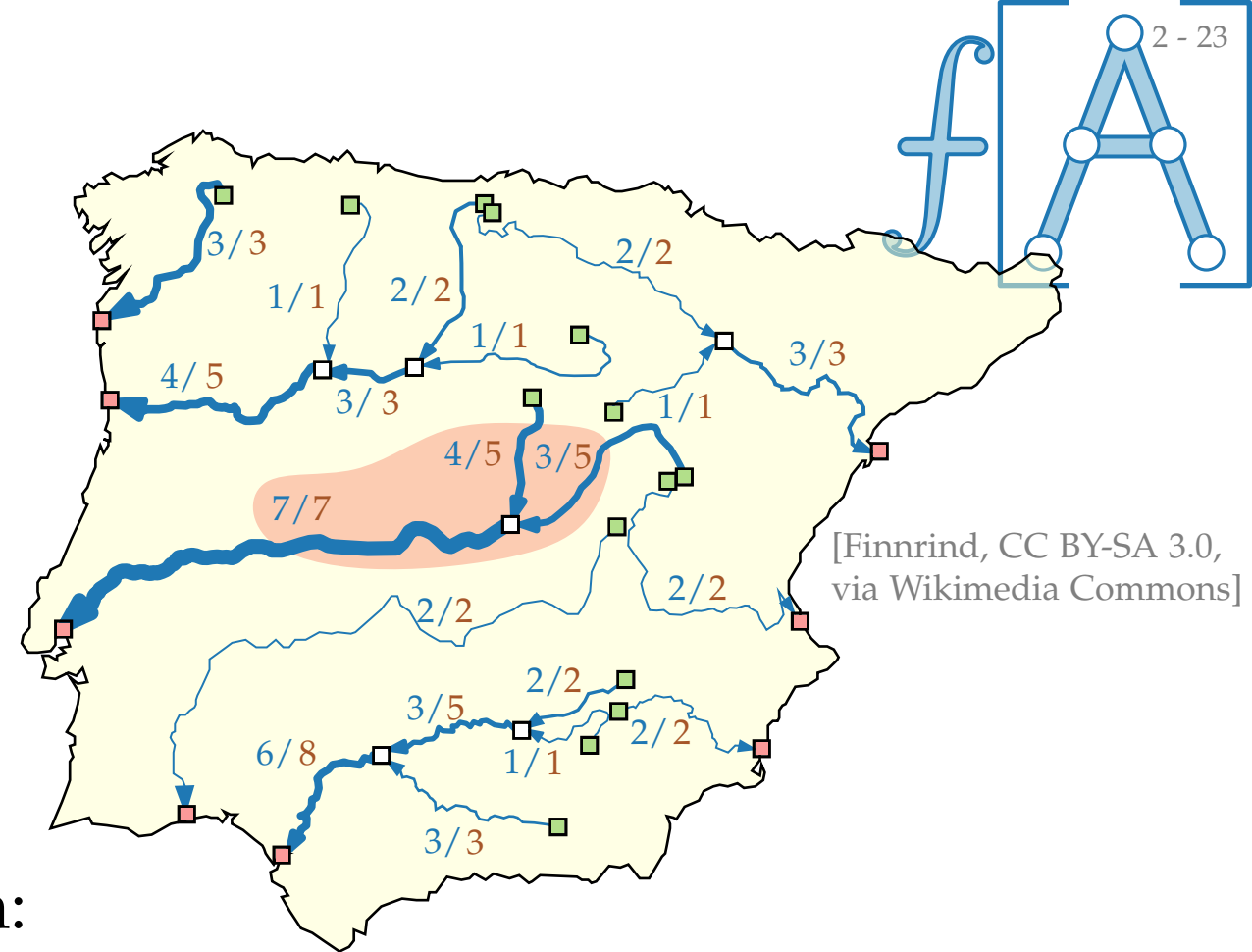
- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus (S \cup T) \end{aligned}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus



Flussnetzwerke

Flussnetzwerk $(G = (V, E); S; T; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

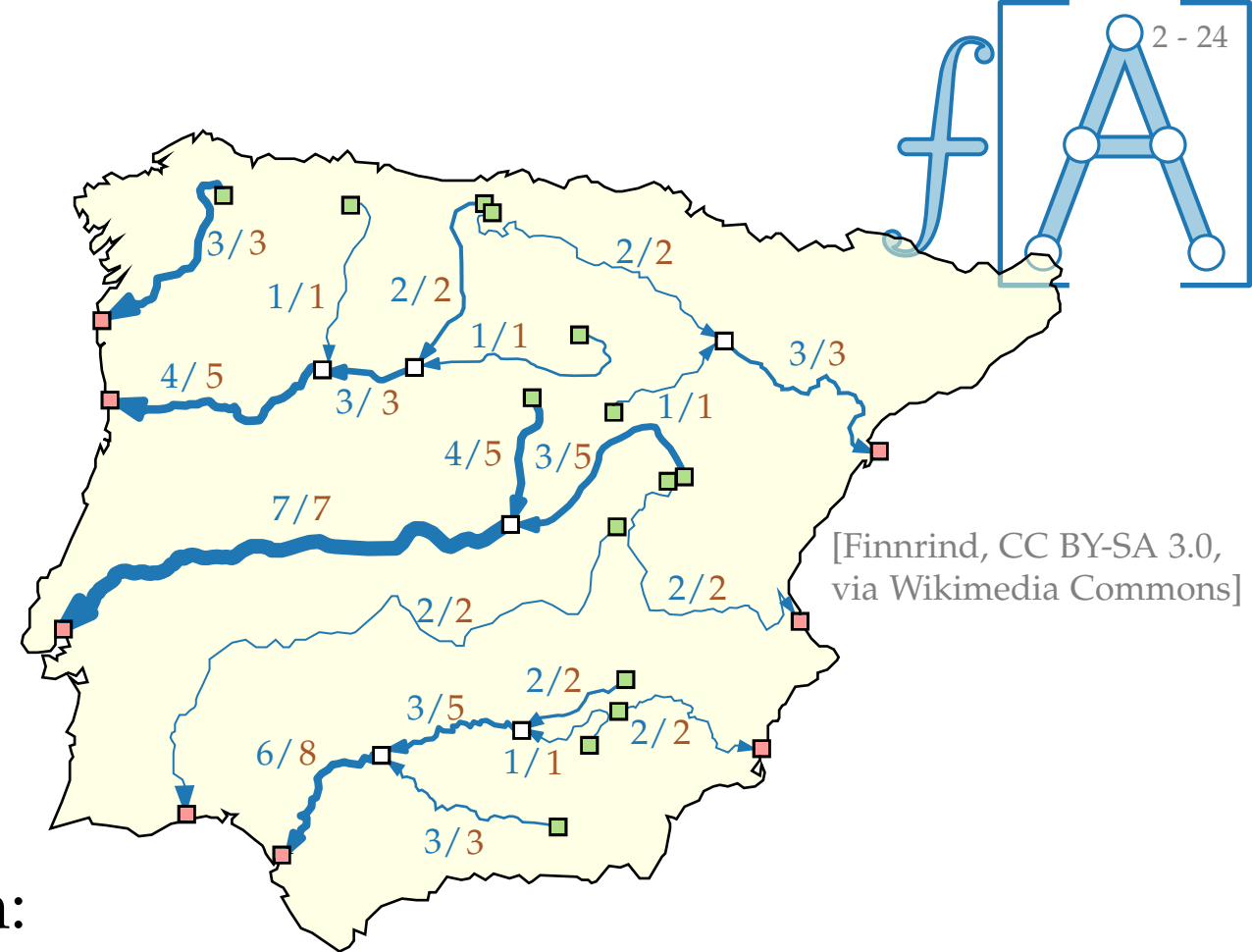
$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

$$\sum_{(u,v) \in E} f(u, v) - \sum_{(v,u) \in E} f(v, u) = 0 \quad \forall v \in V \setminus (S \cup T)$$

Kapazitäten einhalten

Flusserhaltung: rein = raus

Ein **größter** S - T -Fluss ist ein S - T -Fluss, der $\sum_{(u,v) \in E, v \in T} f(u, v)$ maximiert.



s - t -Flussnetzwerke

Flussnetzwerk ($G = (V, E); S; T; c$) mit

- gerichteter Graph $G = (V, E)$
- *Quellen* $S \subseteq V$, *Senken* $T \subseteq V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **S - T -Fluss** wenn:

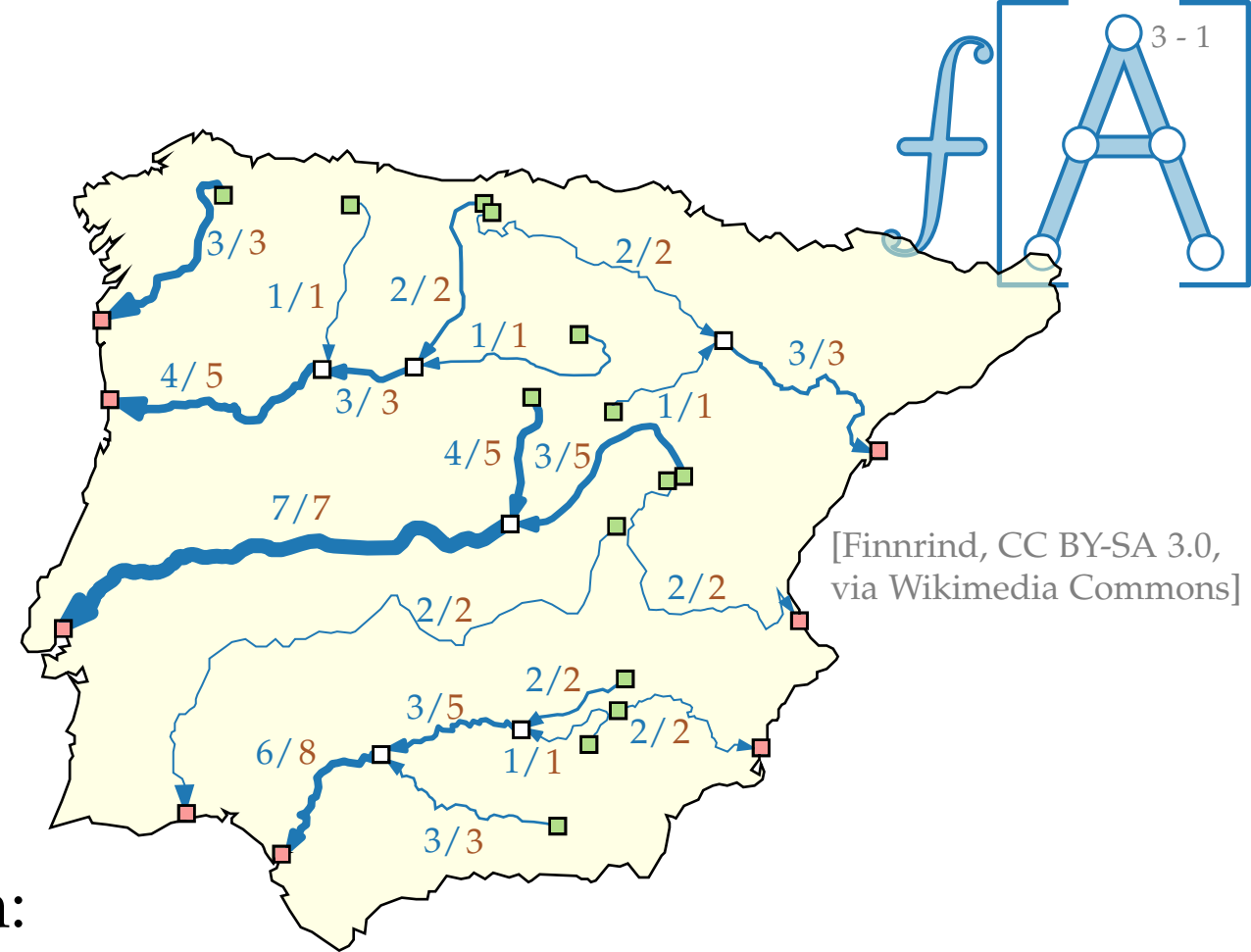
$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

$$\sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 \quad \forall v \in V \setminus (S \cup T)$$

Kapazitäten einhalten

Flusserhaltung: rein = raus

Ein **größter** S - T -Fluss ist ein S - T -Fluss, der $\sum_{(u, v) \in E, v \in T} f(u, v)$ maximiert.



s - t -Flussnetzwerke

Flussnetzwerk ($G = (V, E); s; t; c$) mit

- gerichteter Graph $G = (V, E)$
- *Quelle* $s \in V$, *Senke* $t \in V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **s - t -Fluss** wenn:

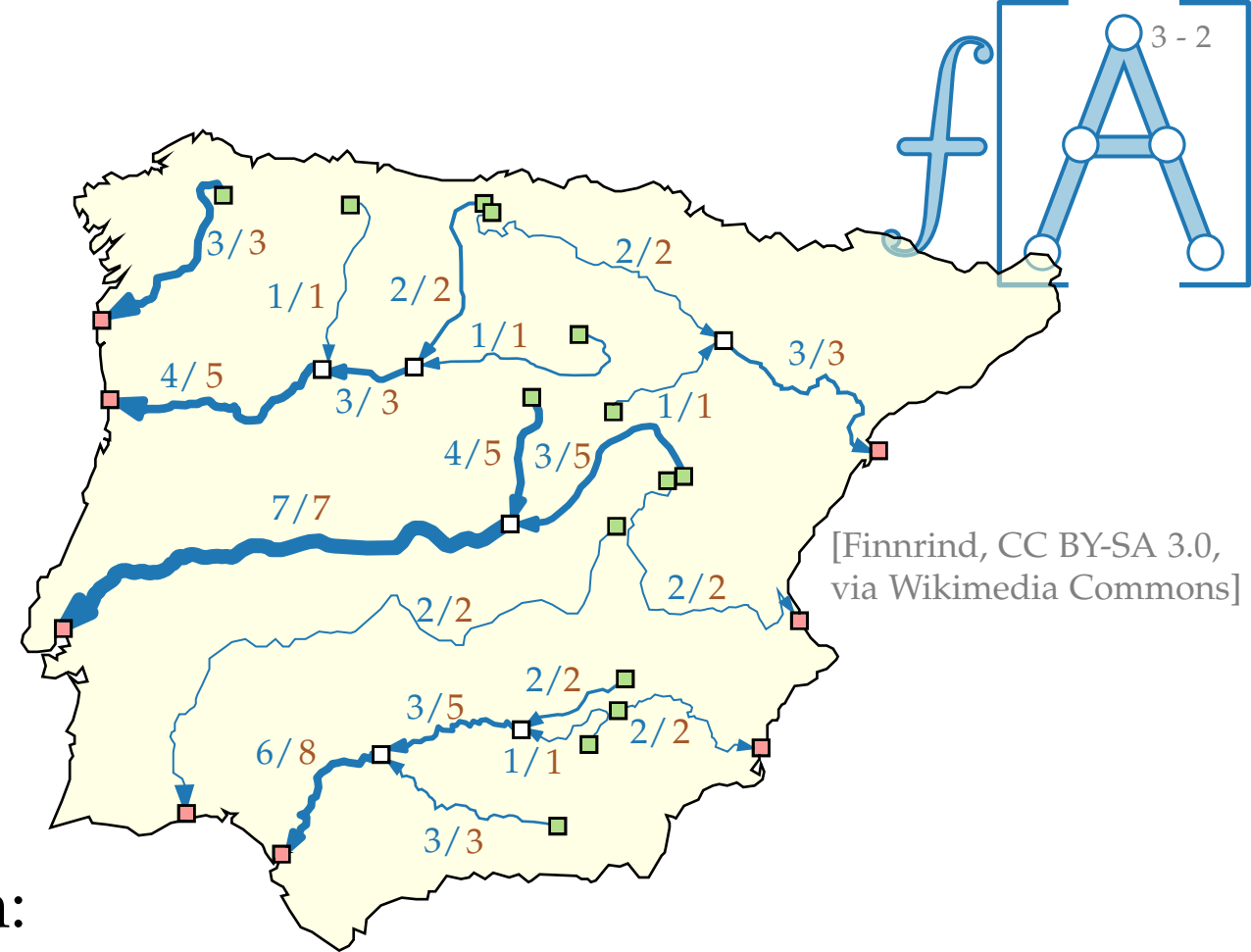
$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

$$\sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 \quad \forall v \in V \setminus (s \cup t)$$

Kapazitäten einhalten

Flusserhaltung: rein = raus

Ein **größter** s - t -Fluss ist ein s - t -Fluss, der $\sum_{(u, v) \in E, v \in t} f(u, v)$ maximiert.



s - t -Flussnetzwerke

Flussnetzwerk ($G = (V, E); s; t; c$) mit

- gerichteter Graph $G = (V, E)$
- *Quelle* $s \in V$, *Senke* $t \in V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **s - t -Fluss** wenn:

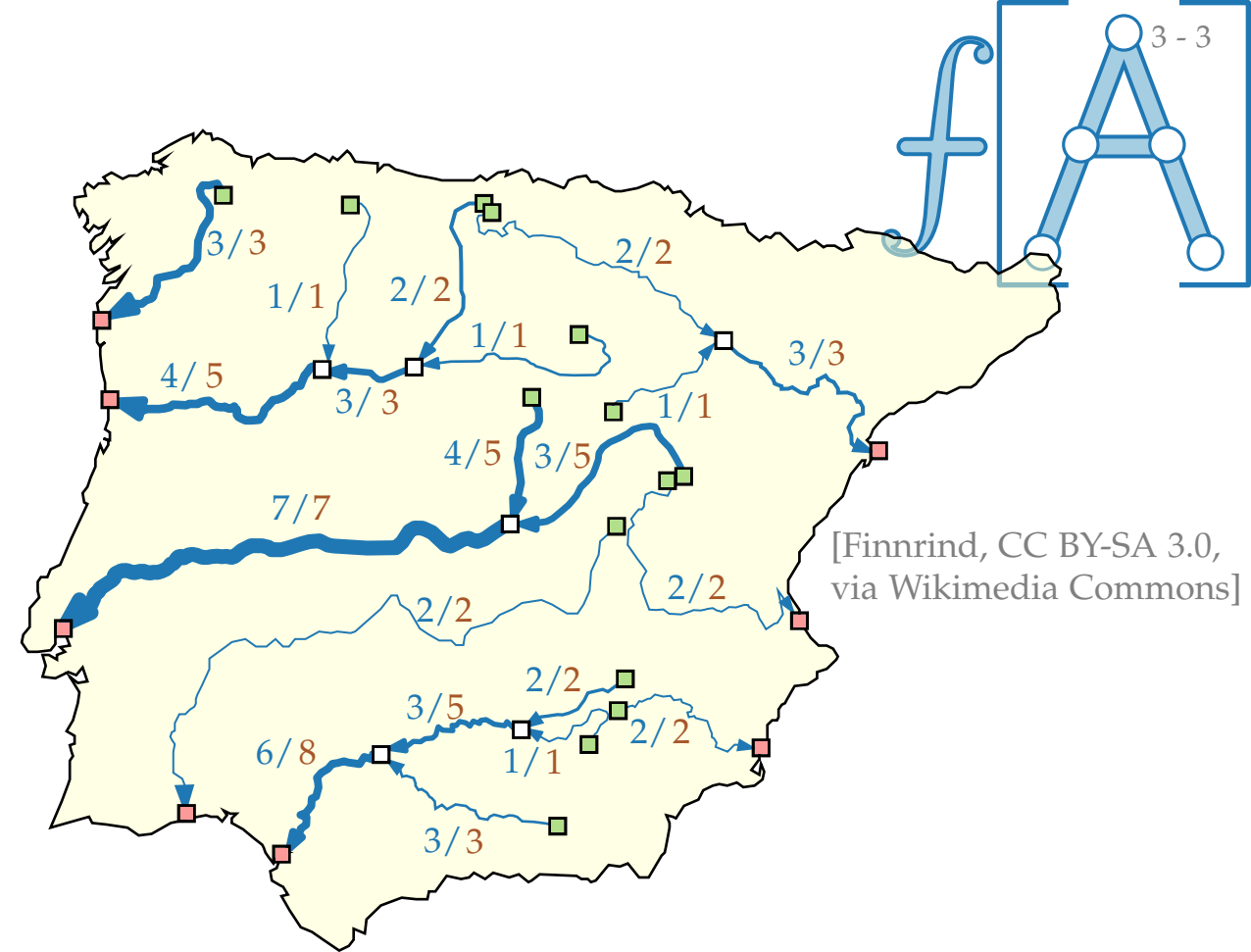
$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

$$\sum_{(u,v) \in E} f(u, v) - \sum_{(v,u) \in E} f(v, u) = 0 \quad \forall v \in V \setminus \{s, t\}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus

Ein **größter** S - T -Fluss ist ein S - T -Fluss, der $\sum_{(u,v) \in E, v \in T} f(u, v)$ maximiert.



s - t -Flussnetzwerke

Flussnetzwerk ($G = (V, E); s; t; c$) mit

- gerichteter Graph $G = (V, E)$
- *Quelle* $s \in V$, *Senke* $t \in V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **s - t -Fluss** wenn:

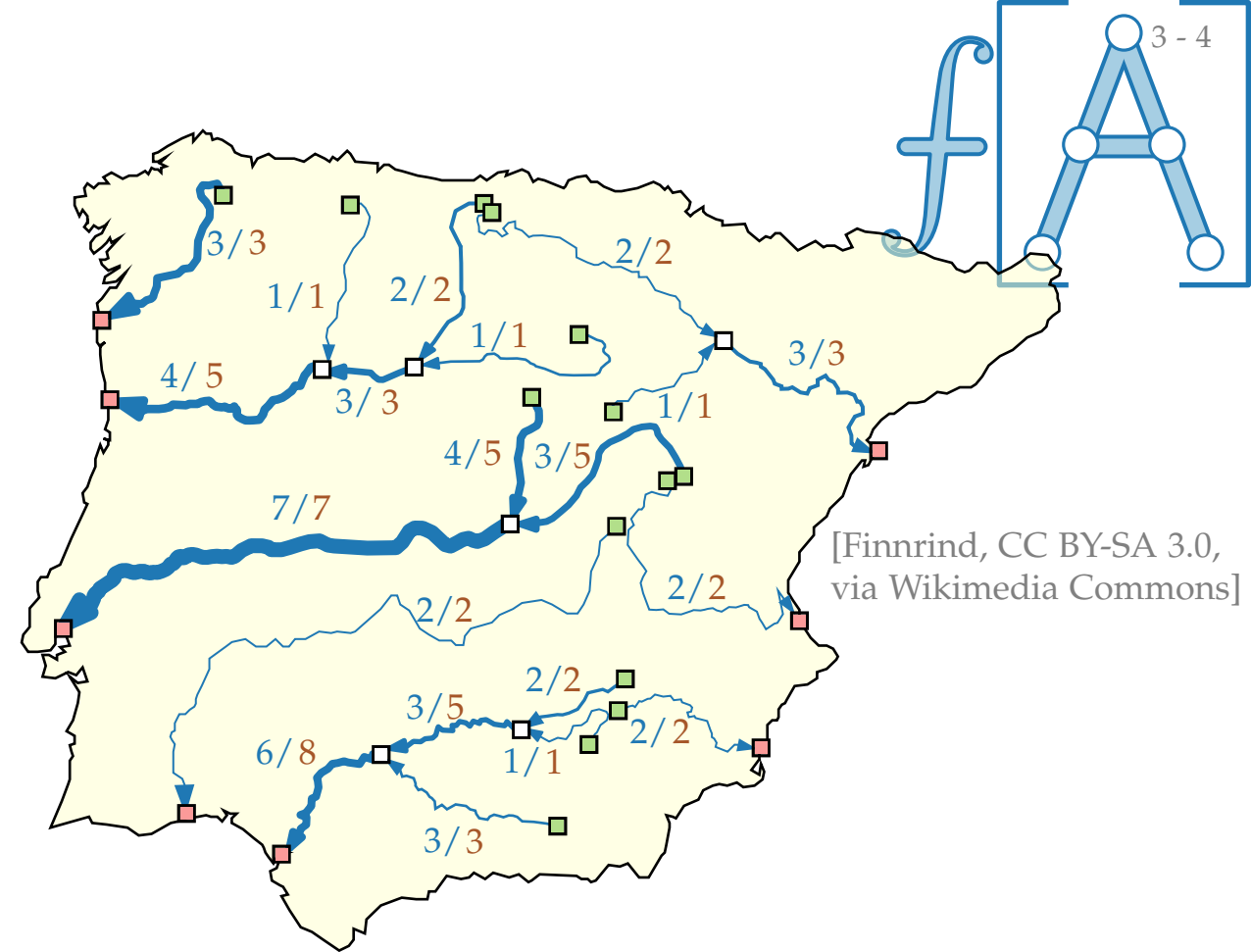
$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

$$\sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 \quad \forall v \in V \setminus \{s, t\}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus

Ein **größter** s - t -Fluss ist ein s - t -Fluss, der $\sum_{(u, t) \in E} f(u, t)$ maximiert.



s - t -Flussnetzwerke

Flussnetzwerk ($G = (V, E); s; t; c$) mit

- gerichteter Graph $G = (V, E)$
- *Quelle* $s \in V$, *Senke* $t \in V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt s - t -**Fluss** wenn:

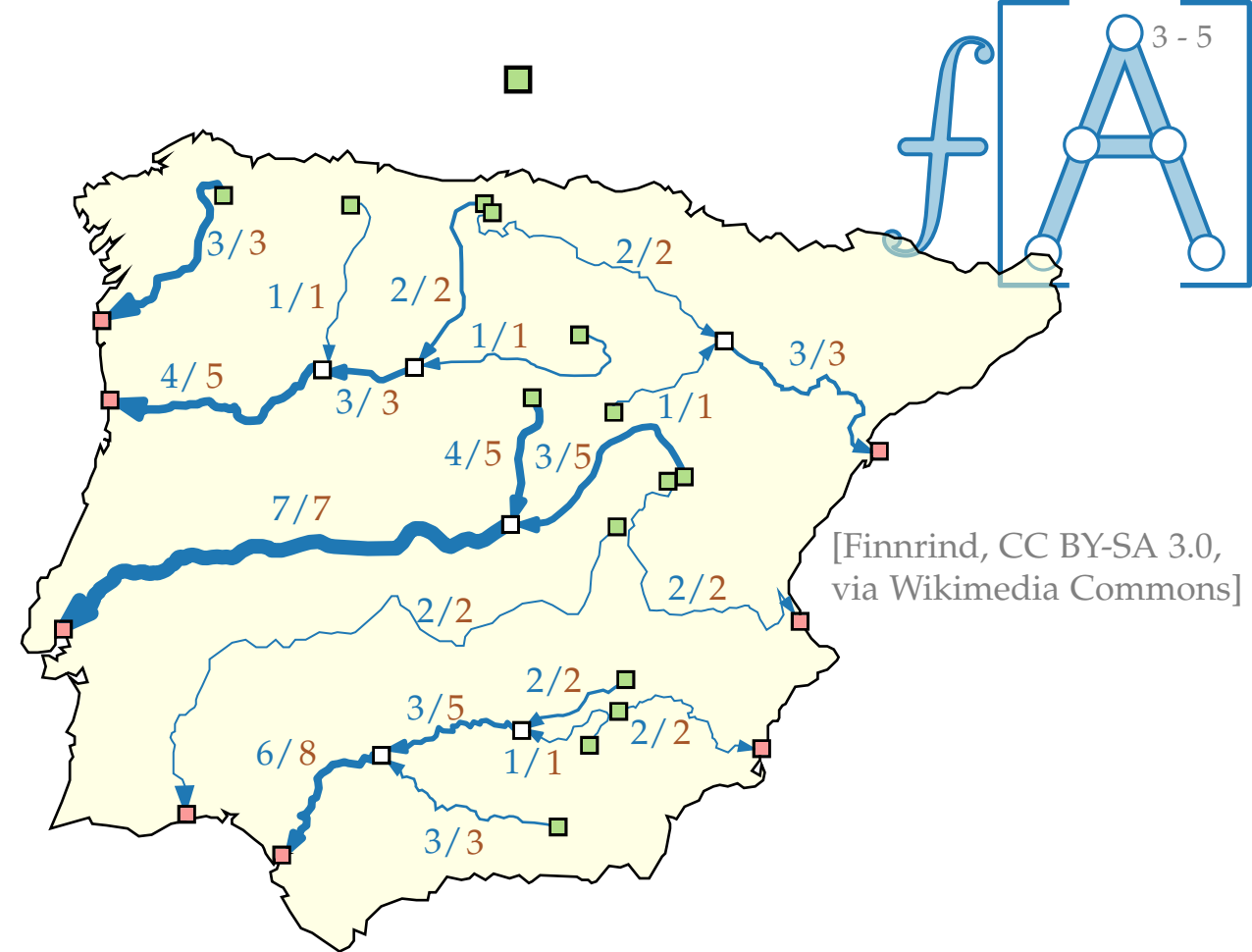
$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

$$\sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 \quad \forall v \in V \setminus \{s, t\}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus

Ein **größter** s - t -Fluss ist ein s - t -Fluss, der $\sum_{(u, t) \in E} f(u, t)$ maximiert.



s - t -Flussnetzwerke

Flussnetzwerk ($G = (V, E); s; t; c$) mit

- gerichteter Graph $G = (V, E)$
- *Quelle* $s \in V$, *Senke* $t \in V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **s - t -Fluss** wenn:

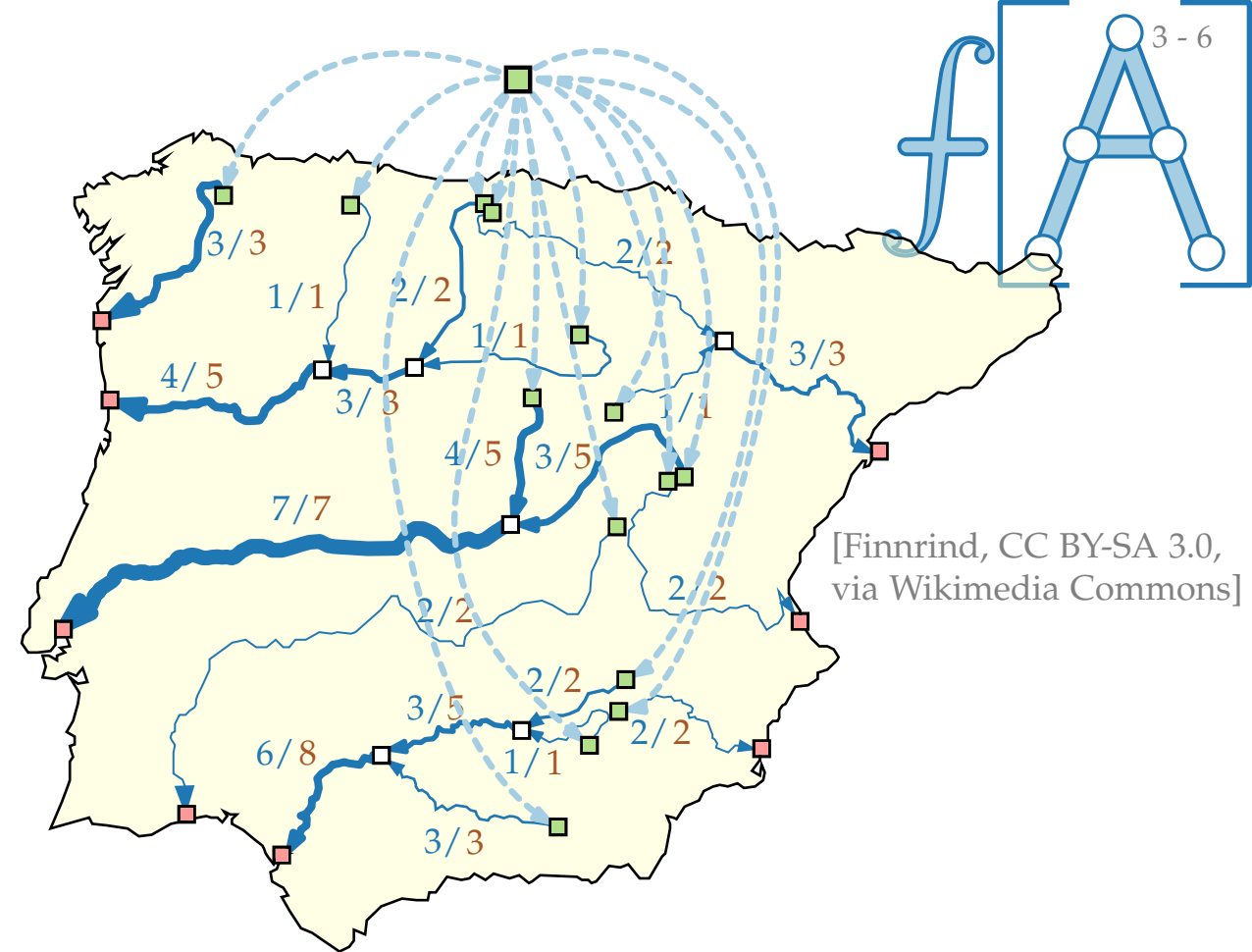
$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

$$\sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 \quad \forall v \in V \setminus \{s, t\}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus

Ein **größter** s - t -Fluss ist ein s - t -Fluss, der $\sum_{(u, t) \in E} f(u, t)$ maximiert.



s - t -Flussnetzwerke

Flussnetzwerk $(G = (V, E); s; t; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quelle* $s \in V$, *Senke* $t \in V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **s-t-Fluss** wenn:

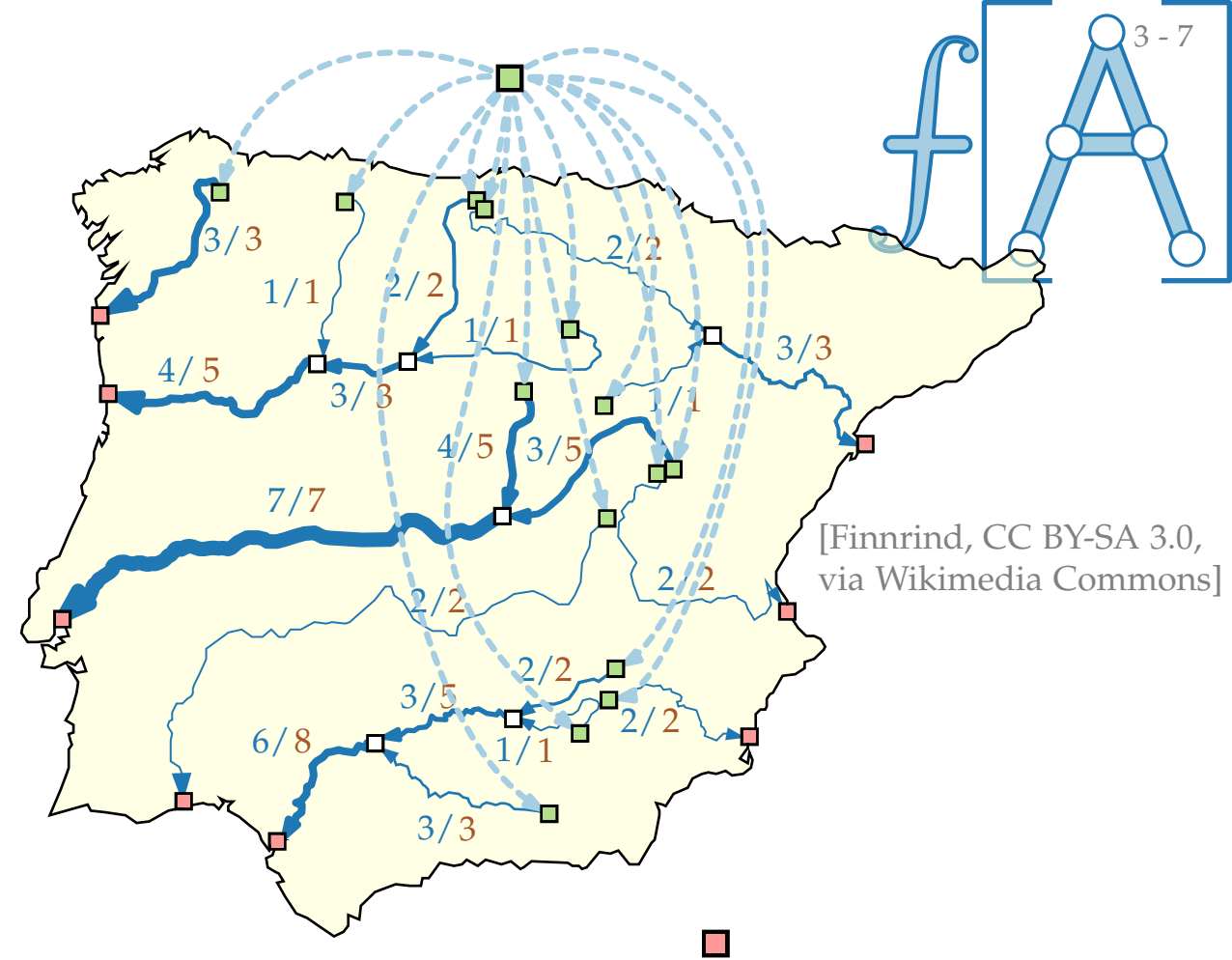
$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

$$\sum_{(u,v) \in E} f(u,v) - \sum_{(v,u) \in E} f(v,u) = 0 \quad \forall v \in V \setminus \{s, t\}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus

Ein **größter** s - t -Fluss ist ein s - t -Fluss, der $\sum_{(u,t) \in E} f(u,t)$ maximiert.



[Finnrind, CC BY-SA 3.0,
via Wikimedia Commons]

s - t -Flussnetzwerke

Flussnetzwerk ($G = (V, E); s; t; c$) mit

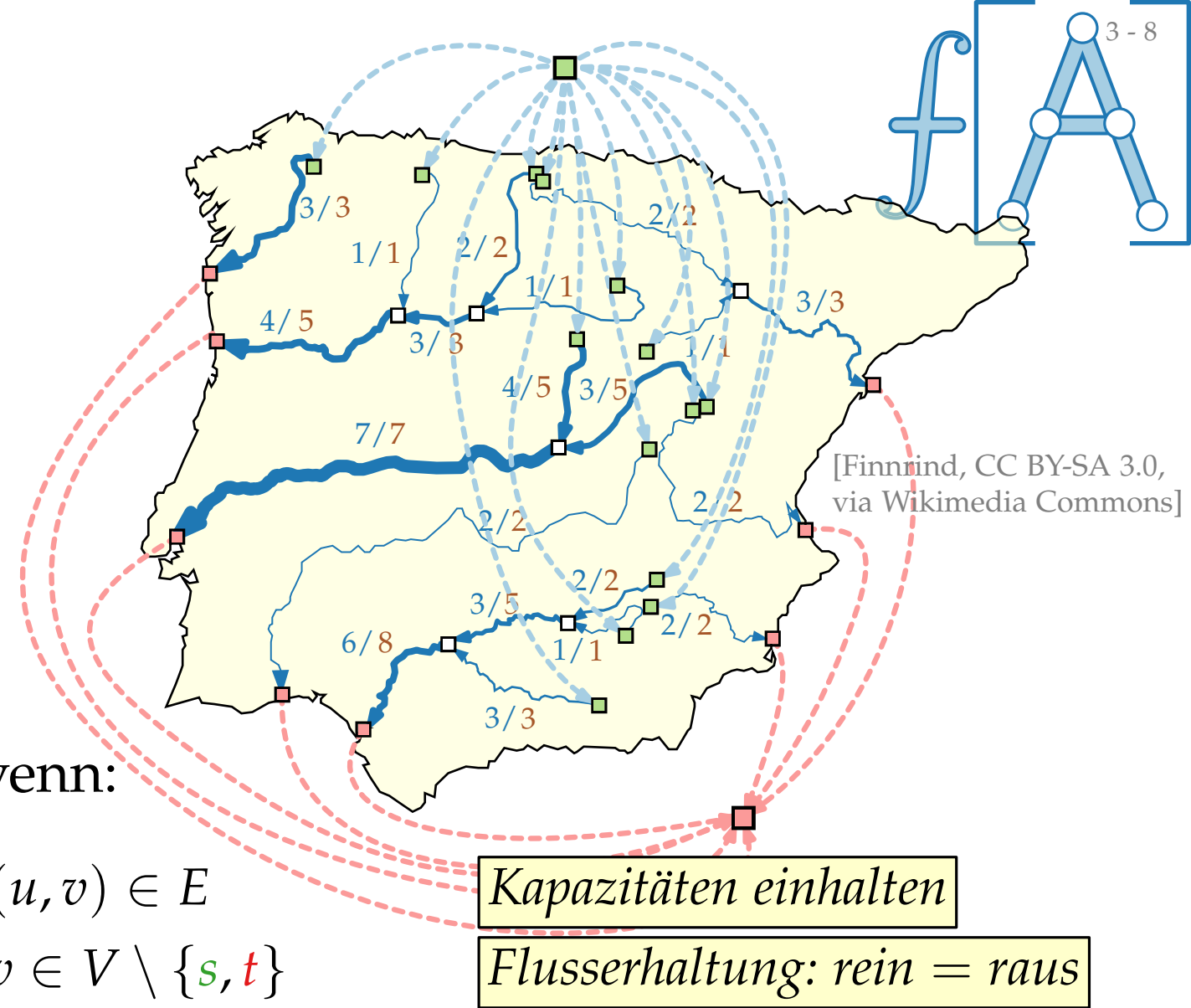
- gerichteter Graph $G = (V, E)$
- *Quelle* $s \in V$, *Senke* $t \in V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **s - t -Fluss** wenn:

$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

$$\sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 \quad \forall v \in V \setminus \{s, t\}$$

Ein **größter** s - t -Fluss ist ein s - t -Fluss, der $\sum_{(u, t) \in E} f(u, t)$ maximiert.



s - t -Flussnetzwerke

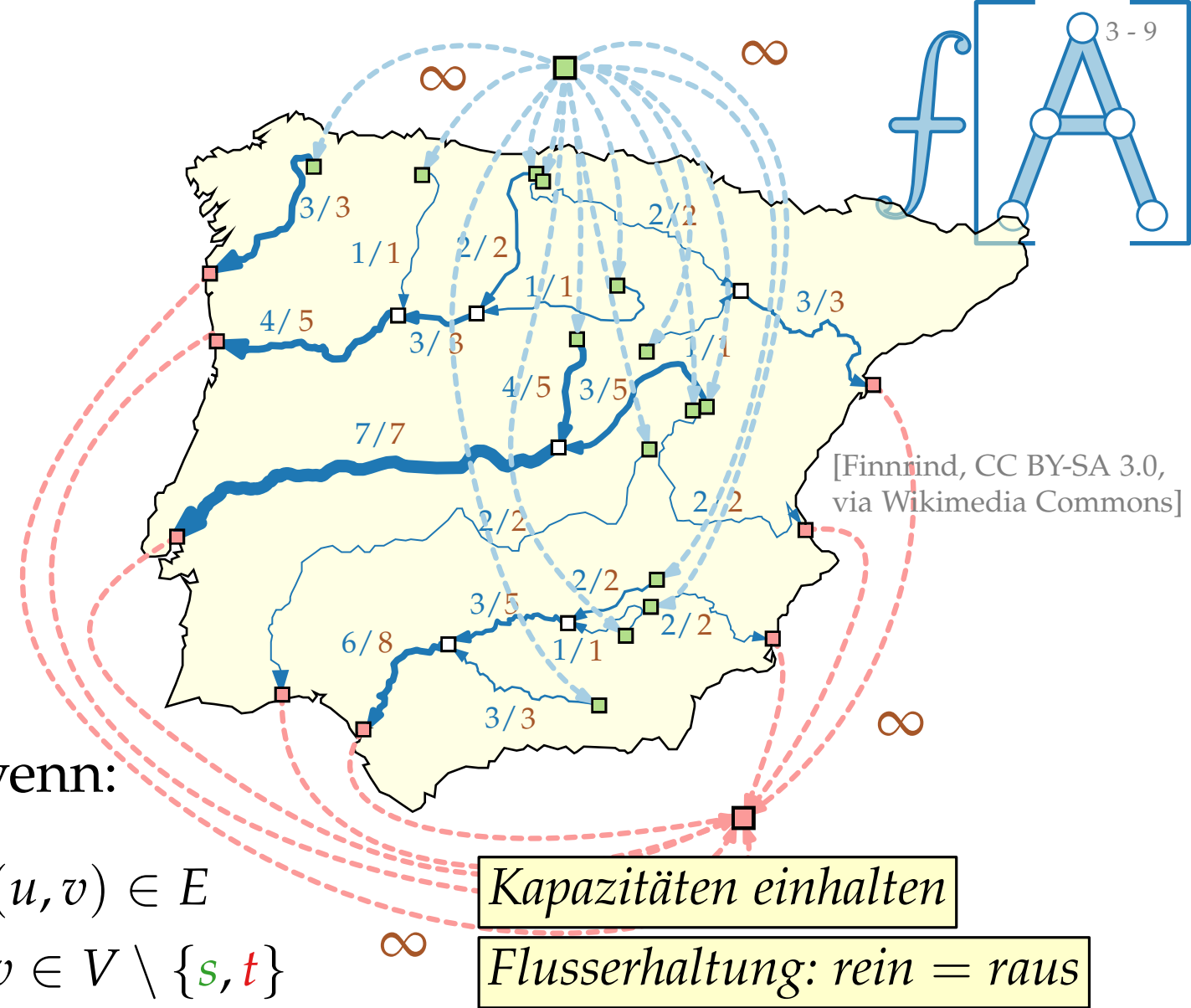
Flussnetzwerk ($G = (V, E); s; t; c$) mit

- gerichteter Graph $G = (V, E)$
- *Quelle* $s \in V$, *Senke* $t \in V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **s - t -Fluss** wenn:

$$\begin{aligned} 0 \leq f(u, v) \leq c(u, v) & \quad \forall (u, v) \in E \\ \sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0 & \quad \forall v \in V \setminus \{s, t\} \end{aligned}$$

Ein **größter** s - t -Fluss ist ein s - t -Fluss, der $\sum_{(u, t) \in E} f(u, t)$ maximiert.



s-t-Flussnetzwerke

Flussnetzwerk $(G = (V, E); s; t; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quelle* $s \in V$, *Senke* $t \in V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **s-t-Fluss** wenn:

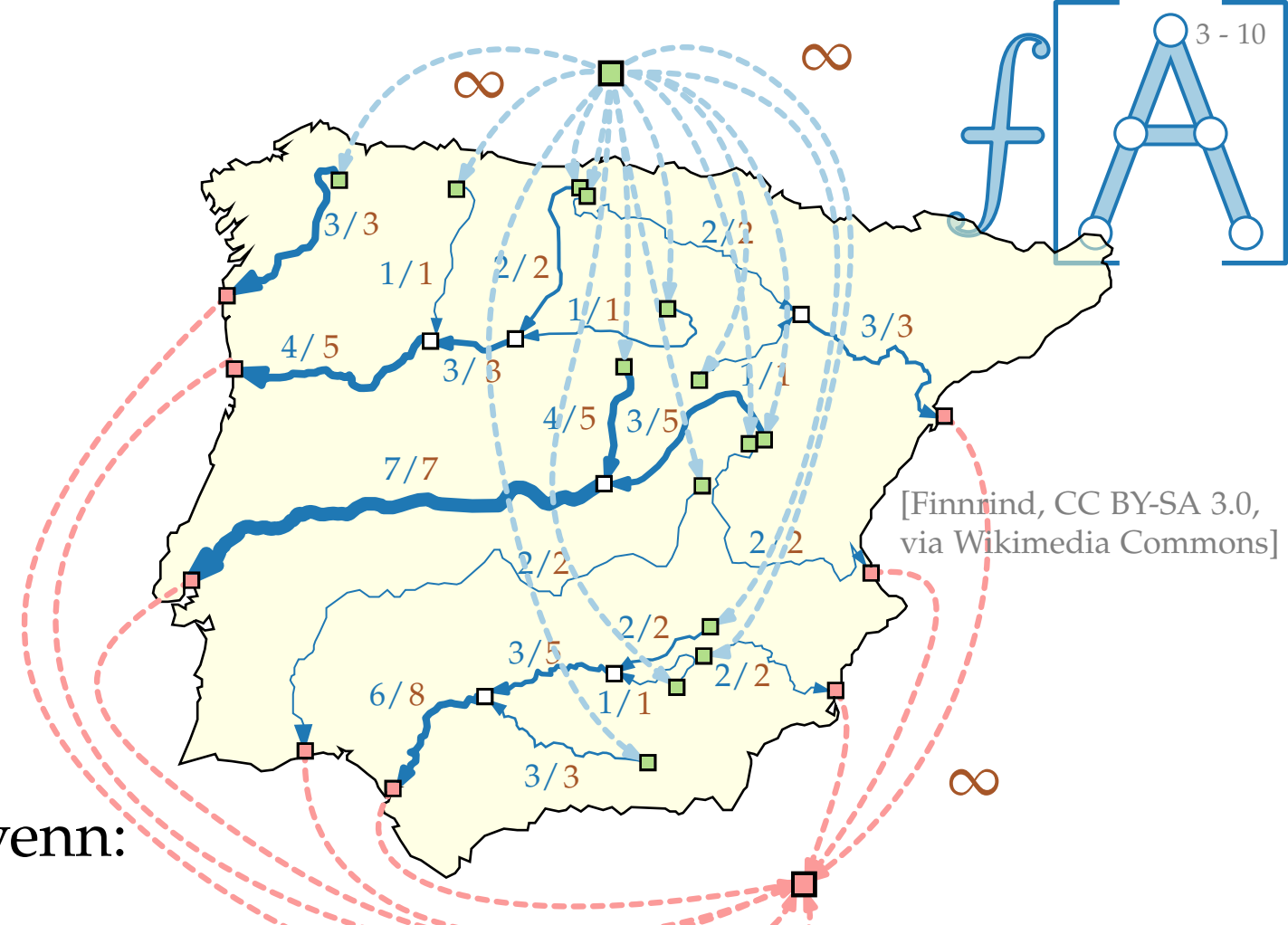
$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

$$\sum_{(u,v) \in E} f(u,v) - \sum_{(v,u) \in E} f(v,u) = 0 \quad \forall v \in V \setminus \{s, t\}$$

Kapazitäten einhalten

Flusserhaltung: rein = raus

Ein **größter** s - t -Fluss ist ein s - t -Fluss, der $\sum_{(u,t) \in E} f(u,t)$ maximiert.

$$- \text{Zufluss}_f(t)$$


s - t -Flussnetzwerke

Flussnetzwerk $(G = (V, E); s; t; c)$ mit

- gerichteter Graph $G = (V, E)$
- *Quelle* $s \in V$, *Senke* $t \in V$
- *Kantenkapazität* $c: E \rightarrow \mathbb{R}_0^+$

Eine Funktion $f: E \rightarrow \mathbb{R}_0^+$ heißt **s - t -Fluss** wenn:

$$0 \leq f(u, v) \leq c(u, v)$$

$$\forall (u, v) \in E$$

$$\sum_{(u, v) \in E} f(u, v) - \sum_{(v, u) \in E} f(v, u) = 0$$

$$\forall v \in V \setminus \{s, t\}$$

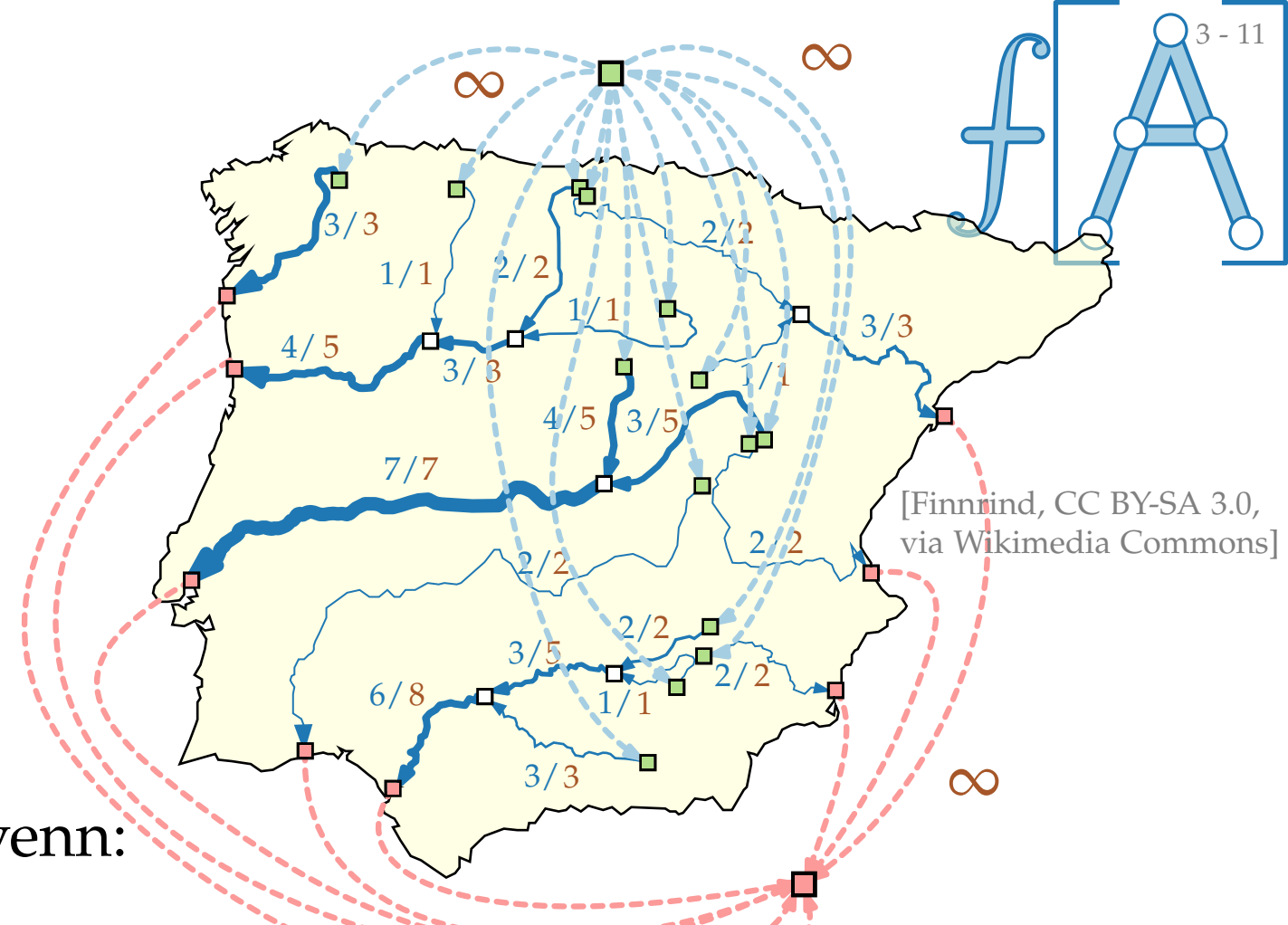
Kapazitäten einhalten

Flusserhaltung: rein = raus

Nettozufluss $f(v)$

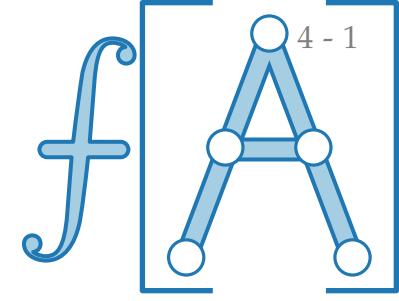
Ein **größter** s - t -Fluss ist ein s - t -Fluss, der $\sum_{(u, t) \in E} f(u, t)$ maximiert.

Zufluss $f(t)$

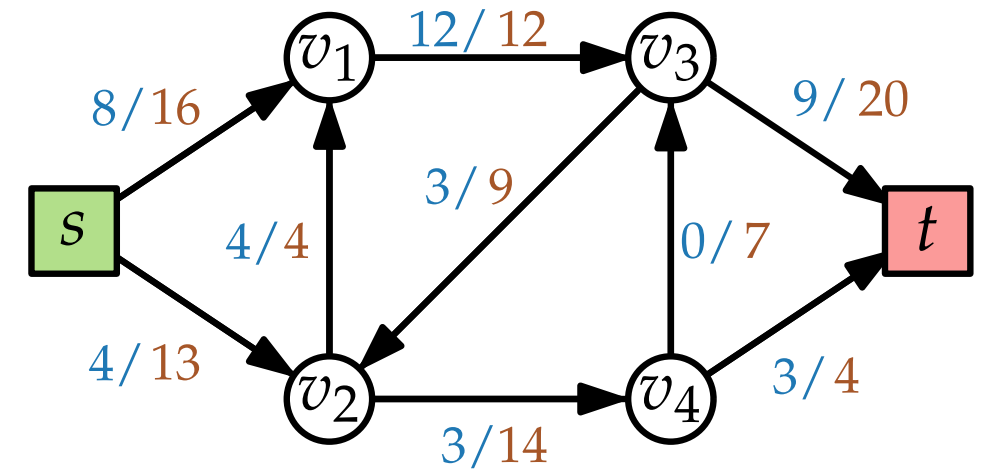


[Finnrind, CC BY-SA 3.0, via Wikimedia Commons]

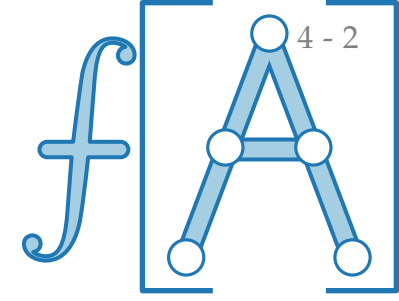
Residualnetzwerk



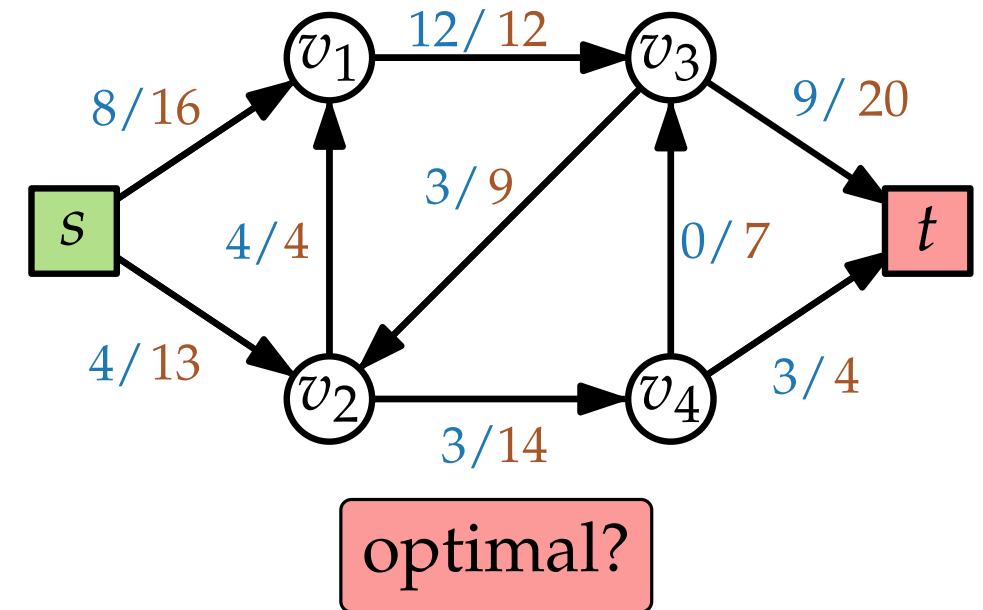
Flussnetzwerk ($G = (V, E); s, t; c$)



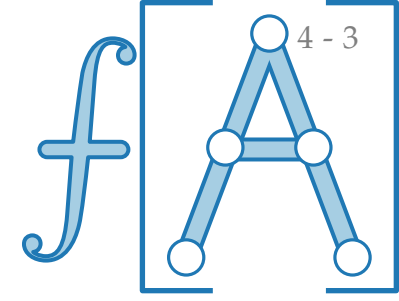
Residualnetzwerk



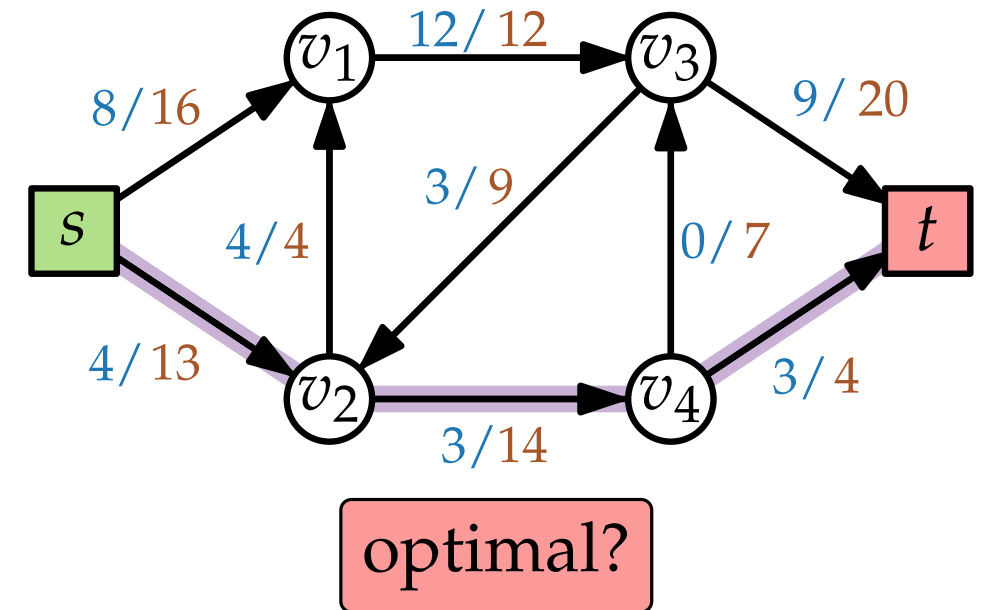
Flussnetzwerk $(G = (V, E); s, t; c)$



Residualnetzwerk

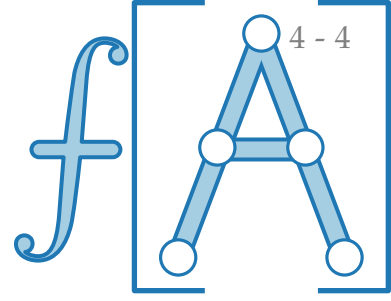


Flussnetzwerk $(G = (V, E); s, t; c)$

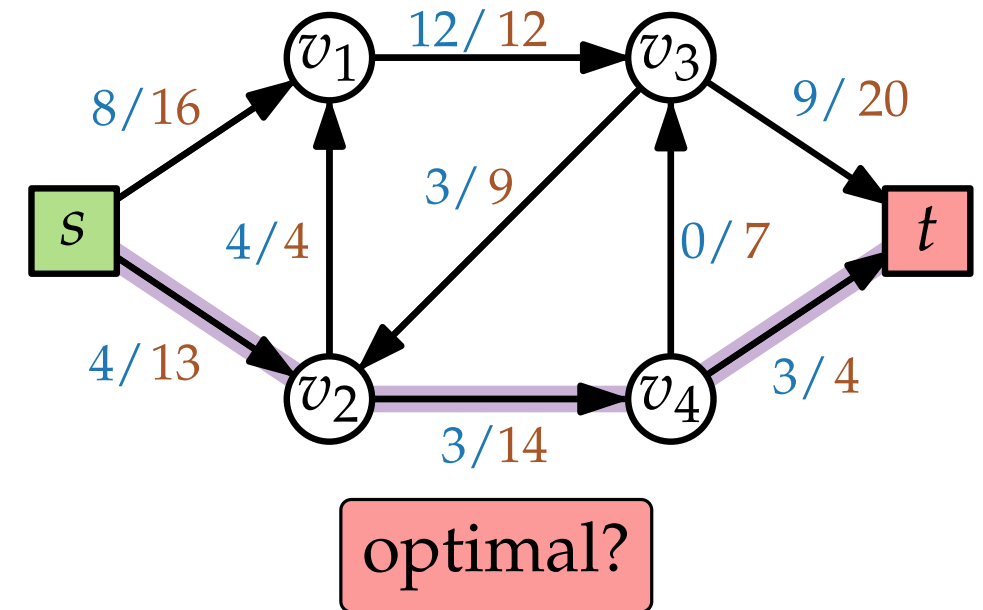


Residualnetzwerk

Idee?

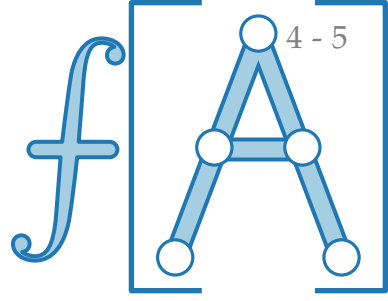


Flussnetzwerk $(G = (V, E); s, t; c)$

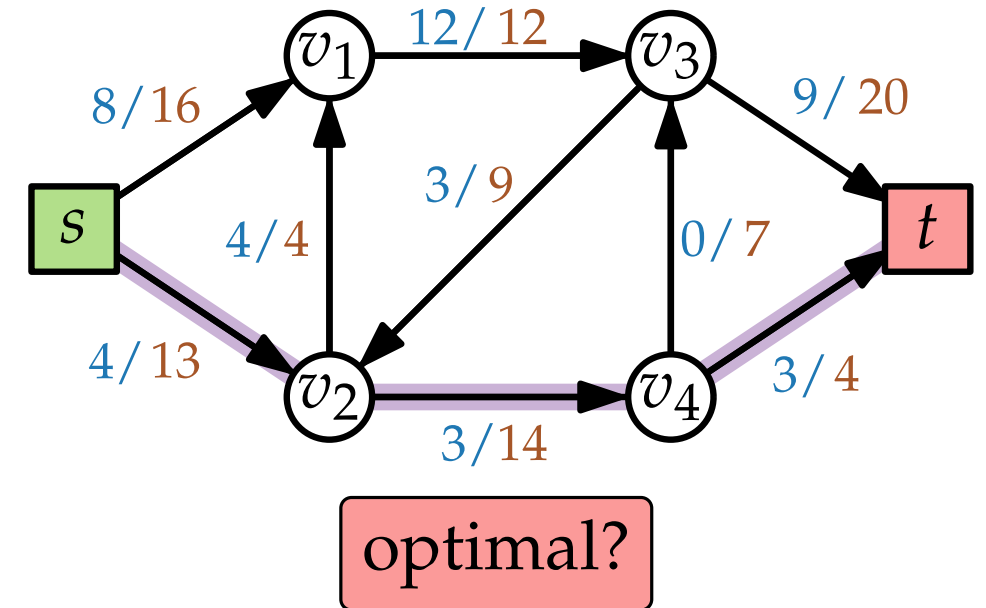


Residualnetzwerk

Idee? Versuche, Fluss schrittweise zu erhöhen.



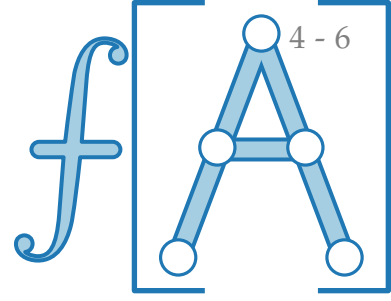
Flussnetzwerk $(G = (V, E); s, t; c)$



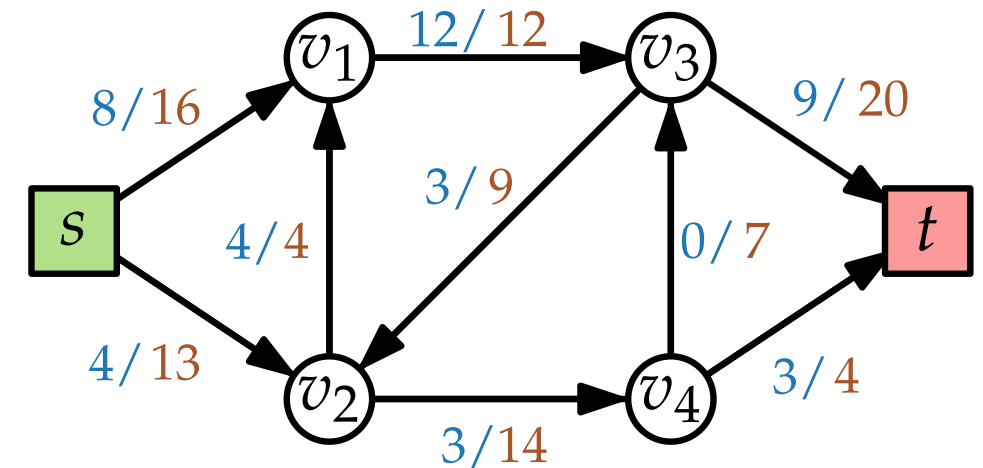
Residualnetzwerk

Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:



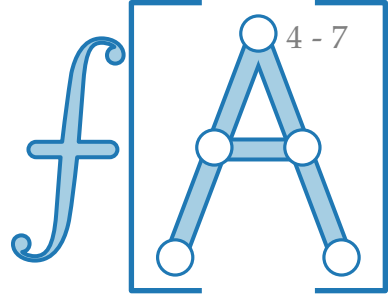
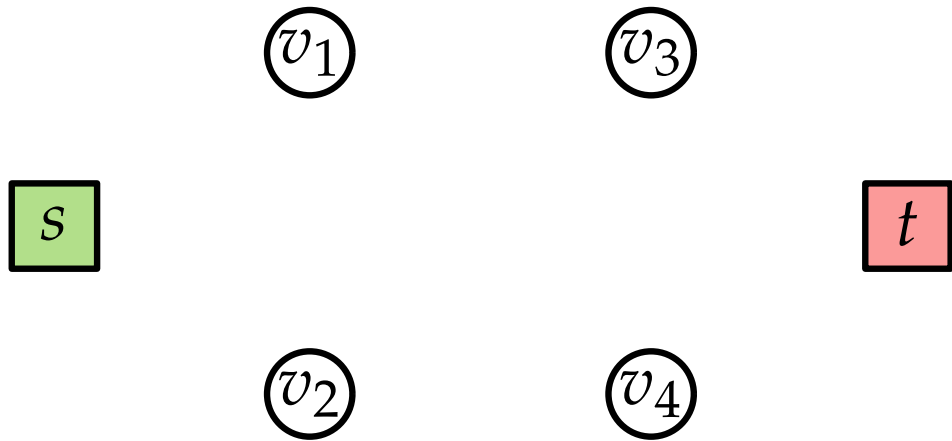
Flussnetzwerk $(G = (V, E); s, t; c)$



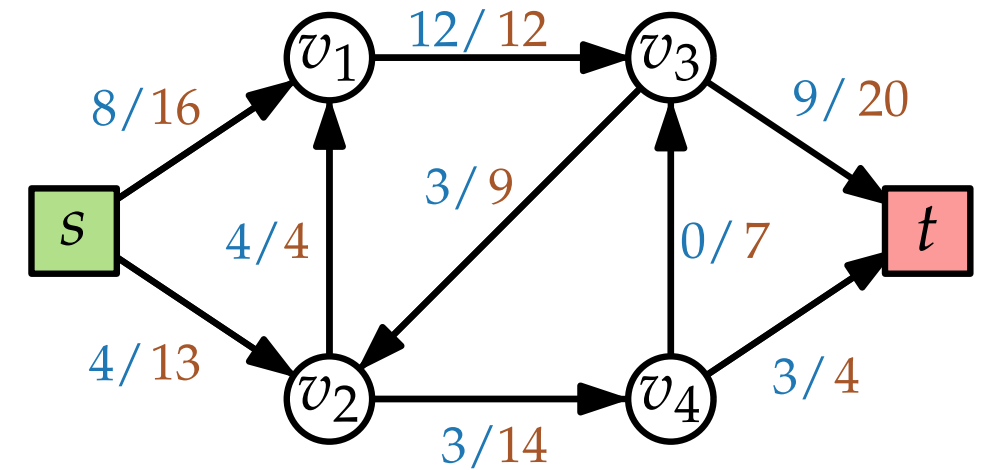
Residualnetzwerk

Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:



Flussnetzwerk $(G = (V, E); s, t; c)$

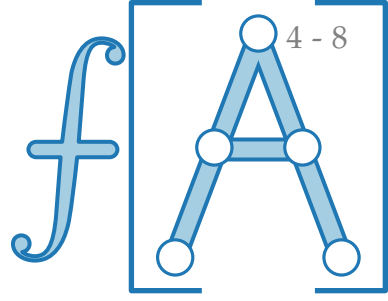


Residualnetzwerk

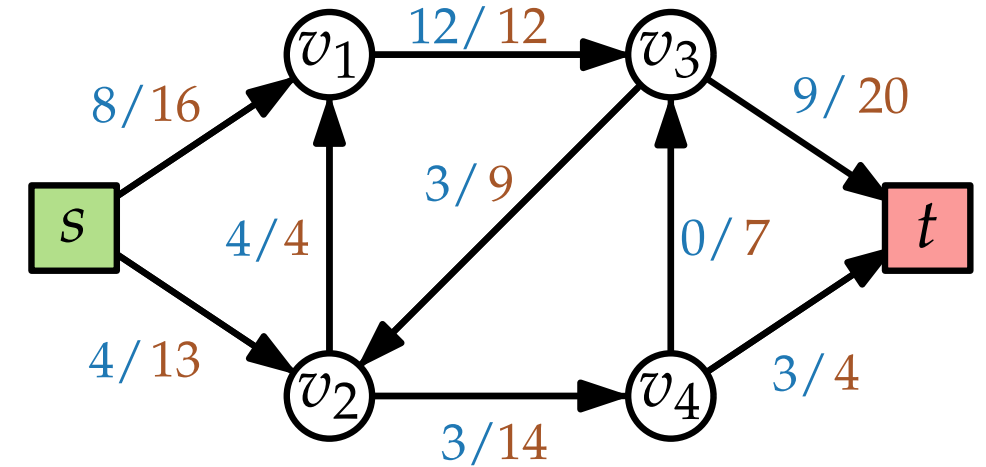
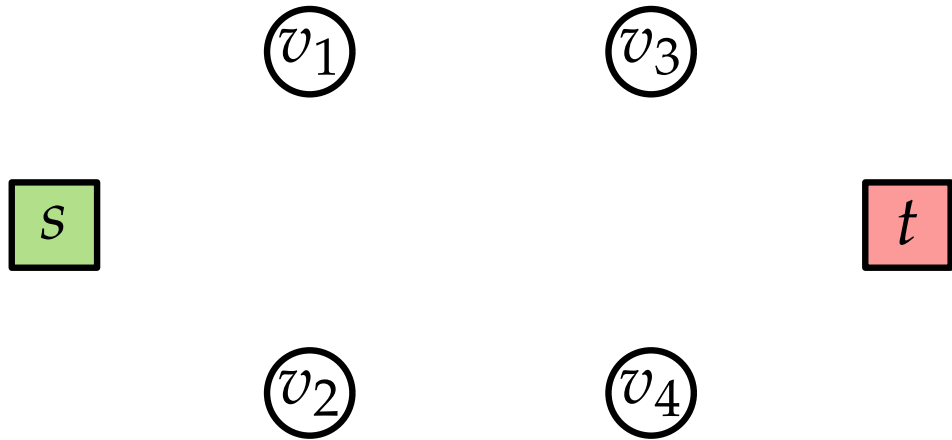
Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:

■ $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$



Flussnetzwerk $(G = (V, E); s, t; c)$

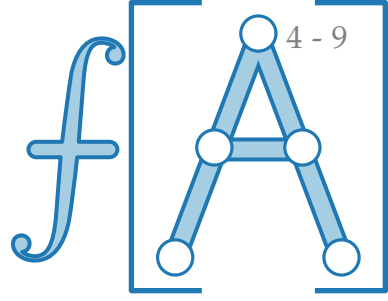
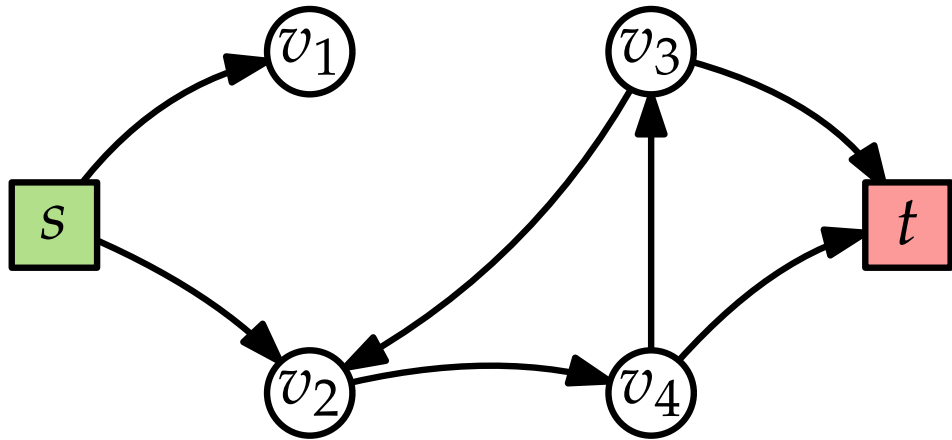


Residualnetzwerk

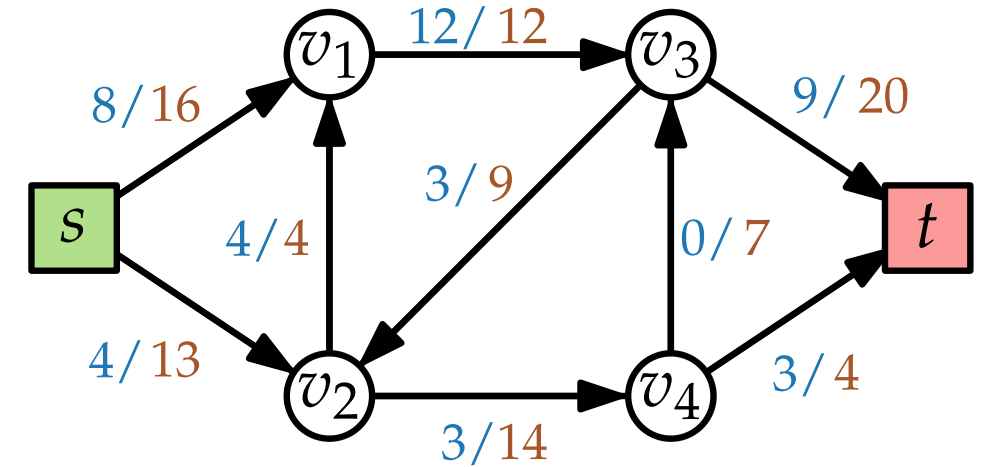
Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:

■ $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$



Flussnetzwerk $(G = (V, E); s, t; c)$



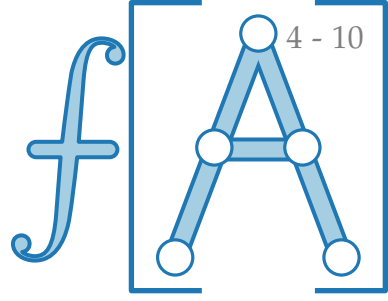
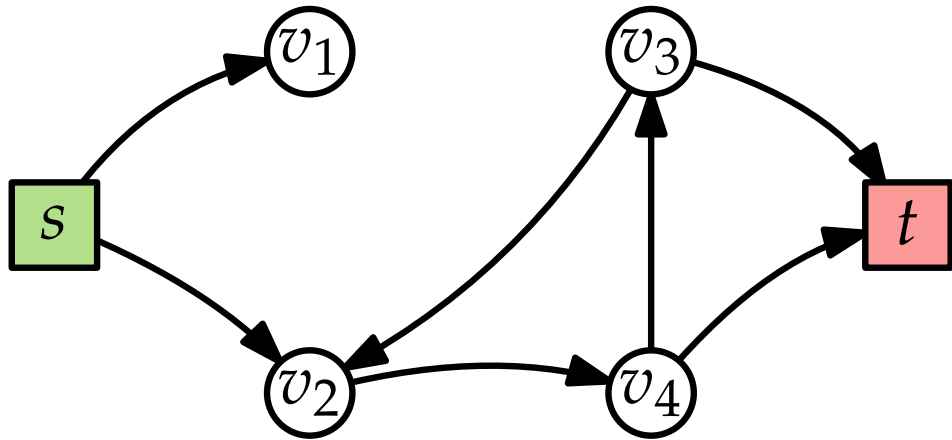
Residualnetzwerk

Idee? Versuche, Fluss schrittweise zu erhöhen.

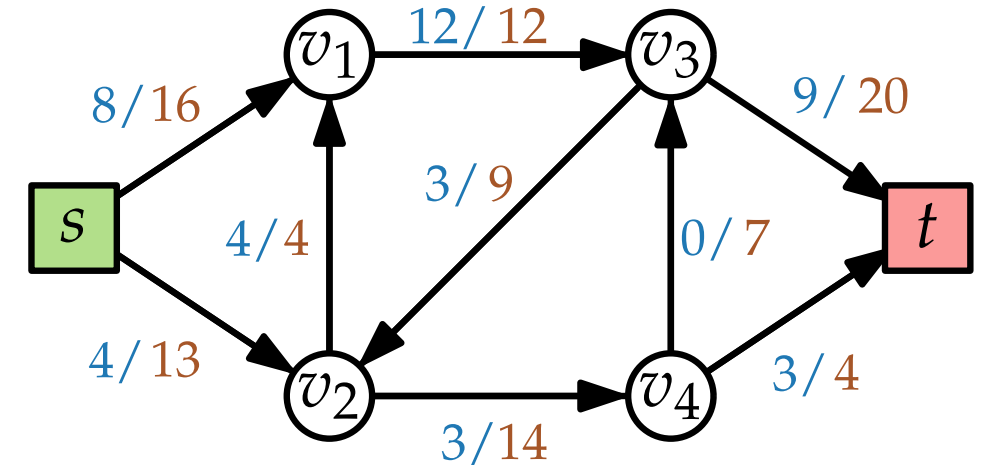
Residualnetzwerk $G_f = (V, E')$:

■ $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$

■ $f(v, v') > 0 \Rightarrow (v', v) \in E'$



Flussnetzwerk $(G = (V, E); s, t; c)$



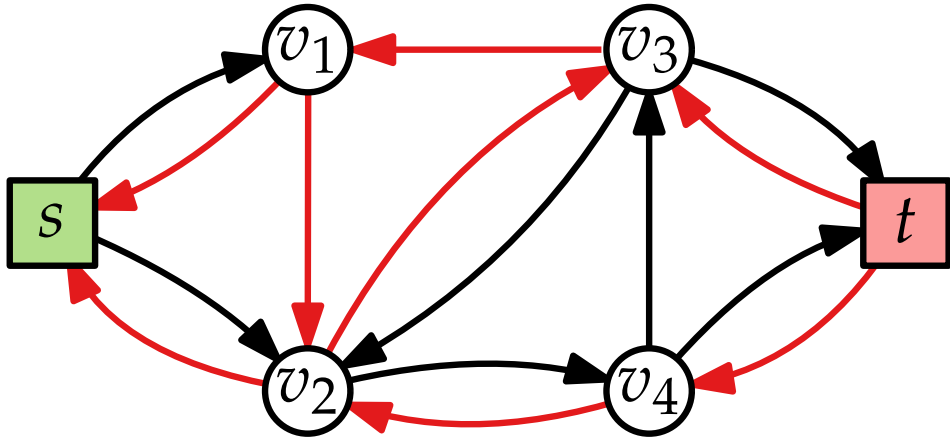
Residualnetzwerk

Idee? Versuche, Fluss schrittweise zu erhöhen.

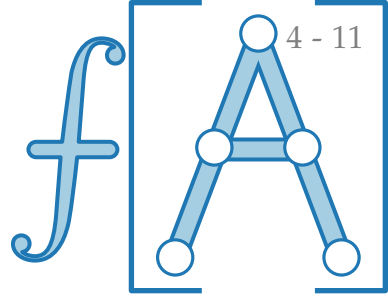
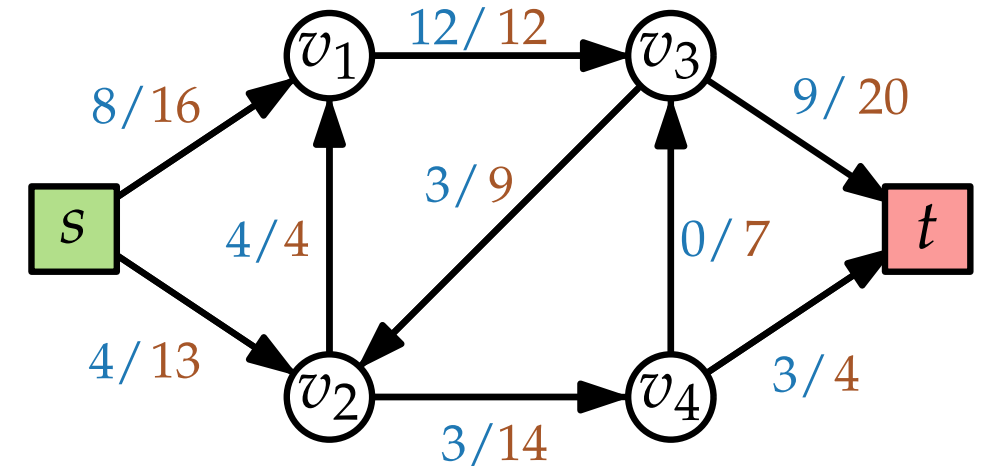
Residualnetzwerk $G_f = (V, E')$:

■ $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$

■ $f(v, v') > 0 \Rightarrow (v', v) \in E'$



Flussnetzwerk ($G = (V, E); s, t; c$)

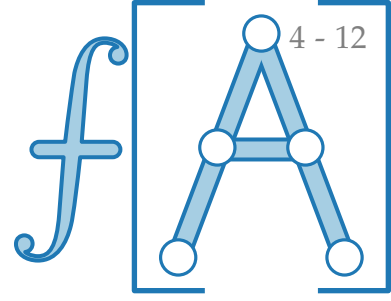
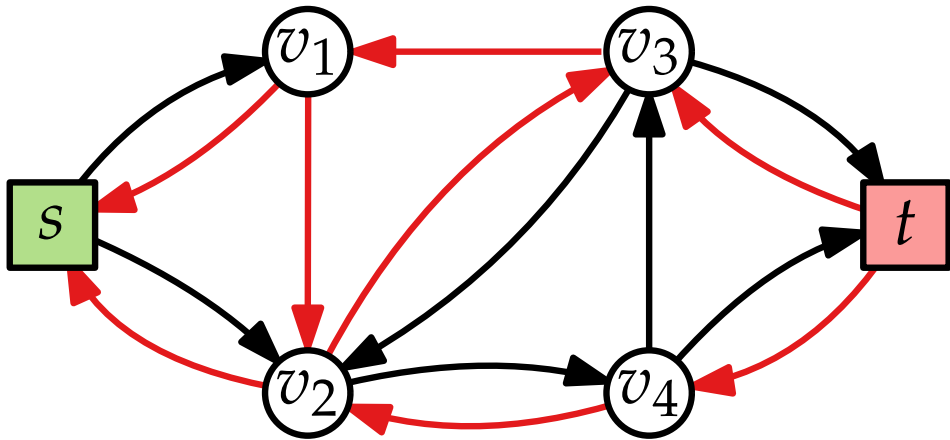


Residualnetzwerk

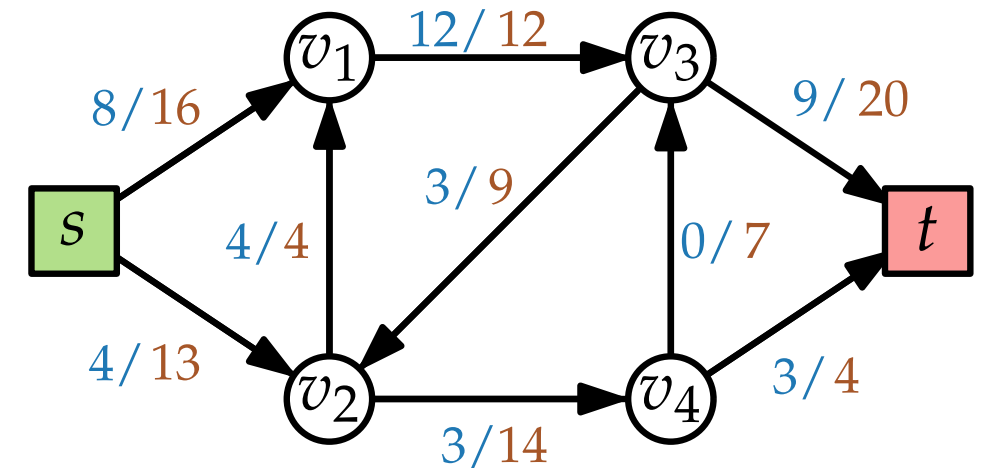
Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:

- $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$
 $C(v, v') = c(v, v') - f(v, v')$
- $f(v, v') > 0 \Rightarrow (v', v) \in E'$



Flussnetzwerk ($G = (V, E); s, t; c$)

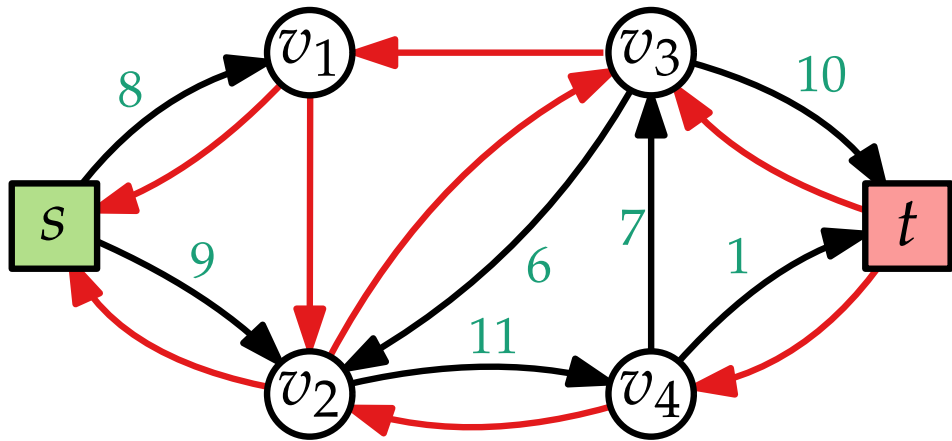


Residualnetzwerk

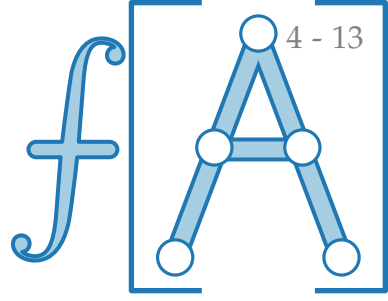
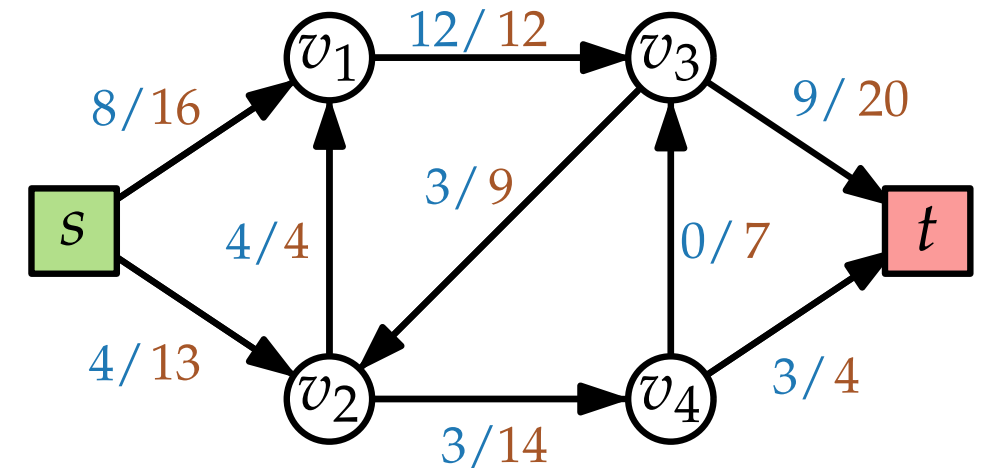
Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:

- $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$
 $C(v, v') = c(v, v') - f(v, v')$
- $f(v, v') > 0 \Rightarrow (v', v) \in E'$



Flussnetzwerk ($G = (V, E); s, t; c$)

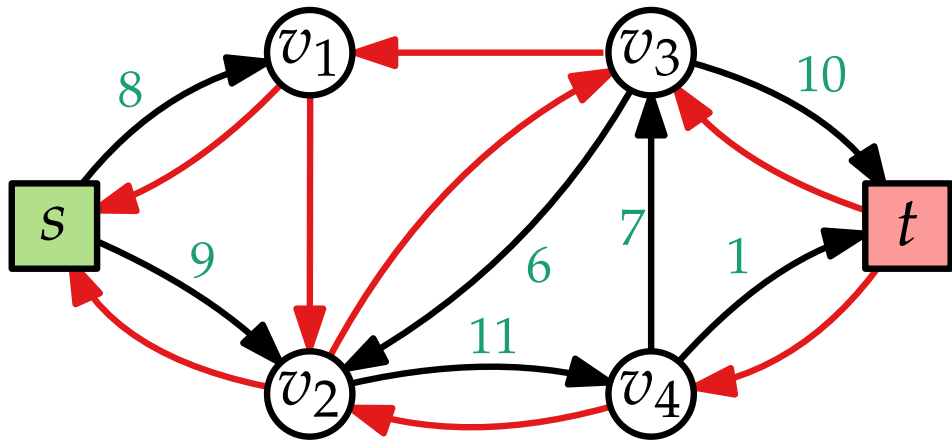


Residualnetzwerk

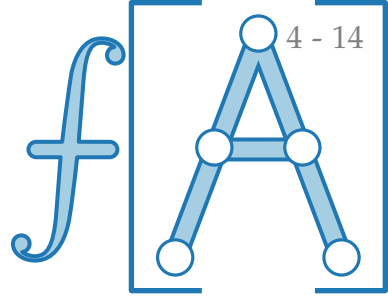
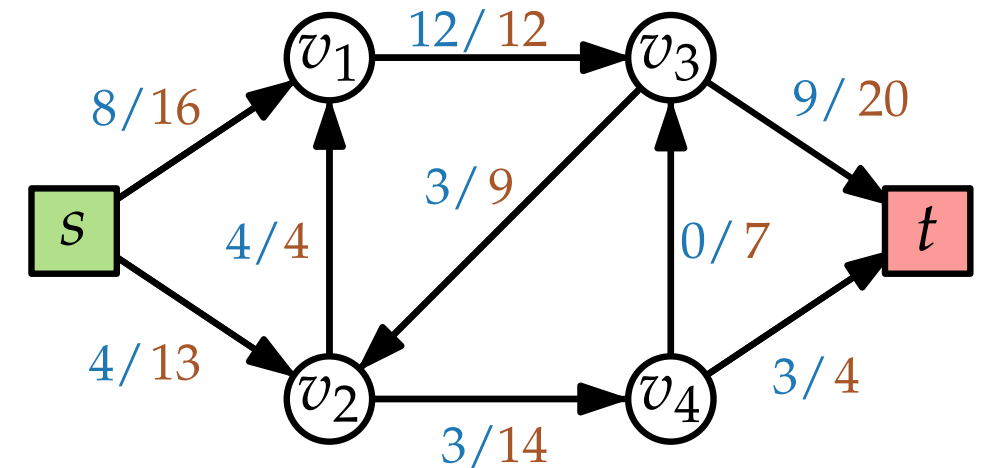
Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:

- $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$
 $C(v, v') = c(v, v') - f(v, v')$
- $f(v, v') > 0 \Rightarrow (v', v) \in E'$
 $C(v', v) = f(v, v')$



Flussnetzwerk ($G = (V, E); s, t; c$)

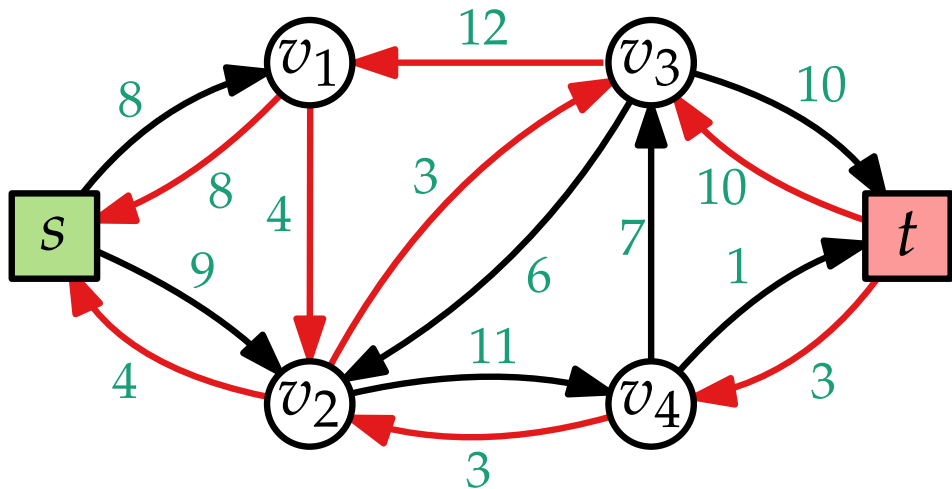


Residualnetzwerk

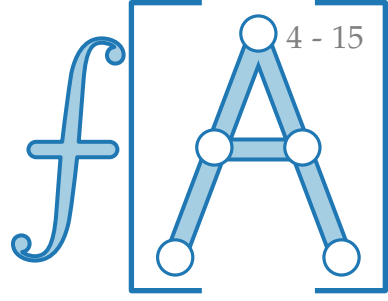
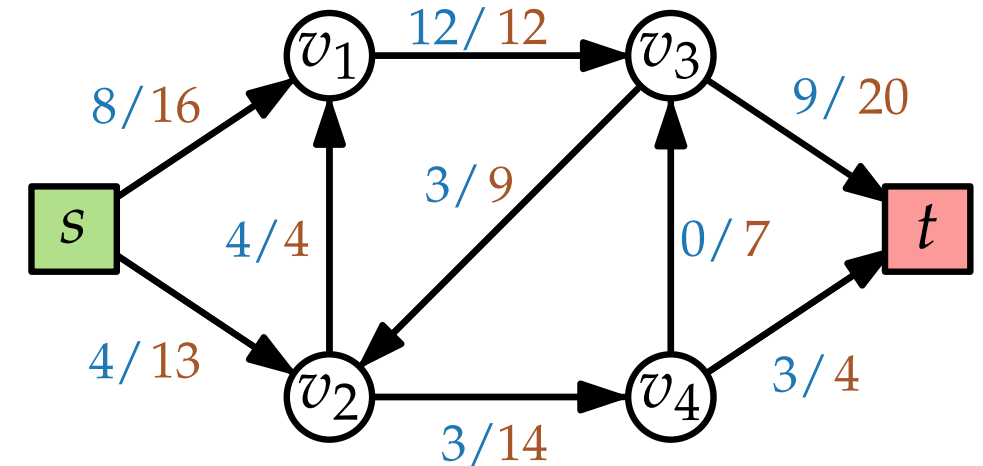
Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:

- $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$
 $C(v, v') = c(v, v') - f(v, v')$
- $f(v, v') > 0 \Rightarrow (v', v) \in E'$
 $C(v', v) = f(v, v')$



Flussnetzwerk $(G = (V, E); s, t; c)$

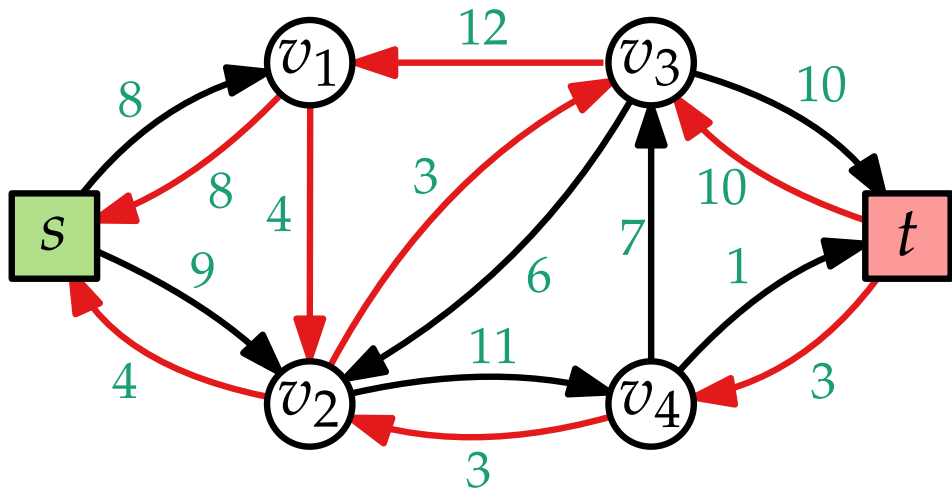


Residualnetzwerk

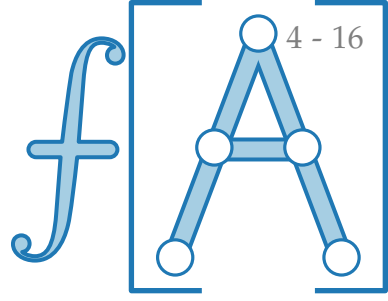
Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:

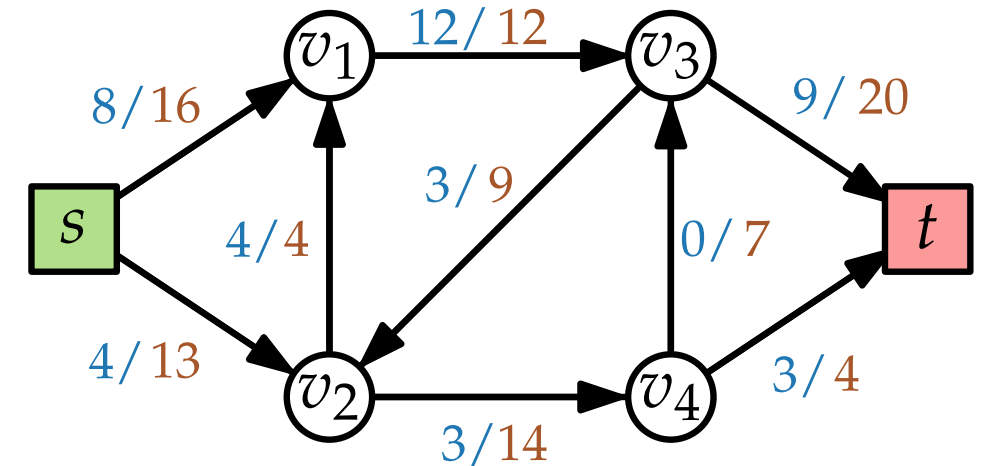
- $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$
 $C(v, v') = c(v, v') - f(v, v')$
- $f(v, v') > 0 \Rightarrow (v', v) \in E'$
 $C(v', v) = f(v, v')$



Augmentierender Weg W



Flussnetzwerk $(G = (V, E); s, t; c)$

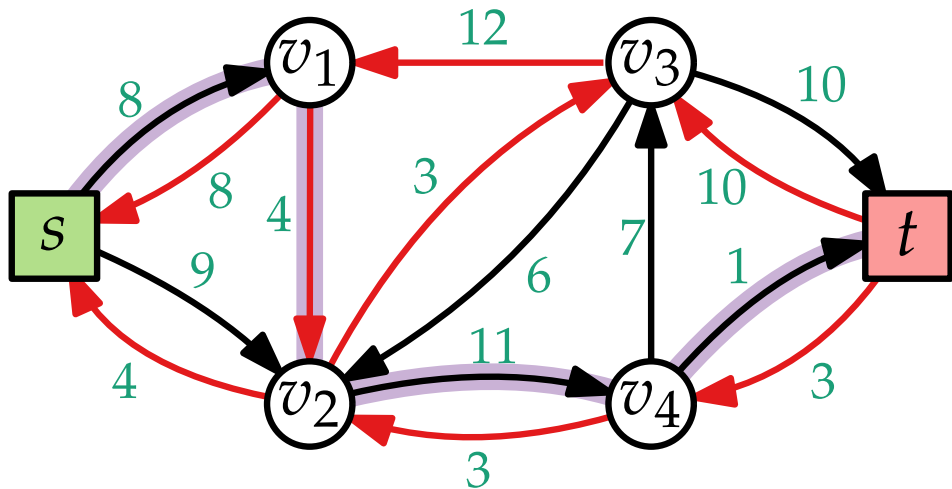


Residualnetzwerk

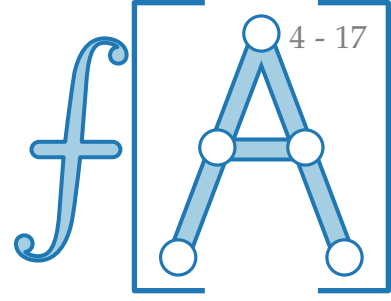
Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:

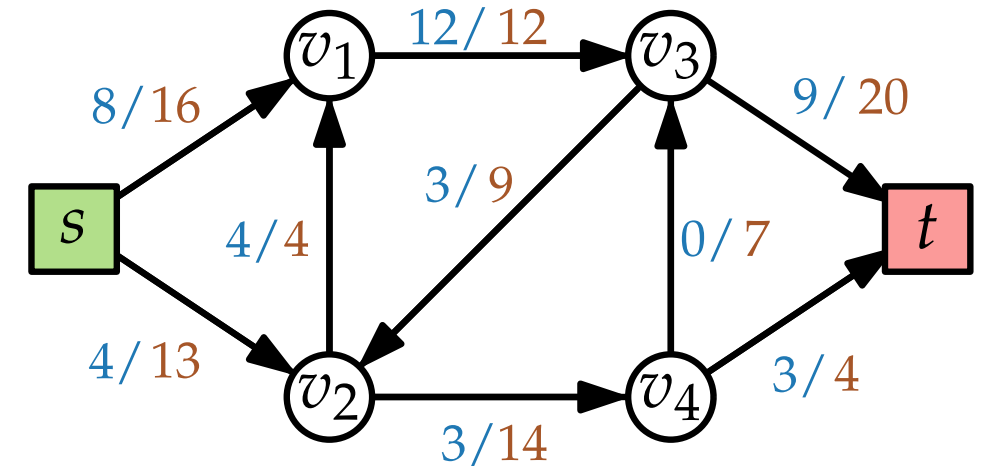
- $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$
 $C(v, v') = c(v, v') - f(v, v')$
- $f(v, v') > 0 \Rightarrow (v', v) \in E'$
 $C(v', v) = f(v, v')$



Augmentierender Weg W



Flussnetzwerk ($G = (V, E); s, t; c$)

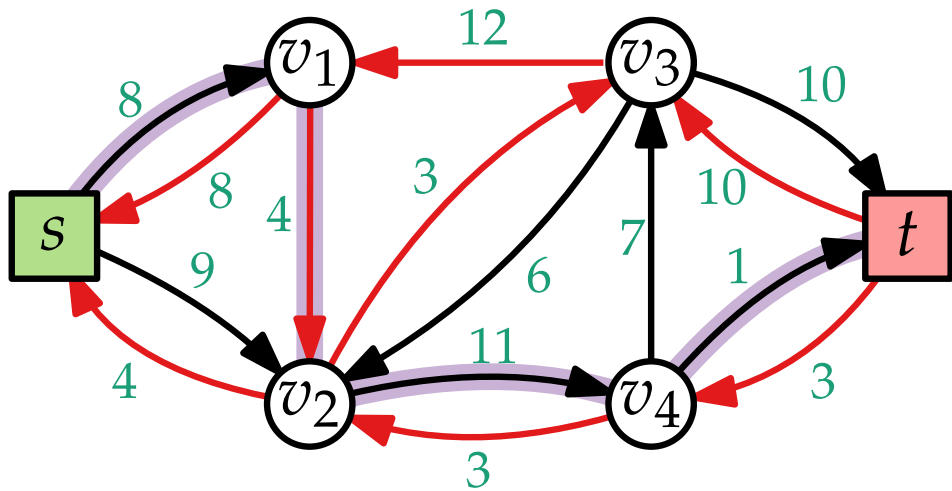


Residualnetzwerk

Idee? Versuche, Fluss schrittweise zu erhöhen.

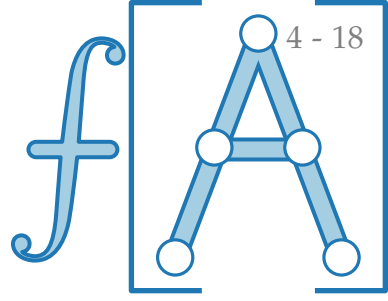
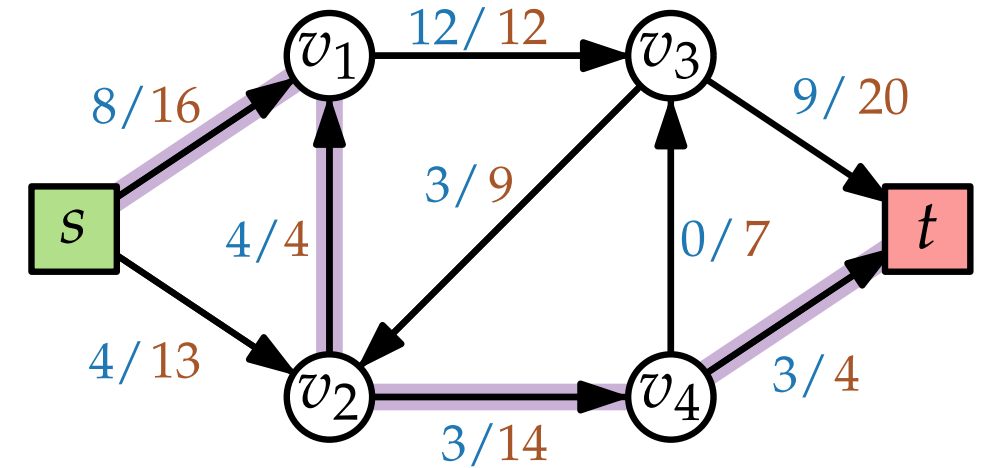
Residualnetzwerk $G_f = (V, E')$:

- $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$
 $C(v, v') = c(v, v') - f(v, v')$
- $f(v, v') > 0 \Rightarrow (v', v) \in E'$
 $C(v', v) = f(v, v')$



Augmentierender Weg W

Flussnetzwerk ($G = (V, E); s, t; c$)

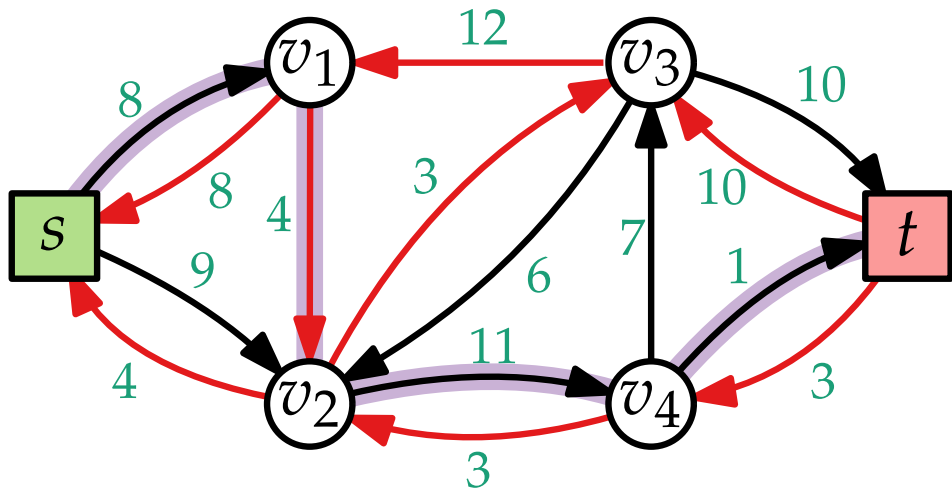


Residualnetzwerk

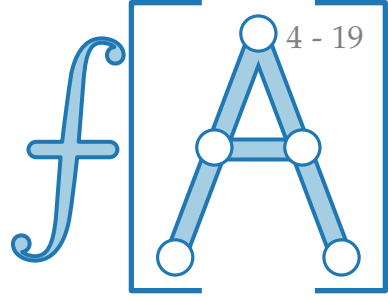
Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:

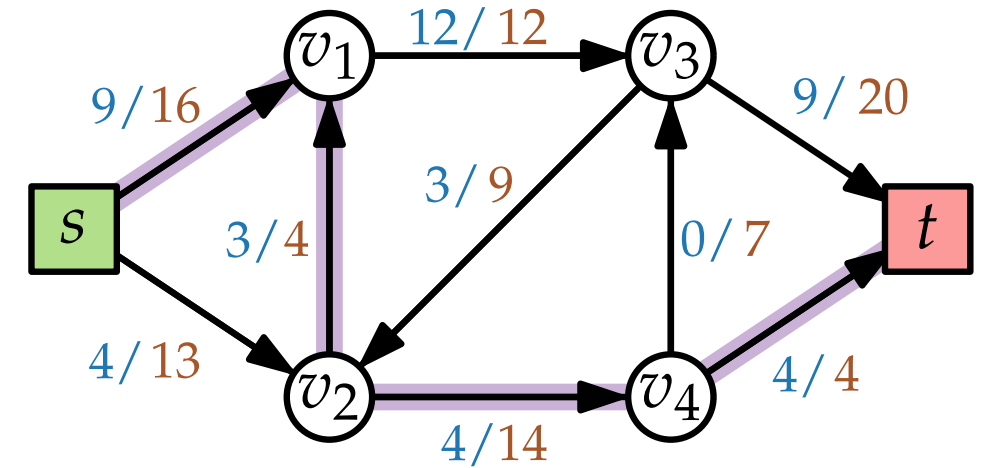
- $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$
 $C(v, v') = c(v, v') - f(v, v')$
- $f(v, v') > 0 \Rightarrow (v', v) \in E'$
 $C(v', v) = f(v, v')$



Augmentierender Weg W



Flussnetzwerk $(G = (V, E); s, t; c)$

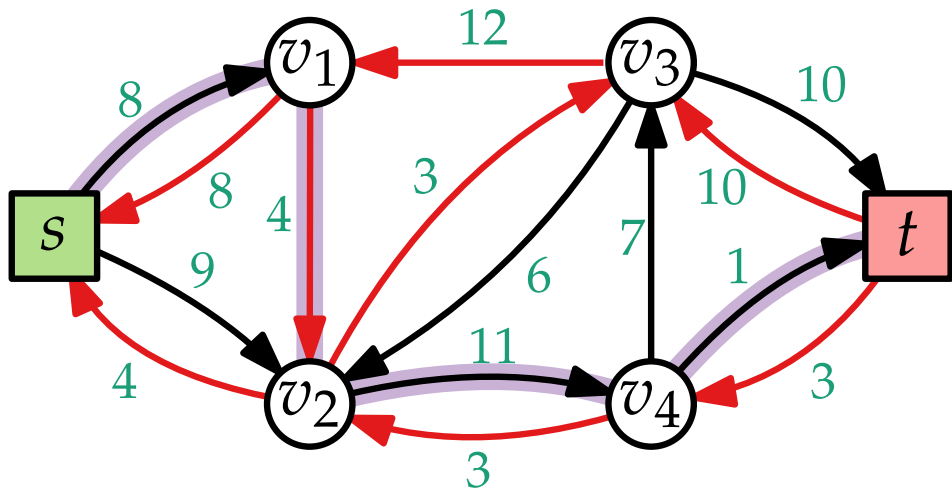


Residualnetzwerk

Idee? Versuche, Fluss schrittweise zu erhöhen.

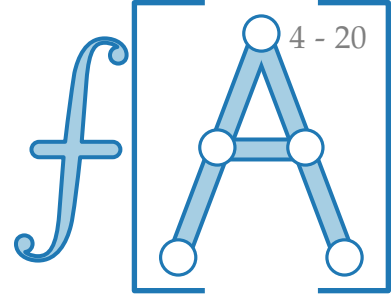
Residualnetzwerk $G_f = (V, E')$:

- $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$
 $C(v, v') = c(v, v') - f(v, v')$
- $f(v, v') > 0 \Rightarrow (v', v) \in E'$
 $C(v', v) = f(v, v')$

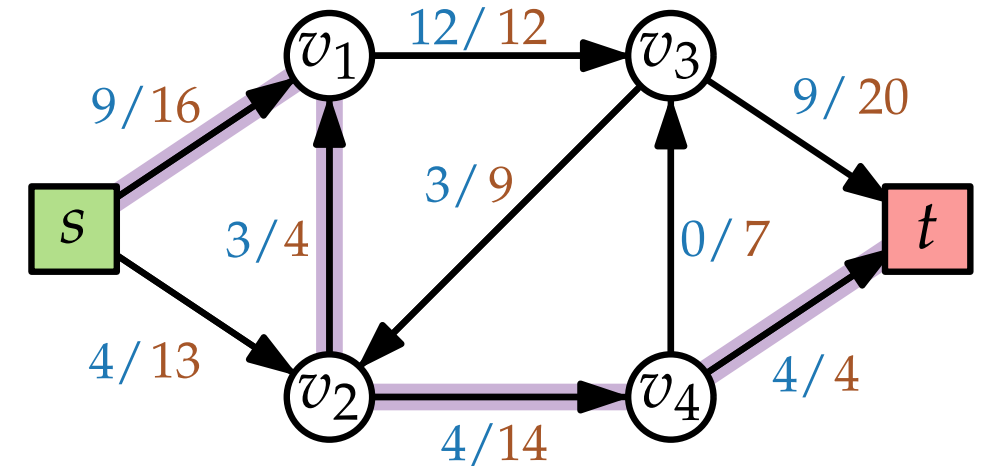


Augmentierender Weg W

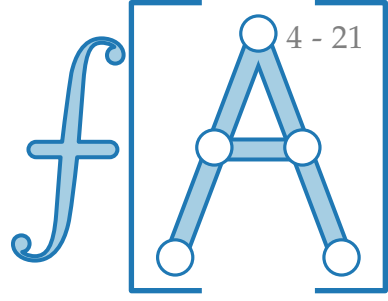
Zu zeigen.



Flussnetzwerk $(G = (V, E); s, t; c)$



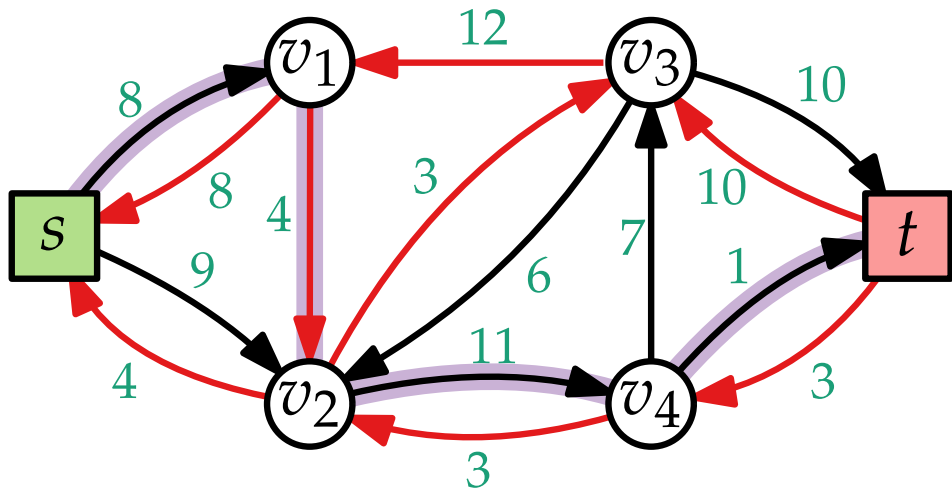
Residualnetzwerk



Idee? Versuche, Fluss schrittweise zu erhöhen.

Residualnetzwerk $G_f = (V, E')$:

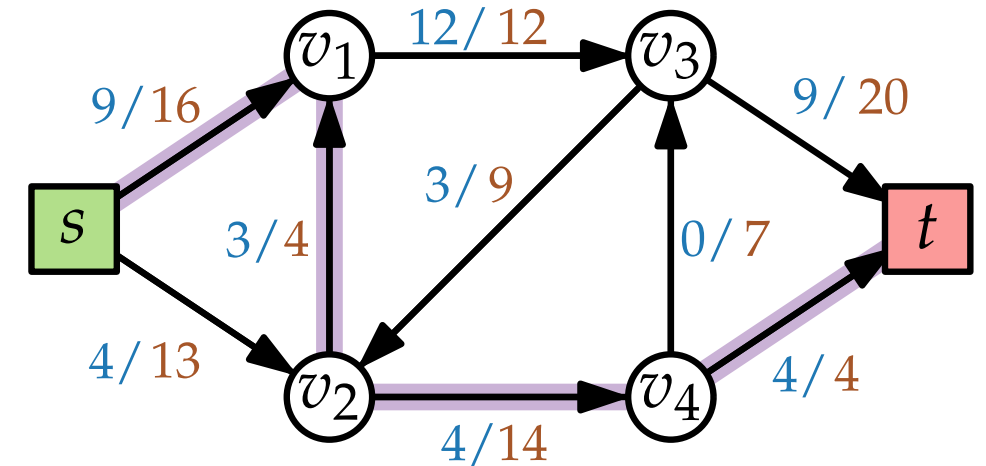
- $f(v, v') < c(v, v') \Rightarrow (v, v') \in E'$
 $C(v, v') = c(v, v') - f(v, v')$
- $f(v, v') > 0 \Rightarrow (v', v) \in E'$
 $C(v', v) = f(v, v')$



Augmentierender Weg W

Zu zeigen. Kein augmentierender Weg $\Rightarrow f$ optimal

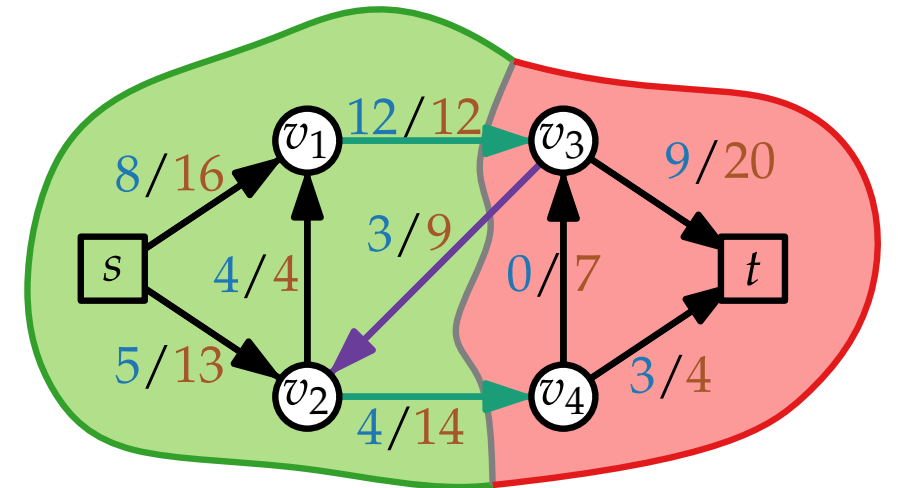
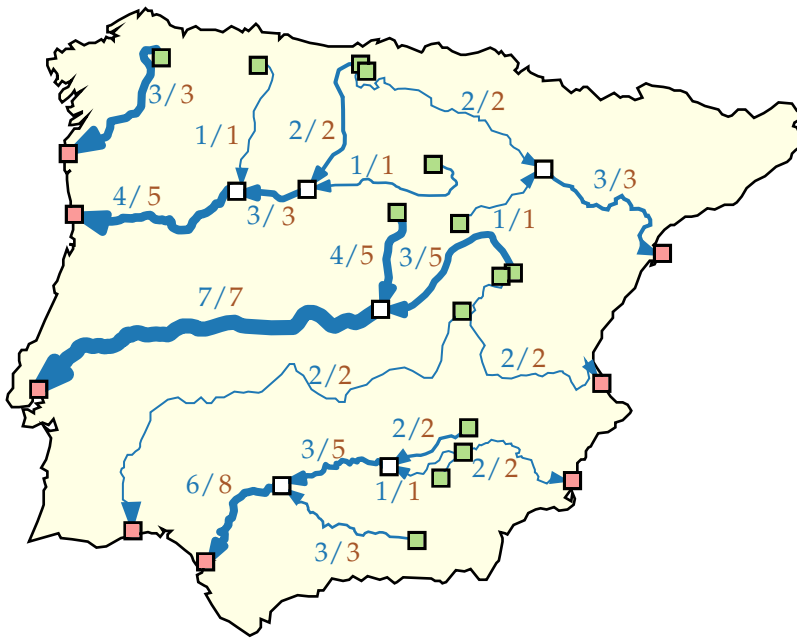
Flussnetzwerk ($G = (V, E); s, t; c$)



Fortgeschrittene Algorithmen

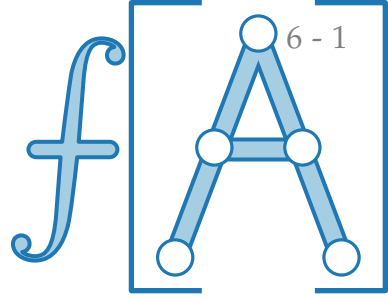
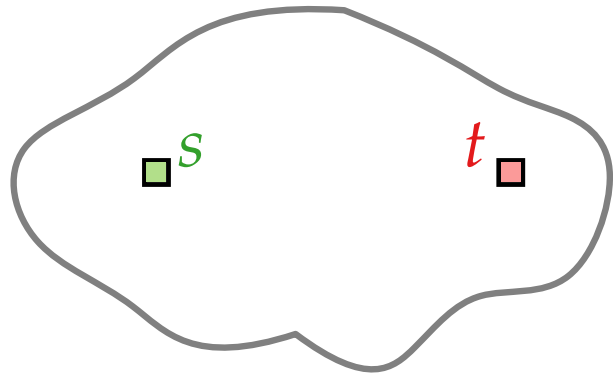
8. Vorlesung Graphalgorithmen II: Flussnetzwerke

Teil II: Schnitte



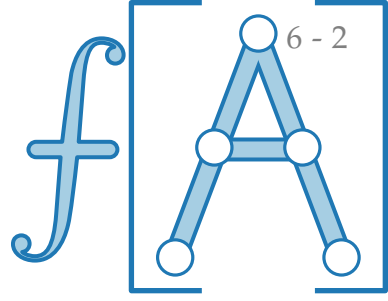
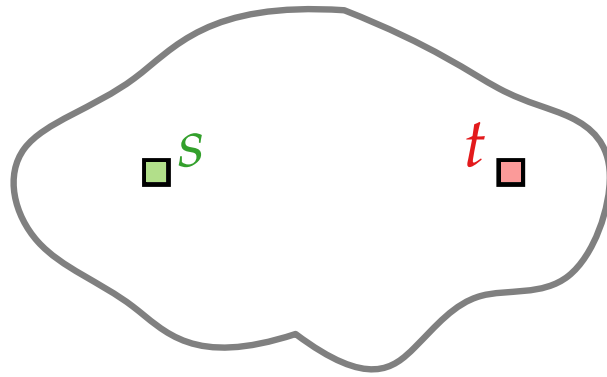
Schnitte

Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.



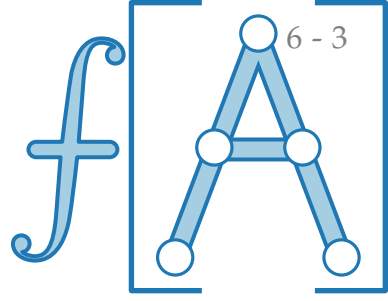
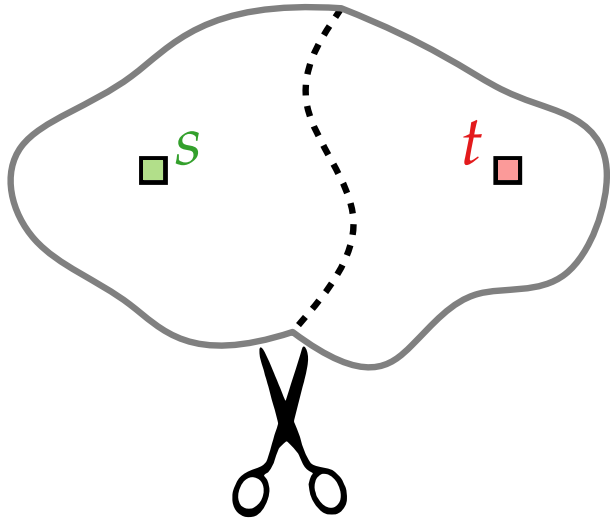
Schnitte

Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein s - t -Schnitt, falls $s \in S$, $t \in T$.



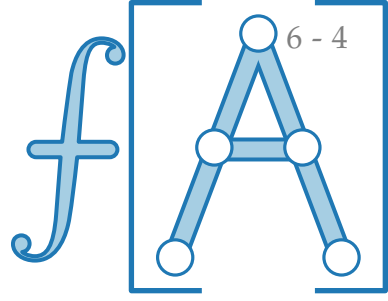
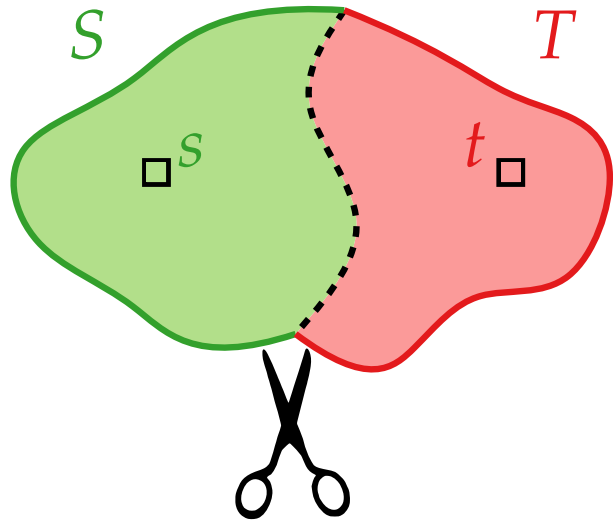
Schnitte

Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein s - t -**Schnitt**, falls $s \in S$, $t \in T$.



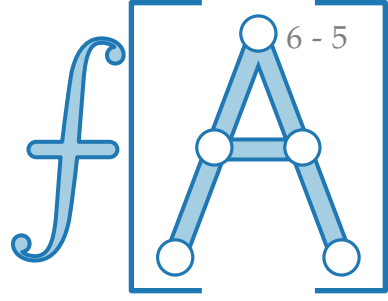
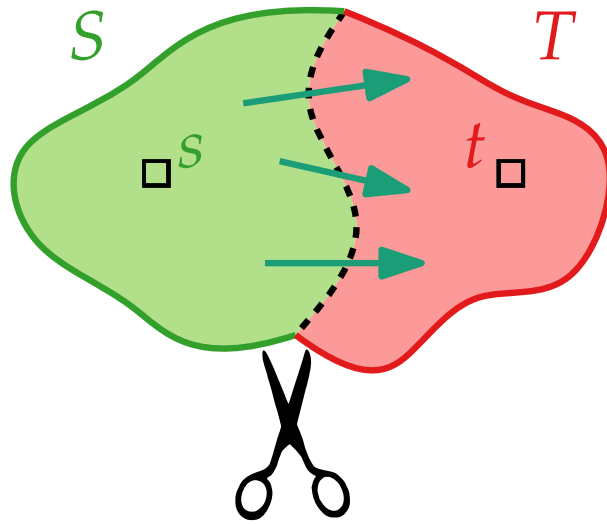
Schnitte

Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein s - t -**Schnitt**, falls $s \in S$, $t \in T$.

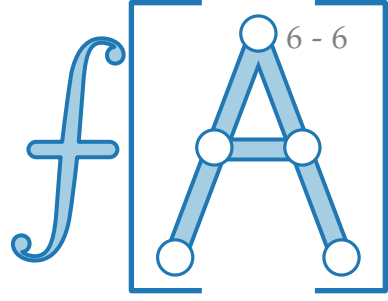


Schnitte

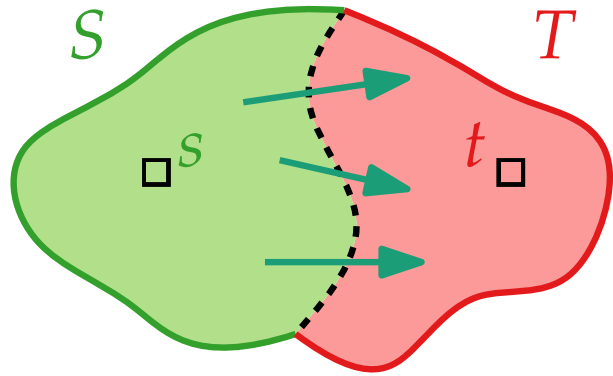
Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein s - t -**Schnitt**, falls $s \in S$, $t \in T$.



Schnitte

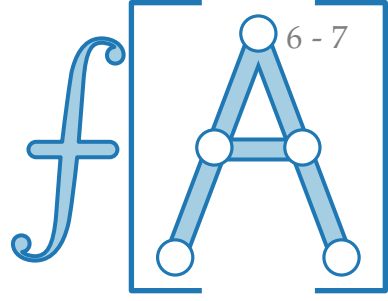


Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein **s - t -Schnitt**, falls $s \in S, t \in T$.

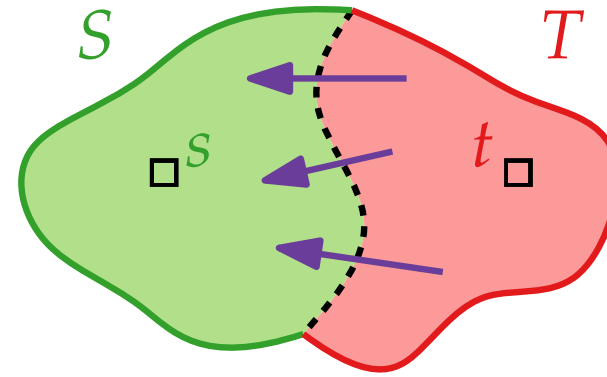
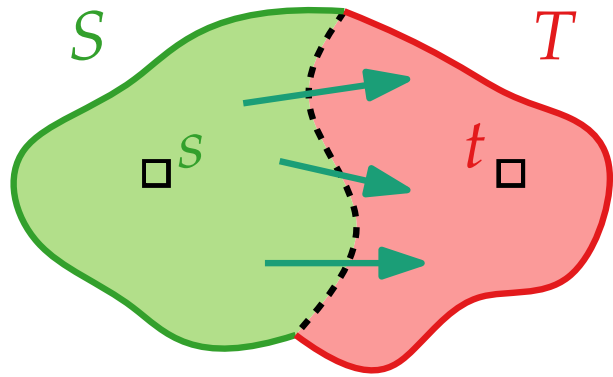


$$\text{Raus}(S) = \{uv \in E \mid u \in S, v \in T\}$$

Schnitte

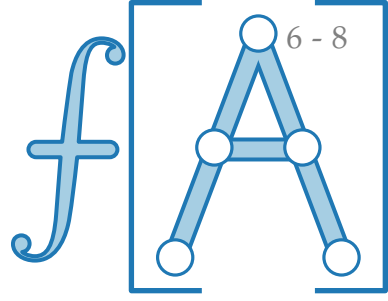


Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein **s - t -Schnitt**, falls $s \in S, t \in T$.

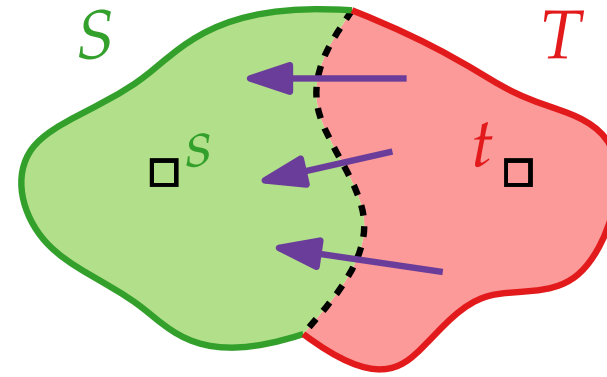
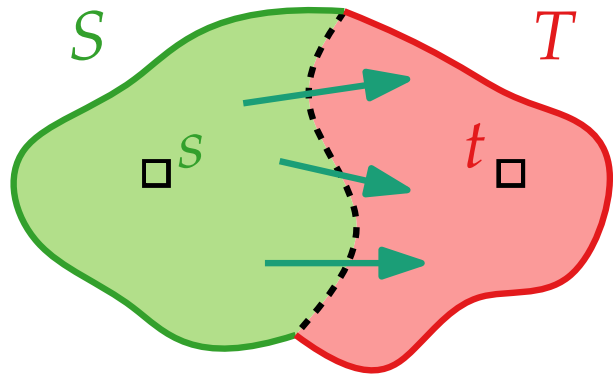


$$\text{Raus}(S) = \{uv \in E \mid u \in S, v \in T\}$$

Schnitte

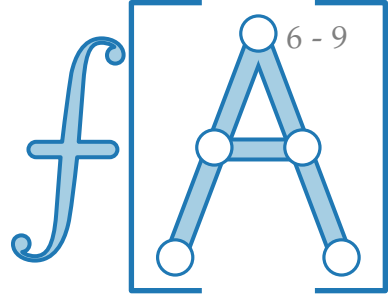


Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein **s - t -Schnitt**, falls $s \in S, t \in T$.

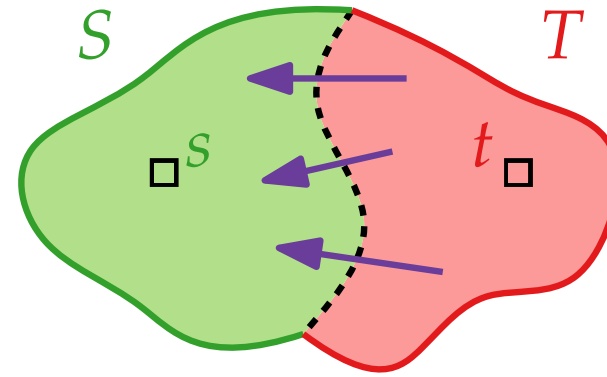
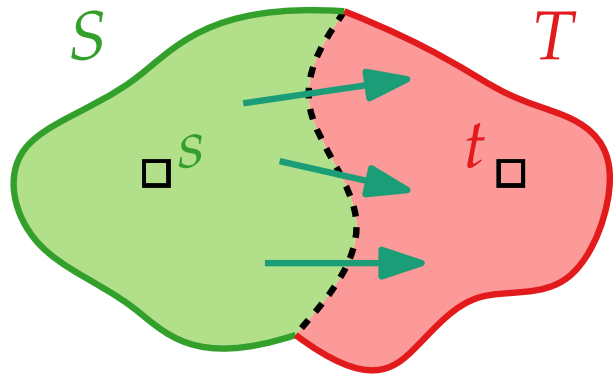


$$\begin{aligned} \text{Raus}(S) &= \{uv \in E \mid u \in S, v \in T\} \\ \{uv \in E \mid u \in T, v \in S\} &= \text{Rein}(S) \end{aligned}$$

Schnitte



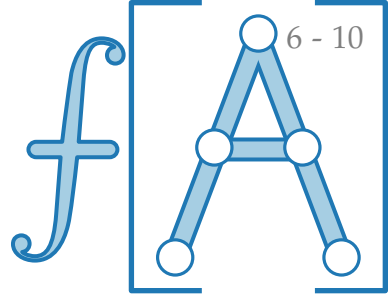
Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein **s - t -Schnitt**, falls $s \in S, t \in T$.



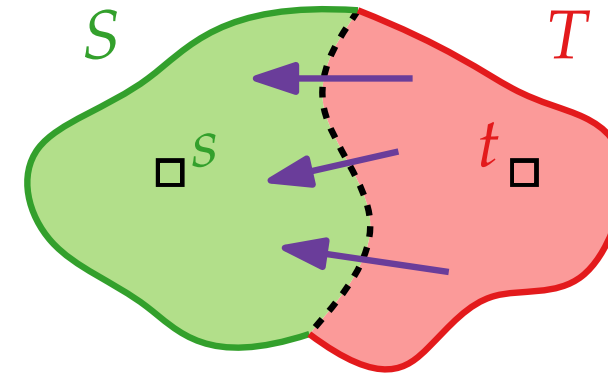
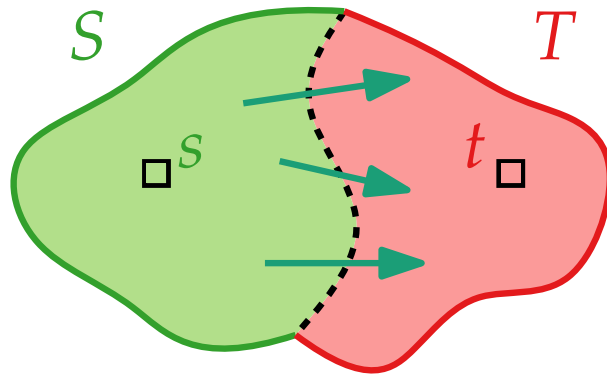
$$\begin{aligned}\text{Raus}(S) &= \{uv \in E \mid u \in S, v \in T\} \\ \{uv \in E \mid u \in T, v \in S\} &= \text{Rein}(S)\end{aligned}$$

$$\text{Zufluss}_f(S) = f(\text{Rein}(S))$$

Schnitte



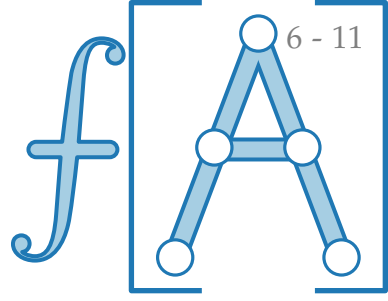
Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein **s - t -Schnitt**, falls $s \in S$, $t \in T$.



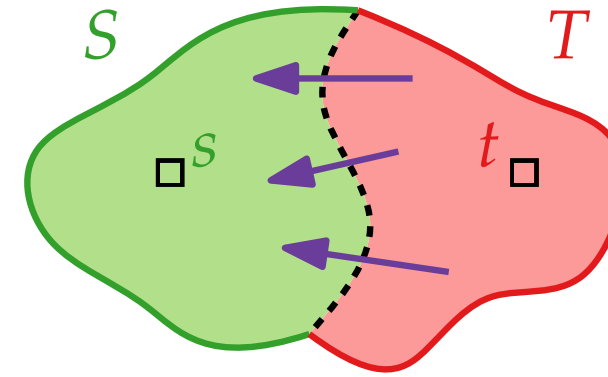
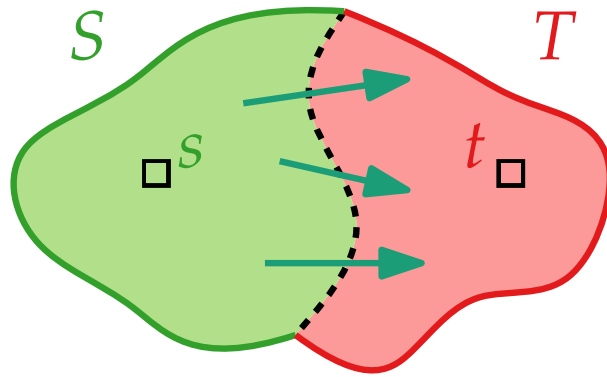
$$\begin{aligned} \text{Raus}(S) &= \{uv \in E \mid u \in S, v \in T\} \\ \{uv \in E \mid u \in T, v \in S\} &= \text{Rein}(S) \end{aligned}$$

$$\text{Zufluss}_f(S) = f(\text{Rein}(S)) = \sum_{e \in \text{Rein}(S)} f(e)$$

Schnitte



Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein **s - t -Schnitt**, falls $s \in S$, $t \in T$.

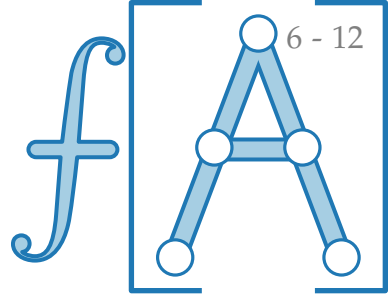


$$\begin{aligned} \text{Raus}(S) &= \{uv \in E \mid u \in S, v \in T\} \\ \{uv \in E \mid u \in T, v \in S\} &= \text{Rein}(S) \end{aligned}$$

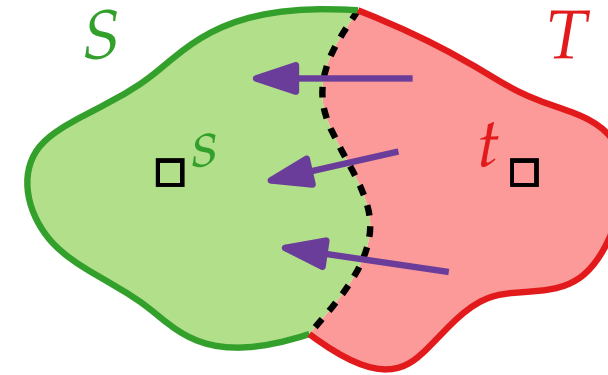
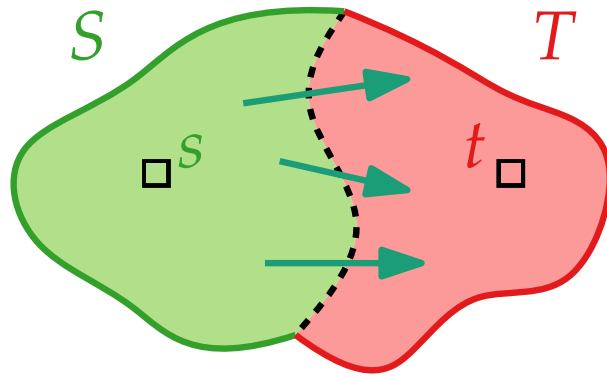
$$\text{Zufluss}_f(S) = f(\text{Rein}(S)) = \sum_{e \in \text{Rein}(S)} f(e)$$

$$\text{Abfluss}_f(S) = f(\text{Raus}(S))$$

Schnitte



Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein **s - t -Schnitt**, falls $s \in S$, $t \in T$.

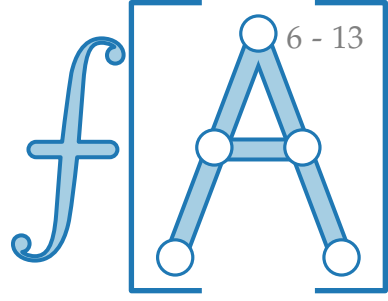


$$\begin{aligned} \text{Raus}(S) &= \{uv \in E \mid u \in S, v \in T\} \\ \{uv \in E \mid u \in T, v \in S\} &= \text{Rein}(S) \end{aligned}$$

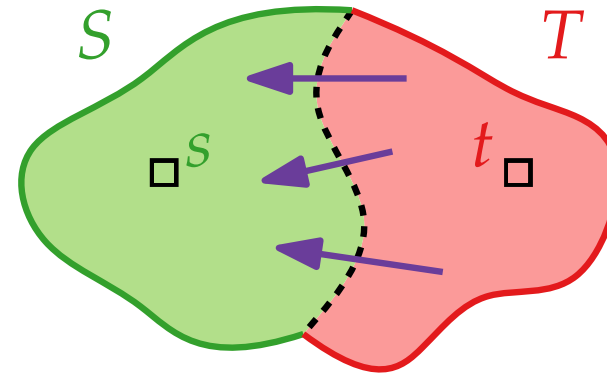
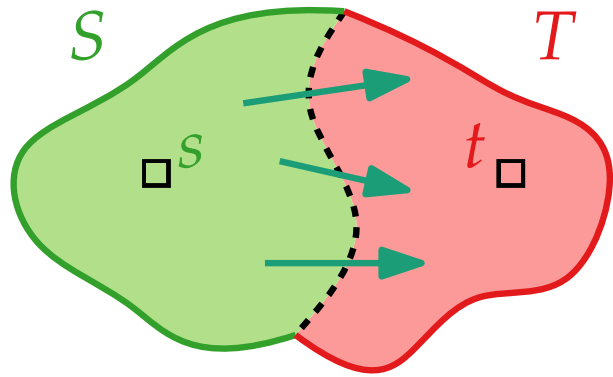
$$\text{Zufluss}_f(S) = f(\text{Rein}(S)) = \sum_{e \in \text{Rein}(S)} f(e)$$

$$\text{Abfluss}_f(S) = f(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} f(e)$$

Schnitte



Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein **s - t -Schnitt**, falls $s \in S$, $t \in T$.



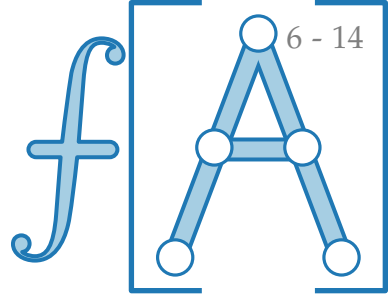
$$\begin{aligned} \text{Raus}(S) &= \{uv \in E \mid u \in S, v \in T\} \\ \{uv \in E \mid u \in T, v \in S\} &= \text{Rein}(S) \end{aligned}$$

$$\text{Zufluss}_f(S) = f(\text{Rein}(S)) = \sum_{e \in \text{Rein}(S)} f(e)$$

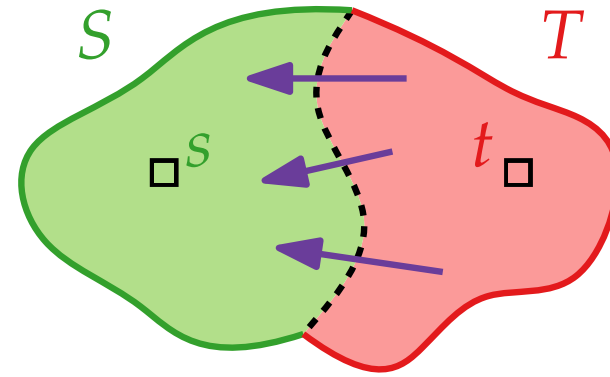
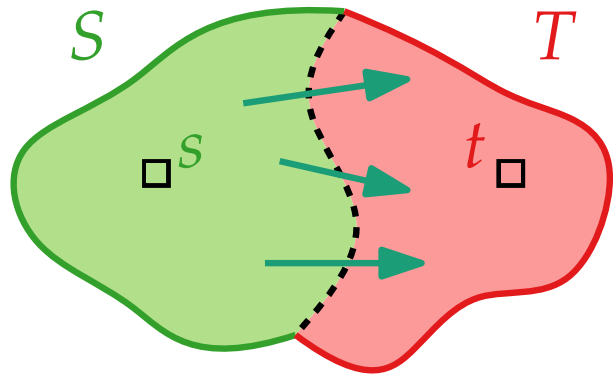
$$\text{Abfluss}_f(S) = f(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} f(e)$$

$$\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$$

Schnitte



Def. Sei $G = (V, E)$ ein gerichteter Graph, $s, t \in V$.
Eine Zerlegung (S, T) von V ist ein **s - t -Schnitt**, falls $s \in S$, $t \in T$.



$$\begin{aligned} \text{Raus}(S) &= \{uv \in E \mid u \in S, v \in T\} \\ \{uv \in E \mid u \in T, v \in S\} &= \text{Rein}(S) \end{aligned}$$

$$\text{Zufluss}_f(S) = f(\text{Rein}(S)) = \sum_{e \in \text{Rein}(S)} f(e)$$

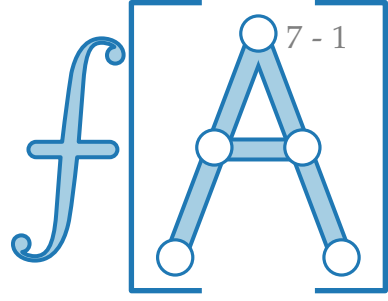
$$\text{Abfluss}_f(S) = f(\text{Raus}(S)) = \sum_{e \in \text{Raus}(S)} f(e)$$

$$\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$$

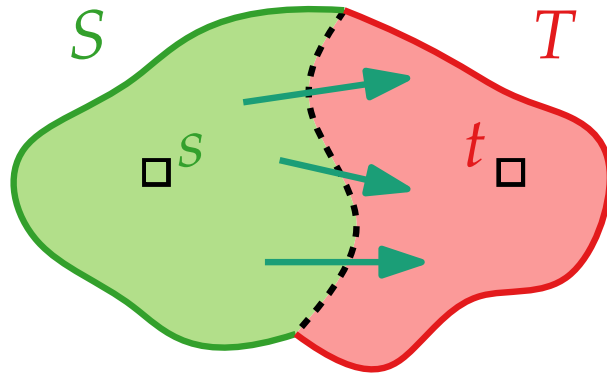
$$\text{Nettoabfluss}_f(S) = \text{Abfluss}_f(S) - \text{Zufluss}_f(S)$$

Nettozuflüsse von Schnitten und Knoten

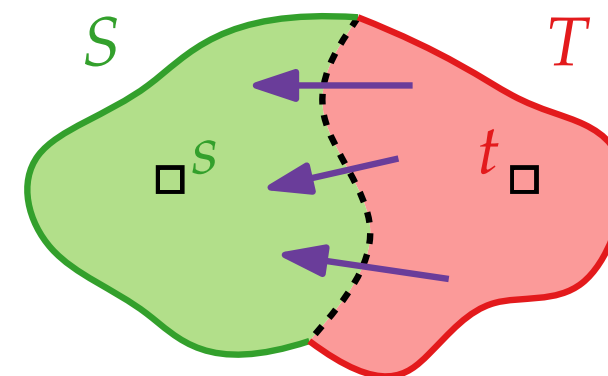
Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$



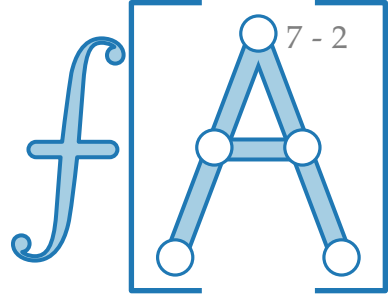
$\text{Raus}(S) \subseteq E$



$\text{Rein}(S) \subseteq E$

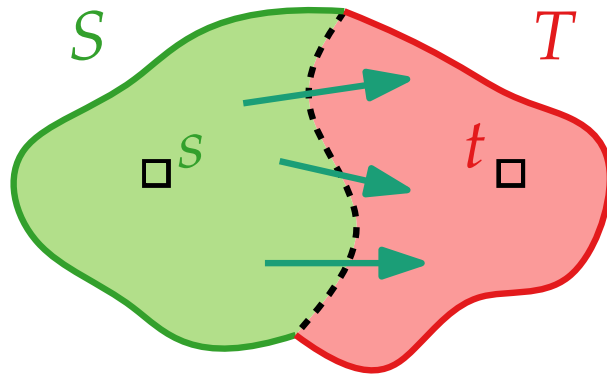


Nettozuflüsse von Schnitten und Knoten

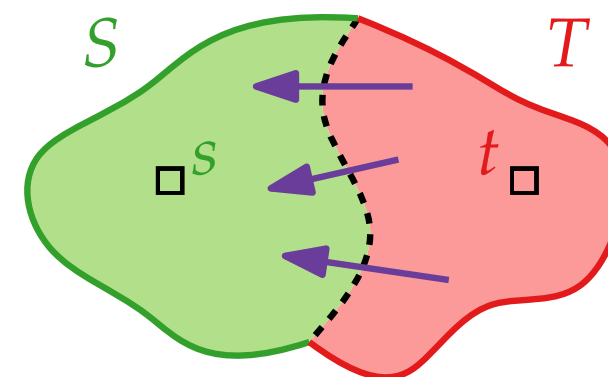


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

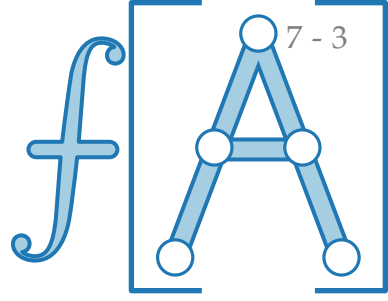


$\text{Rein}(S) \subseteq E$



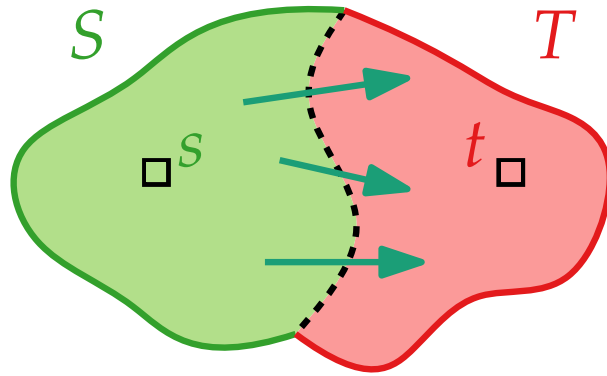
Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) =$

Nettozuflüsse von Schnitten und Knoten

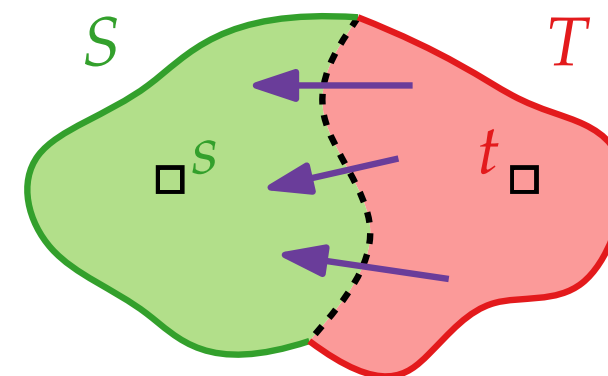


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

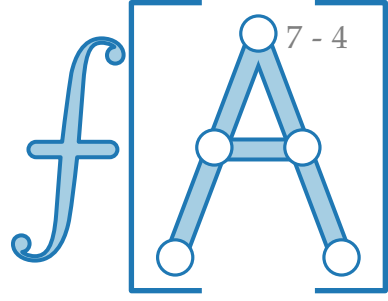


$\text{Rein}(S) \subseteq E$



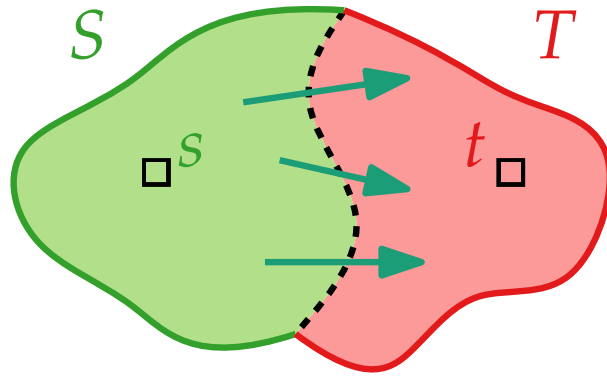
Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S}$

Nettozuflüsse von Schnitten und Knoten

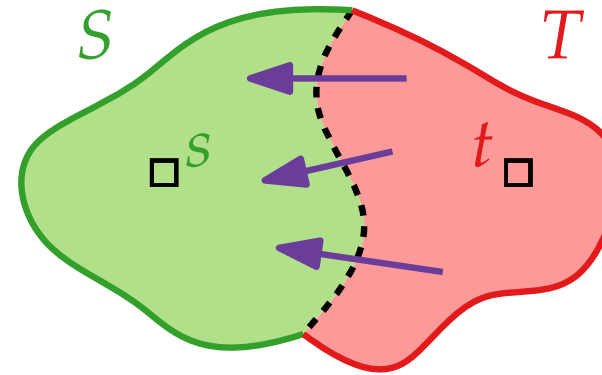


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

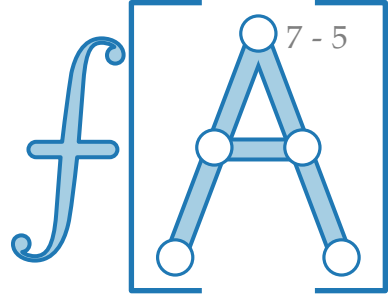


$\text{Rein}(S) \subseteq E$



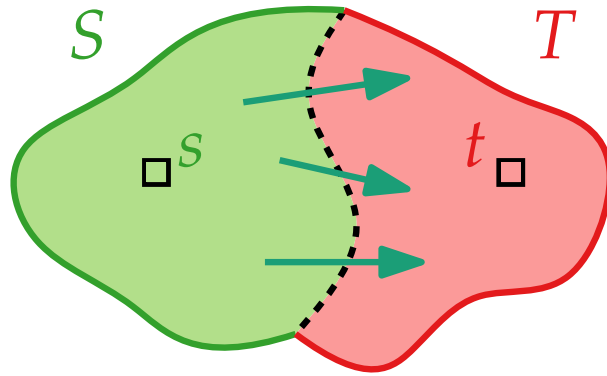
Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v).$

Nettozuflüsse von Schnitten und Knoten

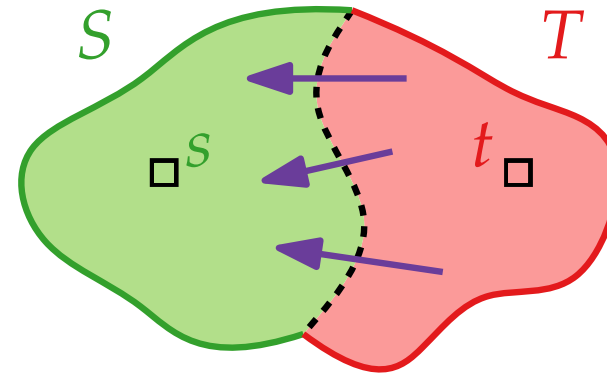


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$



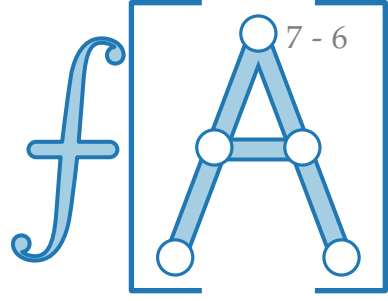
$\text{Rein}(S) \subseteq E$



Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v).$

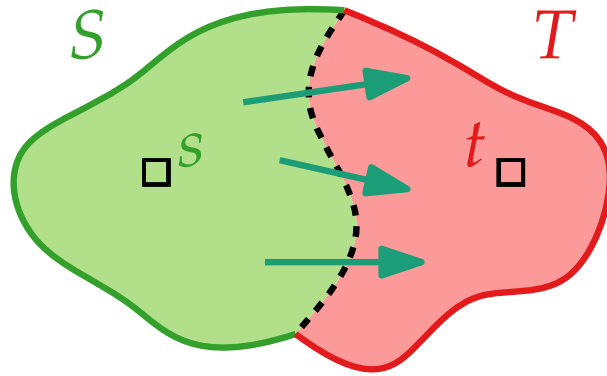
Beweis. $\sum_{v \in S} \text{Nettozufluss}_f(v) =$

Nettozuflüsse von Schnitten und Knoten

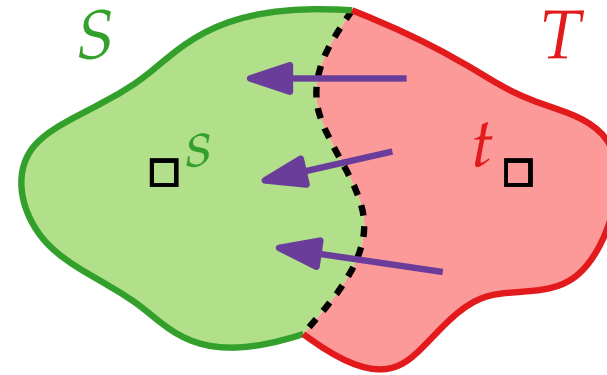


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$



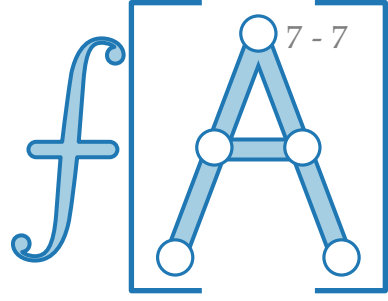
$\text{Rein}(S) \subseteq E$



Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v).$

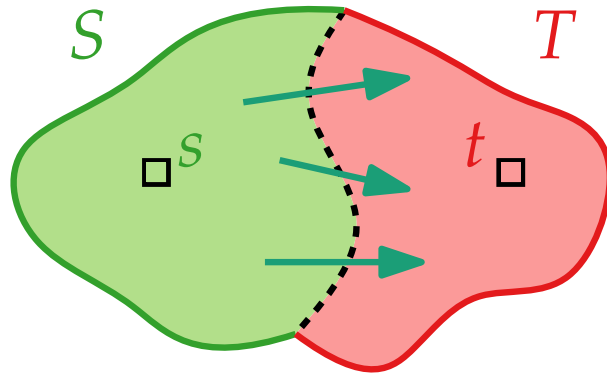
Beweis. $\sum_{v \in S} \text{Nettozufluss}_f(v) = \sum_{v \in S} \left(\text{Zufluss}_f(v) - \text{Abfluss}_f(v) \right)$

Nettozuflüsse von Schnitten und Knoten

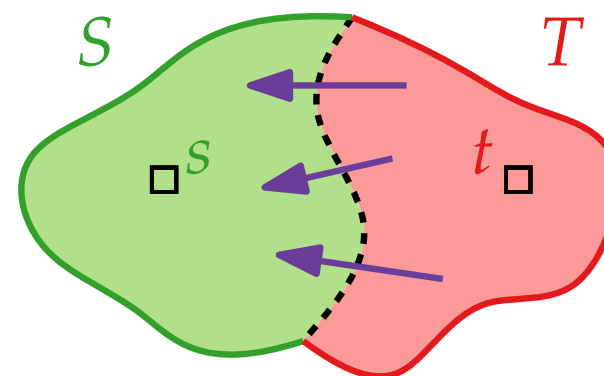


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$



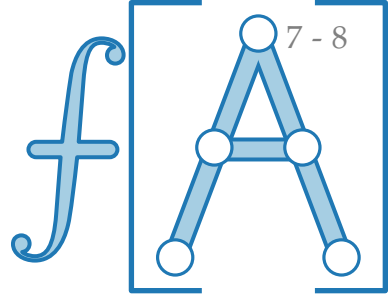
$\text{Rein}(S) \subseteq E$



Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

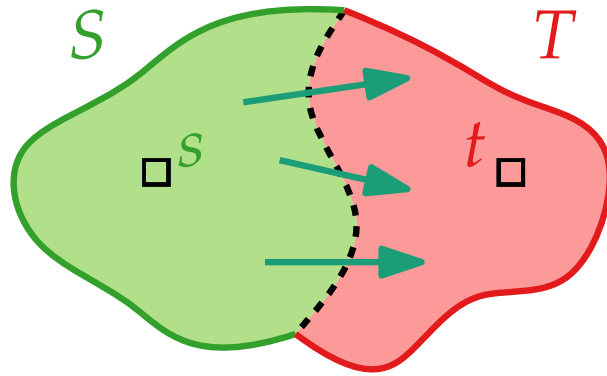
Beweis.
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} \left(\text{Zufluss}_f(v) - \text{Abfluss}_f(v) \right) \\ &= \sum_{v \in S} \left(\sum_{e=uv} \text{Zufluss}_f(v) - \sum_{e=vw} \text{Abfluss}_f(v) \right) \end{aligned}$$

Nettozuflüsse von Schnitten und Knoten

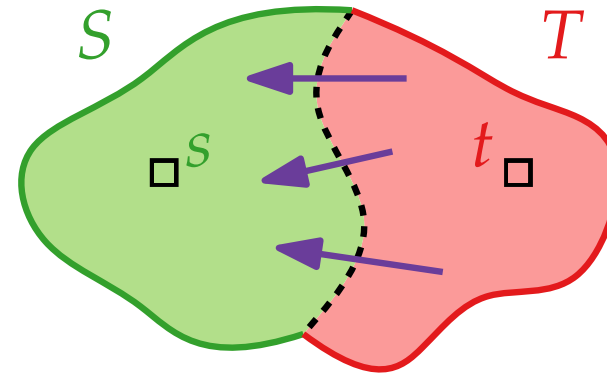


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$



$\text{Rein}(S) \subseteq E$

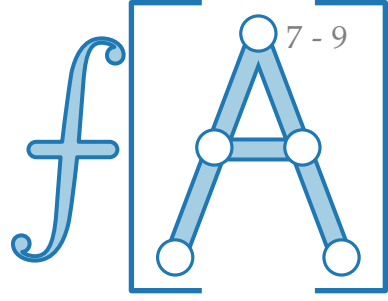


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v).$

Beweis.

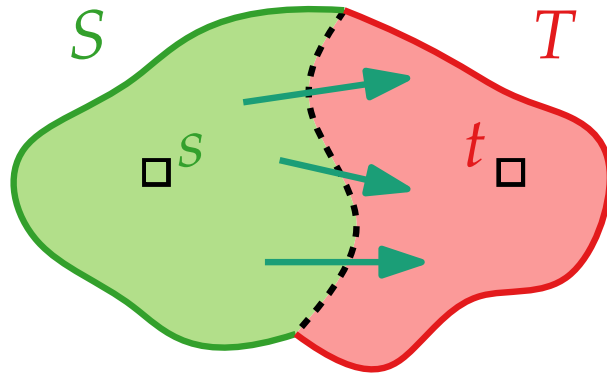
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} \left(\text{Zufluss}_f(v) - \text{Abfluss}_f(v) \right) \\ &= \sum_{v \in S} \left(\sum_{e=uv} \text{Zufluss}_f(v) - \sum_{e=vw} \text{Abfluss}_f(v) \right) \\ &= \end{aligned}$$

Nettozuflüsse von Schnitten und Knoten

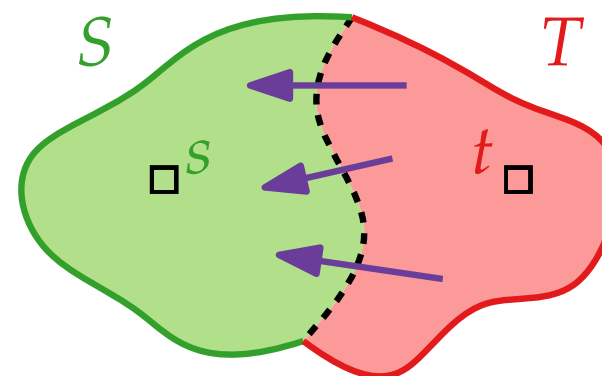


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

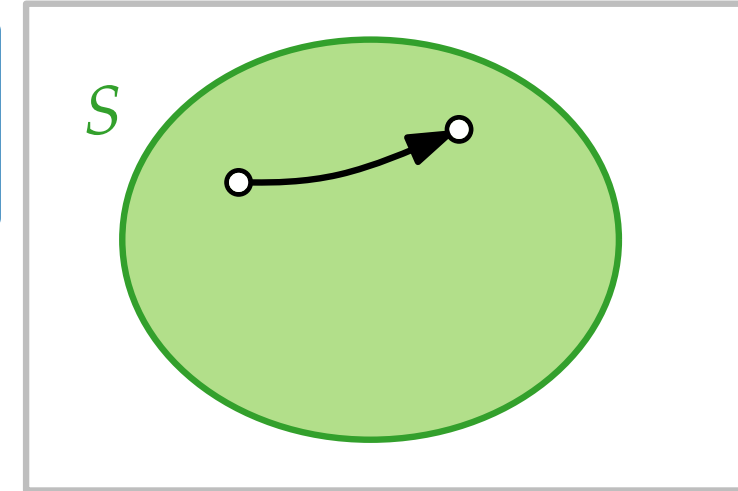


$\text{Rein}(S) \subseteq E$

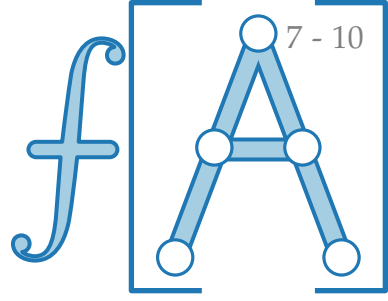


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Beweis.

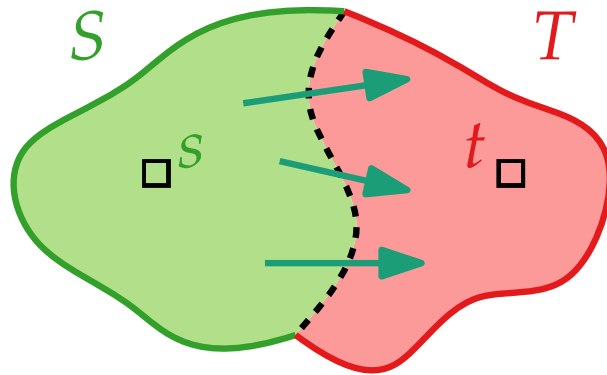
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} \left(\text{Zufluss}_f(v) - \text{Abfluss}_f(v) \right) \\ &= \sum_{v \in S} \left(\sum_{e=uv} f(e) - \sum_{e=vw} f(e) \right) \\ &= \end{aligned}$$


Nettozuflüsse von Schnitten und Knoten

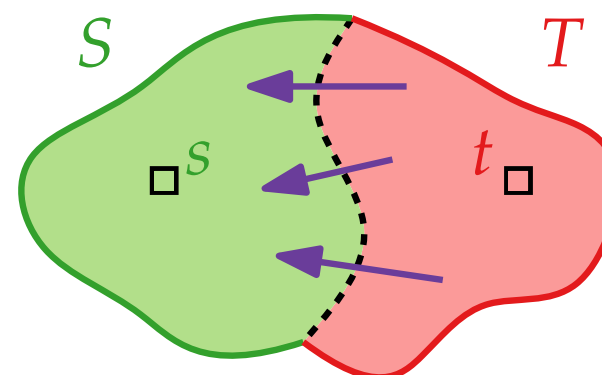


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

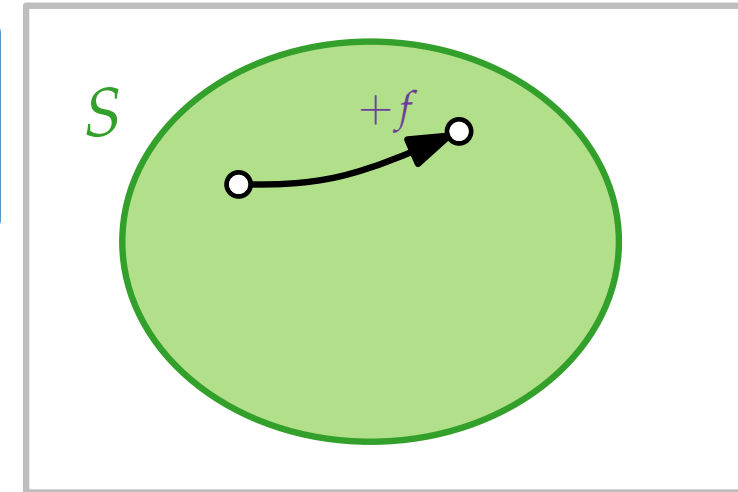


$\text{Rein}(S) \subseteq E$

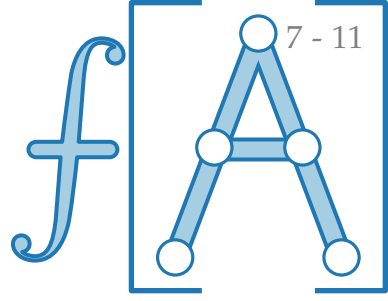


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Beweis.

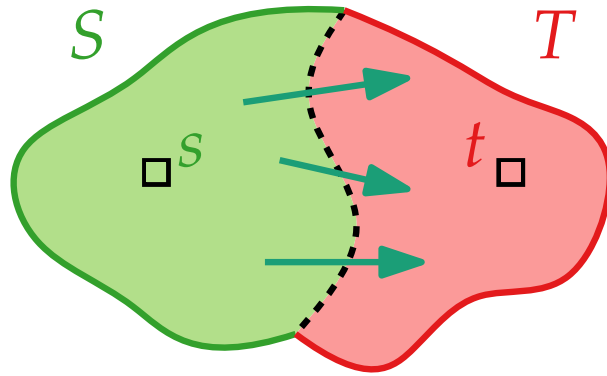
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} \left(\text{Zufluss}_f(v) - \text{Abfluss}_f(v) \right) \\ &= \sum_{v \in S} \left(\sum_{e=uv} f(e) - \sum_{e=vw} f(e) \right) \\ &= \end{aligned}$$


Nettozuflüsse von Schnitten und Knoten

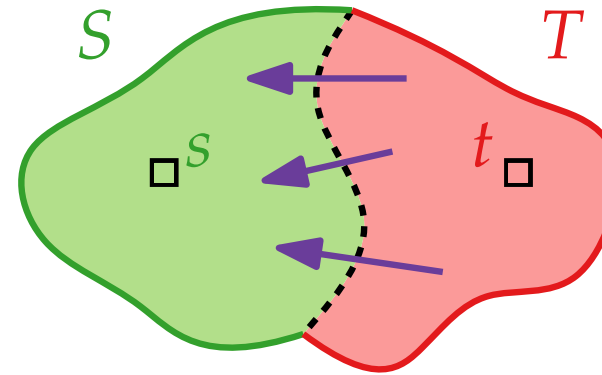


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

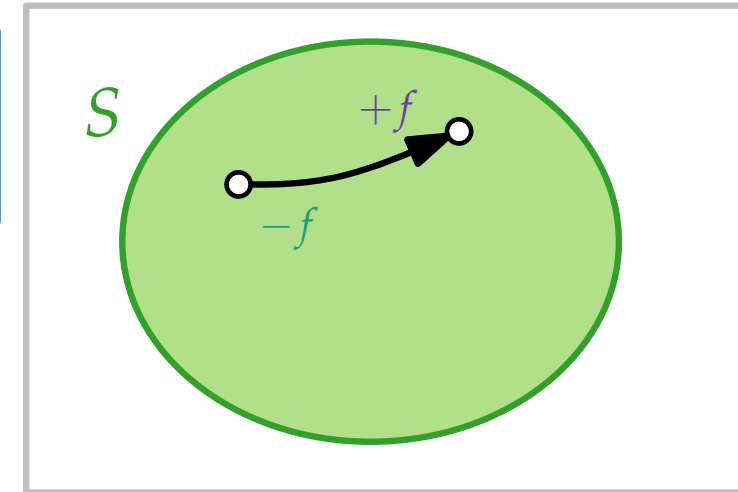


$\text{Rein}(S) \subseteq E$

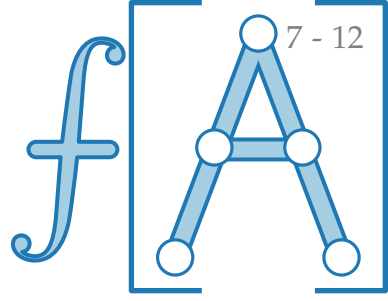


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Beweis.

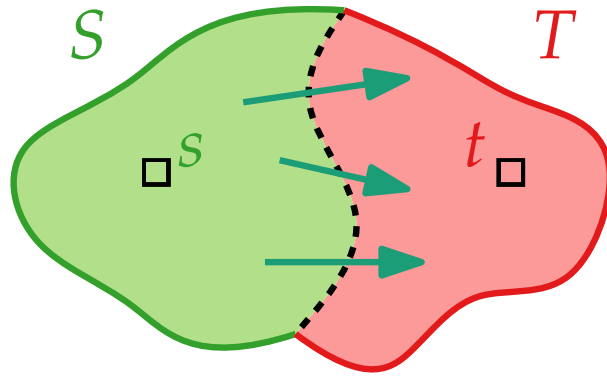
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} \left(\text{Zufluss}_f(v) - \text{Abfluss}_f(v) \right) \\ &= \sum_{v \in S} \left(\sum_{e=uv} f(e) - \sum_{e=vw} f(e) \right) \\ &= \end{aligned}$$


Nettozuflüsse von Schnitten und Knoten

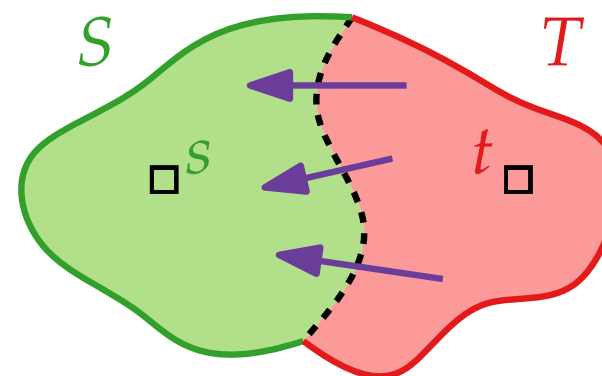


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

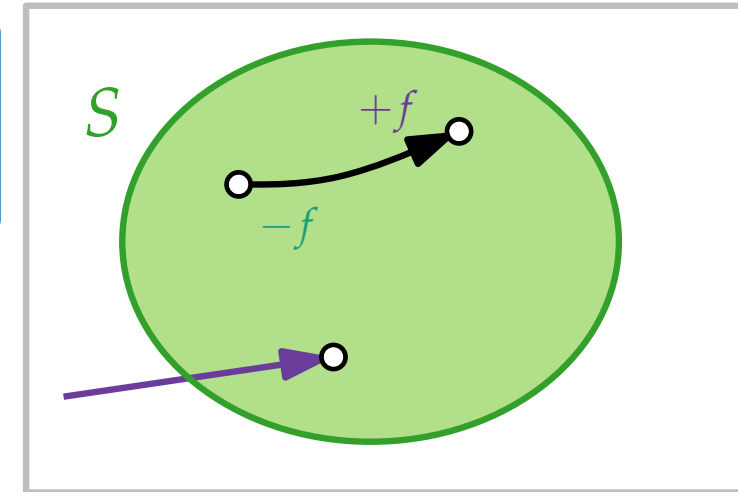


$\text{Rein}(S) \subseteq E$

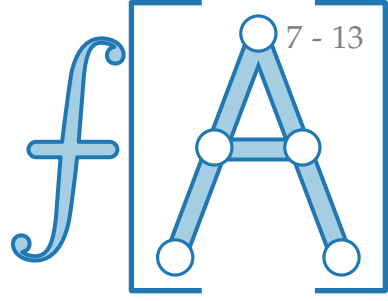


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Beweis.
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} (\text{Zufluss}_f(v) - \text{Abfluss}_f(v)) \\ &= \sum_{v \in S} (\sum_{e=uv} f(e) - \sum_{e=vw} f(e)) \\ &= \end{aligned}$$

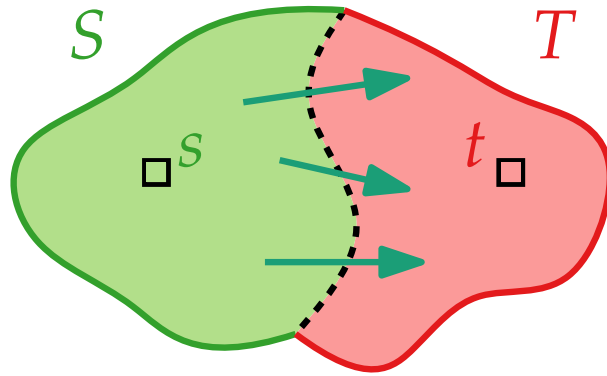


Nettozuflüsse von Schnitten und Knoten

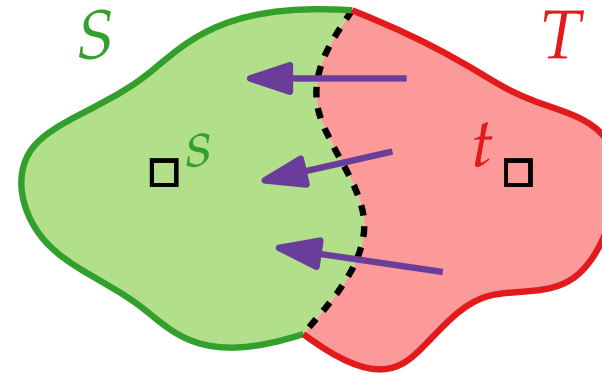


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

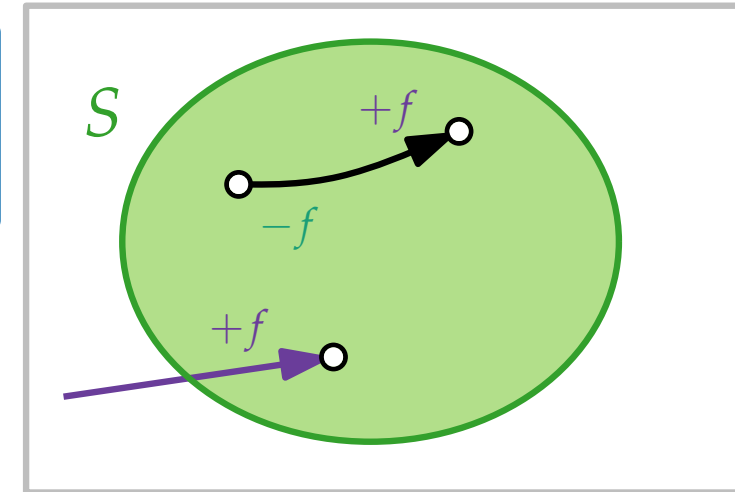


$\text{Rein}(S) \subseteq E$

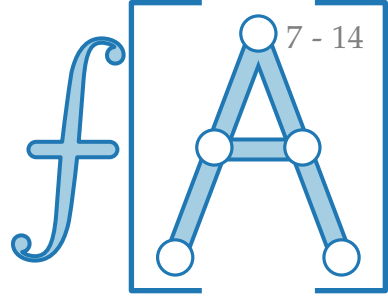


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Beweis.
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} \left(\text{Zufluss}_f(v) - \text{Abfluss}_f(v) \right) \\ &= \sum_{v \in S} \left(\sum_{e=uv} f(e) - \sum_{e=vw} f(e) \right) \\ &= \end{aligned}$$

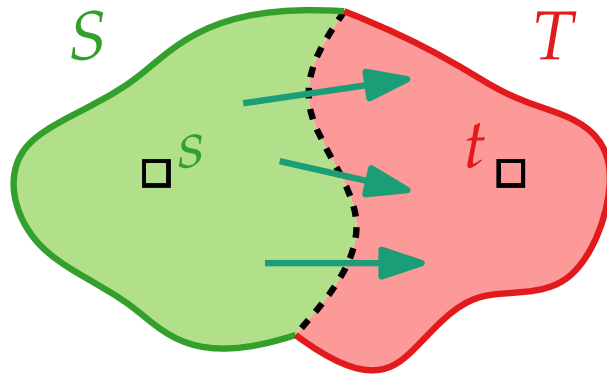


Nettozuflüsse von Schnitten und Knoten

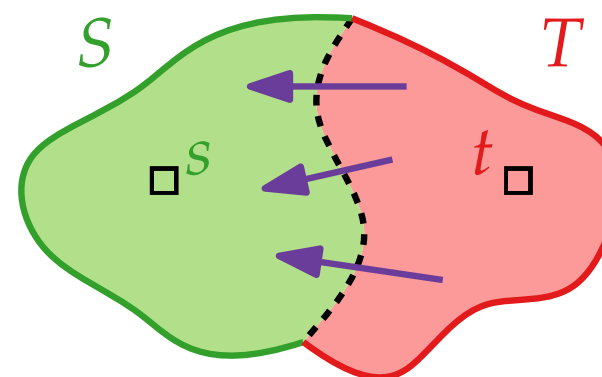


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

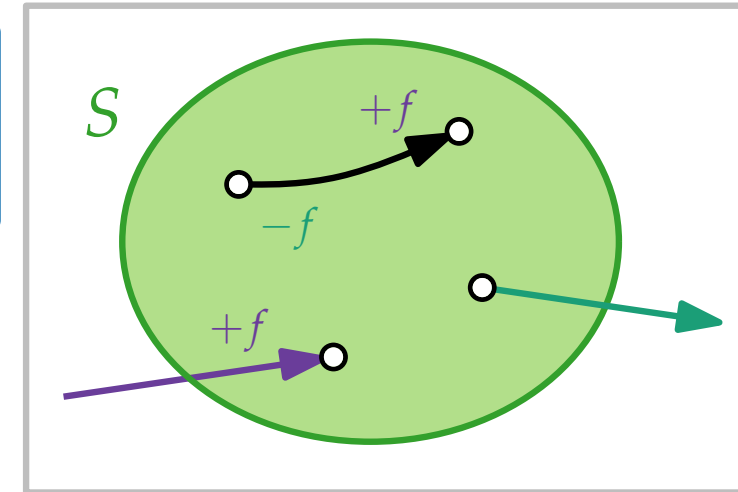


$\text{Rein}(S) \subseteq E$

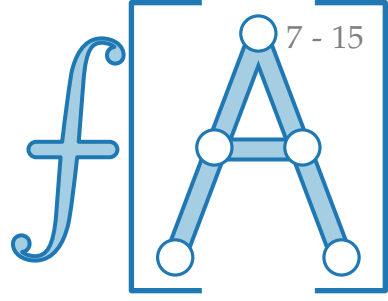


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Beweis.

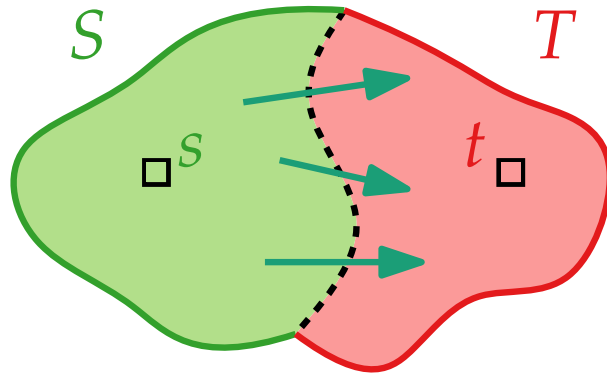
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} \left(\text{Zufluss}_f(v) - \text{Abfluss}_f(v) \right) \\ &= \sum_{v \in S} \left(\sum_{e=uv} f(e) - \sum_{e=vw} f(e) \right) \\ &= \end{aligned}$$


Nettozuflüsse von Schnitten und Knoten

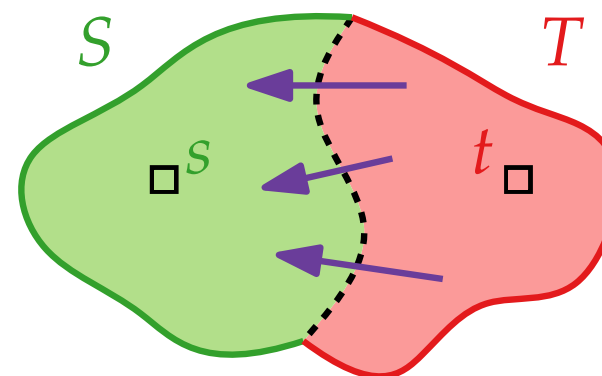


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

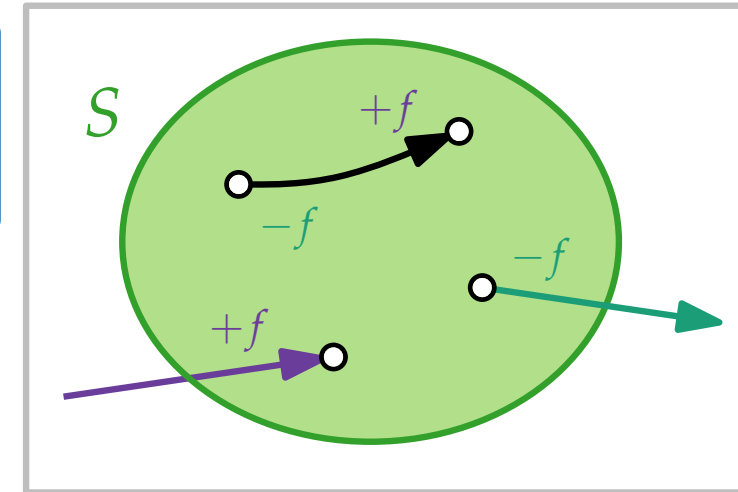


$\text{Rein}(S) \subseteq E$

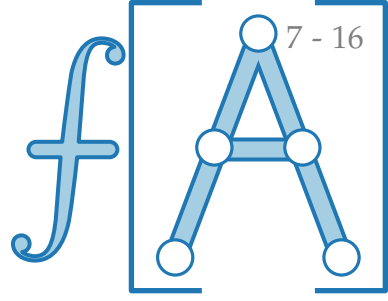


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Beweis.
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} (\text{Zufluss}_f(v) - \text{Abfluss}_f(v)) \\ &= \sum_{v \in S} (\sum_{e=uv} f(e) - \sum_{e=vw} f(e)) \\ &= \end{aligned}$$

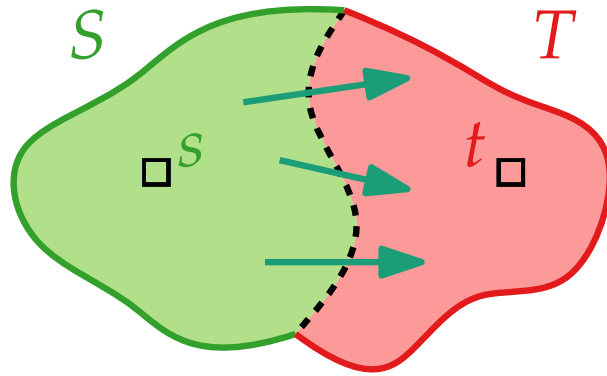


Nettozuflüsse von Schnitten und Knoten

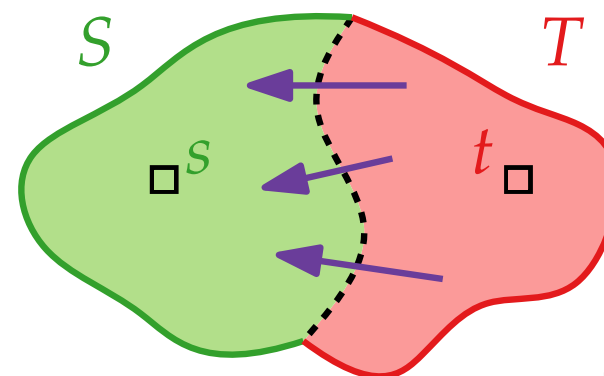


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

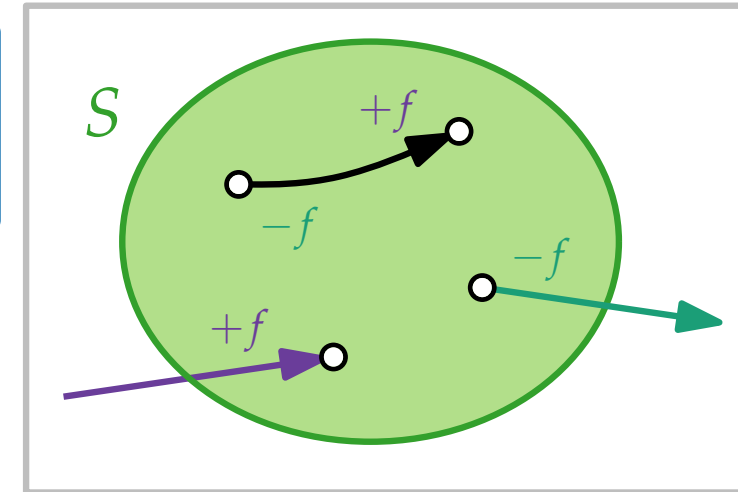


$\text{Rein}(S) \subseteq E$

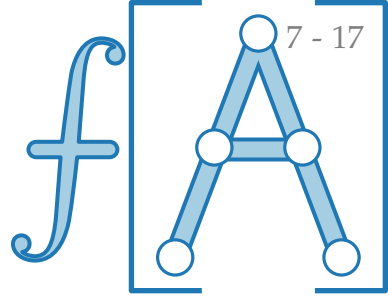


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Beweis.

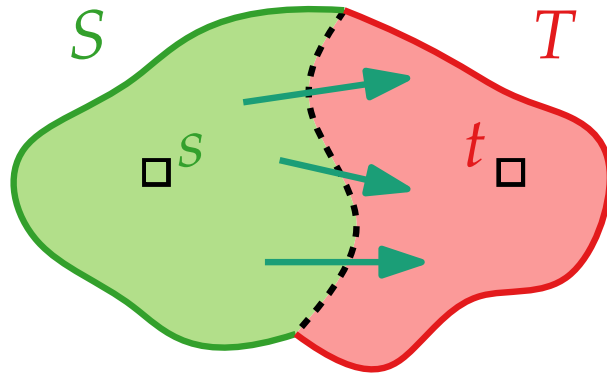
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} (\text{Zufluss}_f(v) - \text{Abfluss}_f(v)) \\ &= \sum_{v \in S} (\sum_{e=uv} f(e) - \sum_{e=vw} f(e)) \\ &= \sum_{e \in \text{Rein}(S)} f(e) - \sum_{e \in \text{Raus}(S)} f(e) \end{aligned}$$


Nettozuflüsse von Schnitten und Knoten

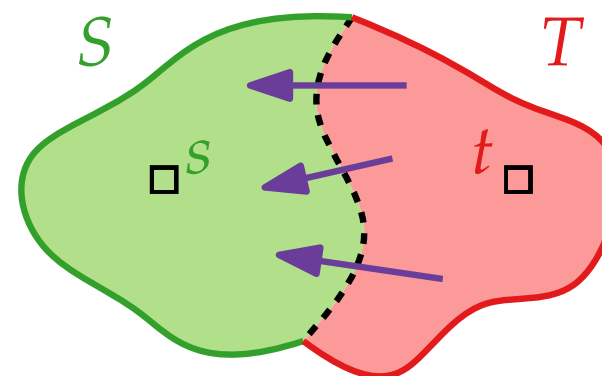


Zur Erinnerung: $\text{Nettozufluss}_f(S) = \text{Zufluss}_f(S) - \text{Abfluss}_f(S)$

$\text{Raus}(S) \subseteq E$

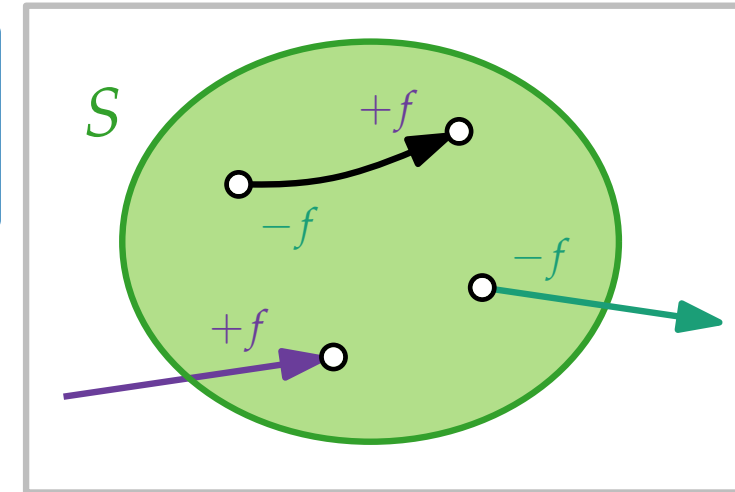


$\text{Rein}(S) \subseteq E$



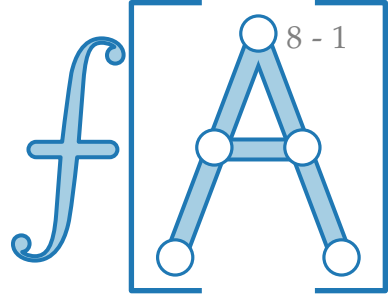
Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Beweis.

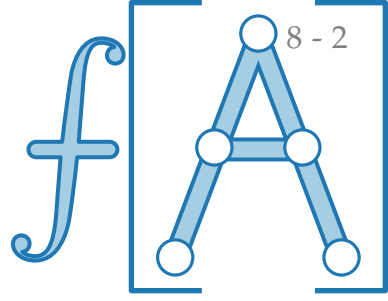
$$\begin{aligned} \sum_{v \in S} \text{Nettozufluss}_f(v) &= \sum_{v \in S} (\text{Zufluss}_f(v) - \text{Abfluss}_f(v)) \\ &= \sum_{v \in S} (\sum_{e=uv} f(e) - \sum_{e=vw} f(e)) \\ &= \sum_{e \in \text{Rein}(S)} f(e) - \sum_{e \in \text{Raus}(S)} f(e) = \text{Nettozufluss}_f(S) \quad \square \end{aligned}$$


Noch mehr Schnitte

Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v).$



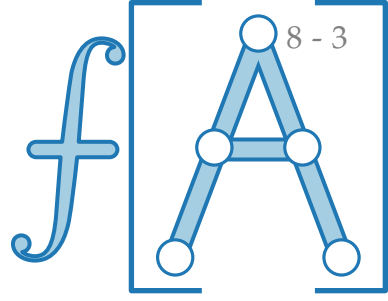
Noch mehr Schnitte



Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| =$

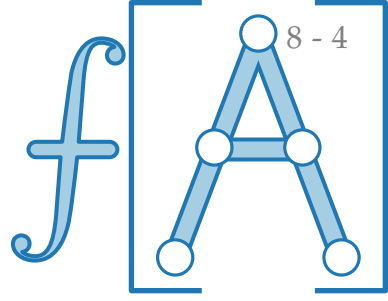
Noch mehr Schnitte



Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T)$.

Noch mehr Schnitte

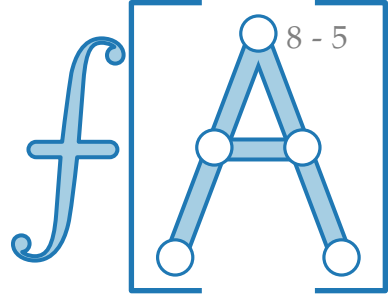


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T)$.

Beweis. $|f| \stackrel{\text{Def.}}{=}$

Noch mehr Schnitte

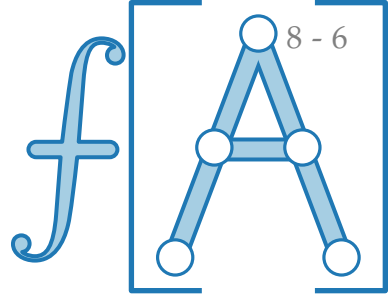


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T)$.

Beweis. $|f| \stackrel{\text{Def.}}{=} \text{Nettozufluss}_f(t)$
 $=$

Noch mehr Schnitte

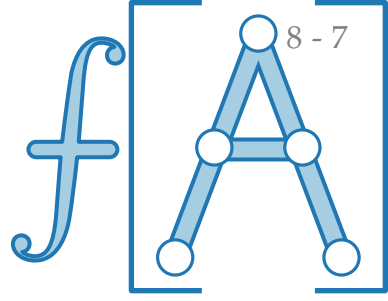


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T)$.

Beweis. $|f| \stackrel{\text{Def.}}{=} \text{Nettozufluss}_f(t)$
 $= \sum_{v \in T} \text{Nettozufluss}_f(v)$

Noch mehr Schnitte

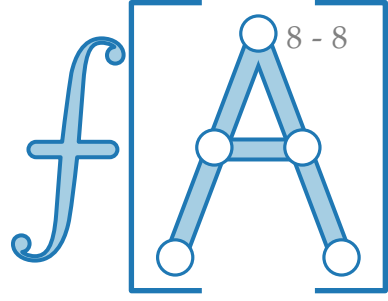


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T)$.

Beweis. $|f| \stackrel{\text{Def.}}{=} \text{Nettozufluss}_f(t)$
 $= \sum_{v \in T} \text{Nettozufluss}_f(v)$
da $\text{Nettozufluss}_f(v) = 0$ für alle $v \neq s, t$

Noch mehr Schnitte

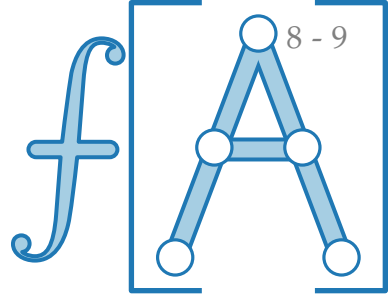


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T)$.

Beweis. $|f| \stackrel{\text{Def.}}{=} \text{Nettozufluss}_f(t)$
 $= \sum_{v \in T} \text{Nettozufluss}_f(v)$
da $\text{Nettozufluss}_f(v) = 0$ für alle $v \neq s, t$
 $=$

Noch mehr Schnitte

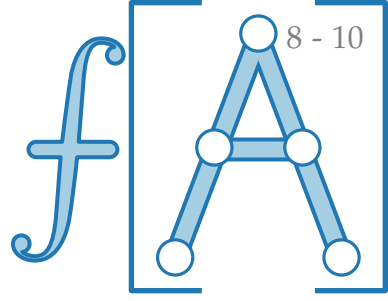


Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T)$.

Beweis. $|f| \stackrel{\text{Def.}}{=} \text{Nettozufluss}_f(t)$
 $= \sum_{v \in T} \text{Nettozufluss}_f(v)$
da $\text{Nettozufluss}_f(v) = 0$ für alle $v \neq s, t$
 $\Rightarrow$

Noch mehr Schnitte



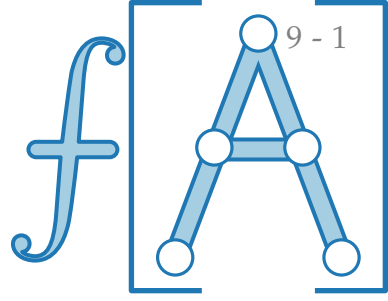
Lemma 1. Sei $G = (V, E)$ Graph, $S \subseteq V$ und $f: E \rightarrow \mathbb{R}$. Dann:
 $\text{Nettozufluss}_f(S) = \sum_{v \in S} \text{Nettozufluss}_f(v)$.

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T)$.

Beweis. $|f| \stackrel{\text{Def.}}{=} \text{Nettozufluss}_f(t)$
 $= \sum_{v \in T} \text{Nettozufluss}_f(v)$
da $\text{Nettozufluss}_f(v) = 0$ für alle $v \neq s, t$
 $= \text{Nettozufluss}_f(T)$ □

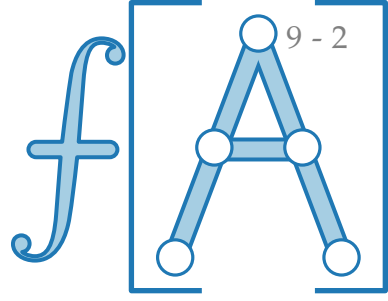
Kapazität von Schnitten

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T)$.

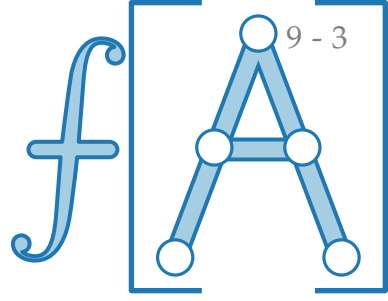


Kapazität von Schnitten

Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

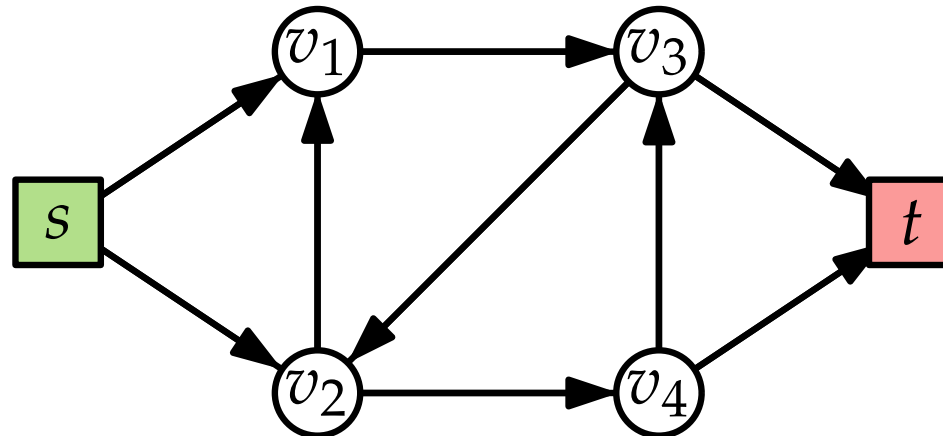


Kapazität von Schnitten

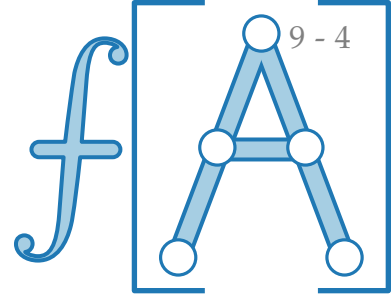


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

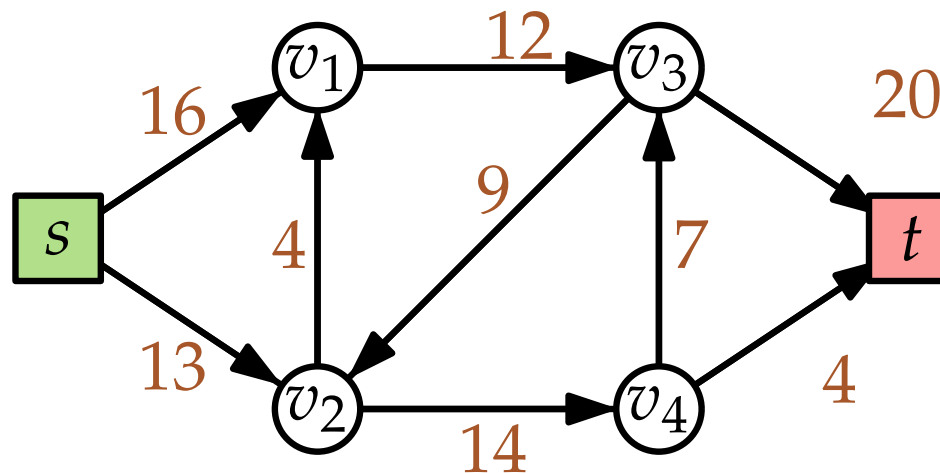


Kapazität von Schnitten

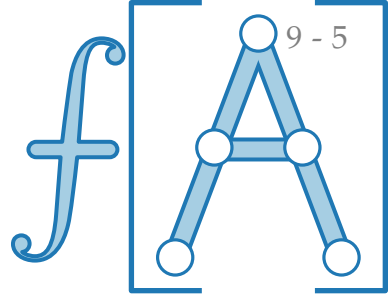


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

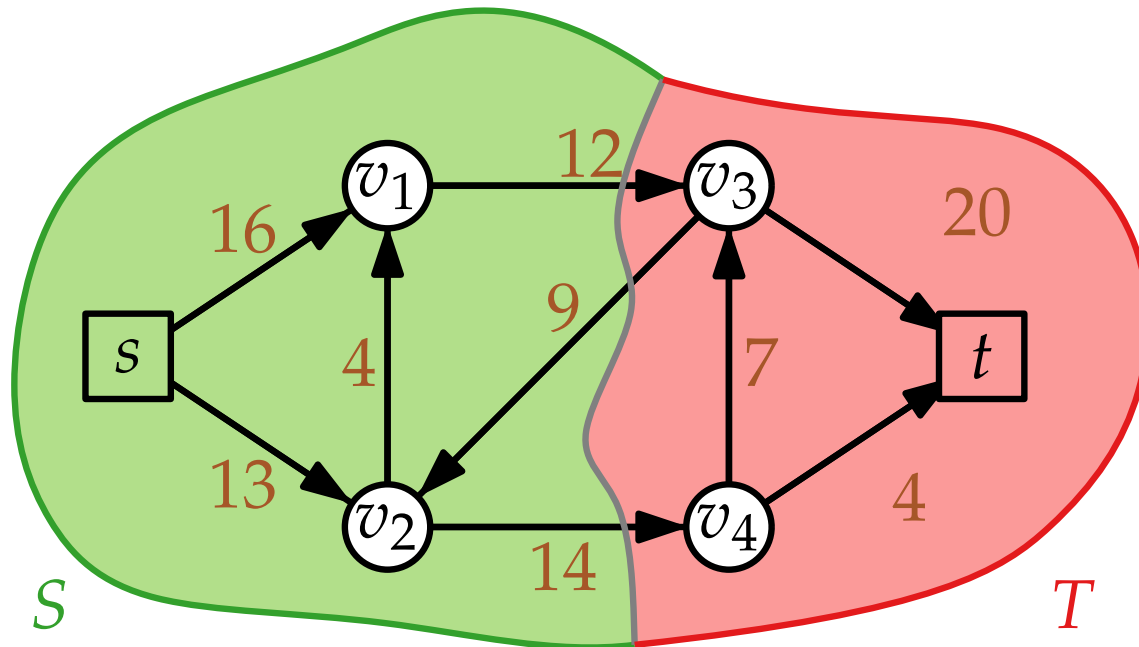


Kapazität von Schnitten

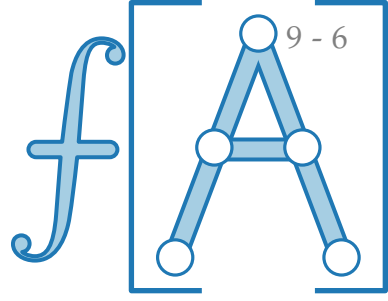


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

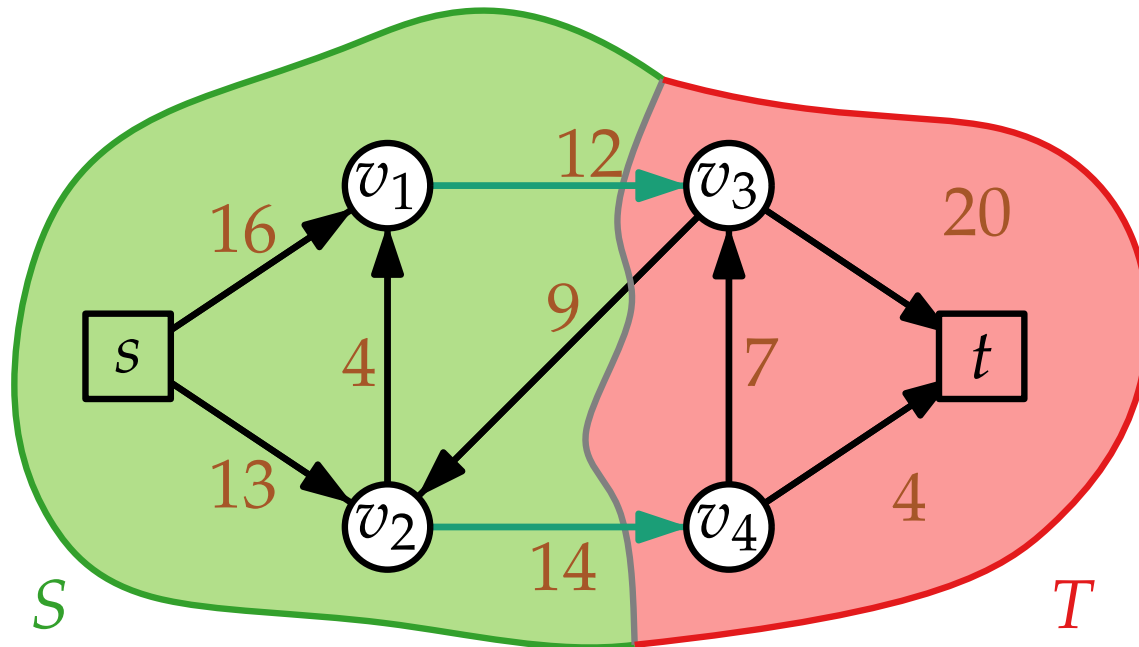


Kapazität von Schnitten

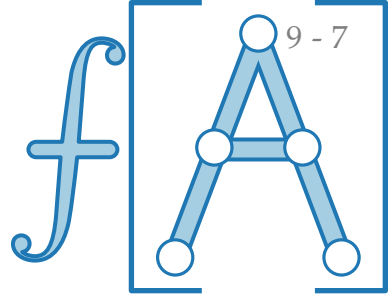


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

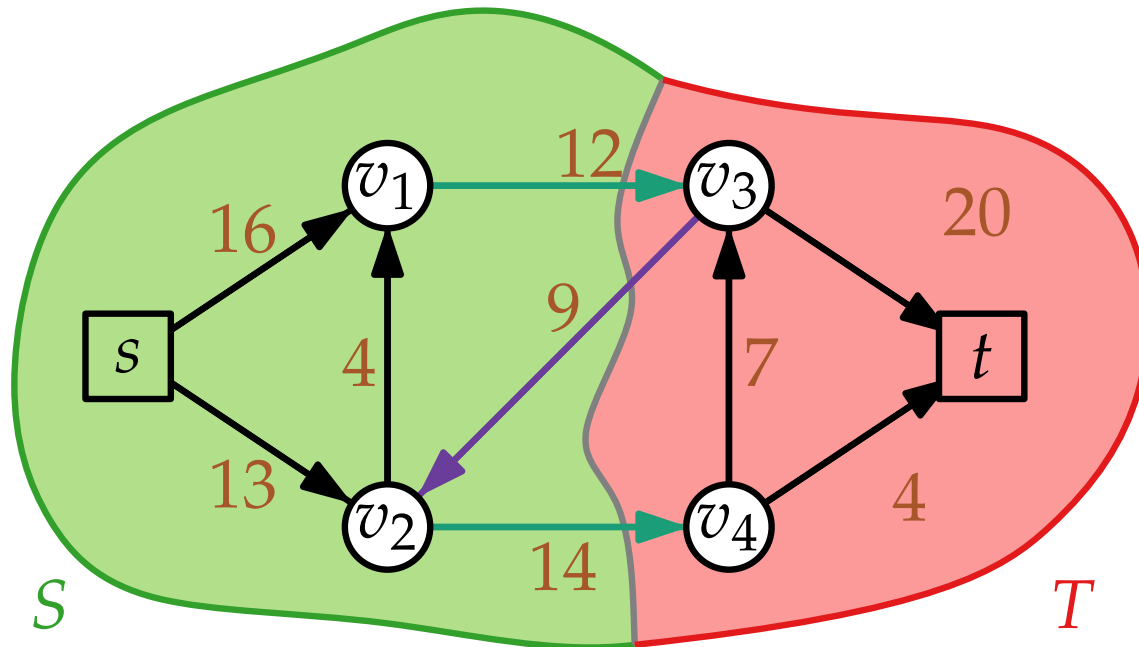


Kapazität von Schnitten

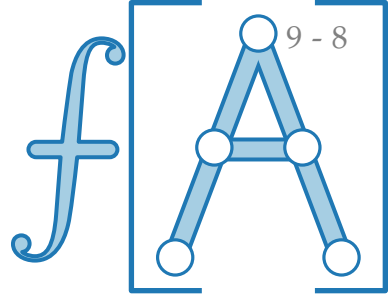


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

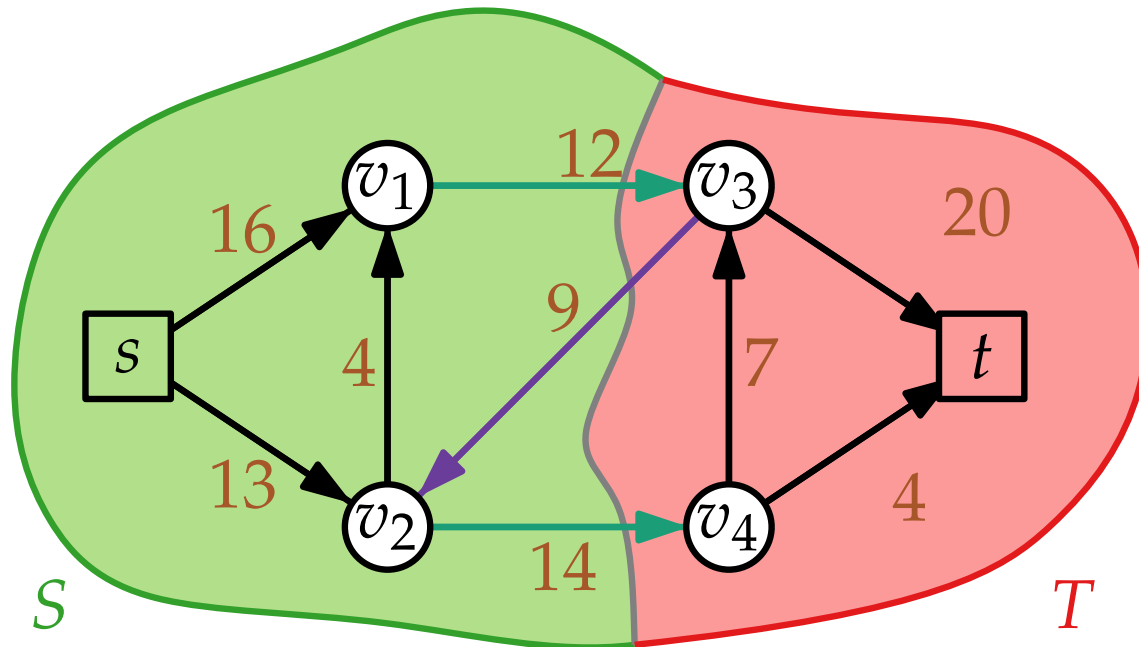


Kapazität von Schnitten

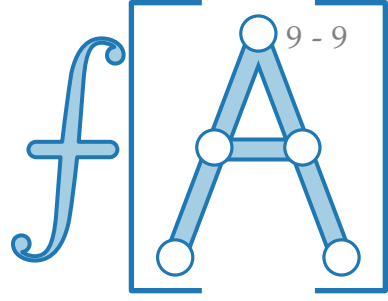


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

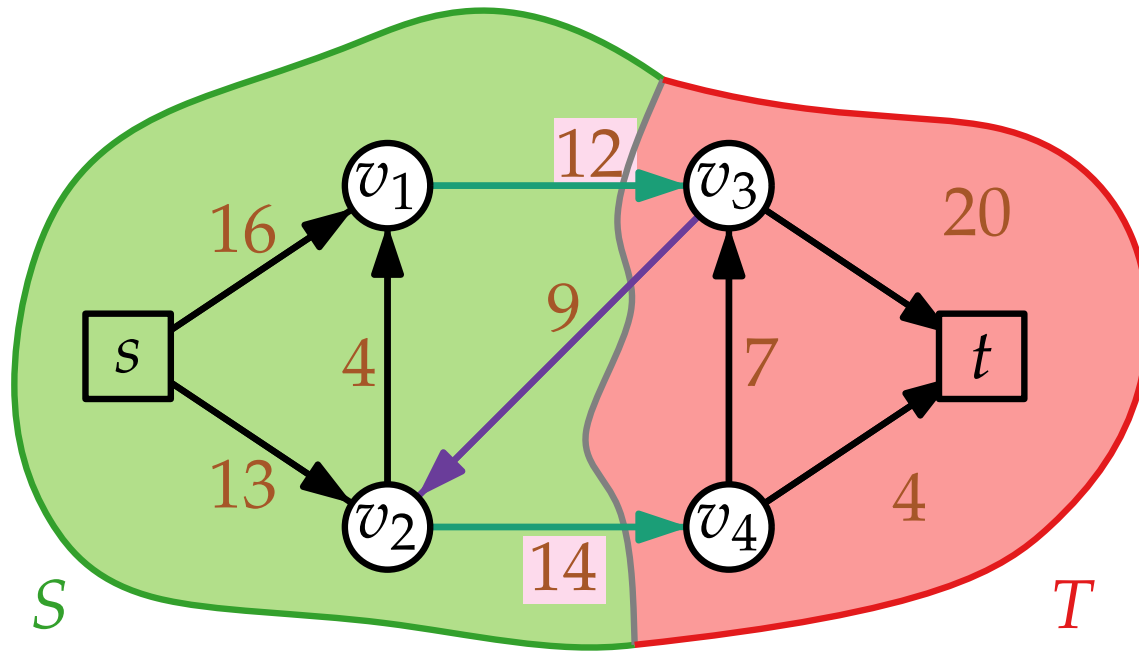


Kapazität von Schnitten

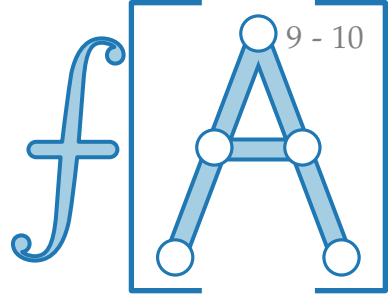


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

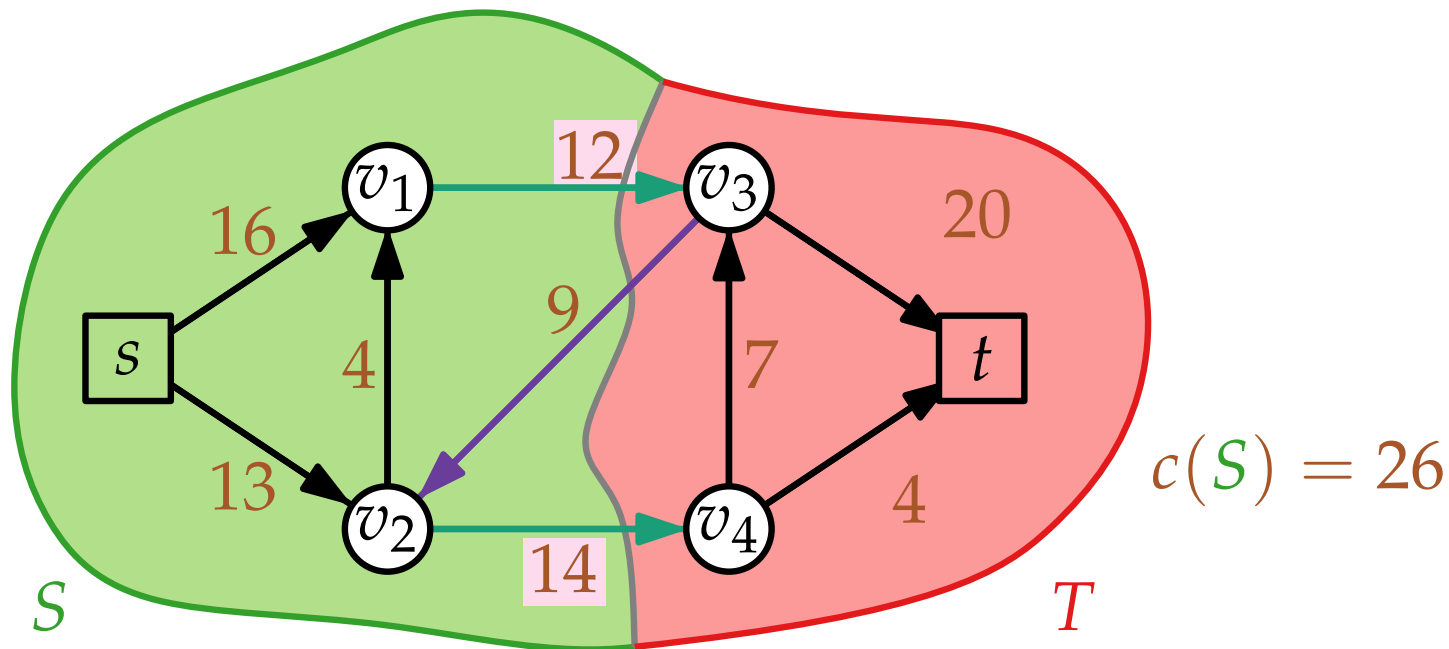


Kapazität von Schnitten

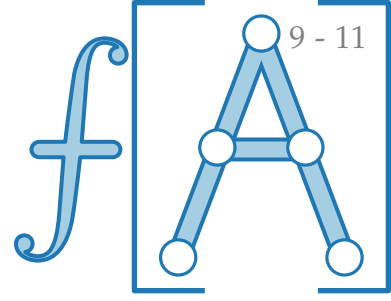


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .



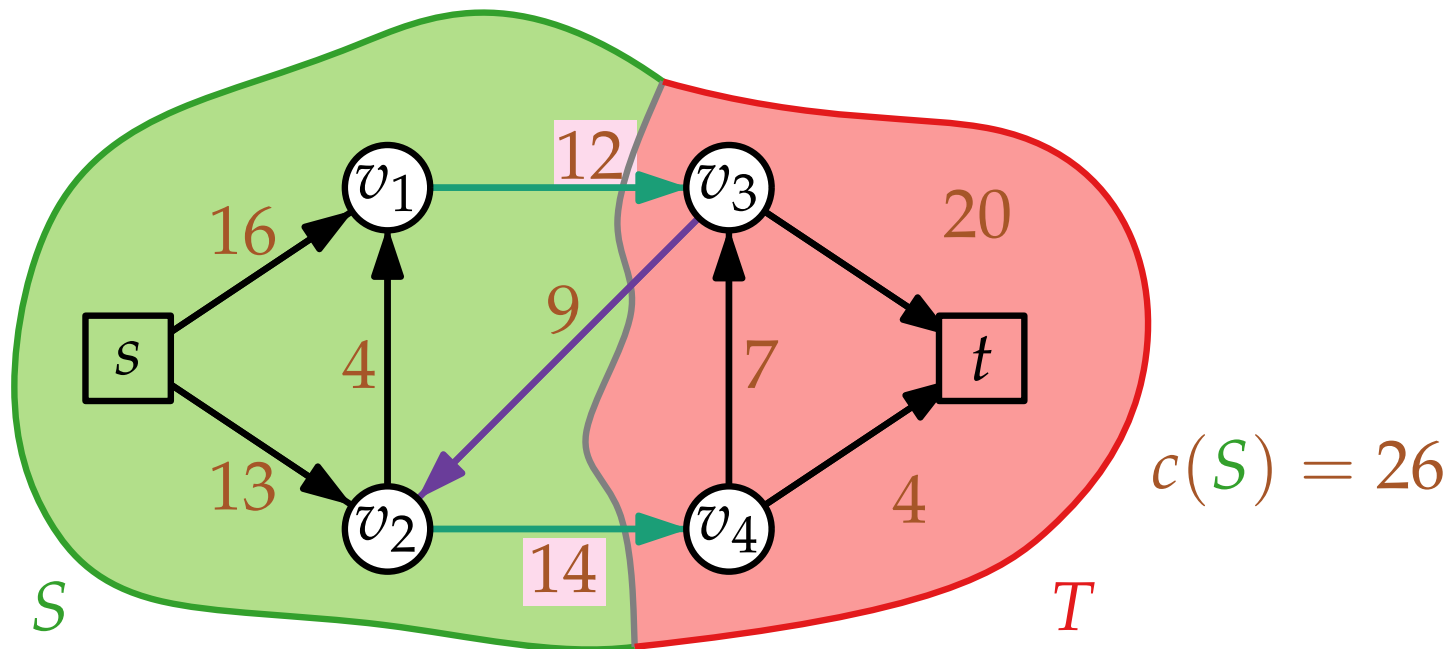
Kapazität von Schnitten



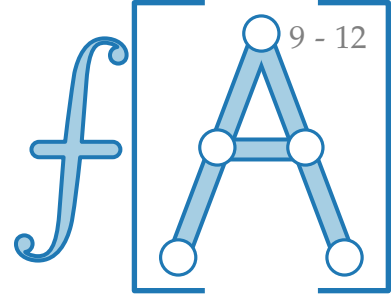
Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.



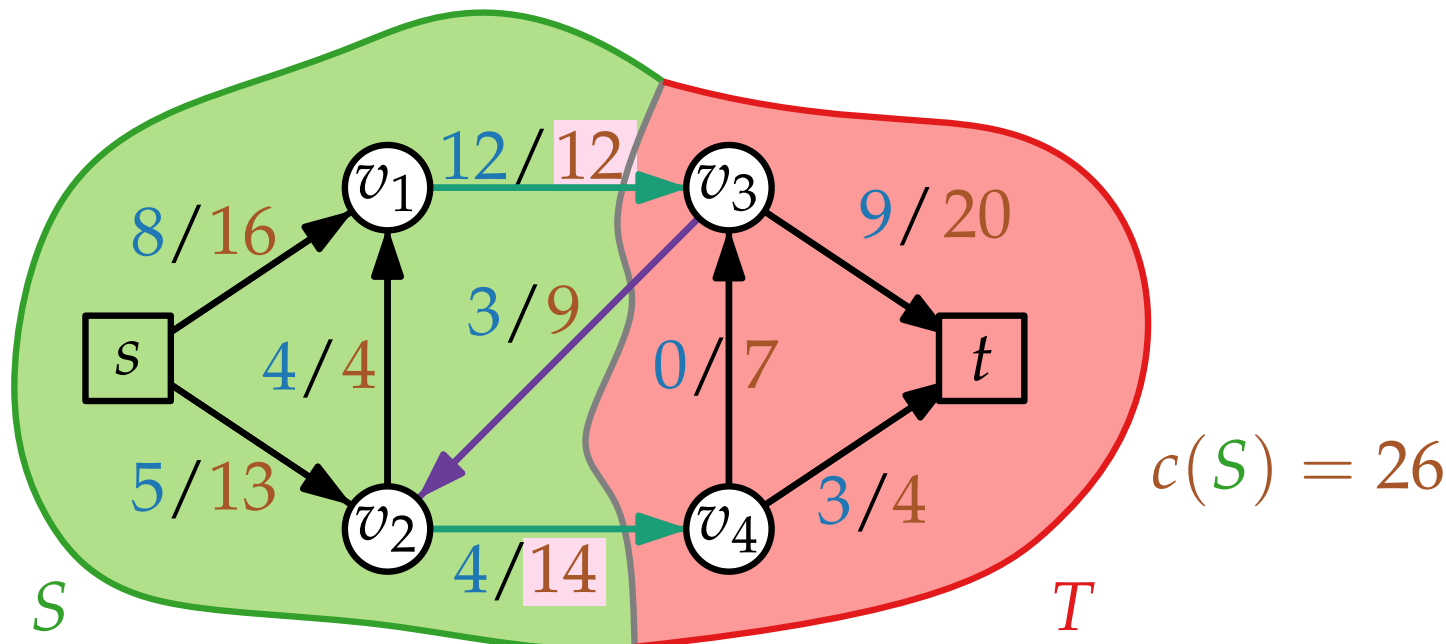
Kapazität von Schnitten



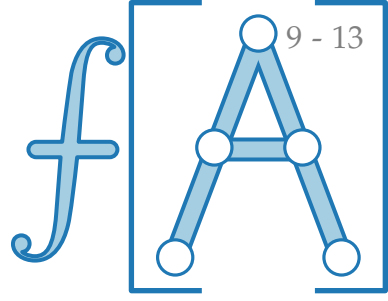
Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.



Kapazität von Schnitten

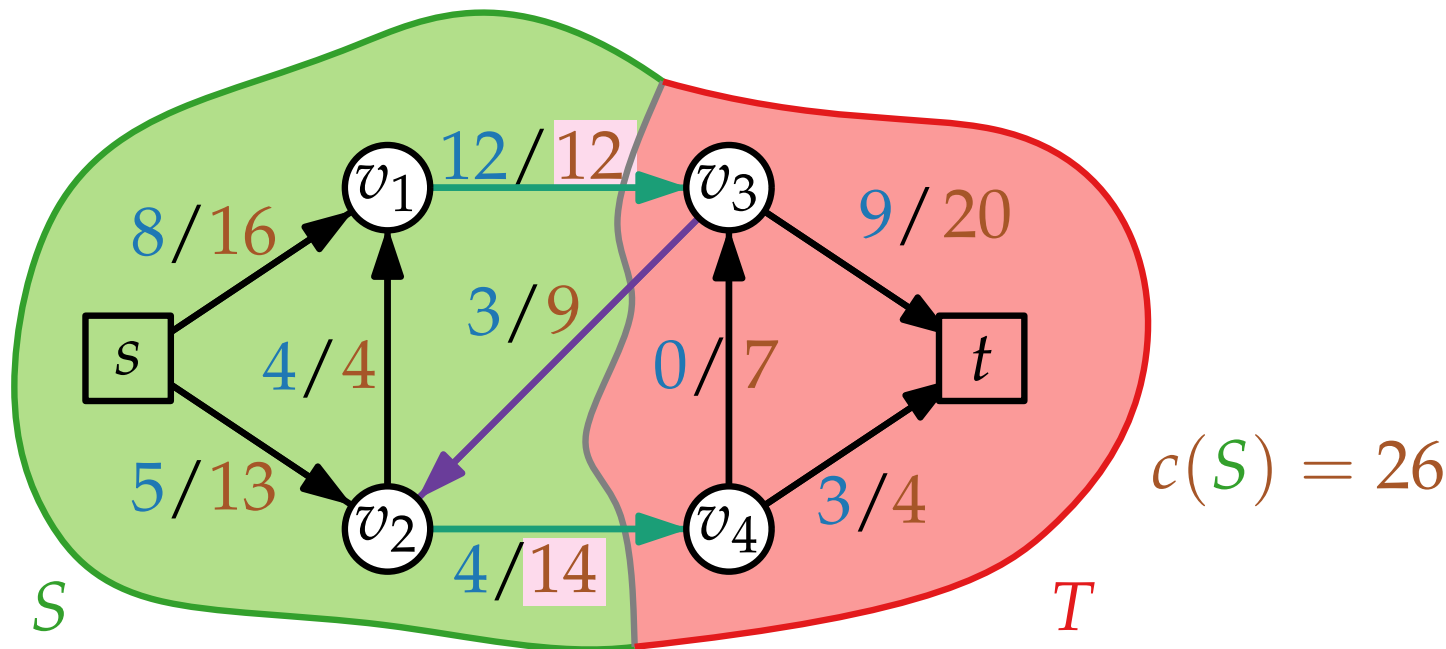


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

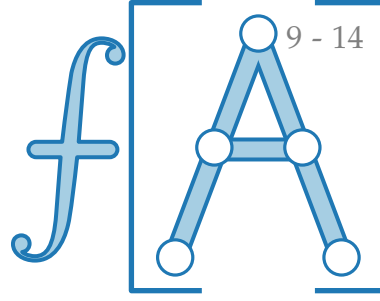
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| =$



Kapazität von Schnitten

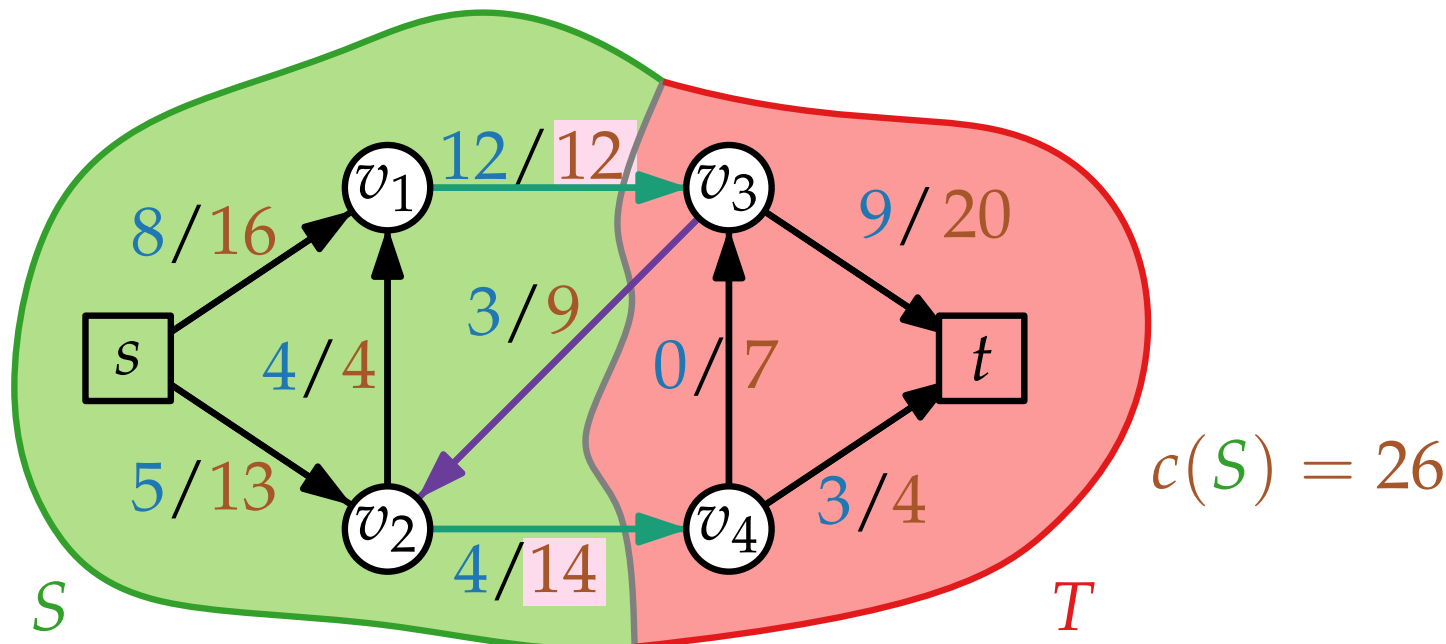


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

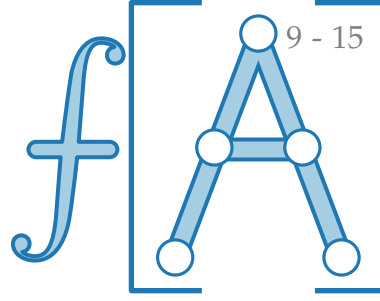
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| =$



Kapazität von Schnitten

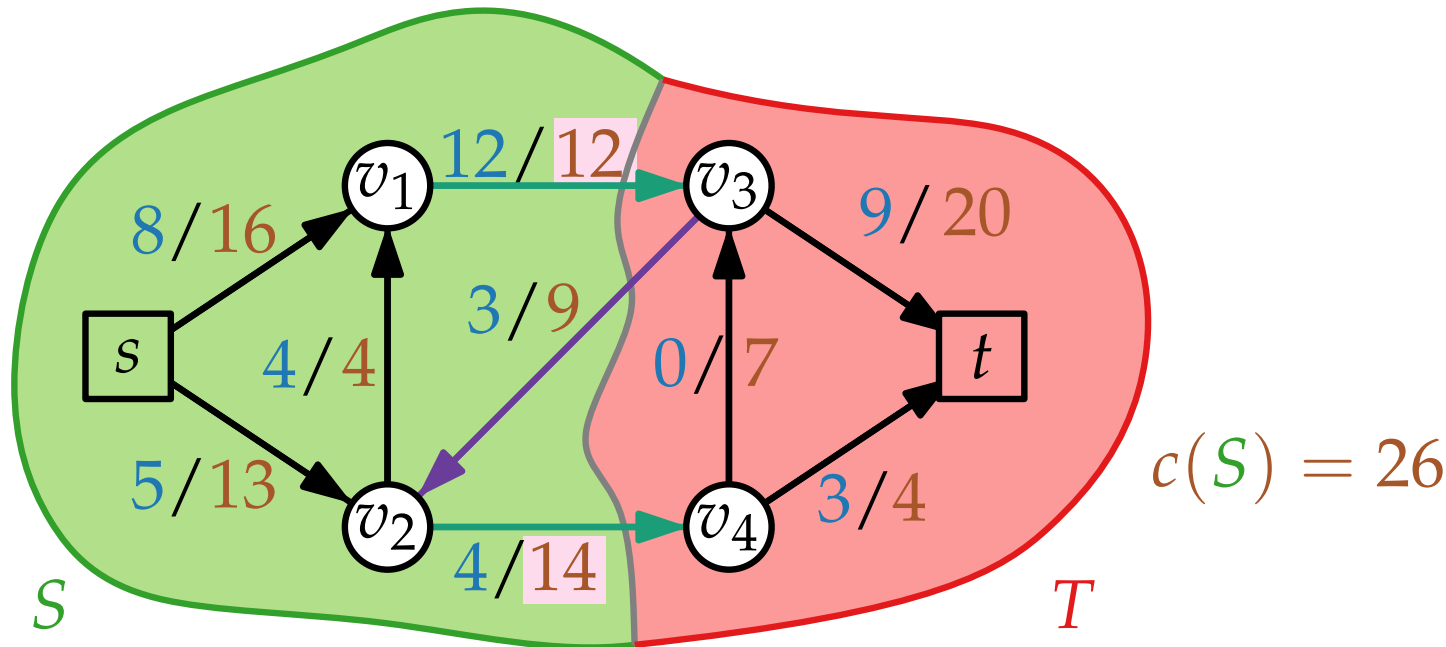


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

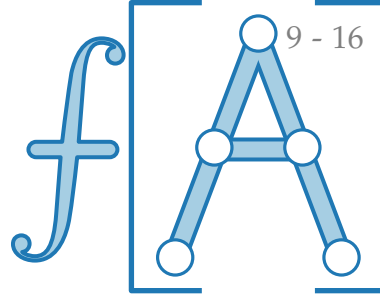
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S)$



Kapazität von Schnitten

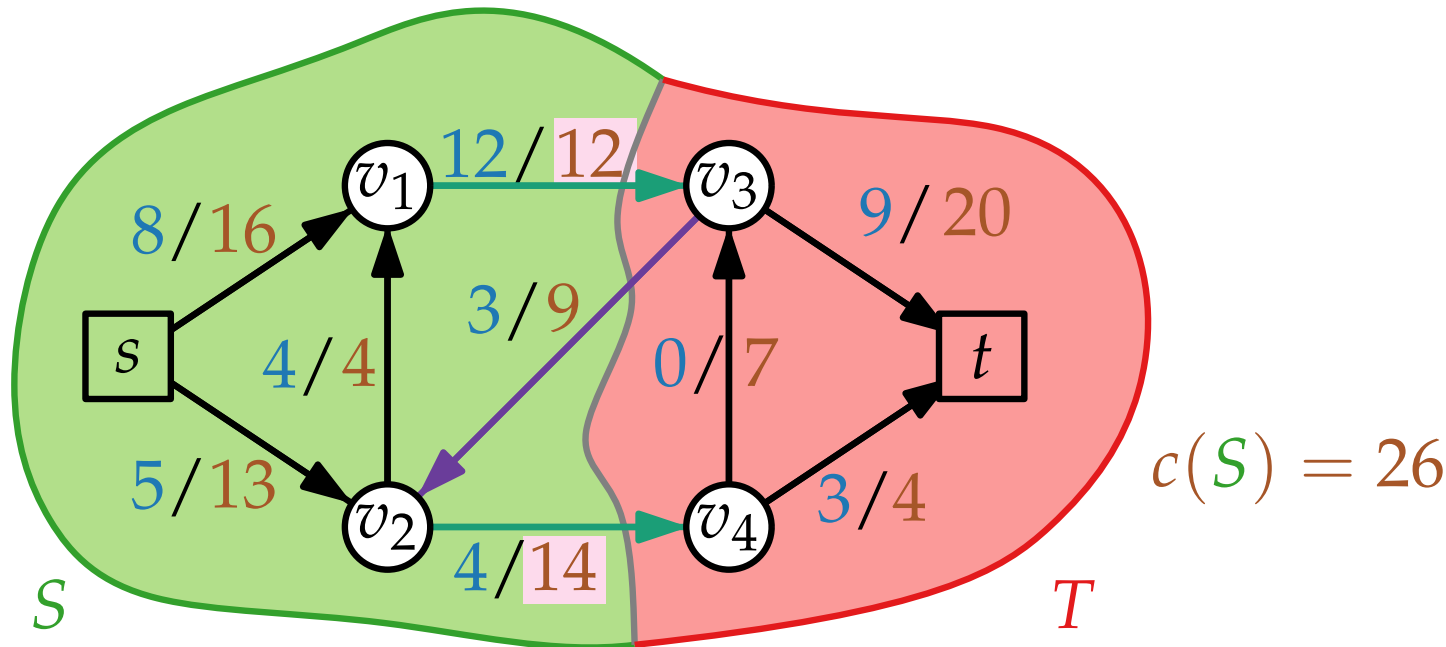


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

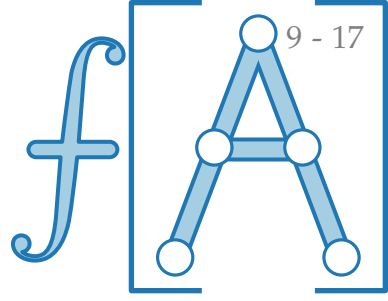
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann
ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) =$



Kapazität von Schnitten

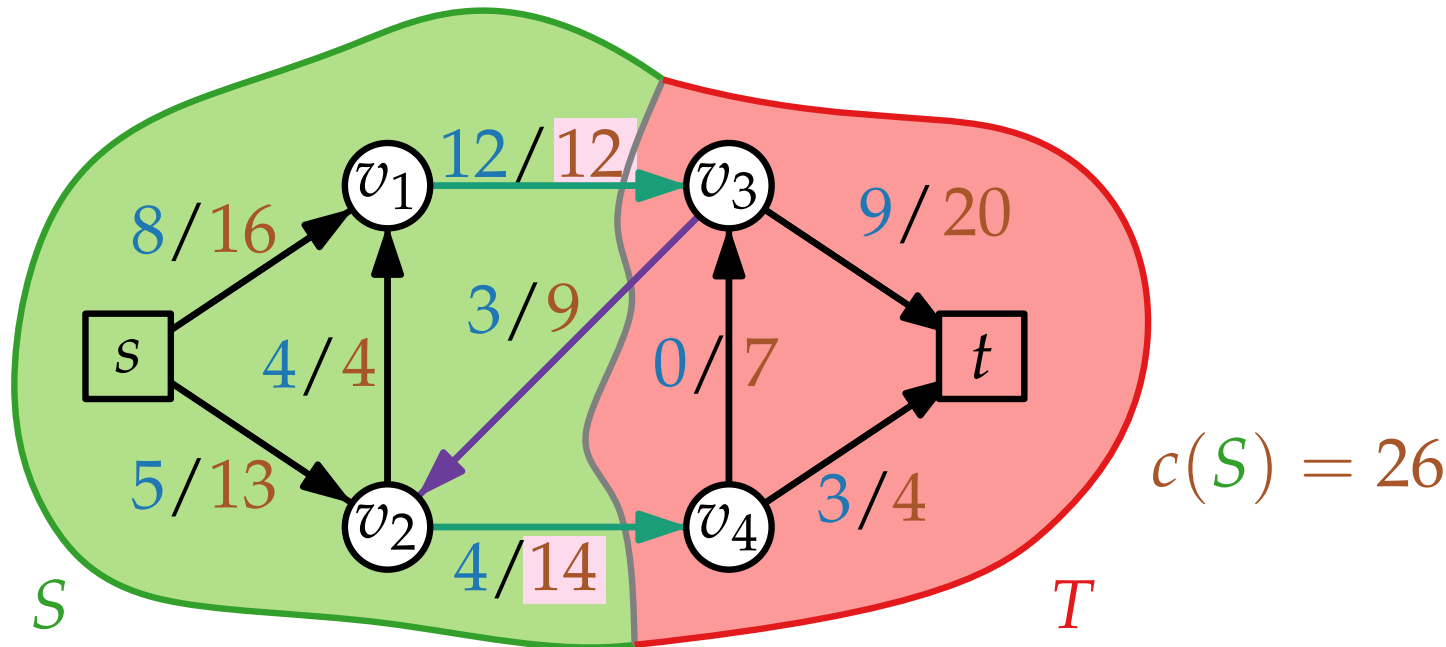


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

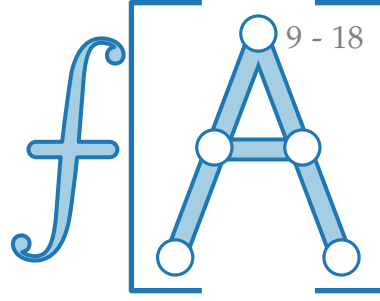
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$



Kapazität von Schnitten

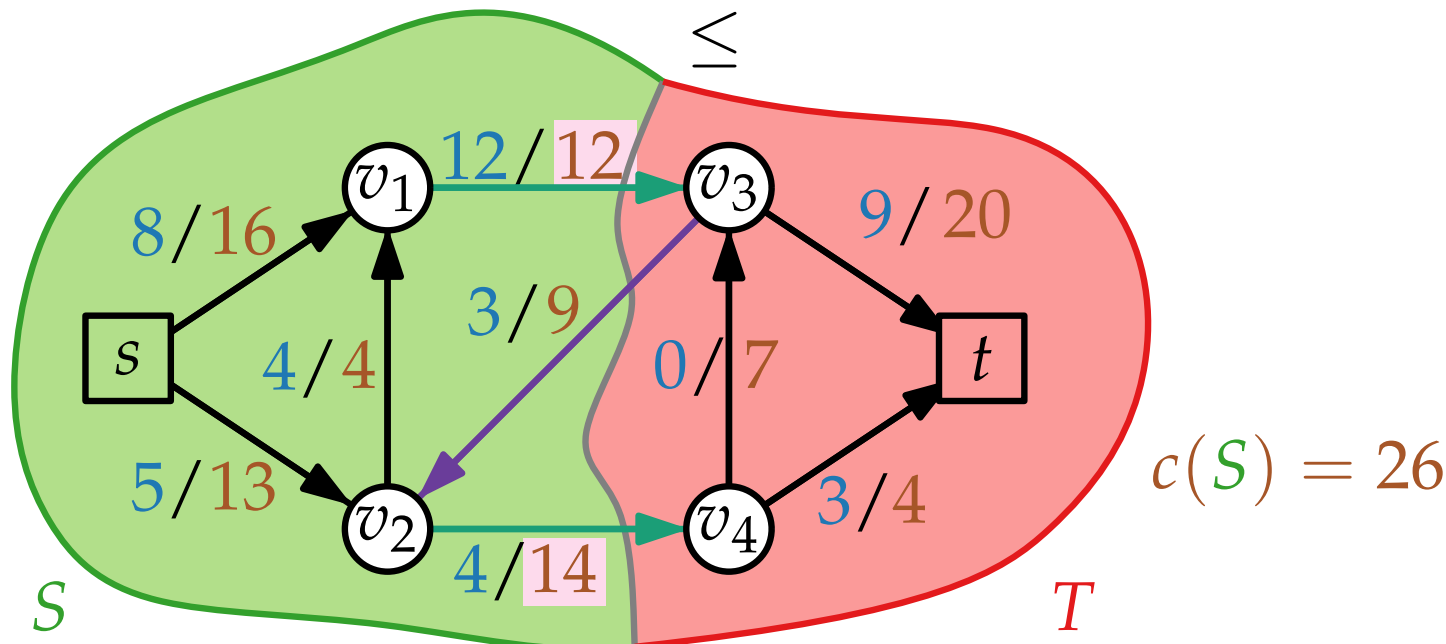


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

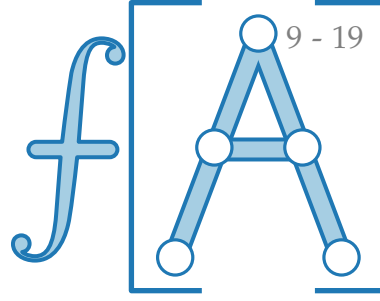
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$



Kapazität von Schnitten

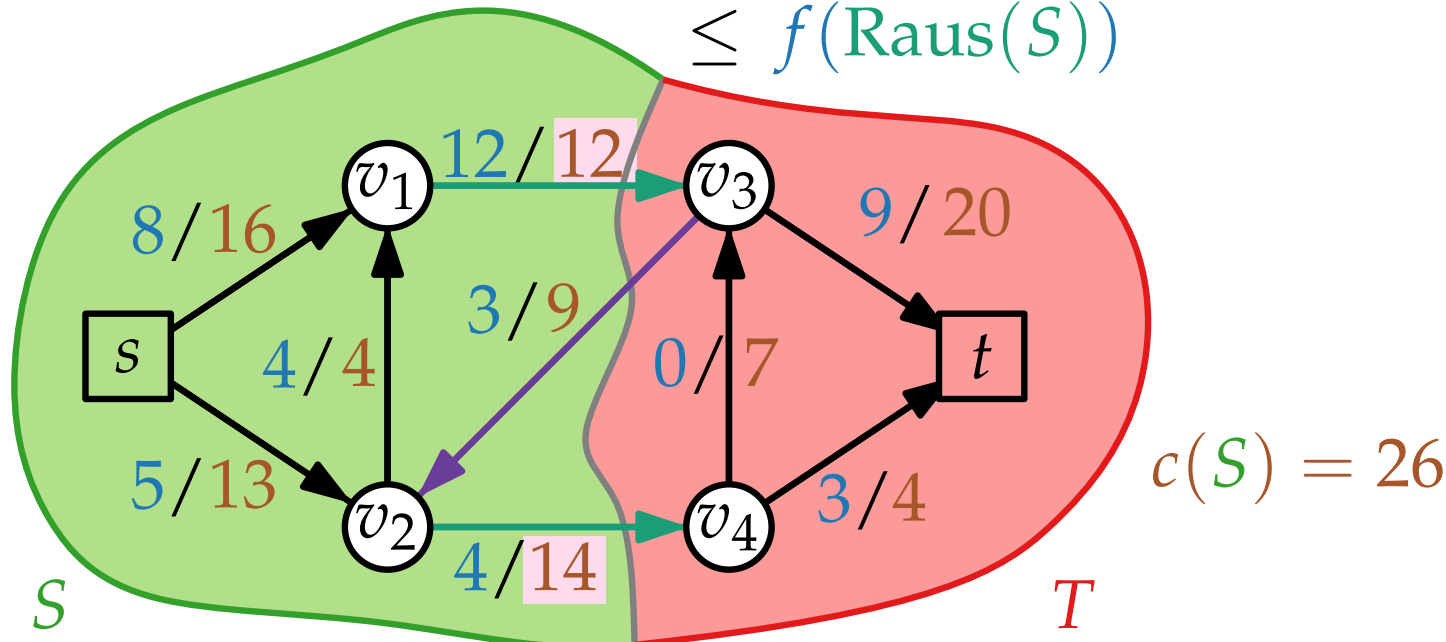


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

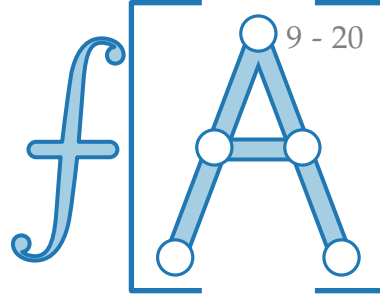
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S)) \leq f(\text{Raus}(S))$



Kapazität von Schnitten

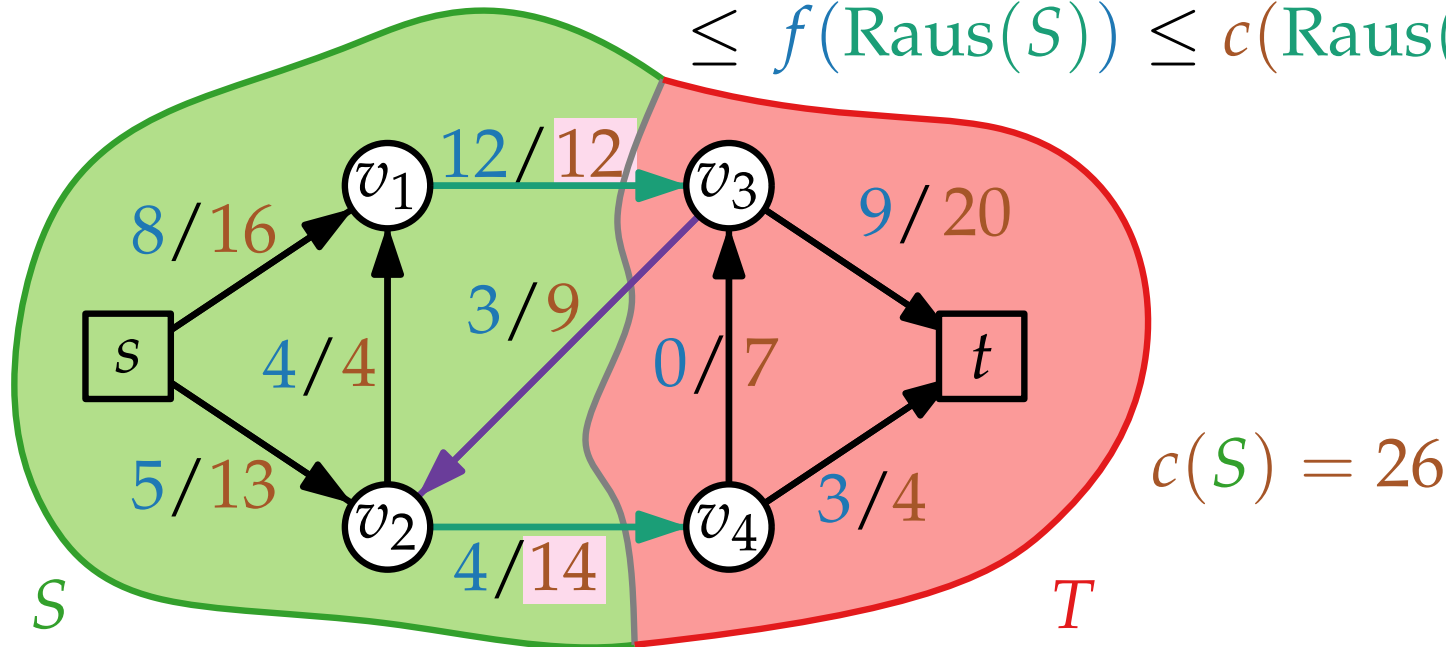


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

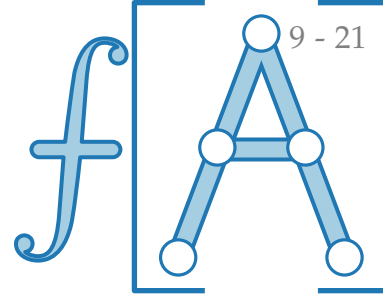
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$
 $\leq f(\text{Raus}(S)) \leq c(\text{Raus}(S))$



Kapazität von Schnitten

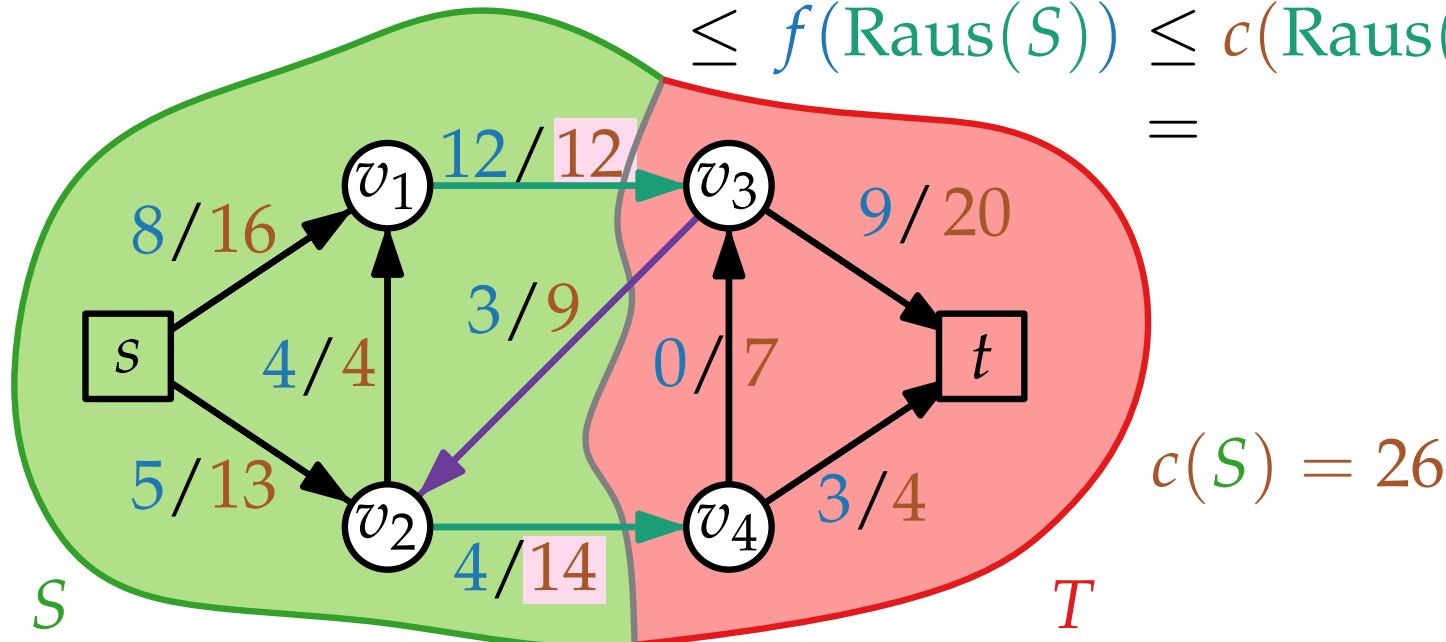


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

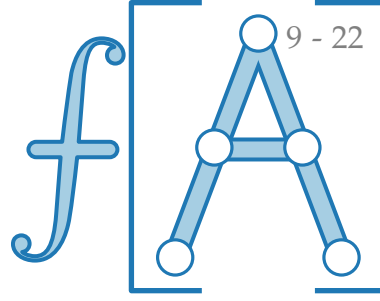
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$
 $\leq f(\text{Raus}(S)) \leq c(\text{Raus}(S))$



Kapazität von Schnitten

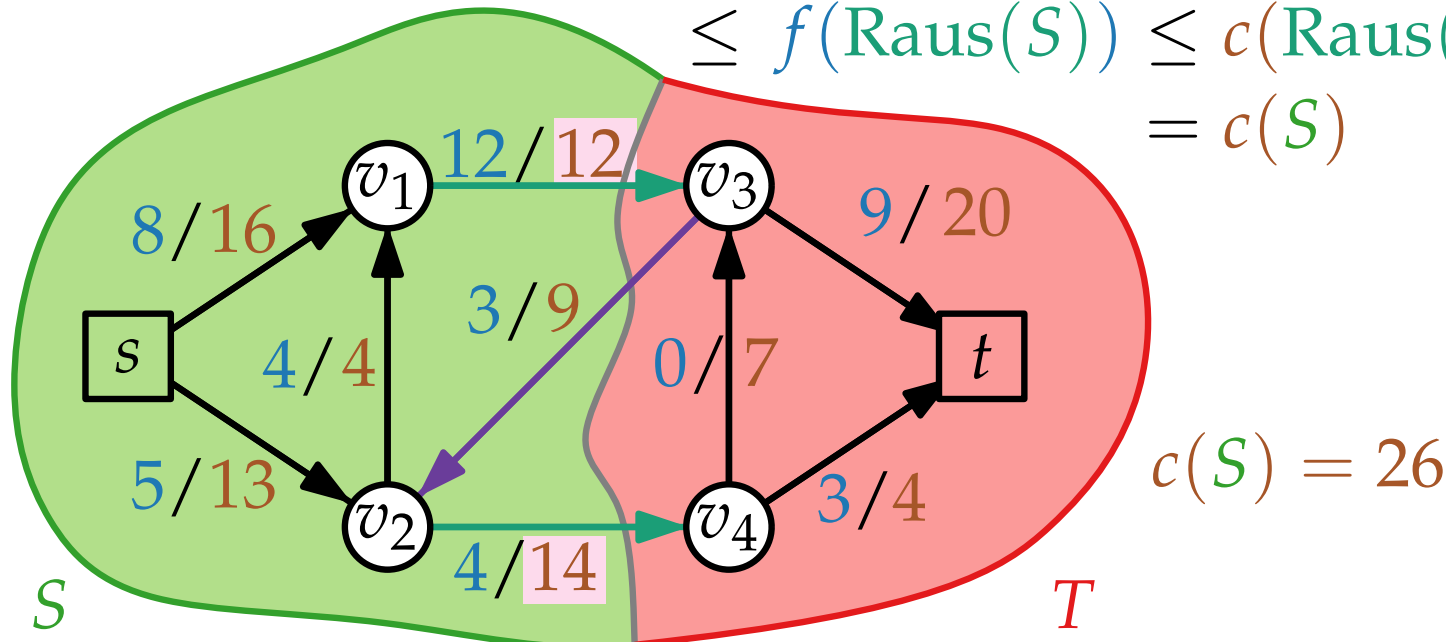


Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

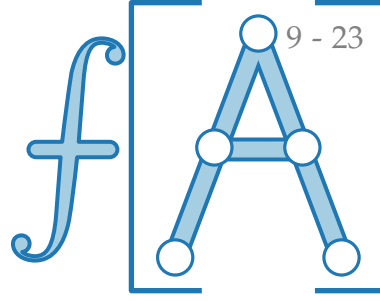
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$
 $\leq f(\text{Raus}(S)) \leq c(\text{Raus}(S)) = c(S) \quad \square$



Kapazität von Schnitten



Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

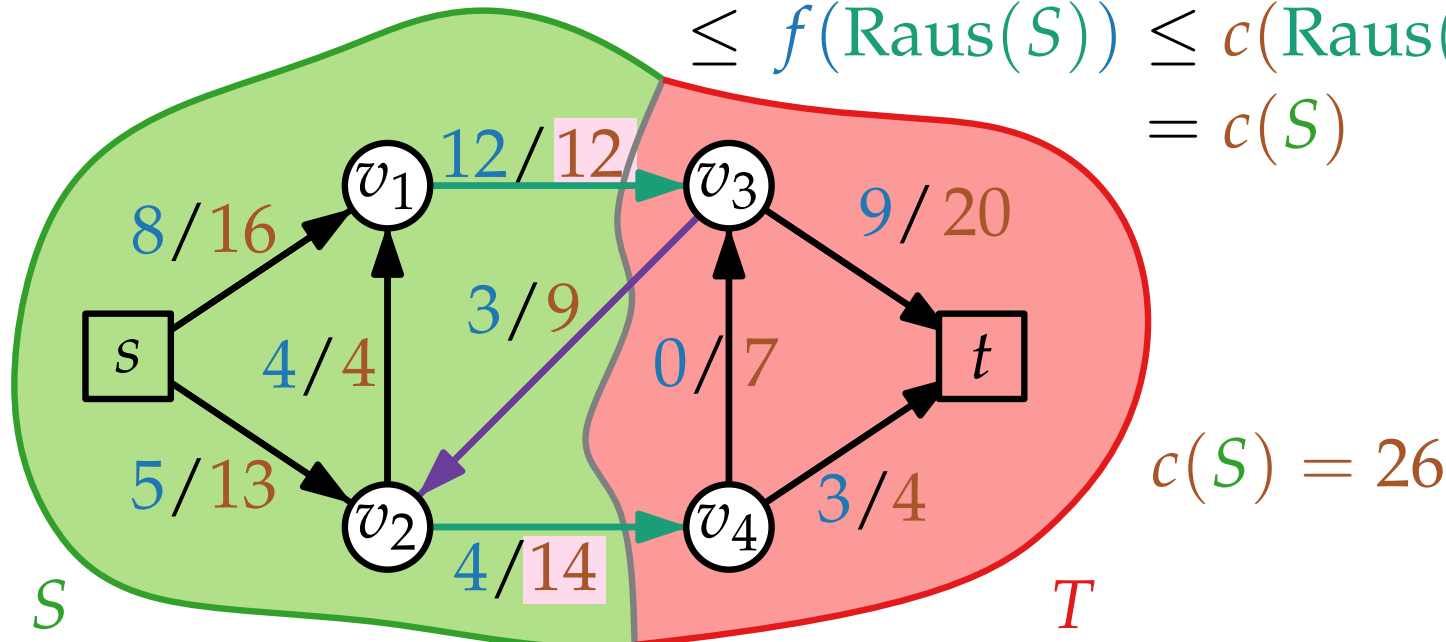
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

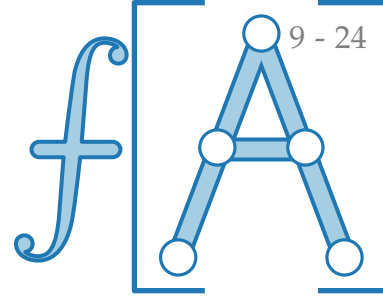
Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$
 $\leq f(\text{Raus}(S)) \leq c(\text{Raus}(S)) = c(S) \quad \square$

Speziell:

$$\min_S c(S) \geq \max_f |f|$$



Kapazität von Schnitten



Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

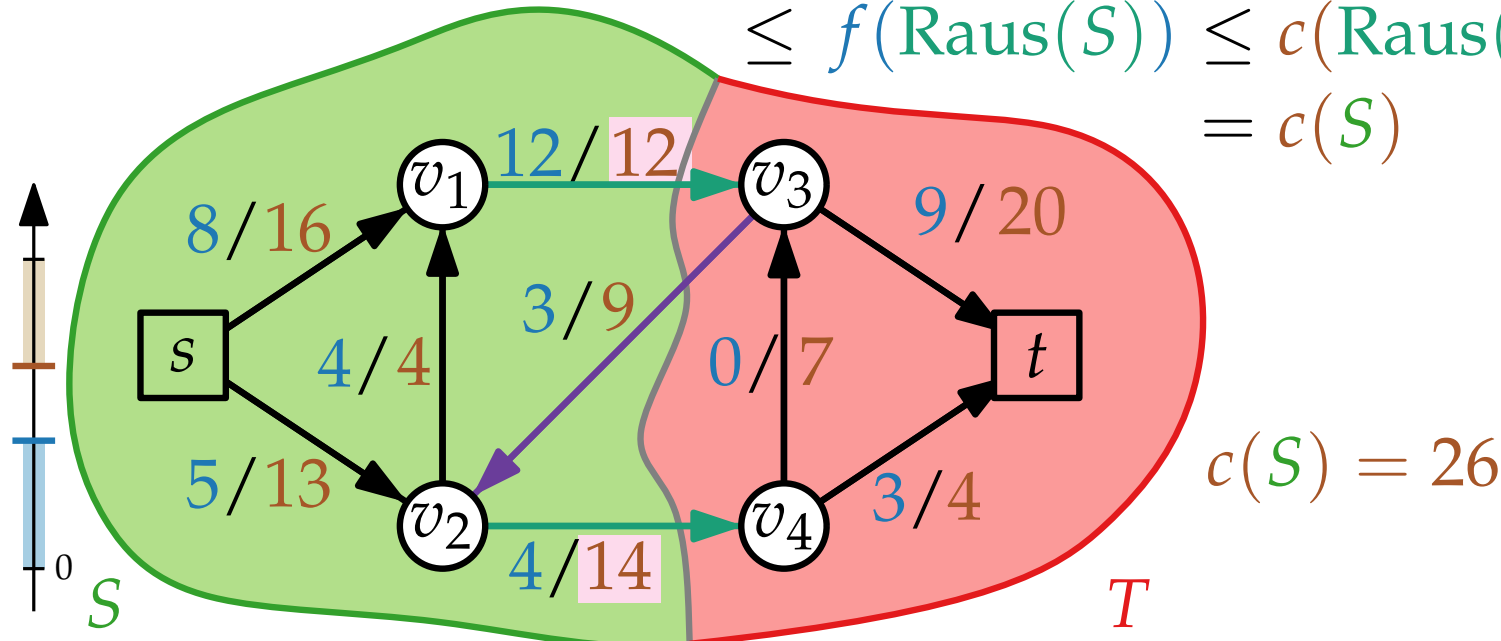
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

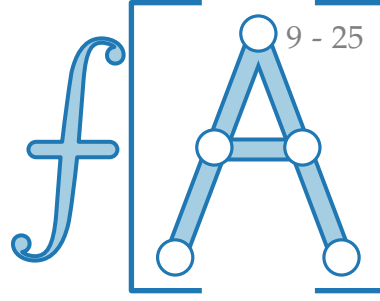
Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$
 $\leq f(\text{Raus}(S)) \leq c(\text{Raus}(S)) = c(S) \quad \square$

Speziell:

$$\min_S c(S) \geq \max_f |f|$$



Kapazität von Schnitten



Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

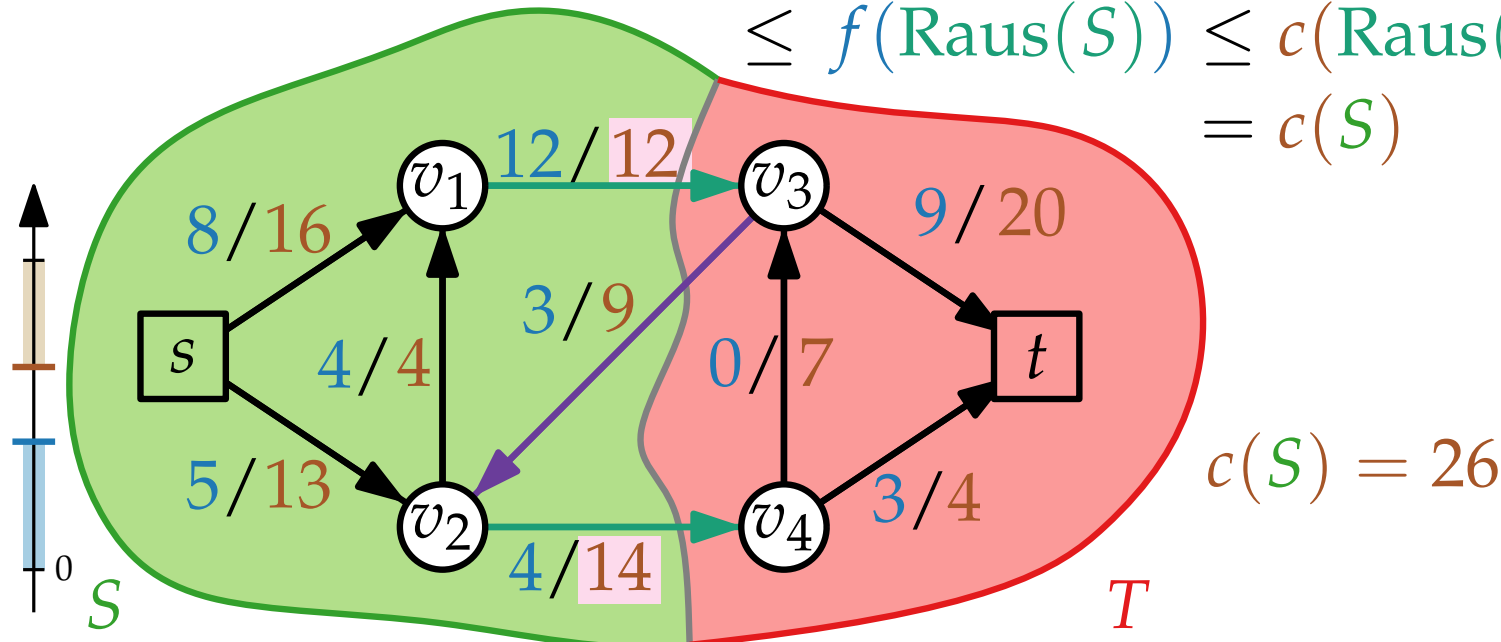
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$
 $\leq f(\text{Raus}(S)) \leq c(\text{Raus}(S)) = c(S) \quad \square$

Speziell:

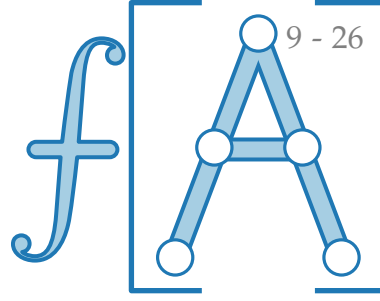
$$\min_S c(S) \geq \max_f |f|$$



Korollar.

Wenn $|f| = c(S)$
 $\Rightarrow$

Kapazität von Schnitten



Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

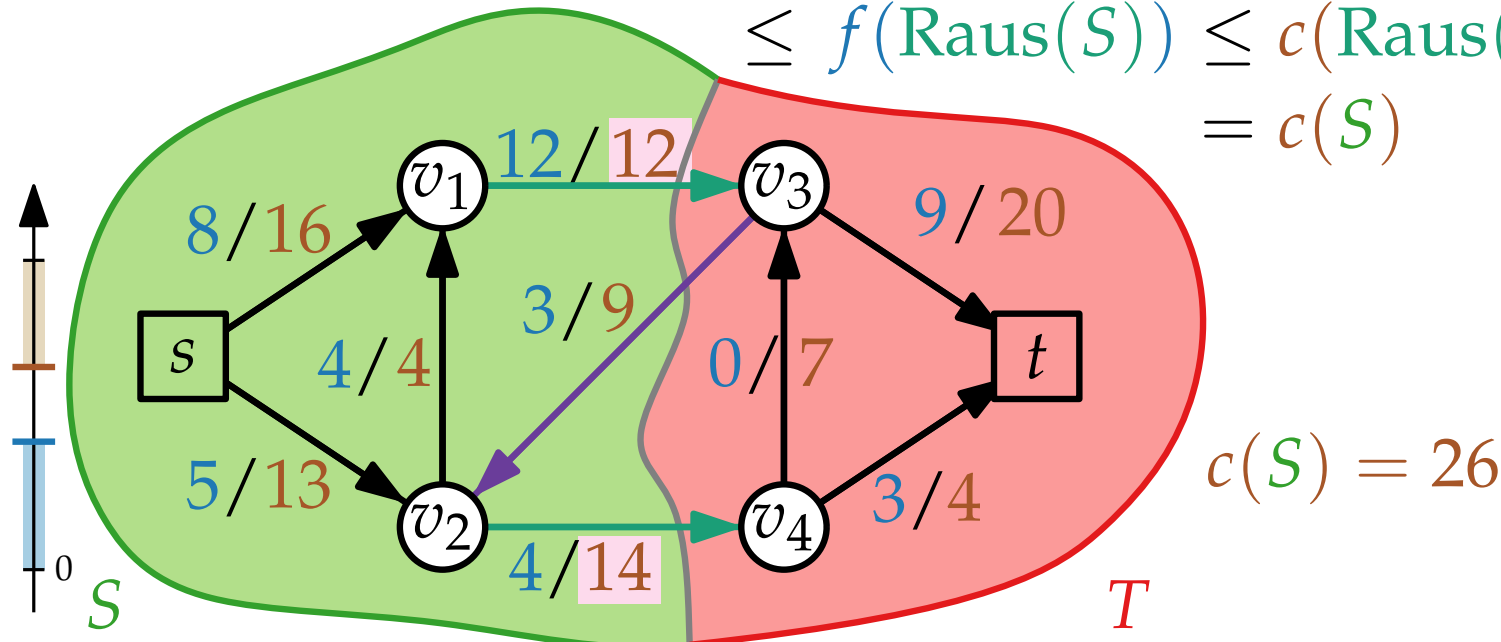
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$
 $\leq f(\text{Raus}(S)) \leq c(\text{Raus}(S)) = c(S) \quad \square$

Speziell:

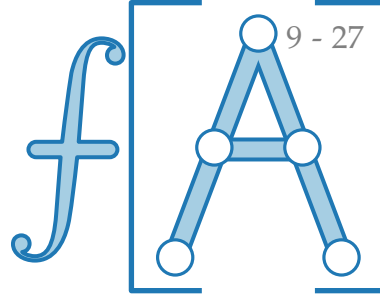
$$\min_S c(S) \geq \max_f |f|$$



Korollar.

Wenn $|f| = c(S)$
 $\Rightarrow f$ max.,

Kapazität von Schnitten



Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

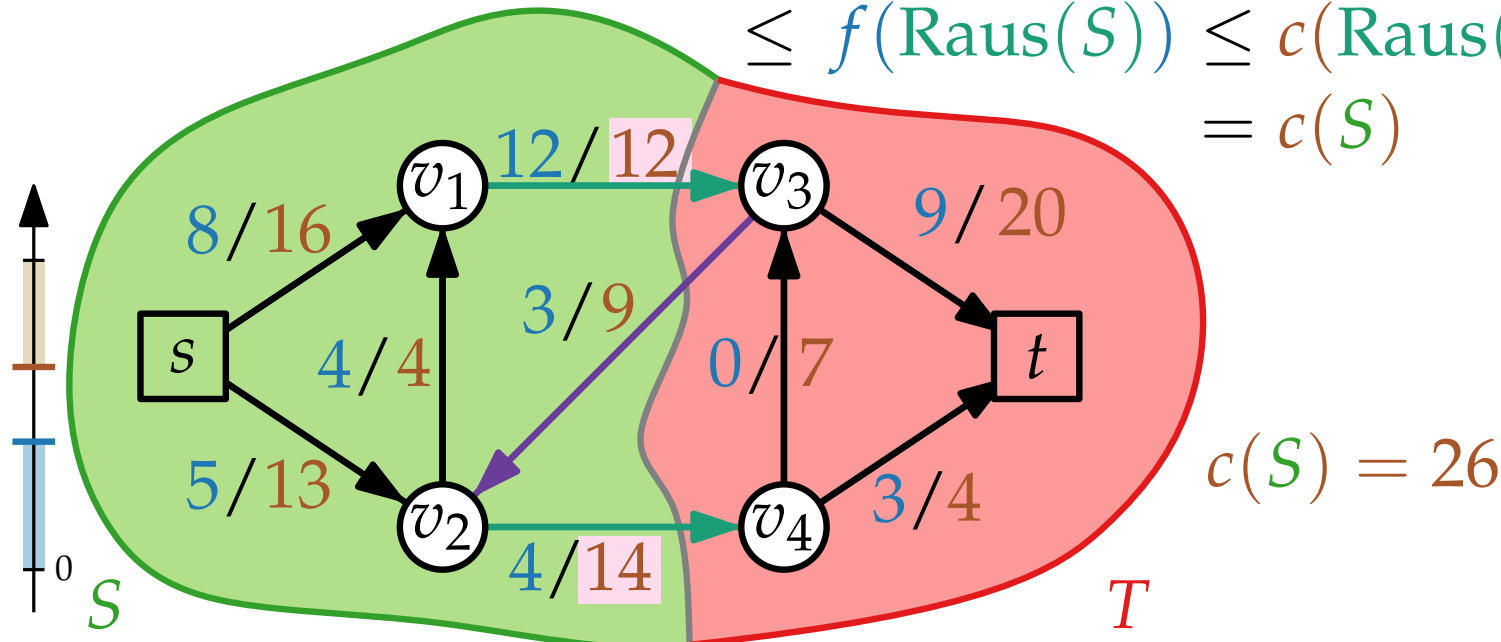
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$
 $\leq f(\text{Raus}(S)) \leq c(\text{Raus}(S)) = c(S) \quad \square$

Speziell:

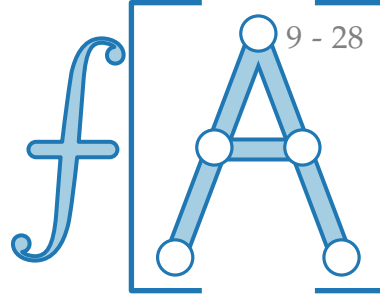
$$\min_S c(S) \geq \max_f |f|$$



Korollar.

Wenn $|f| = c(S)$
 $\Rightarrow f$ max., (S, T) min.

Kapazität von Schnitten



Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

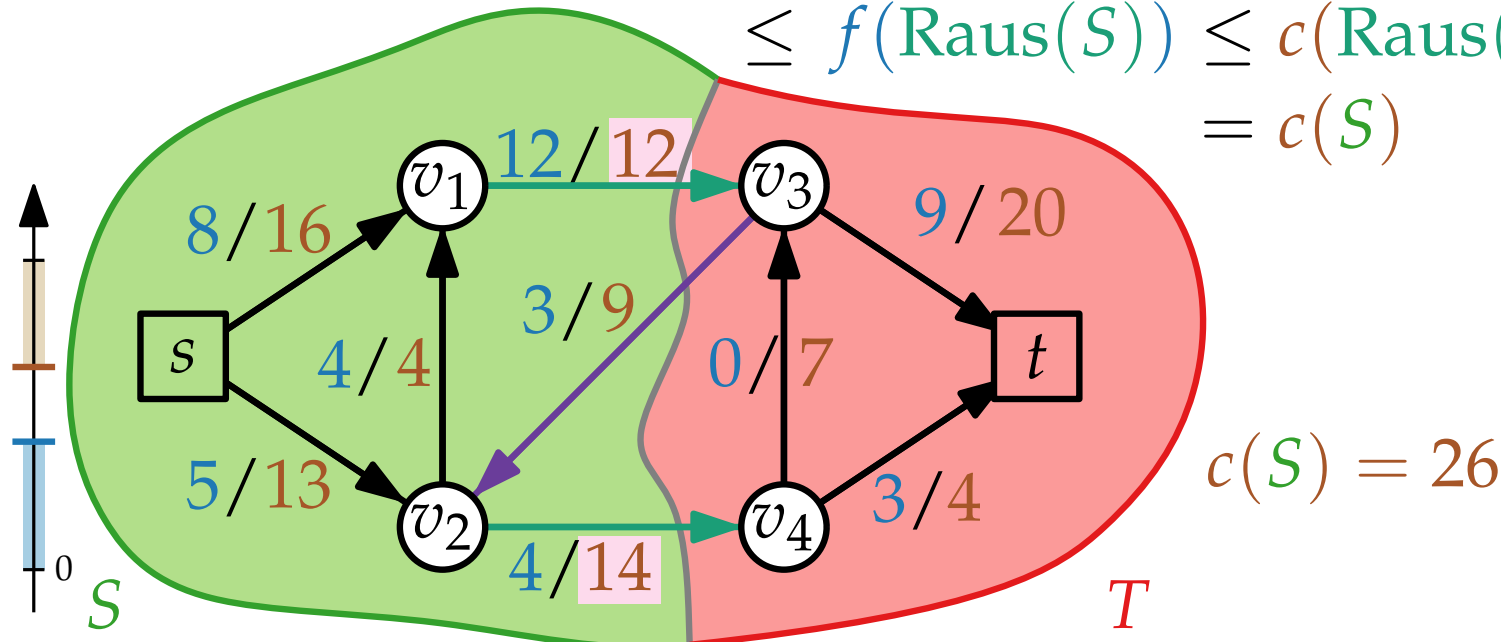
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$
 $\leq f(\text{Raus}(S)) \leq c(\text{Raus}(S)) = c(S) \quad \square$

Speziell:

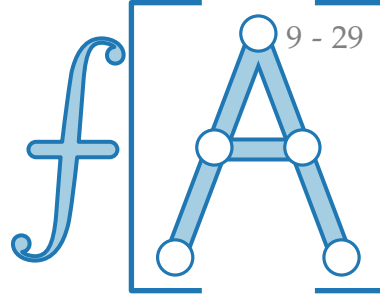
$$\min_S c(S) \geq \max_f |f|$$



Korollar.

Wenn $|f| = c(S)$
 $\Rightarrow f$ max., (S, T) min. !!

Kapazität von Schnitten



Lemma 2. G Graph, $s, t \in V$, f s - t -Fluss, (S, T) s - t -Schnitt.
Dann gilt $|f| = \text{Nettozufluss}_f(T) =: \text{Nettoabfluss}_f(S)$.

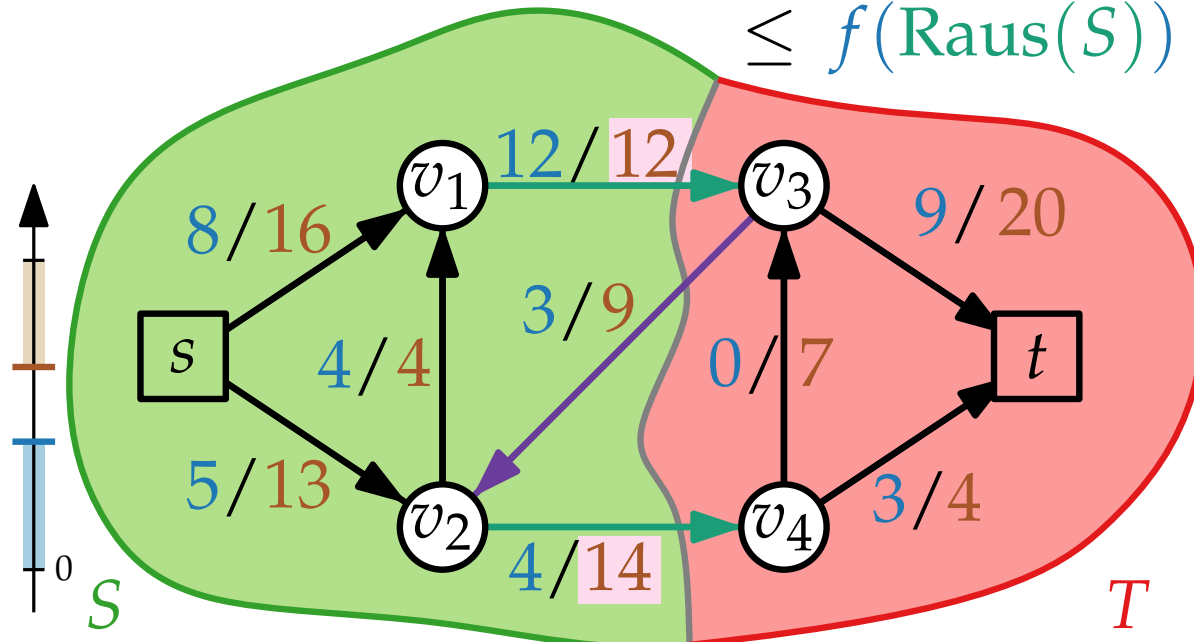
Def. G Graph mit Kap. $c: E \rightarrow \mathbb{R}_{>0}$, (S, T) s - t -Schnitt. Dann ist $c(S) := c(\text{Raus}(S))$ die **Kapazität** von (S, T) .

Lemma 3. f zuläss. s - t -Fluss, (S, T) s - t -Schnitt $\Rightarrow |f| \leq c(S)$.

Beweis. $|f| = \text{Nettoabfluss}_f(S) = f(\text{Raus}(S)) - f(\text{Rein}(S))$
 $\leq f(\text{Raus}(S)) \leq c(\text{Raus}(S)) = c(S)$ □

Speziell:

$$\min_S c(S) \geq \max_f |f|$$

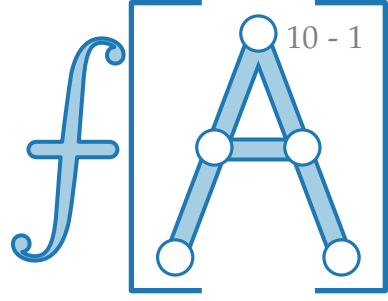


Korollar.

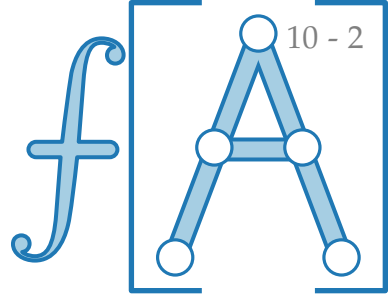
Wenn $|f| = c(S)$
 $\Rightarrow f$ max., (S, T) min. !!

Das Max-Flow-Min-Cut-Theorem

Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.
Dann sind folgende Bedingungen äquivalent:



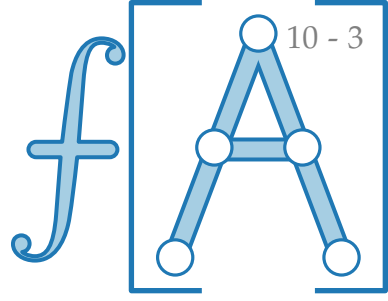
Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.
Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .

Das Max-Flow-Min-Cut-Theorem

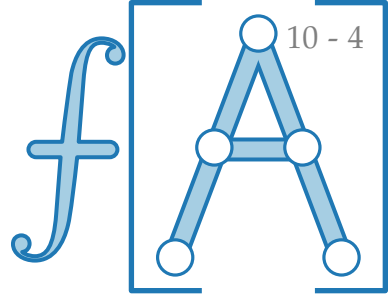


Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.

Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.

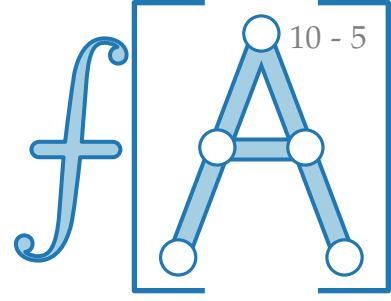
Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.
Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.

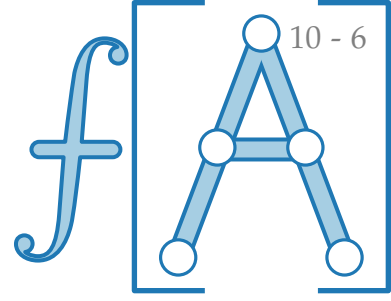
Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

Kurz:

$$\max_{f \text{ zulässiger } s\text{-}t\text{-Fluss}} |f| = \min_{(S,T) \text{ } s\text{-}t\text{-Schnitt}} c(S)$$

Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.

Dann sind folgende Bedingungen äquivalent:

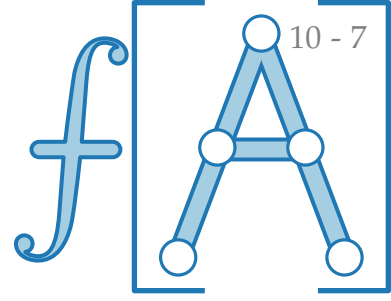
1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

Kurz:

$$\max_{f \text{ zulässiger } s\text{-}t\text{-Fluss}} |f| = \min_{(S, T) \text{ } s\text{-}t\text{-Schnitt}} c(S)$$

Beweis. (1) $\Rightarrow$ (2):

Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.

Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

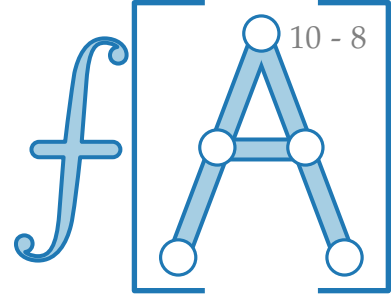
Kurz:

$$\max_{f \text{ zulässiger } s\text{-}t\text{-Fluss}} |f| = \min_{(S,T) \text{ } s\text{-}t\text{-Schnitt}} c(S)$$

Beweis. (1) $\Rightarrow$ (2):

Angenommen, G_f enthält augmentierenden Weg.

Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.

Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

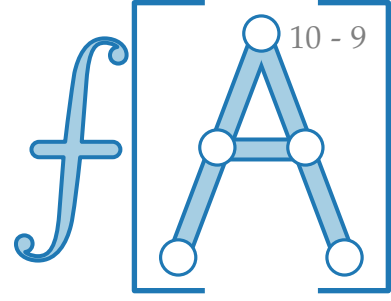
Kurz:

$$\max_{f \text{ zulässiger } s\text{-}t\text{-Fluss}} |f| = \min_{(S,T) \text{ } s\text{-}t\text{-Schnitt}} c(S)$$

Beweis. (1) $\Rightarrow$ (2):

Angenommen, G_f enthält augmentierenden Weg.
Dann könnte f erhöht werden.

Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.

Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

Kurz:

$$\max_{f \text{ zulässiger } s\text{-}t\text{-Fluss}} |f| = \min_{(S,T) \text{ } s\text{-}t\text{-Schnitt}} c(S)$$

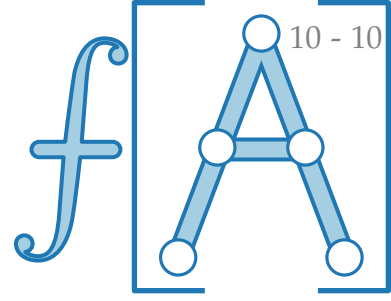
Beweis. (1) $\Rightarrow$ (2):

Angenommen, G_f enthält augmentierenden Weg.

Dann könnte f erhöht werden.

Widerspruch zu $|f|$ maximal. ⚡

Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.

Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

Kurz:

$$\max_{f \text{ zulässiger } s\text{-}t\text{-Fluss}} |f| = \min_{(S,T) \text{ } s\text{-}t\text{-Schnitt}} c(S)$$

Beweis. (1) $\Rightarrow$ (2):

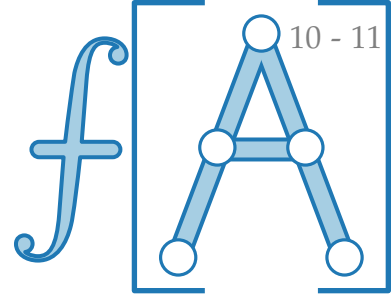
Angenommen, G_f enthält augmentierenden Weg.

Dann könnte f erhöht werden.

Widerspruch zu $|f|$ maximal. ⚡

(3) $\Rightarrow$ (1):

Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.

Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

Kurz:

$$\max_{f \text{ zulässiger } s\text{-}t\text{-Fluss}} |f| = \min_{(S, T) \text{ } s\text{-}t\text{-Schnitt}} c(S)$$

Beweis. (1) $\Rightarrow$ (2):

Angenommen, G_f enthält augmentierenden Weg.

Dann könnte f erhöht werden.

Widerspruch zu $|f|$ maximal. ⚡

(3) $\Rightarrow$ (1):

Erinnerung:

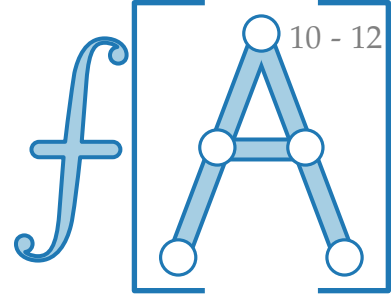
Korollar.

$$|f| = c(S)$$



f maximal

Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.

Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

Kurz:

$$\max_{f \text{ zulässiger } s\text{-}t\text{-Fluss}} |f| = \min_{(S, T) \text{ } s\text{-}t\text{-Schnitt}} c(S)$$

Beweis. (1) $\Rightarrow$ (2):

Angenommen, G_f enthält augmentierenden Weg.

Dann könnte f erhöht werden.

Widerspruch zu $|f|$ maximal. ⚡

(3) $\Rightarrow$ (1): Korollar $\Rightarrow$

Erinnerung:

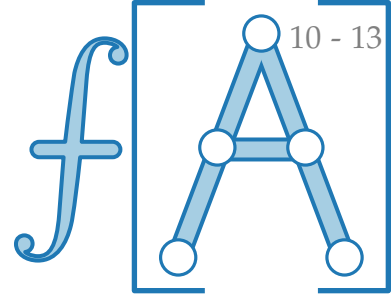
Korollar.

$$|f| = c(S)$$



f maximal

Das Max-Flow-Min-Cut-Theorem



Satz. Sei f ein zulässiger s - t -Fluss in einem gerichteten Graphen G mit Kapazitäten $c: E \rightarrow \mathbb{R}_{>0}$.

Dann sind folgende Bedingungen äquivalent:

1. f ist ein maximaler Fluss in G .
2. G_f enthält keine augmentierenden Wege.
3. Es gibt einen s - t -Schnitt (S, T) mit $|f| = c(S)$.

Kurz:

$$\max_{f \text{ zulässiger } s\text{-}t\text{-Fluss}} |f| = \min_{(S,T) \text{ } s\text{-}t\text{-Schnitt}} c(S)$$

Beweis. (1) $\Rightarrow$ (2):

Angenommen, G_f enthält augmentierenden Weg.

Dann könnte f erhöht werden.

Widerspruch zu $|f|$ maximal. ⚡

(3) $\Rightarrow$ (1): Korollar $\Rightarrow |f|$ maximal.

Erinnerung:

Korollar.

$$|f| = c(S)$$



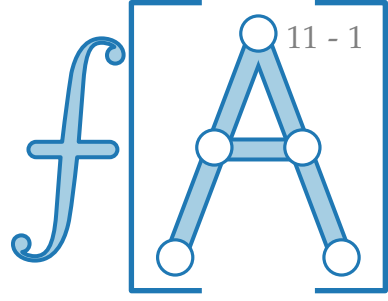
f maximal

Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



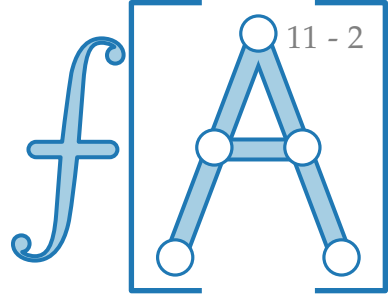
Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.

(2) $\Rightarrow$ (3):



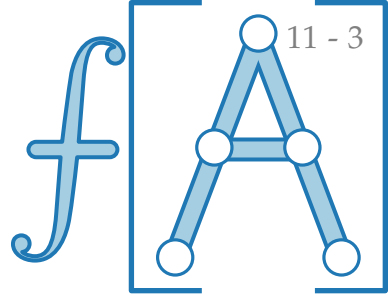
Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.

(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .



Zu zeigen:

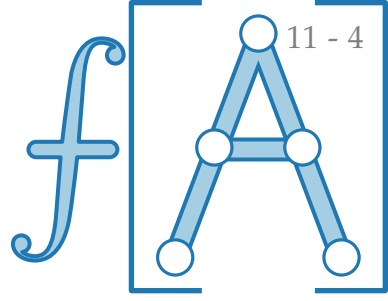
2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.

(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$



Zu zeigen:

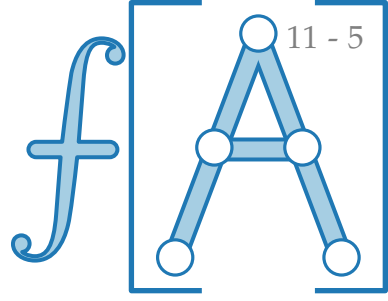
2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.

(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

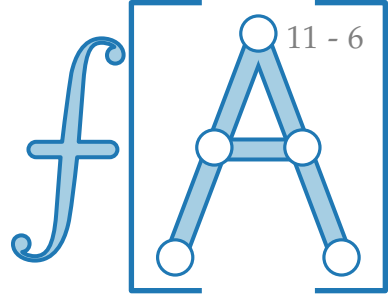


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

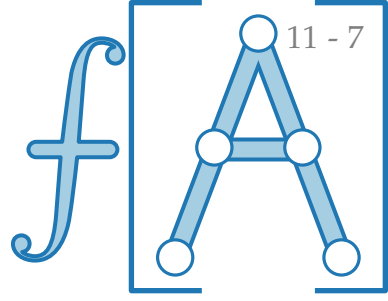
$\Rightarrow \left\{ \right. \left. \right\}$ ist s - t -Schnitt.

Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

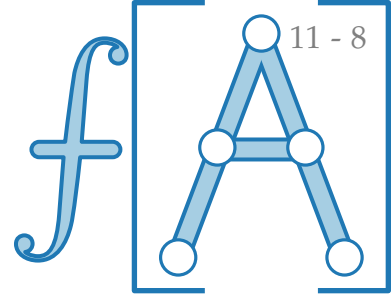
$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.

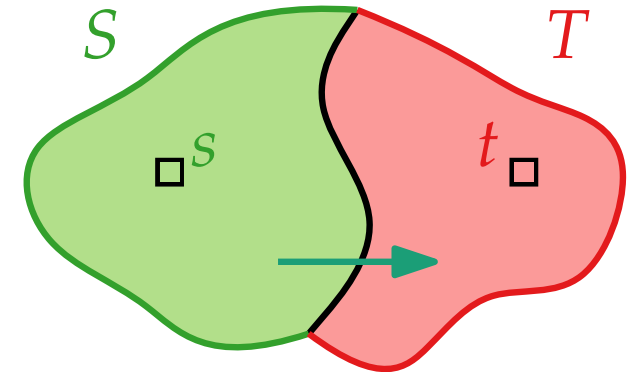


(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

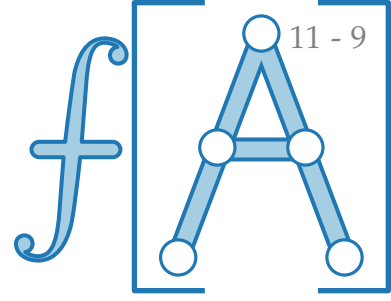


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



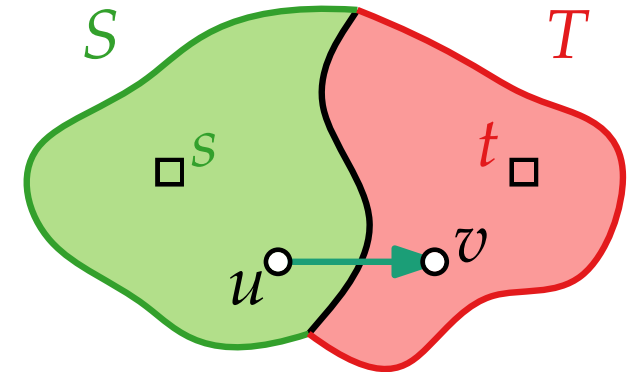
(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

$\Rightarrow$

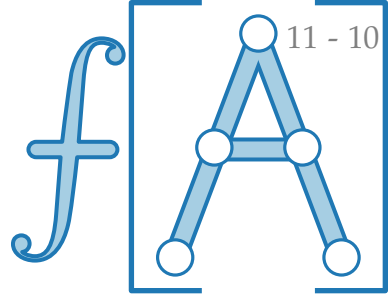


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



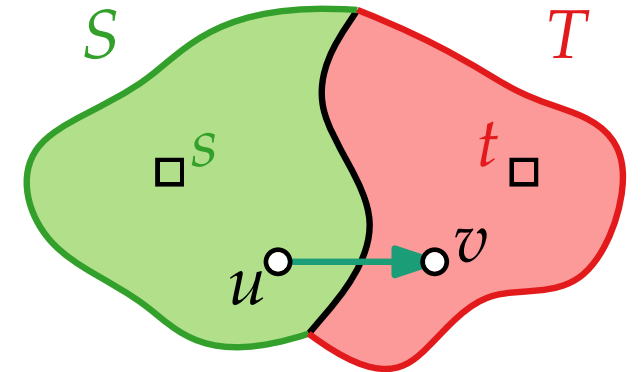
(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

$\Rightarrow f(e) = c(e)$

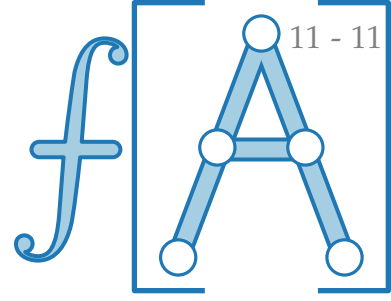


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



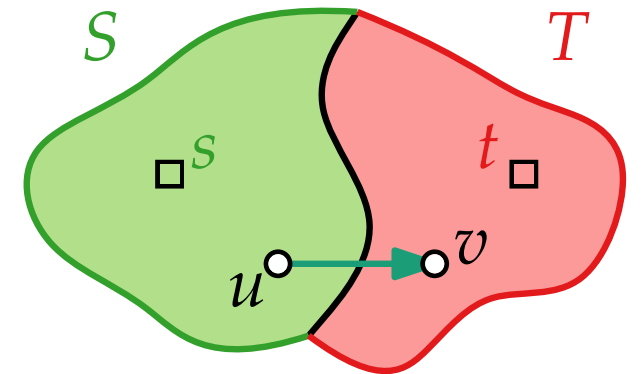
(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

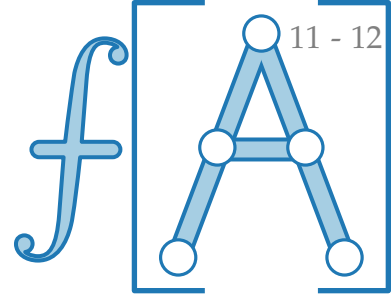


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

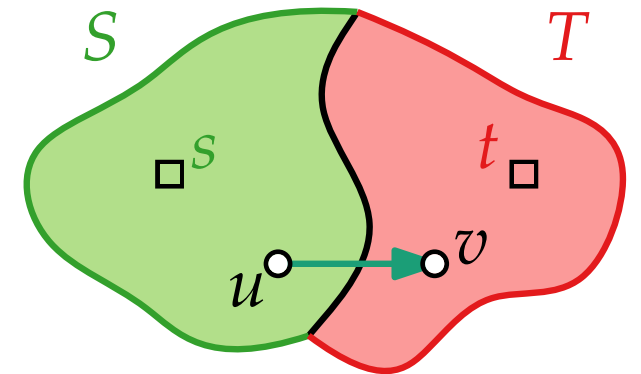
$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow}$

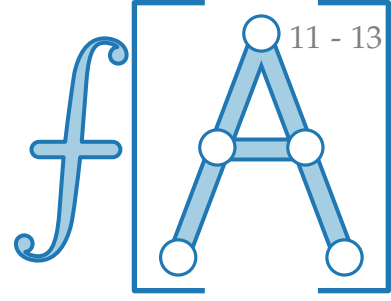


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

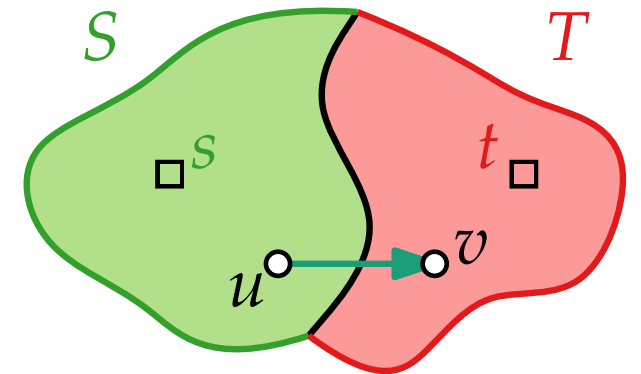
$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Raus}(S)) = c(\mathbf{Raus}(S))$

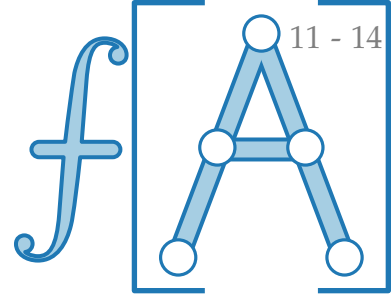


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

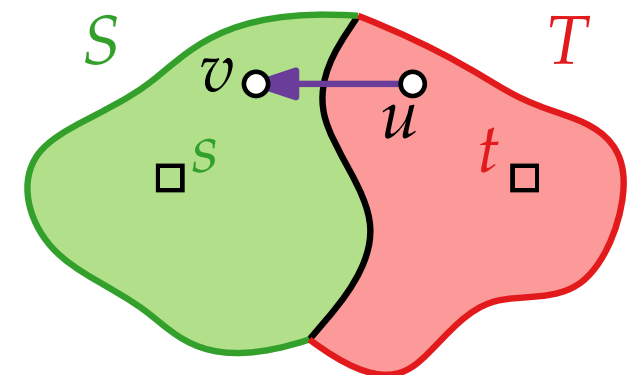
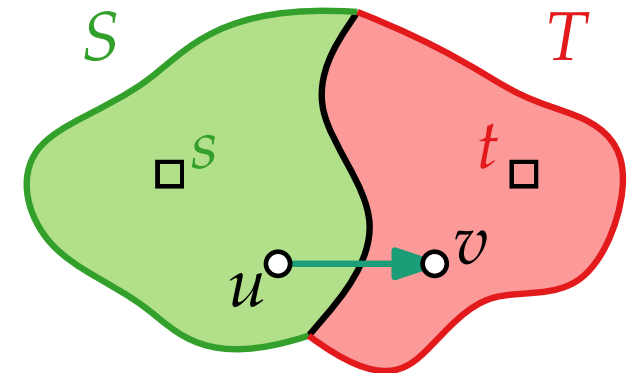
$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Raus}(S)) = c(\mathbf{Raus}(S))$

Nun sei $e = uv \in \mathbf{Rein}(S)$

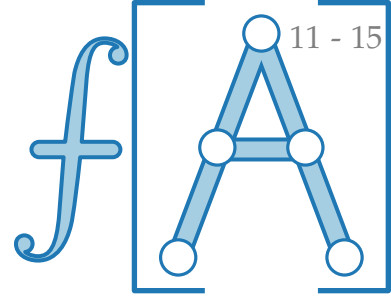


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

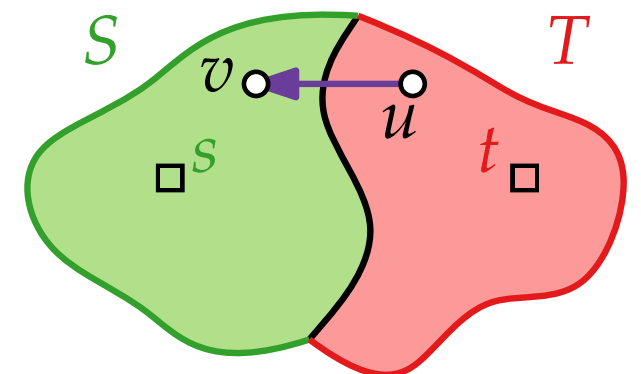
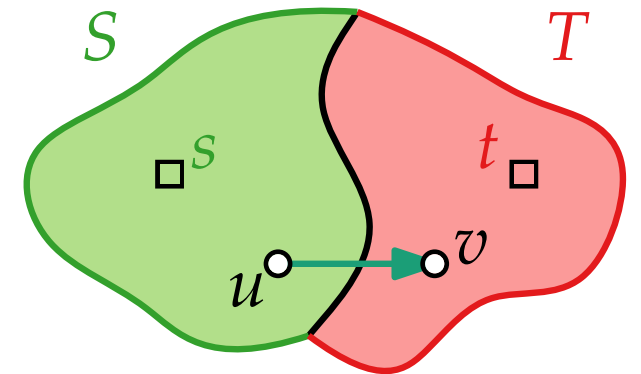
$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Raus}(S)) = c(\mathbf{Raus}(S))$

Nun sei $e = uv \in \mathbf{Rein}(S) \Rightarrow f(e) = 0$

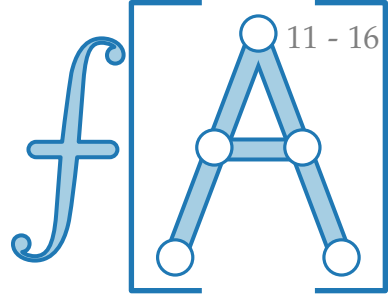


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

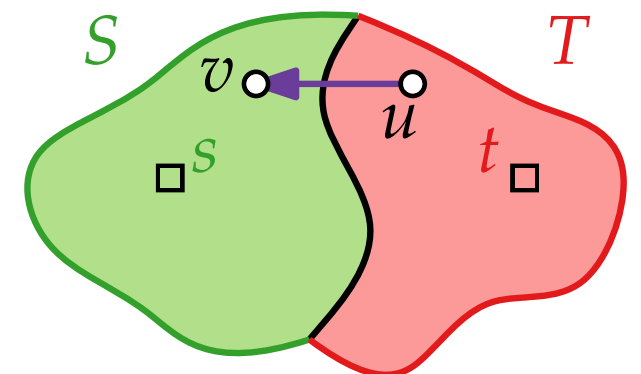
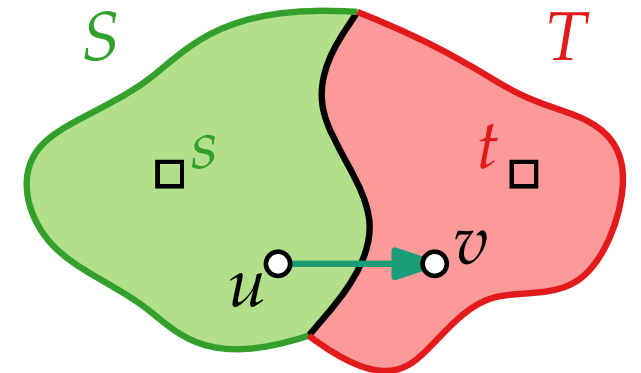
Sei $e = uv \in \mathbf{Raus}(S)$.

$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Raus}(S)) = c(\mathbf{Raus}(S))$

Nun sei $e = uv \in \mathbf{Rein}(S) \Rightarrow f(e) = 0$

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Rein}(S)) = 0$

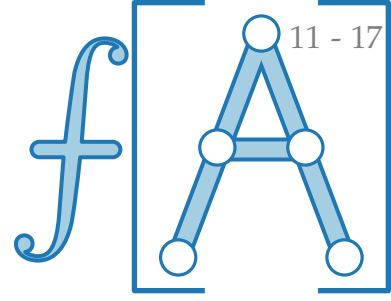


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

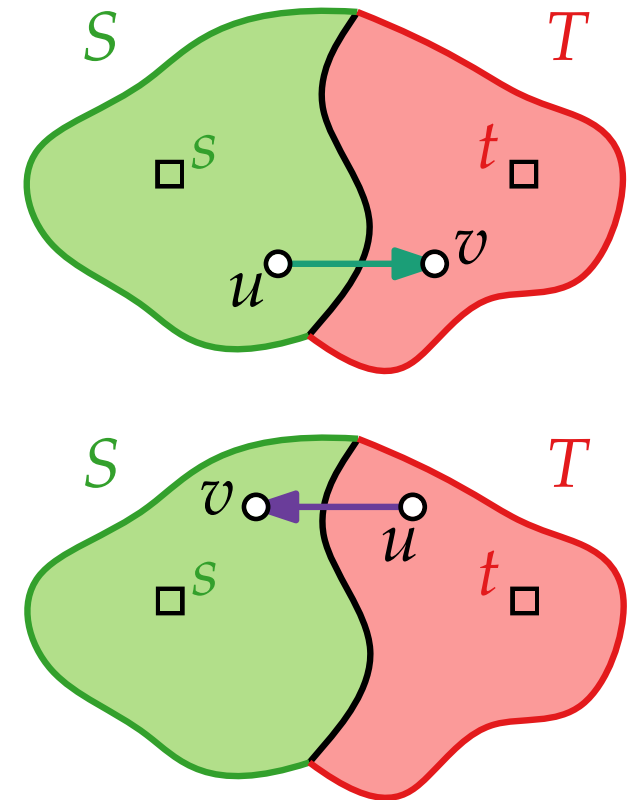
$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Raus}(S)) = c(\mathbf{Raus}(S))$

Nun sei $e = uv \in \mathbf{Rein}(S) \Rightarrow f(e) = 0$

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Rein}(S)) = 0$

Also: $c(S) \stackrel{\text{Def.}}{=}$

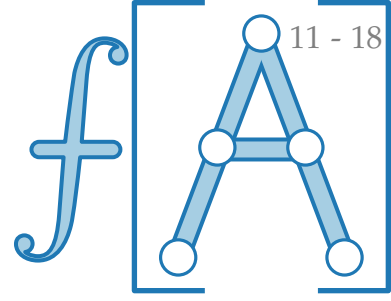


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

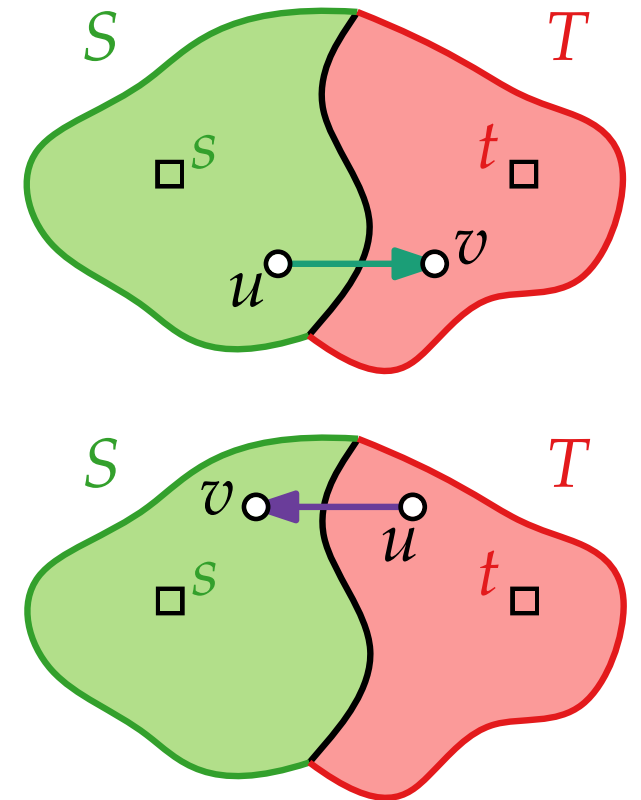
$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Raus}(S)) = c(\mathbf{Raus}(S))$

Nun sei $e = uv \in \mathbf{Rein}(S) \Rightarrow f(e) = 0$

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Rein}(S)) = 0$

Also: $c(S) \stackrel{\text{Def.}}{=} c(\mathbf{Raus}(S)) =$

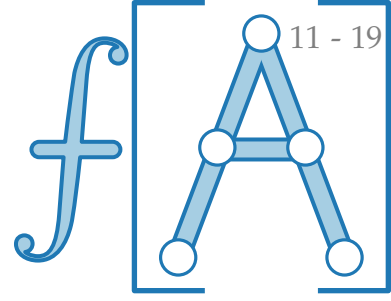


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

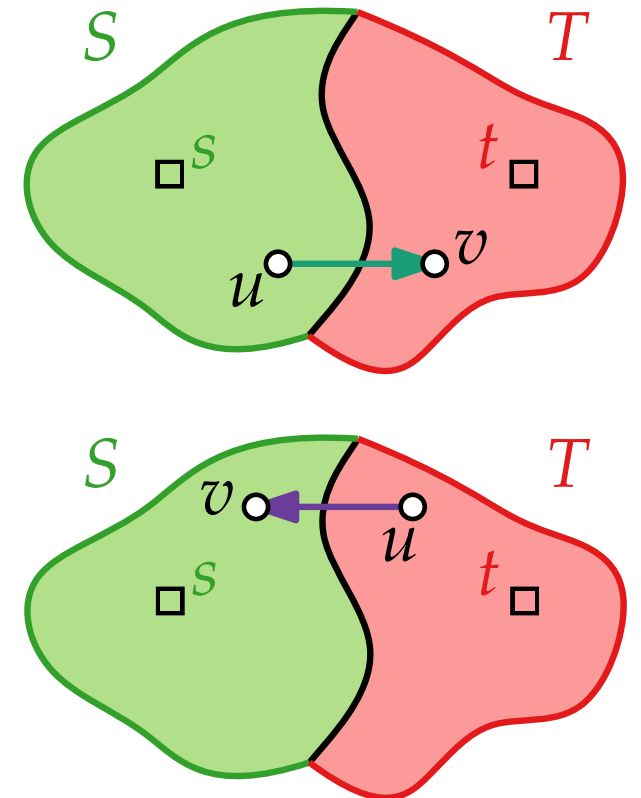
$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Raus}(S)) = c(\mathbf{Raus}(S))$

Nun sei $e = uv \in \mathbf{Rein}(S) \Rightarrow f(e) = 0$

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Rein}(S)) = 0$

Also: $c(S) \stackrel{\text{Def.}}{=} c(\mathbf{Raus}(S)) =$

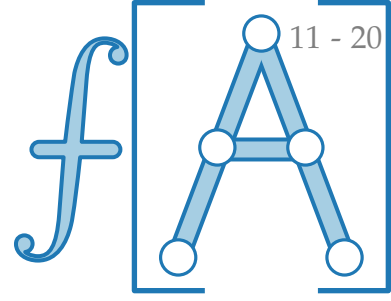


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

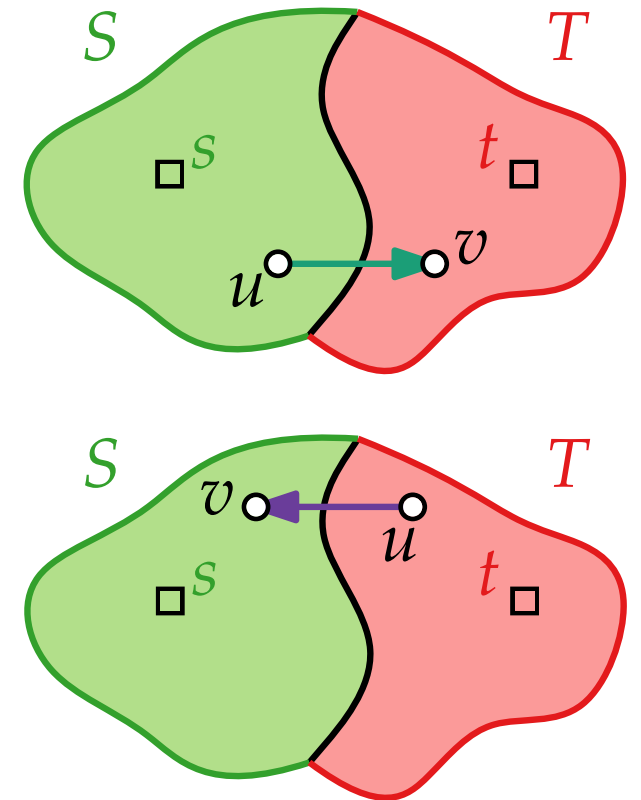
$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Raus}(S)) = c(\mathbf{Raus}(S))$

Nun sei $e = uv \in \mathbf{Rein}(S) \Rightarrow f(e) = 0$

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Rein}(S)) = 0$

Also: $c(S) \stackrel{\text{Def.}}{=} c(\mathbf{Raus}(S)) = f(\mathbf{Raus}(S)) - f(\mathbf{Rein}(S))$

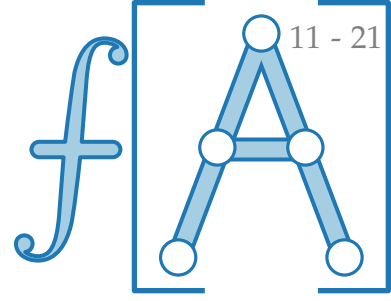


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

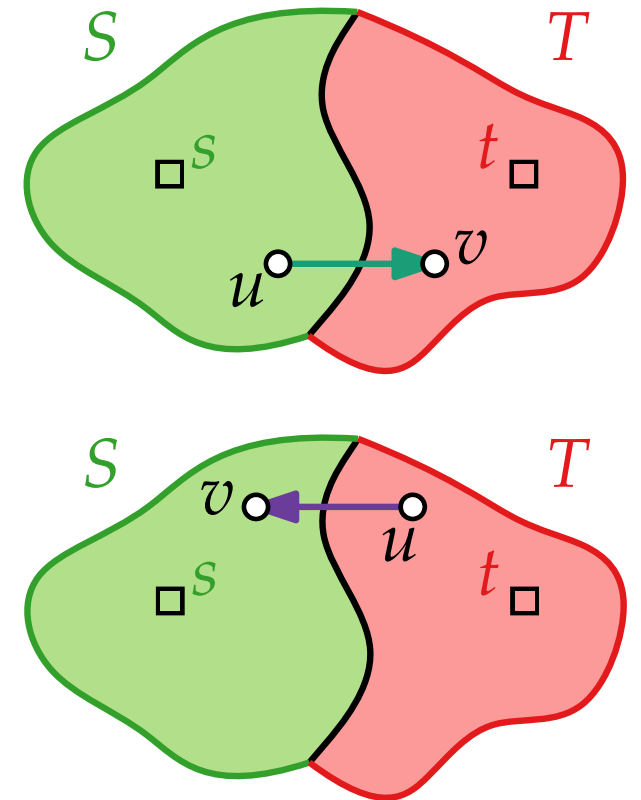
$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Raus}(S)) = c(\mathbf{Raus}(S))$

Nun sei $e = uv \in \mathbf{Rein}(S) \Rightarrow f(e) = 0$

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Rein}(S)) = 0$

Also: $c(S) \stackrel{\text{Def.}}{=} c(\mathbf{Raus}(S)) = f(\mathbf{Raus}(S)) - f(\mathbf{Rein}(S))$
 $= \mathbf{Nettoabfluss}_f(S) \stackrel{\text{Lem. 2}}{=}$

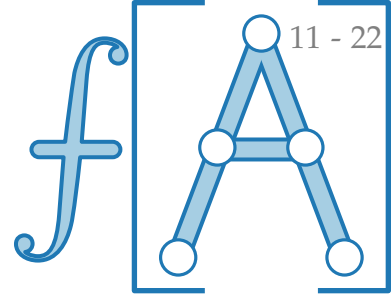


Zu zeigen:

2. G_f enthält keine augmentierenden Wege.



3. Es gibt einen Schnitt (S, T) mit $|f| = c(S)$.



(2) $\Rightarrow$ (3): Es gibt keinen augmentierenden Weg in G_f .

$\Rightarrow$ In G_f ist t von s aus nicht erreichbar.

$\Rightarrow \left\{ \begin{array}{l} S = \{v \mid v \text{ von } s \text{ erreichbar}\} \\ T = \{v \mid v \text{ von } s \text{ nicht erreichbar}\} \end{array} \right\}$ ist s - t -Schnitt.

Sei $e = uv \in \mathbf{Raus}(S)$.

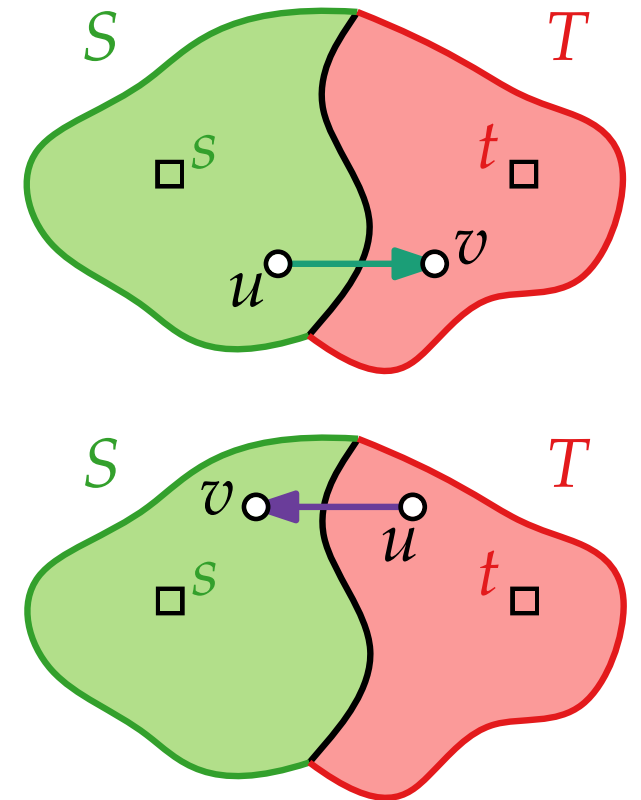
$\Rightarrow f(e) = c(e)$, sonst wäre v von s in G_f erreichbar.

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Raus}(S)) = c(\mathbf{Raus}(S))$

Nun sei $e = uv \in \mathbf{Rein}(S) \Rightarrow f(e) = 0$

$\stackrel{\Sigma}{\Rightarrow} f(\mathbf{Rein}(S)) = 0$

Also: $c(S) \stackrel{\text{Def.}}{=} c(\mathbf{Raus}(S)) = f(\mathbf{Raus}(S)) - f(\mathbf{Rein}(S))$
 $= \mathbf{Nettoabfluss}_f(S) \stackrel{\text{Lem. 2}}{=} |f| \quad \square$

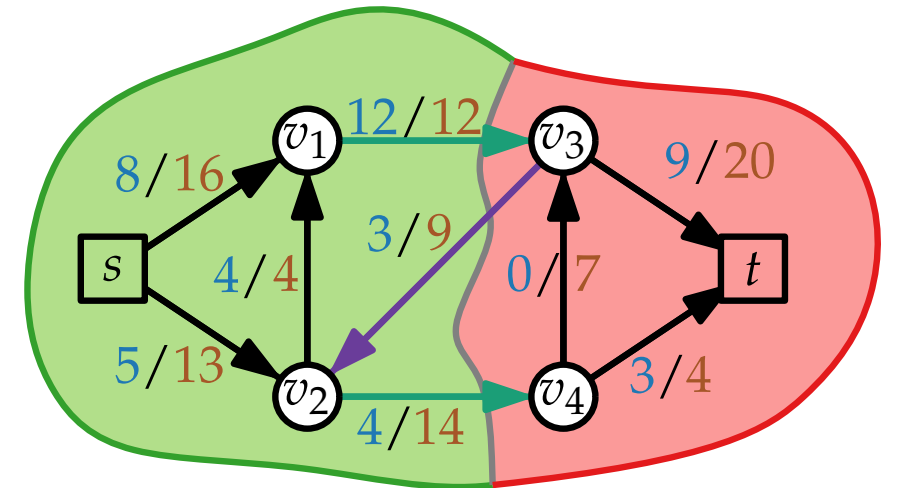
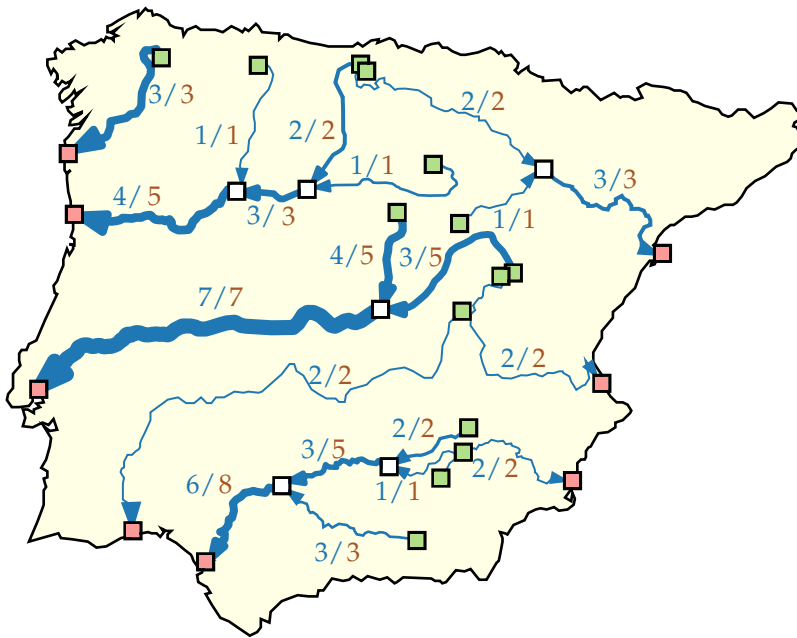




Fortgeschrittene Algorithmen

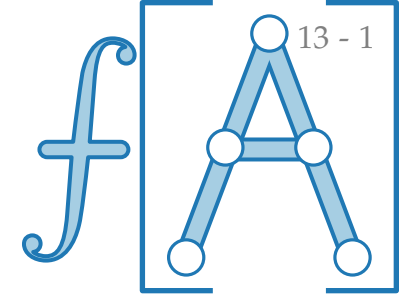
8. Vorlesung Graphalgorithmen II: Flussnetzwerke

Teil III: Algorithmen



FORDFULKERSON '56

FORDFULKERSON($G = (V, E); s; t; c$)



Lester R. Ford Jr.
*1927 Houston, TX
†2017



Delbert R. Fulkerson
*1924 Tamm, IL
†1976 Ithaca, NY

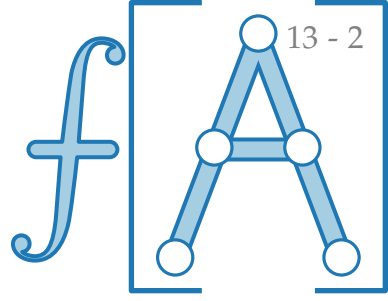


FORDFULKERSON '56

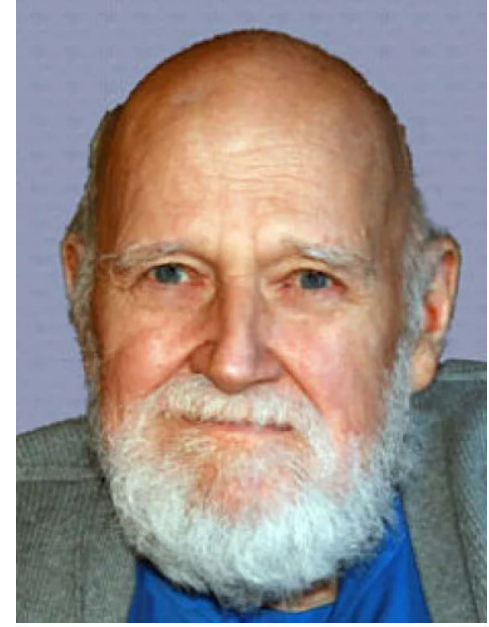
```
FORDFULKERSON( $G = (V, E); s; t; c$ )
```

```
  foreach  $(v, v') \in E$  do  
     $f(v, v') = 0$ 
```

```
return  $f$ 
```



Lester R. Ford Jr.
*1927 Houston, TX
+2017



Delbert R. Fulkerson
*1924 Tamm, IL
+1976 Ithaca, NY



FORDFULKERSON '56

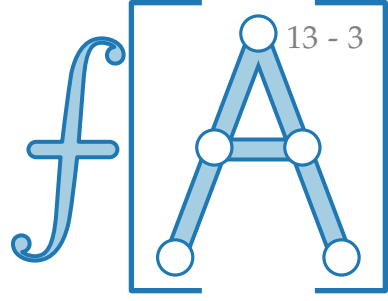
```
FORDFULKERSON( $G = (V, E); s; t; c$ )
```

```
  foreach  $(v, v') \in E$  do
```

```
     $f(v, v') = 0$ 
```

```
return  $f$ 
```

} Initialisierung mit Null-Fluss



FORDFULKERSON '56

```
FORDFULKERSON( $G = (V, E); s; t; c$ )
```

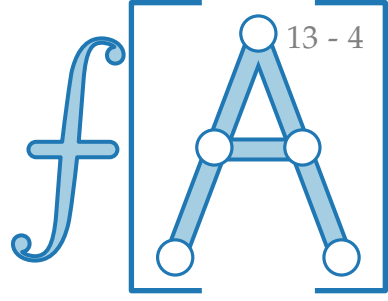
```
  foreach  $(v, v') \in E$  do
```

```
     $f(v, v') = 0$ 
```

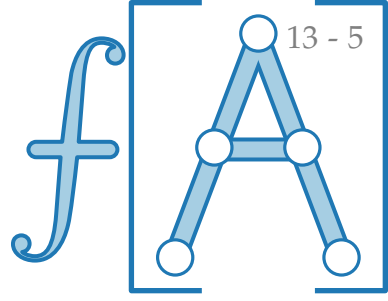
} Initialisierung mit Null-Fluss

```
  return  $f$ 
```

} Größter Fluss



FORDFULKERSON '56



```
FORDFULKERSON( $G = (V, E); s; t; c$ )
```

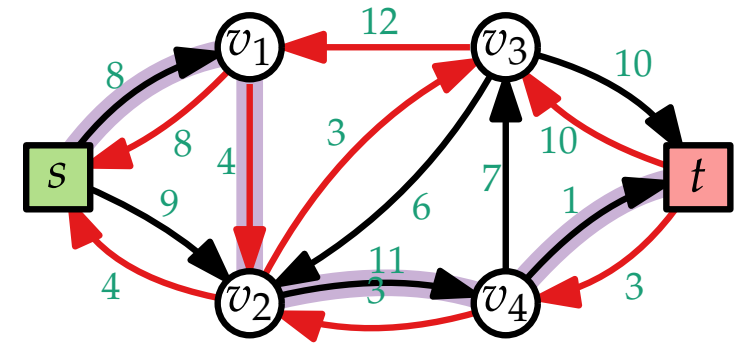
```
  foreach  $(v, v') \in E$  do
```

```
     $f(v, v') = 0$ 
```

```
  while  $G_f$  enthält  $s$ - $t$ -Pfad  $W$  do
```

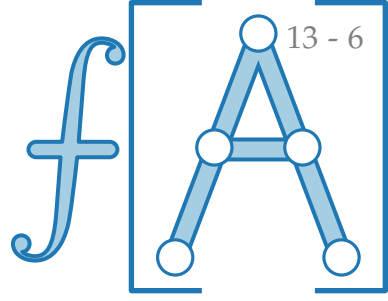
```
  return  $f$ 
```

} Initialisierung mit Null-Fluss



} Größter Fluss

FORDFULKERSON '56



FORDFULKERSON($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ **do**

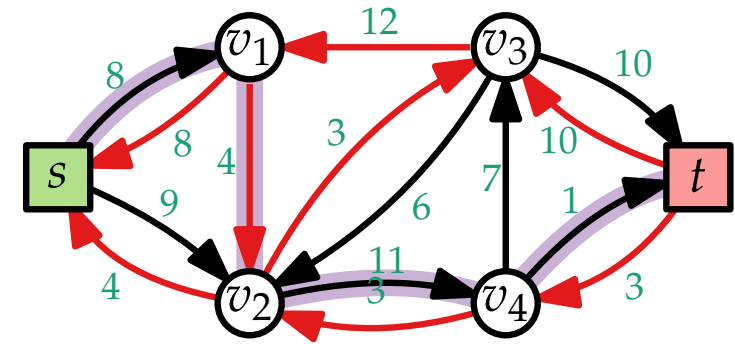
$f(v, v') = 0$

while G_f enthält s - t -Pfad W **do**

$\Delta_W = \min_{(v, v') \in W} C(v, v')$

return f

} Initialisierung mit Null-Fluss



} Größter Fluss

FORDFULKERSON '56

FORDFULKERSON($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ **do**

$f(v, v') = 0$

while G_f enthält s - t -Pfad W **do**

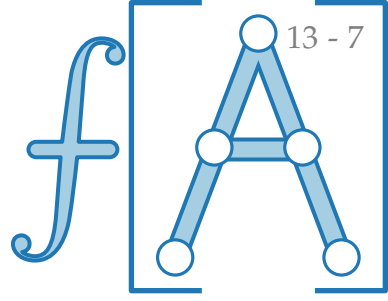
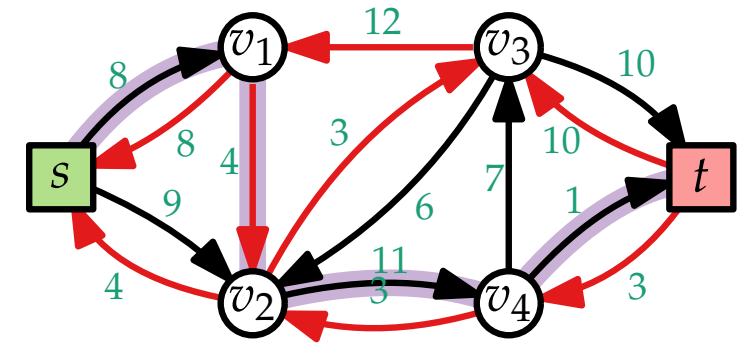
$\Delta_W = \min_{(v, v') \in W} C(v, v')$

return f

} Initialisierung mit Null-Fluss

} Kapazität von W

} Größter Fluss



FORDFULKERSON '56

FORDFULKERSON($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ **do**

$f(v, v') = 0$

while G_f enthält s - t -Pfad W **do**

$\Delta_W = \min_{(v, v') \in W} C(v, v')$

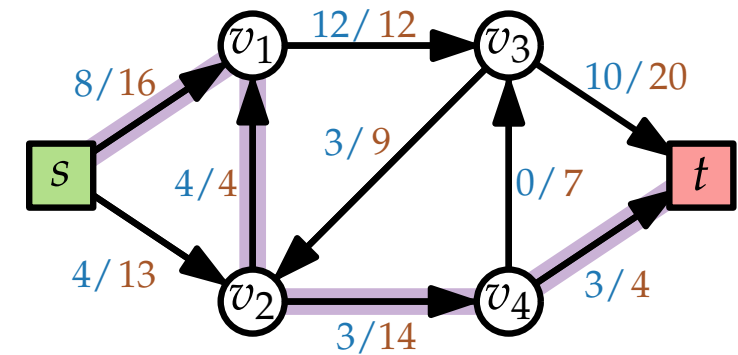
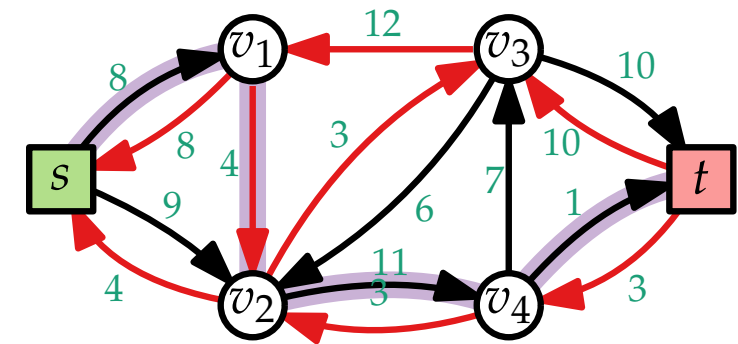
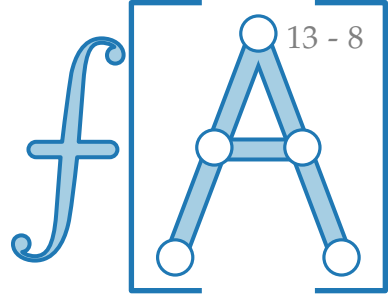
foreach $(v, v') \in W$ **do**

return f

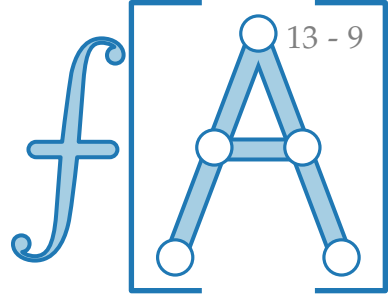
} Initialisierung mit Null-Fluss

} Kapazität von W

} Größter Fluss



FORDFULKERSON '56



FORDFULKERSON($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ **do**

$f(v, v') = 0$

while G_f enthält s - t -Pfad W **do**

$\Delta_W = \min_{(v, v') \in W} C(v, v')$

foreach $(v, v') \in W$ **do**

if $(v, v') \in E$ **then**

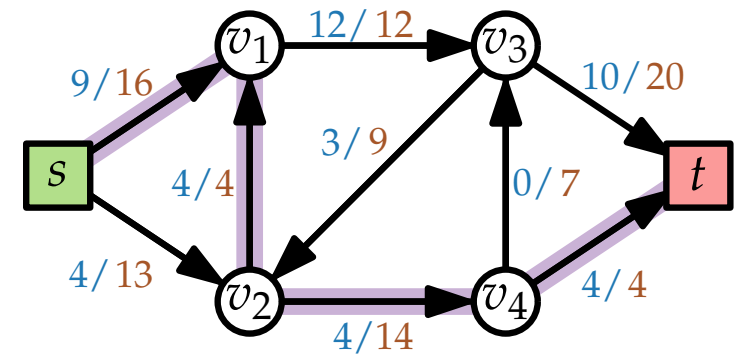
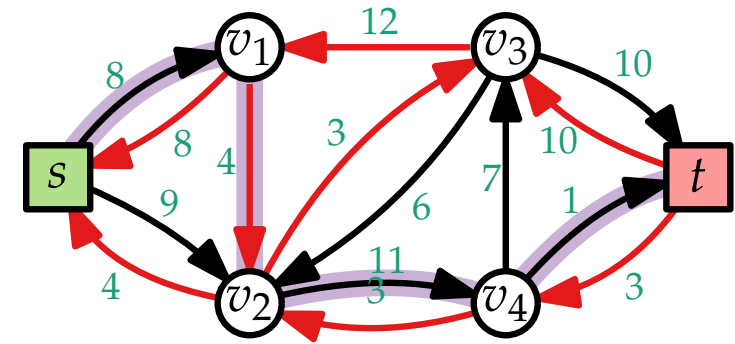
$f(v, v') = f(v, v') + \Delta_W$

return f

} Initialisierung mit Null-Fluss

} Kapazität von W

} Größter Fluss



FORDFULKERSON '56

FORDFULKERSON($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ **do**

$f(v, v') = 0$

while G_f enthält s - t -Pfad W **do**

$\Delta_W = \min_{(v, v') \in W} C(v, v')$

foreach $(v, v') \in W$ **do**

if $(v, v') \in E$ **then**

$f(v, v') = f(v, v') + \Delta_W$

else

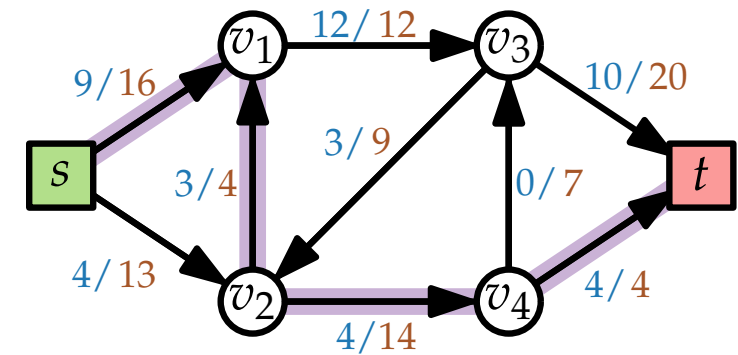
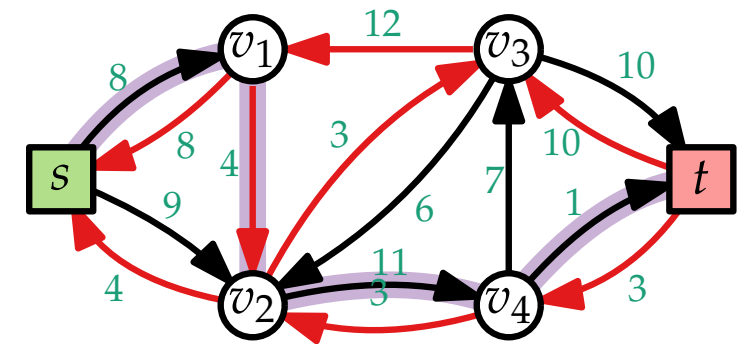
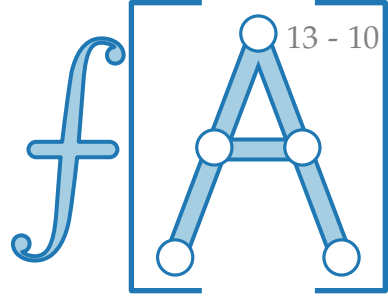
$f(v, v') = f(v, v') - \Delta_W$

return f

} Initialisierung mit Null-Fluss

} Kapazität von W

} Größter Fluss



FORDFULKERSON '56

FORDFULKERSON($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ **do**

$f(v, v') = 0$

while G_f enthält s - t -Pfad W **do**

$\Delta_W = \min_{(v, v') \in W} C(v, v')$

foreach $(v, v') \in W$ **do**

if $(v, v') \in E$ **then**

$f(v, v') = f(v, v') + \Delta_W$

else

$f(v, v') = f(v, v') - \Delta_W$

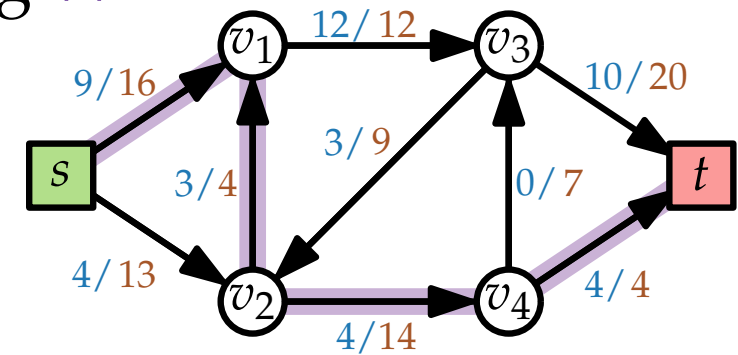
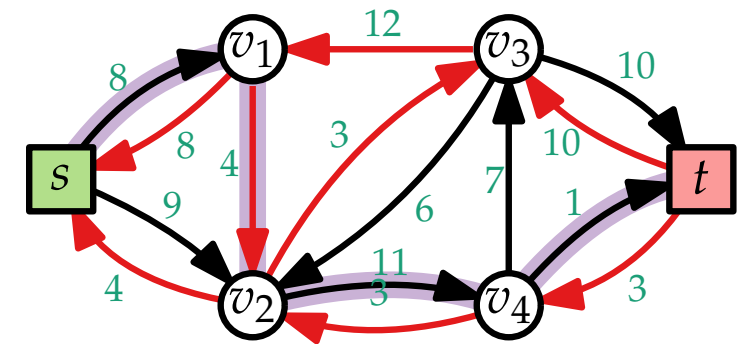
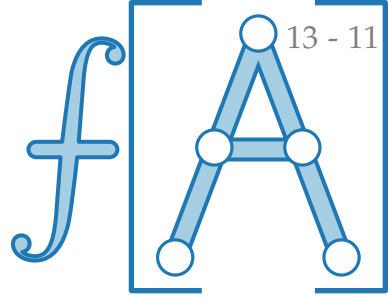
return f

} Initialisierung mit Null-Fluss

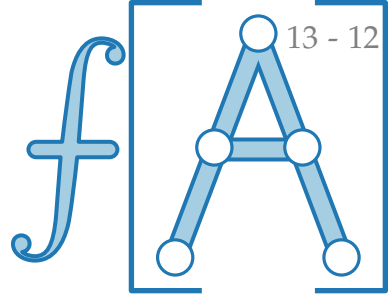
} Kapazität von W

} Erhöhe Fluss entlang W

} Größter Fluss



FORDFULKERSON '56



FORDFULKERSON($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ **do**

$f(v, v') = 0$

while G_f enthält s - t -Pfad W **do**

$\Delta_W = \min_{(v, v') \in W} C(v, v')$

foreach $(v, v') \in W$ **do**

if $(v, v') \in E$ **then**

$f(v, v') = f(v, v') + \Delta_W$

else

$f(v, v') = f(v, v') - \Delta_W$

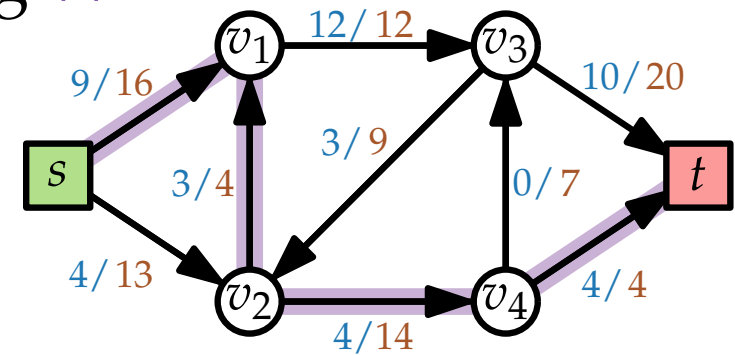
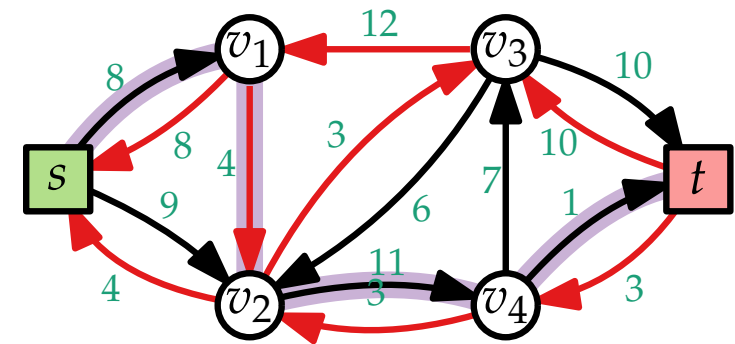
return f

} Initialisierung mit Null-Fluss

} Kapazität von W

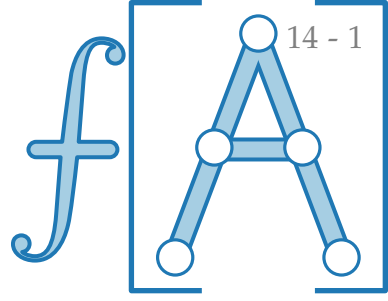
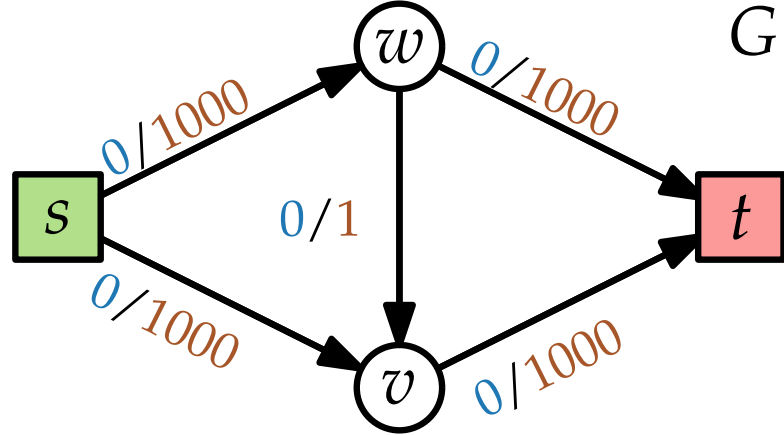
} Erhöhe Fluss entlang W

} Größter Fluss

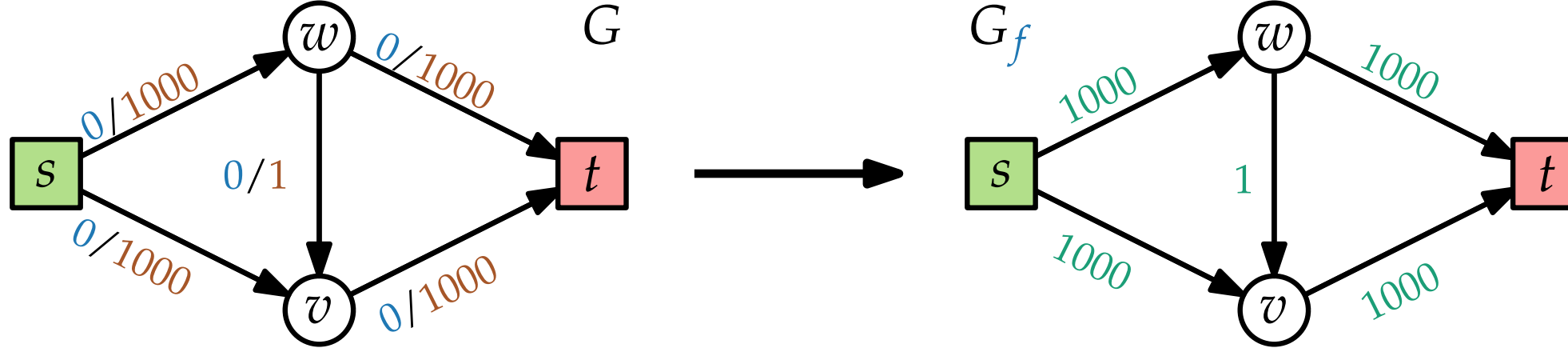


FORDFULKERSON findet einen größten s - t -Fluss in $\mathcal{O}(|f^*| \cdot m)$ Zeit.

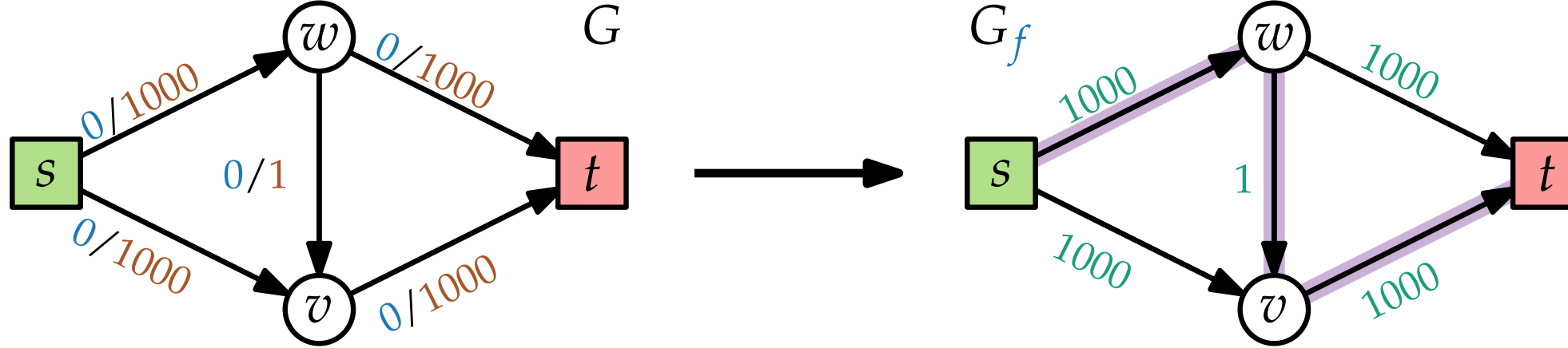
FORDFULKERSON – Beispiel



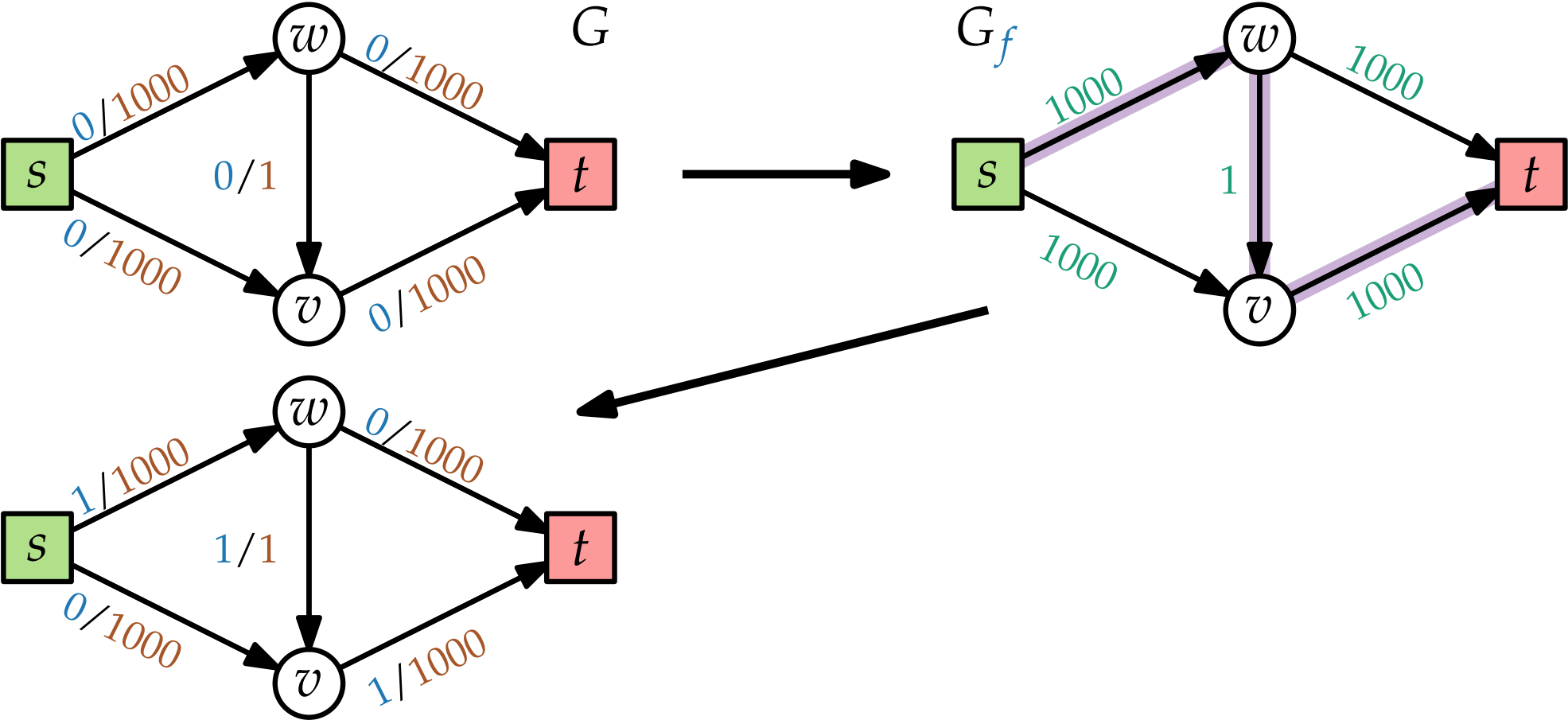
FORDFULKERSON – Beispiel



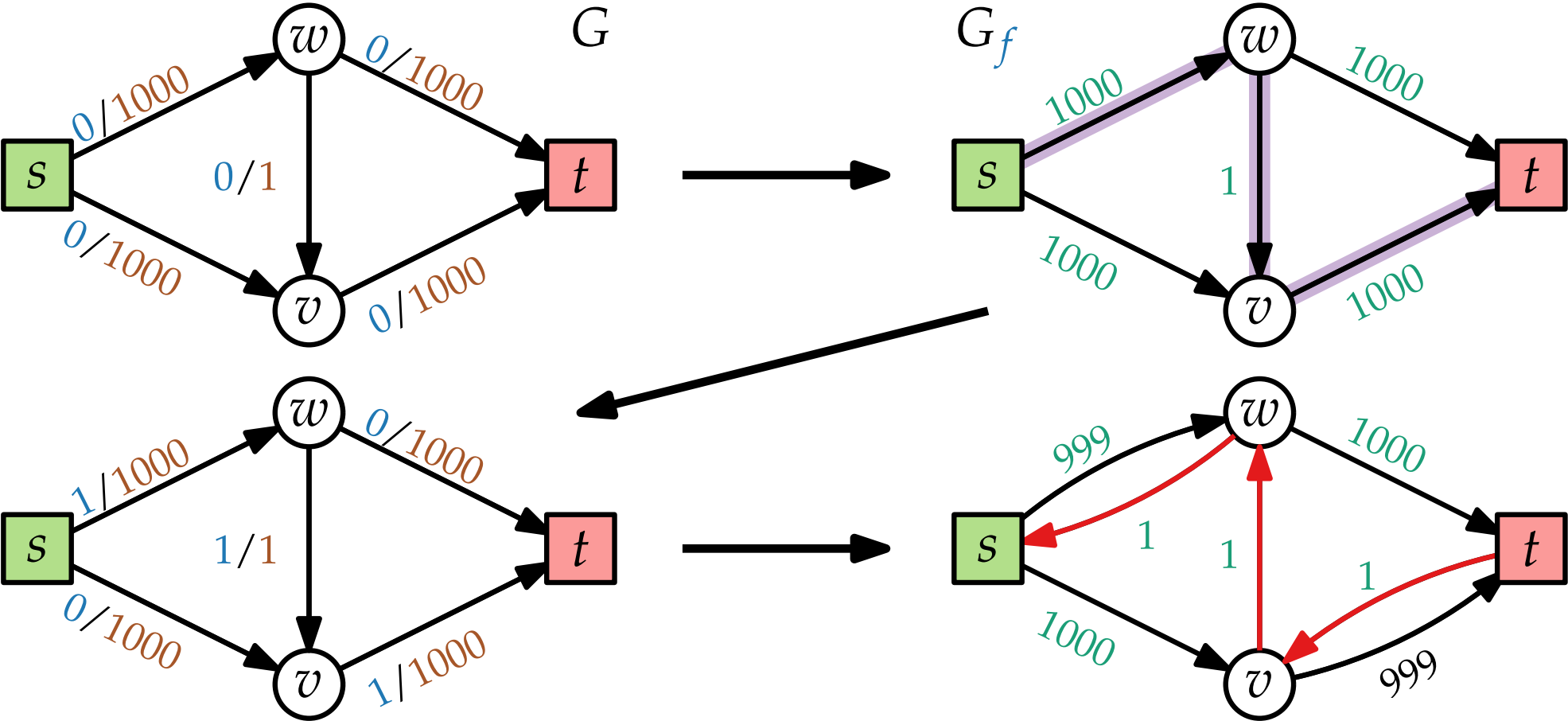
FORDFULKERSON – Beispiel



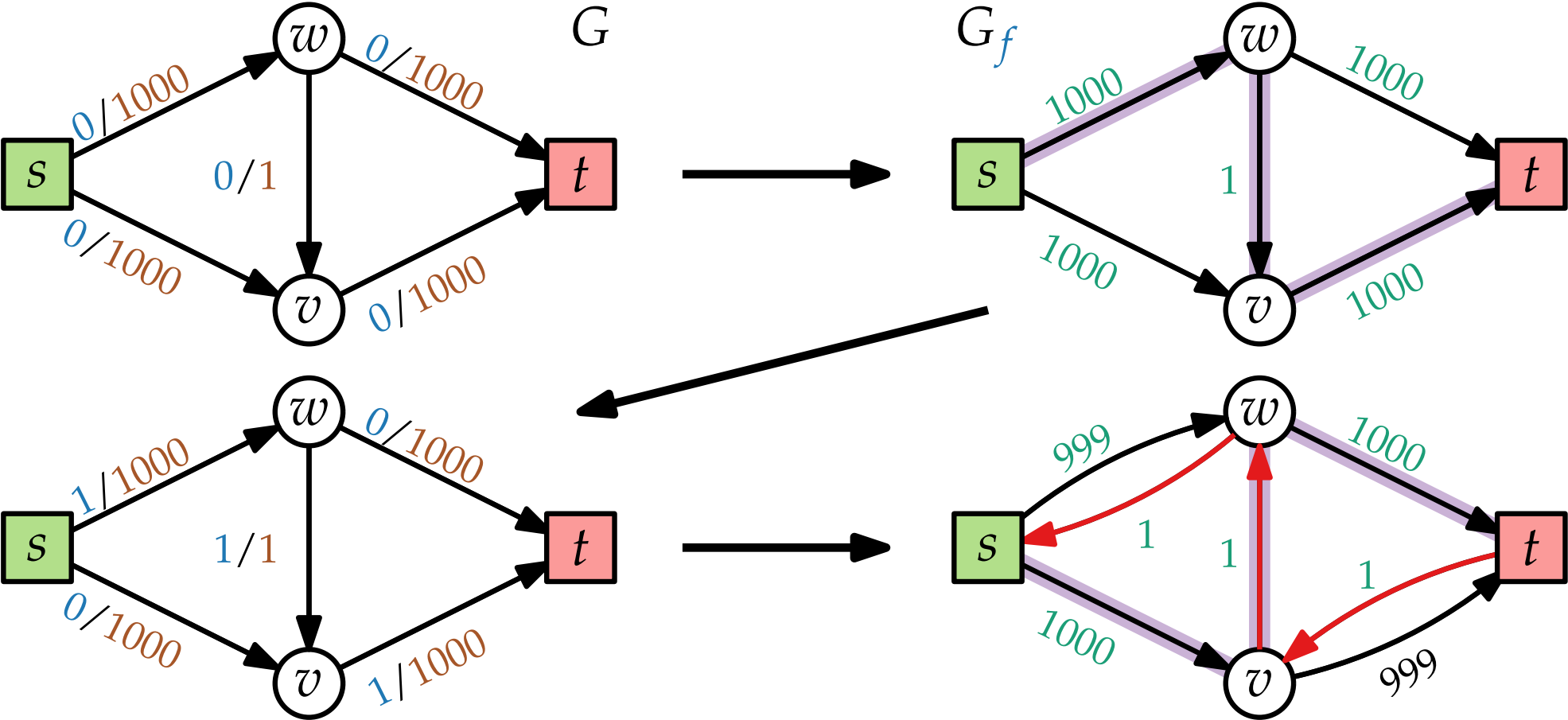
FORDFULKERSON – Beispiel



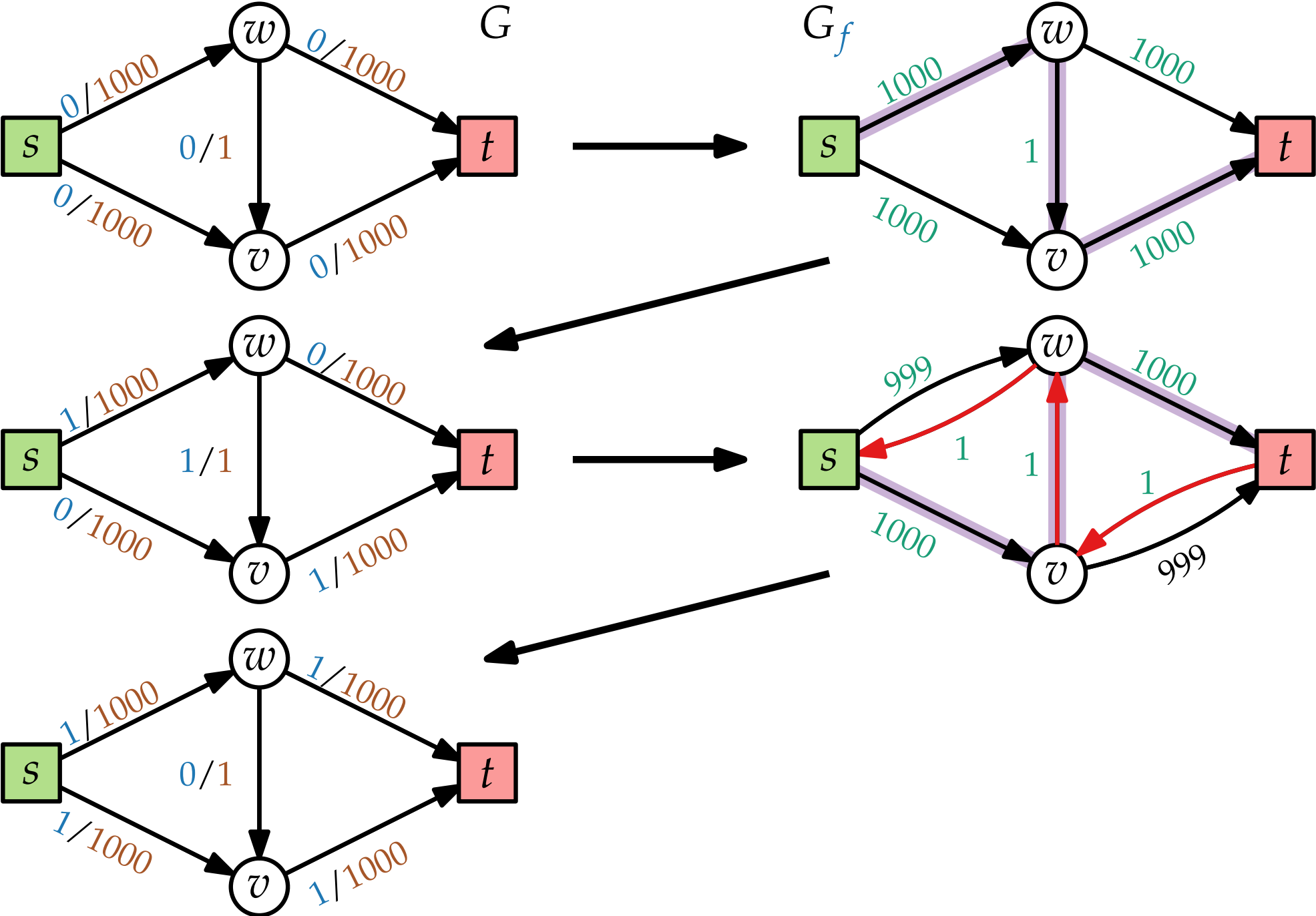
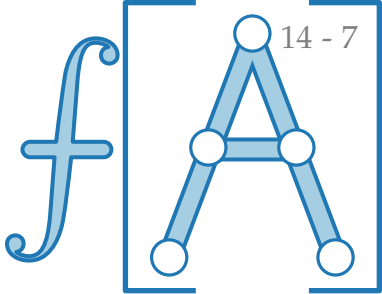
FORDFULKERSON – Beispiel



FORDFULKERSON – Beispiel



FORDFULKERSON – Beispiel



EDMONDSKARP '70 (auch DINIC '72)

FORDFULKERSON($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ **do**

$f(v, v') = 0$

while G_f enthält s - t -Pfad W **do**

$W =$ s - t -Pfad in G_f

$\Delta_W = \min_{(v, v') \in W} C(v, v')$

foreach $(v, v') \in W$ **do**

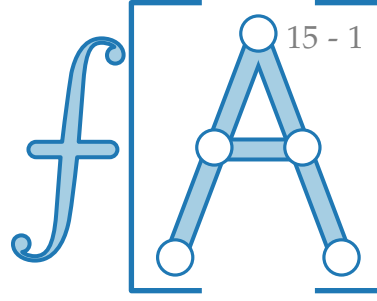
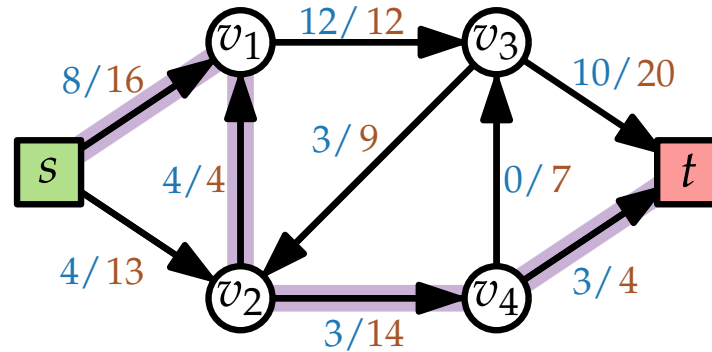
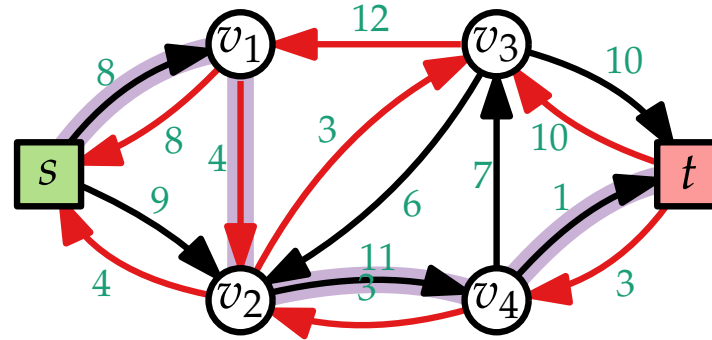
if $(v, v') \in E$ **then**

$f(v, v') = f(v, v') + \Delta_W$

else

$f(v, v') = f(v, v') - \Delta_W$

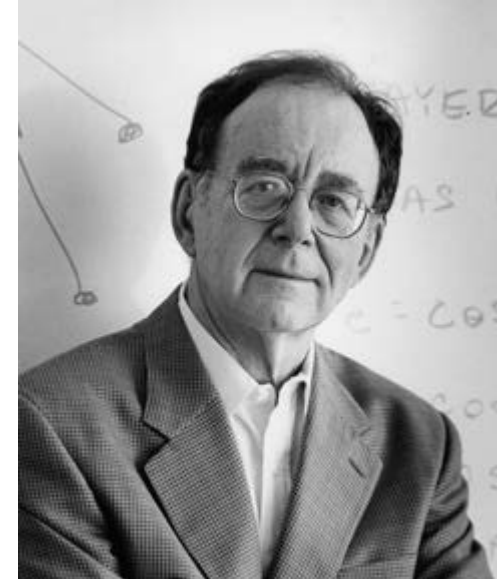
return f



Jack R. Edmonds
*1934 Washington, D.C.



Richard M. Karp
*1935 Boston, MA



EDMONDSKARP '70 (auch DINIC '72)

EDMONDSKARP

~~FORDFULKERSON~~($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ **do**

$f(v, v') = 0$

while G_f enthält s - t -Pfad W **do**

$W =$ s - t -Pfad in G_f

$\Delta_W = \min_{(v, v') \in W} C(v, v')$

foreach $(v, v') \in W$ **do**

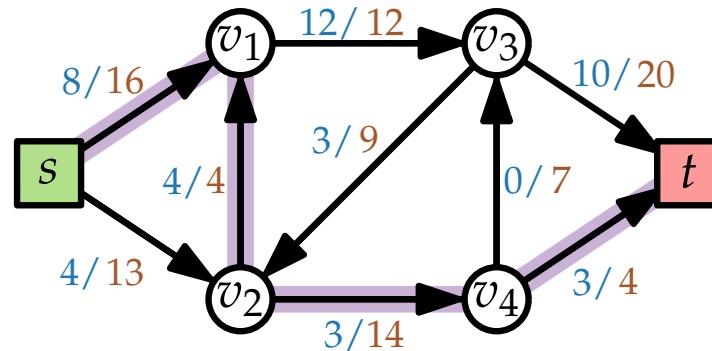
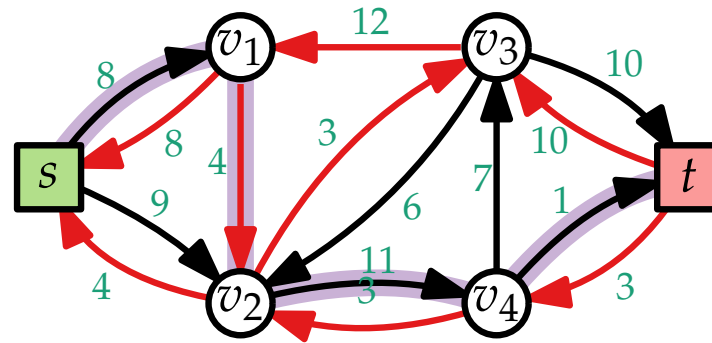
if $(v, v') \in E$ **then**

$f(v, v') = f(v, v') + \Delta_W$

else

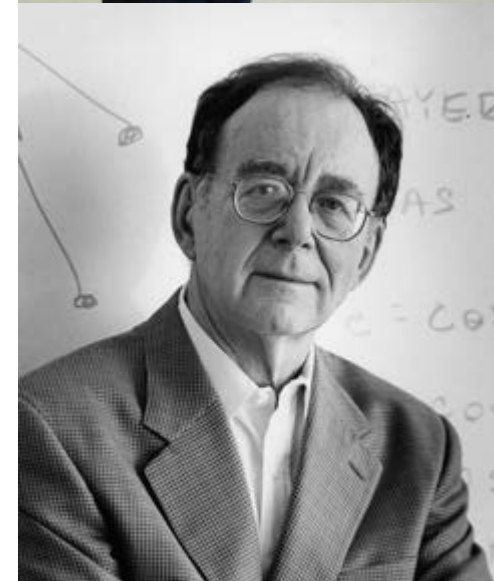
$f(v, v') = f(v, v') - \Delta_W$

return f



Jack R. Edmonds
*1934 Washington, D.C.

Richard M. Karp
*1935 Boston, MA



EDMONDSKARP '70 (auch DINIC '72)

EDMONDSKARP

~~FORDFULKERSON~~($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ **do**

$f(v, v') = 0$

while G_f enthält s - t -Pfad W **do**

$W =$ kürzester s - t -Pfad in G_f

$\Delta_W = \min_{(v, v') \in W} C(v, v')$

foreach $(v, v') \in W$ **do**

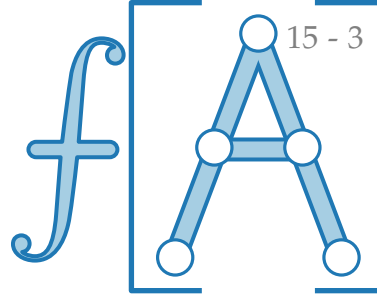
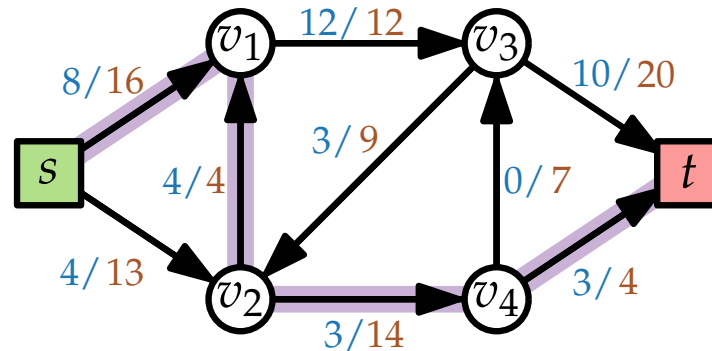
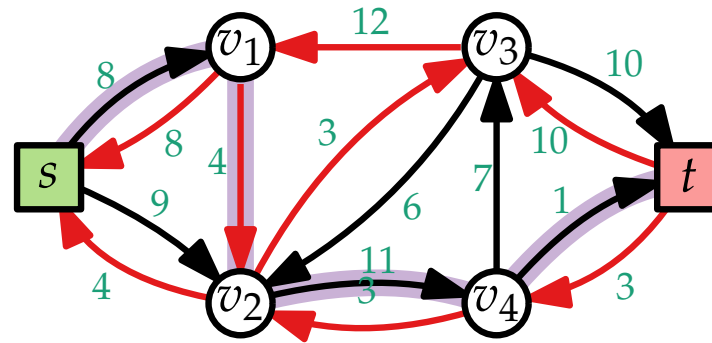
if $(v, v') \in E$ **then**

$f(v, v') = f(v, v') + \Delta_W$

else

$f(v, v') = f(v, v') - \Delta_W$

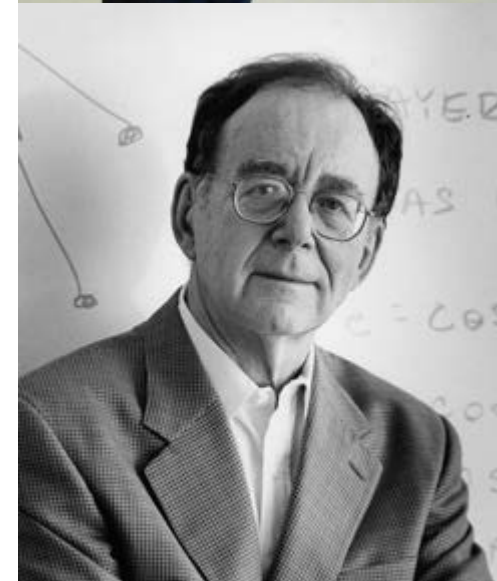
return f



Jack R. Edmonds
*1934 Washington, D.C.



Richard M. Karp
*1935 Boston, MA



EDMONDSKARP '70 (auch DINIC '72)

EDMONDSKARP

~~FORDFULKERSON~~($G = (V, E); s; t; c$)

foreach $(v, v') \in E$ do

$f(v, v') = 0$

while G_f enthält s - t -Pfad W do

$W =$ kürzester s - t -Pfad in G_f

$\Delta_W = \min_{(v, v') \in W} C(v, v')$

 foreach $(v, v') \in W$ do

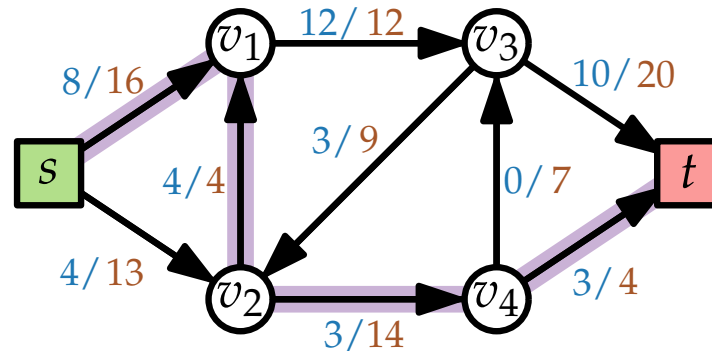
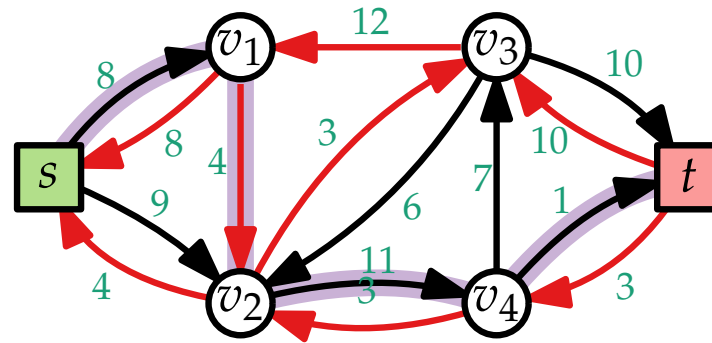
 if $(v, v') \in E$ then

$f(v, v') = f(v, v') + \Delta_W$

 else

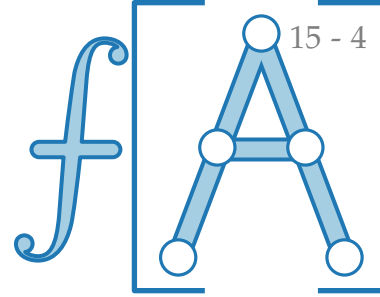
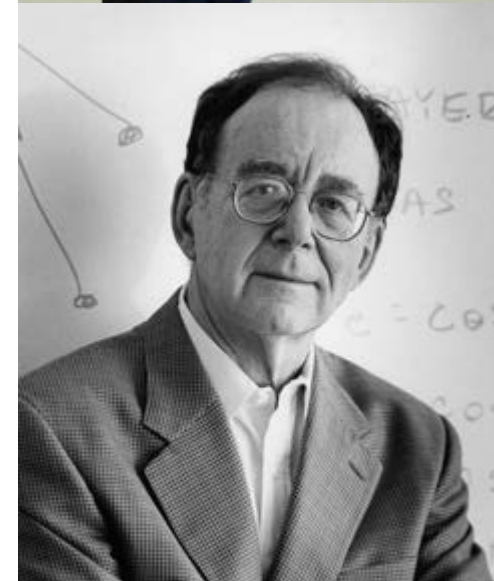
$f(v, v') = f(v, v') - \Delta_W$

return f



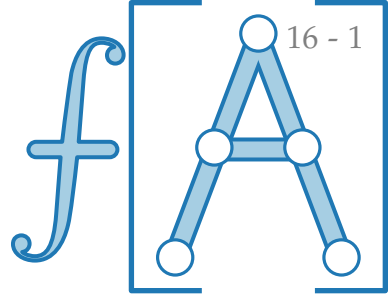
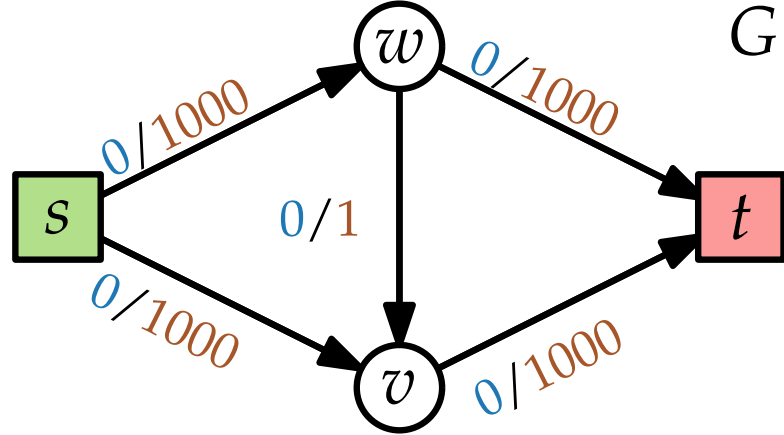
Jack R. Edmonds
*1934 Washington, D.C.

Richard M. Karp
*1935 Boston, MA

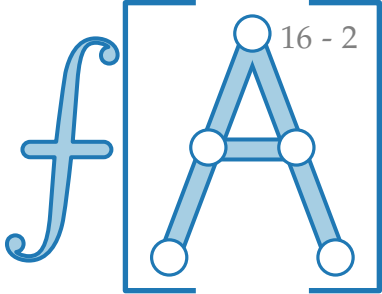
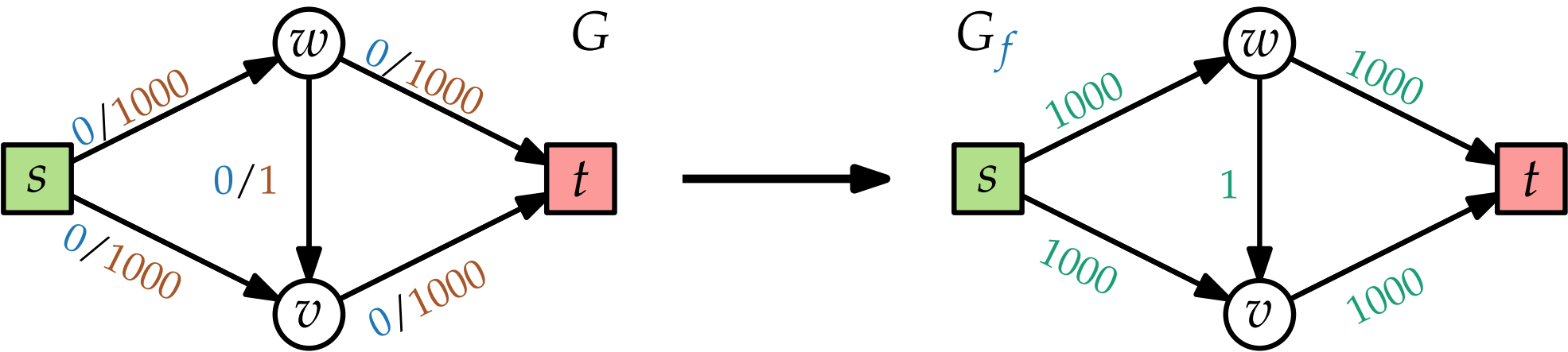


EDMONDSKARP findet einen größten s - t -Fluss in $\mathcal{O}(nm^2)$ Zeit.

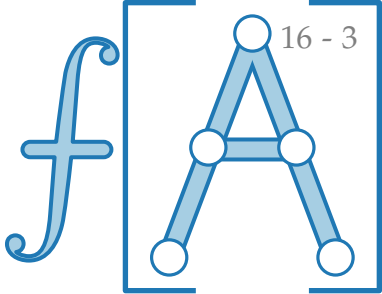
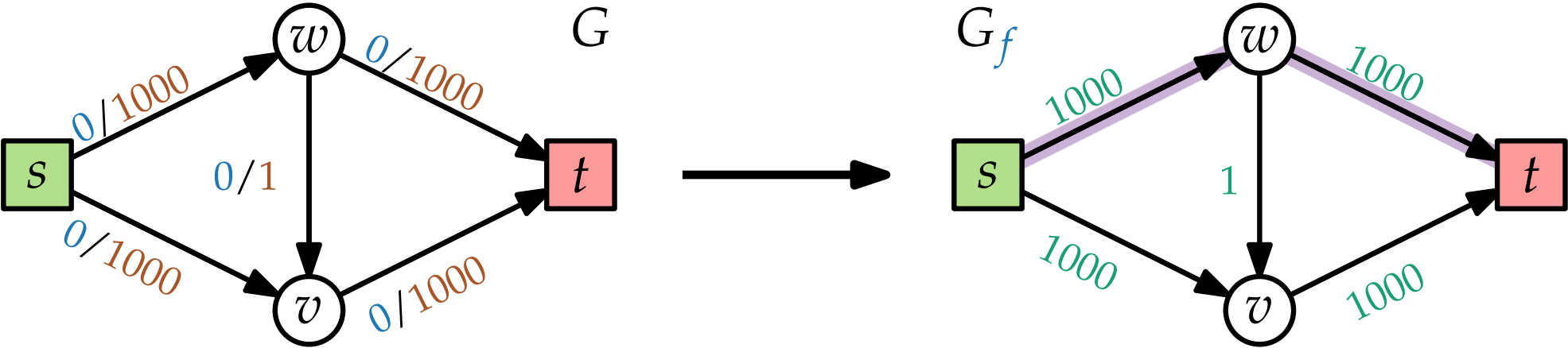
EDMONDSKARP – Beispiel



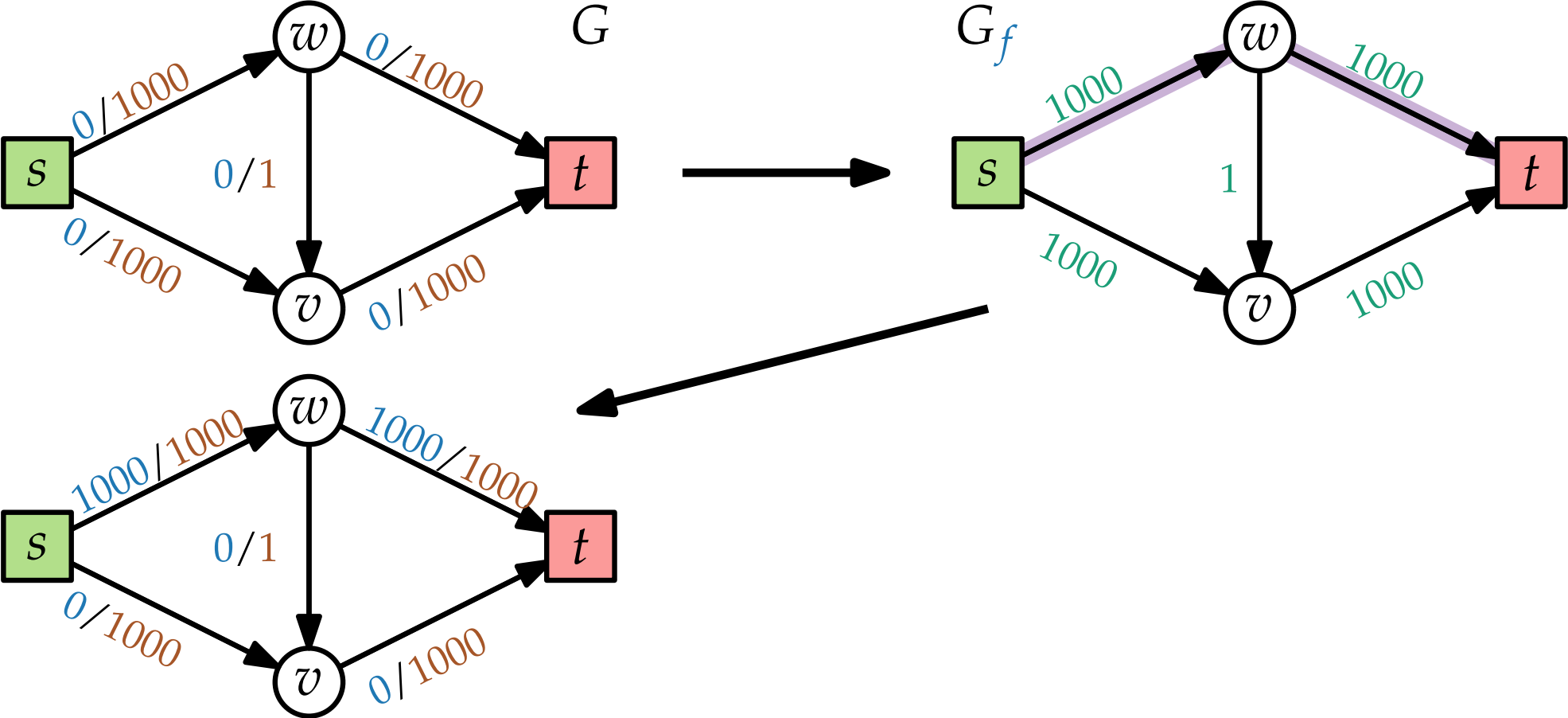
EDMONDSKARP – Beispiel



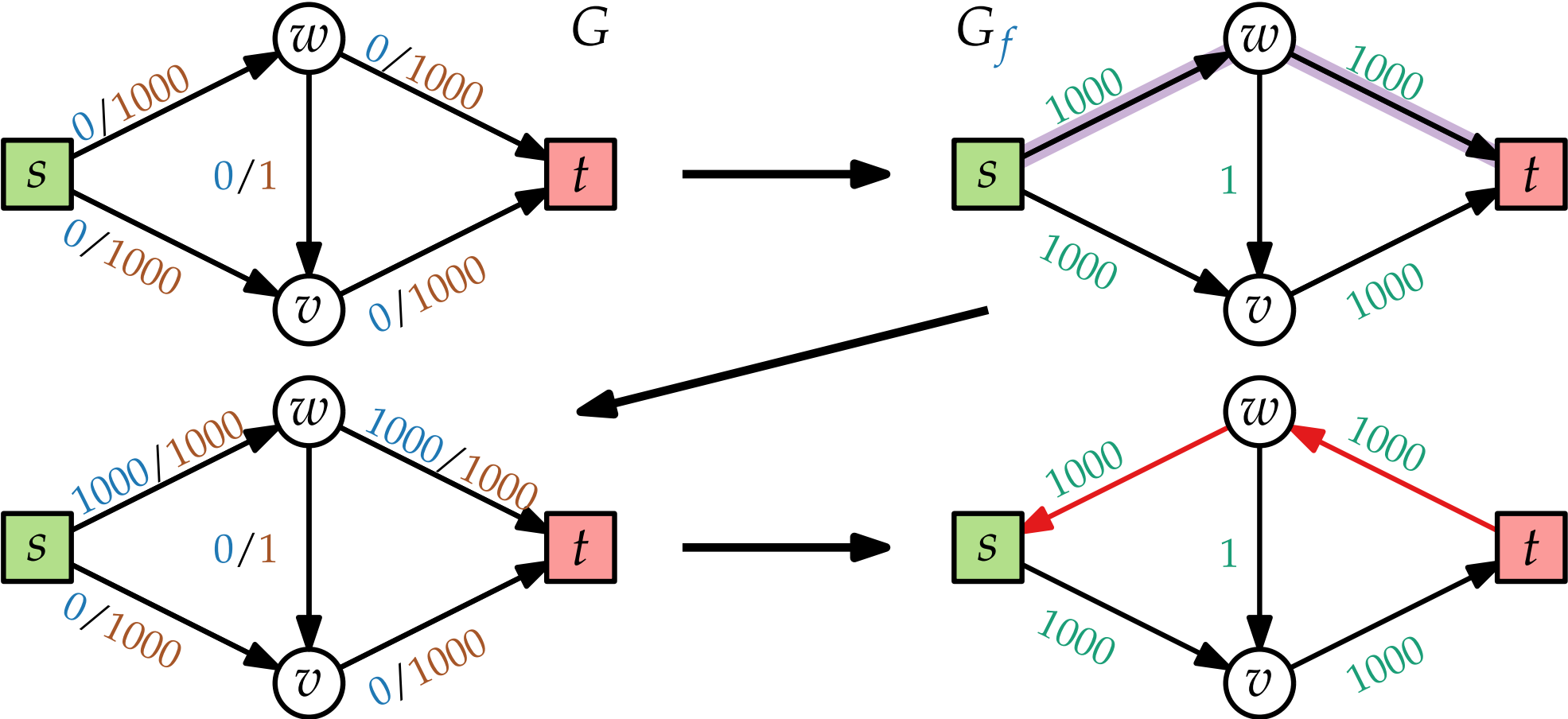
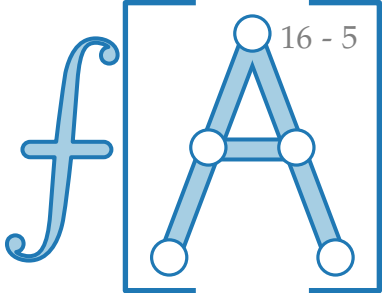
EDMONDSKARP – Beispiel



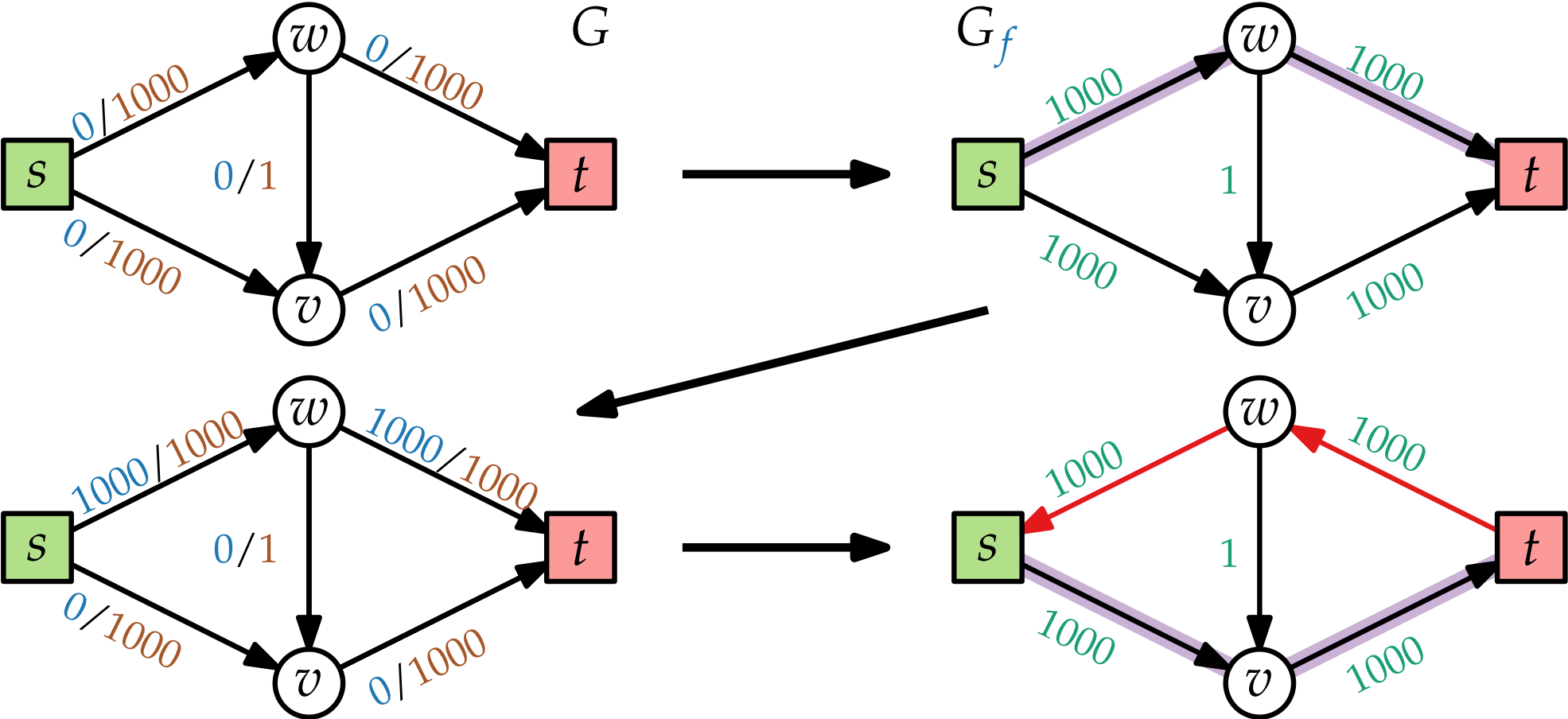
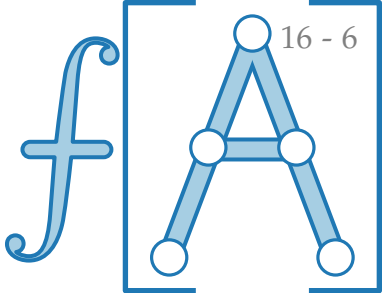
EDMONDSKARP – Beispiel



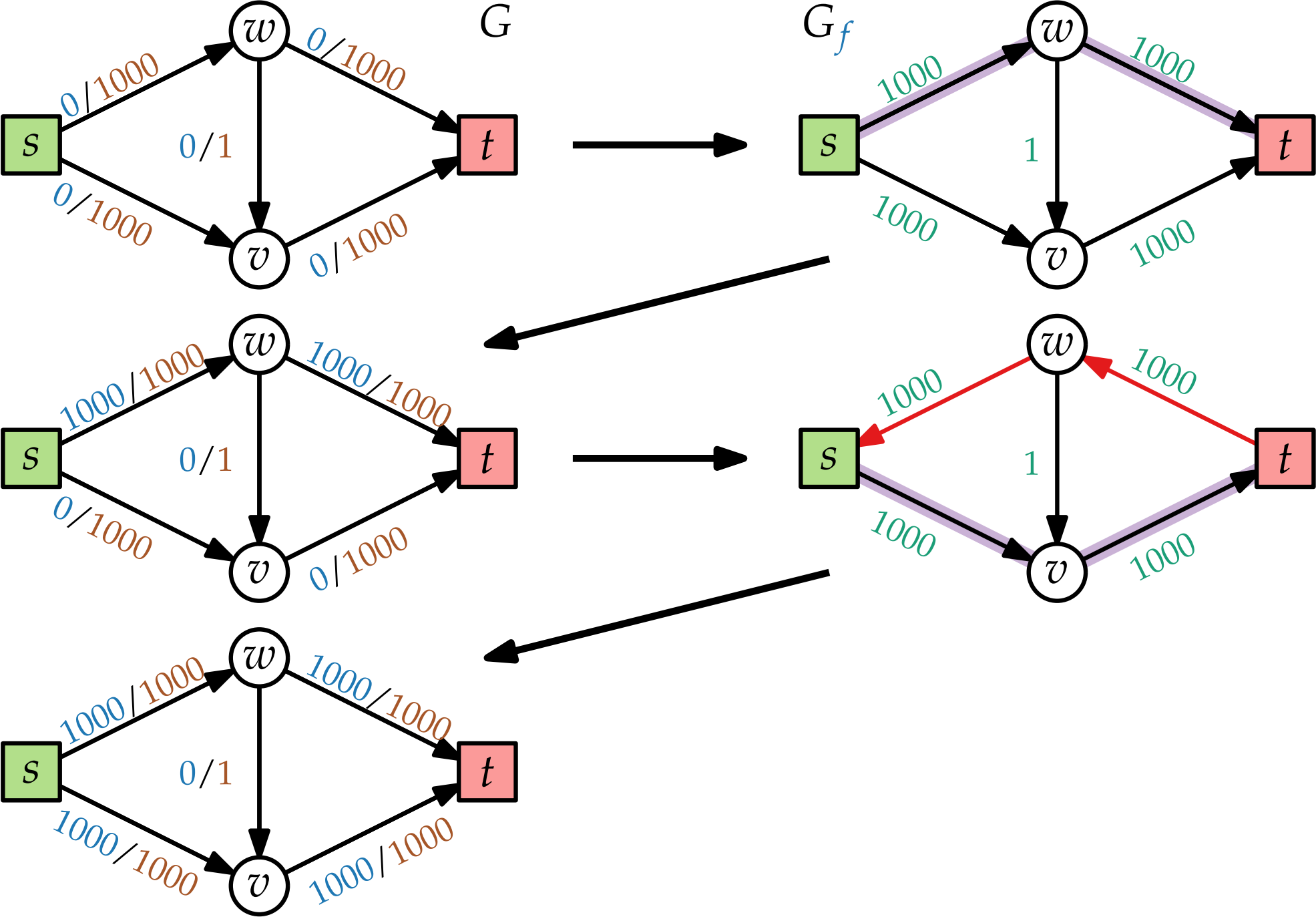
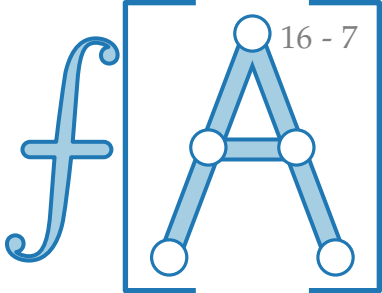
EDMONDSKARP – Beispiel



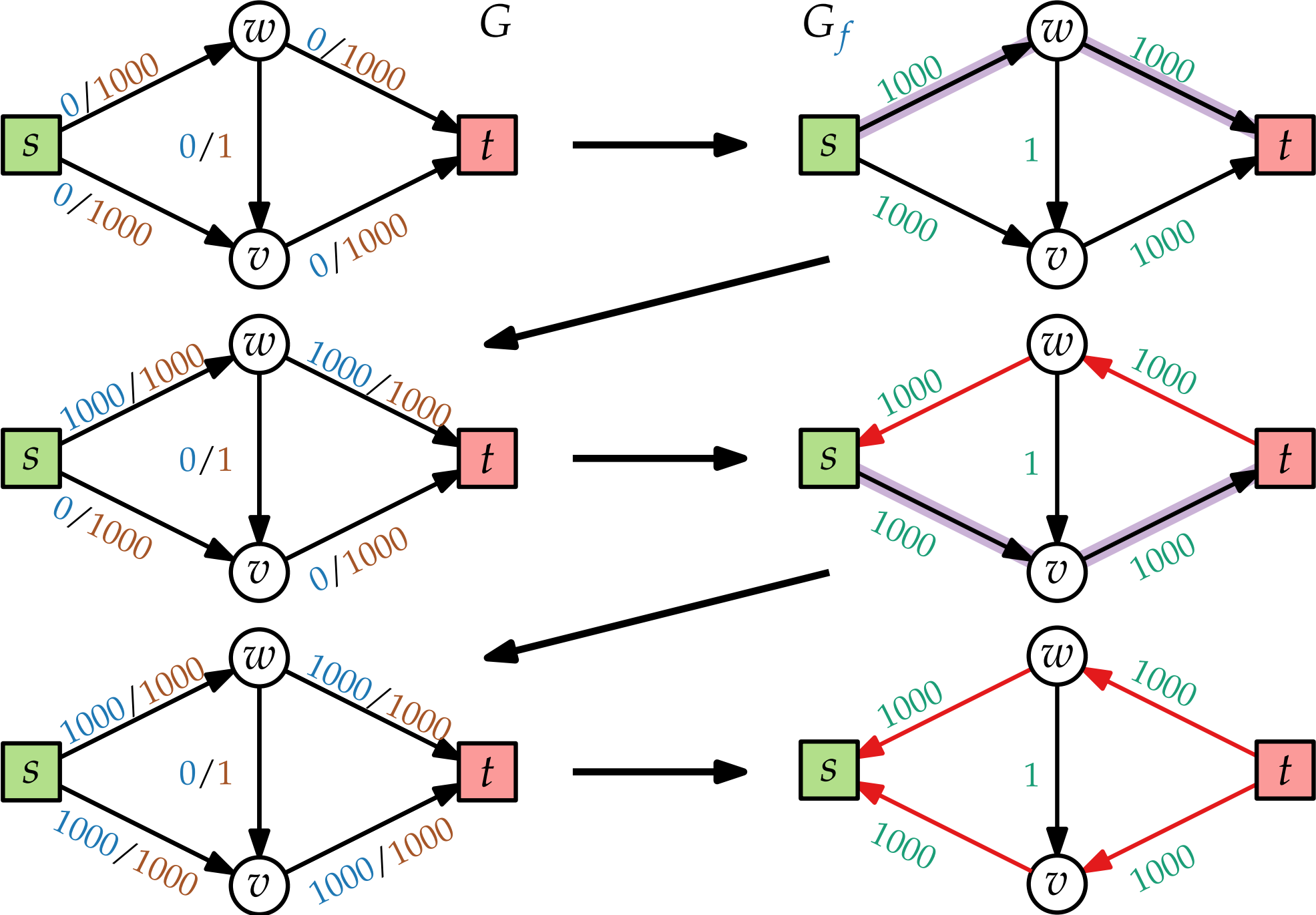
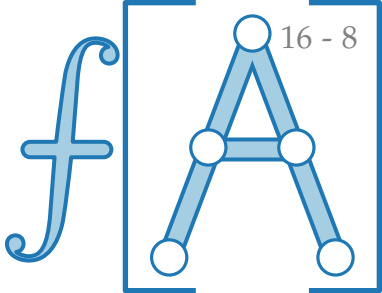
EDMONDSKARP – Beispiel



EDMONDSKARP – Beispiel

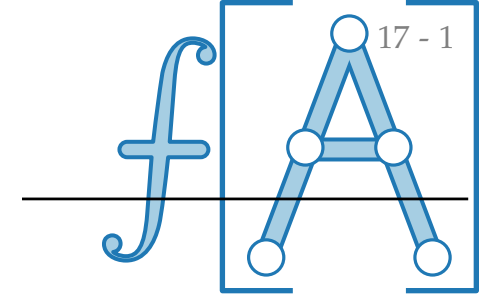


EDMONDSKARP – Beispiel



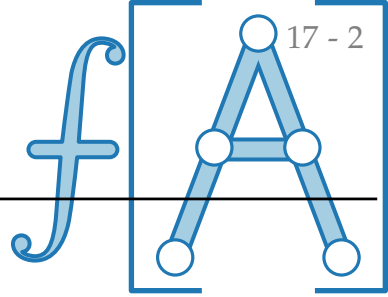
Kurze Geschichte der Berechnung max. Flüsse

<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
-------------	---	----------------	----------------



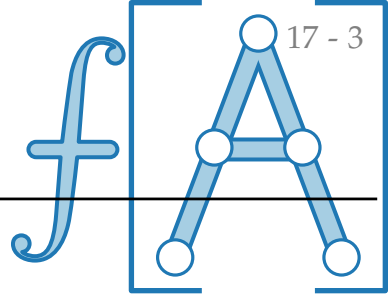
Kurze Geschichte der Berechnung max. Flüsse

<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. s - t -path	Ford & Fulkerson

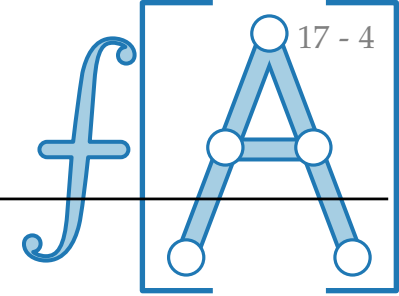


Kurze Geschichte der Berechnung max. Flüsse

<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp

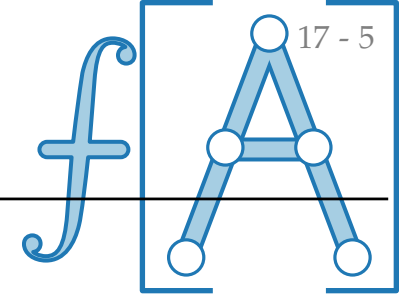


Kurze Geschichte der Berechnung max. Flüsse



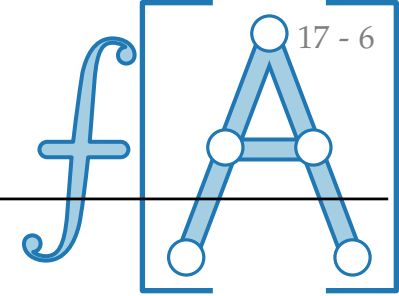
<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic

Kurze Geschichte der Berechnung max. Flüsse



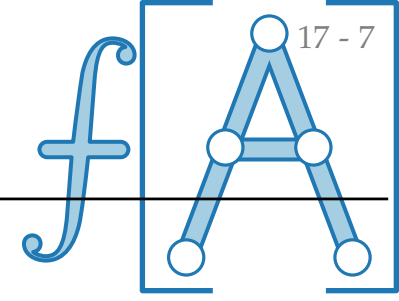
<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari

Kurze Geschichte der Berechnung max. Flüsse



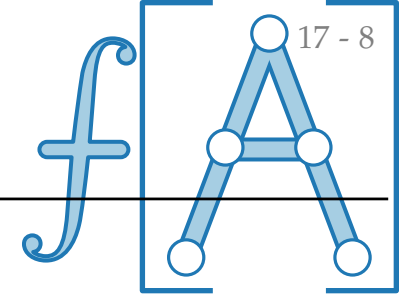
<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan

Kurze Geschichte der Berechnung max. Flüsse



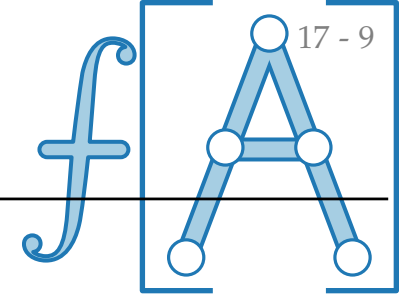
<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan

Kurze Geschichte der Berechnung max. Flüsse



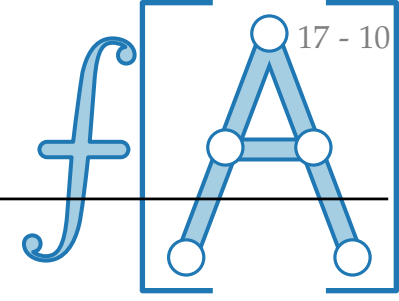
<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan

Kurze Geschichte der Berechnung max. Flüsse



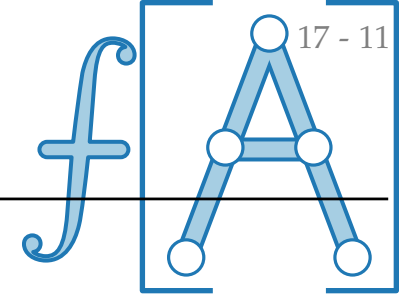
<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2\sqrt{m}$	- " -	Cheriyān & Maheshwari

Kurze Geschichte der Berechnung max. Flüsse



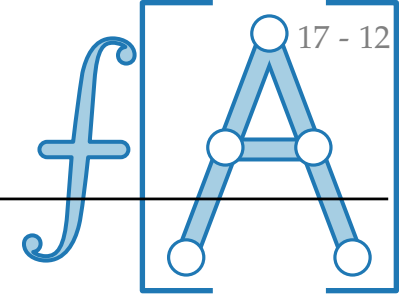
<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2 \sqrt{m}$	- " -	Cheriyān & Maheshwari
1988	$nm \log \frac{n^2}{m}$	- " -	Goldberg & Tarjan

Kurze Geschichte der Berechnung max. Flüsse



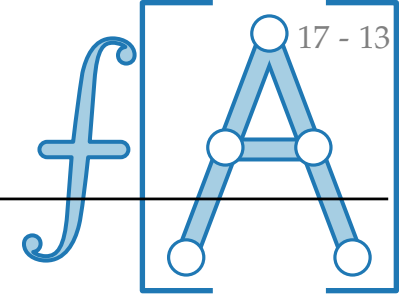
<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2 \sqrt{m}$	- " -	Cheriyān & Maheshwari
1988	$nm \log \frac{n^2}{m}$	- " -	Goldberg & Tarjan
1998	$m \min\{n^{\frac{2}{3}}, \sqrt{m}\} \log \frac{n^2}{m} \log C$	Binary blocking flow	Goldberg & Rao

Kurze Geschichte der Berechnung max. Flüsse



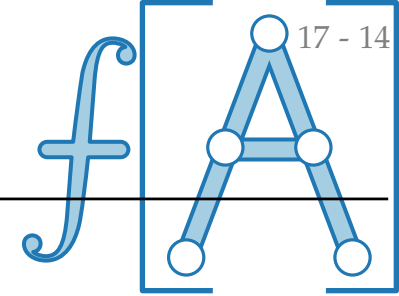
<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2 \sqrt{m}$	- " -	Cheriyān & Maheshwari
1988	$nm \log \frac{n^2}{m}$ max. Kapazität	- " -	Goldberg & Tarjan
1998	$m \min\{n^{\frac{2}{3}}, \sqrt{m}\} \log \frac{n^2}{m} \log C$	Binary blocking flow	Goldberg & Rao

Kurze Geschichte der Berechnung max. Flüsse



<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2 \sqrt{m}$	- " -	Cheriyān & Maheshwari
1988	$nm \log \frac{n^2}{m}$ max. Kapazität	- " -	Goldberg & Tarjan
1998	$m \min\{n^{\frac{2}{3}}, \sqrt{m}\} \log \frac{n^2}{m} \log C$	Binary blocking flow	Goldberg & Rao
1994	nm falls $m > n^{1+\varepsilon}$	Combinatorial game	King, Rao & Tarjan

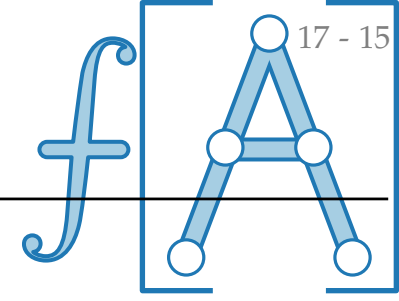
Kurze Geschichte der Berechnung max. Flüsse



17 - 14

<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2 \sqrt{m}$	- " -	Cheriyān & Maheshwari
1988	$nm \log \frac{n^2}{m}$ max. Kapazität	- " -	Goldberg & Tarjan
1998	$m \min\{n^{\frac{2}{3}}, \sqrt{m}\} \log \frac{n^2}{m} \log C$	Binary blocking flow	Goldberg & Rao
1994	nm falls $m > n^{1+\varepsilon}$	Combinatorial game	King, Rao & Tarjan
2013	nm falls $m \leq \mathcal{O}(n^{\frac{16}{15}-\varepsilon})$	Binary blocking flow	Orlin

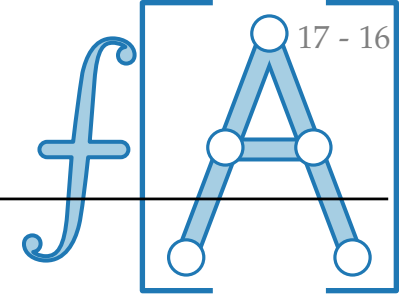
Kurze Geschichte der Berechnung max. Flüsse



<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2 \sqrt{m}$	- " -	Cheriyān & Maheshwari
1988	$nm \log \frac{n^2}{m}$ max. Kapazität	- " -	Goldberg & Tarjan
1998	$m \min\{n^{\frac{2}{3}}, \sqrt{m}\} \log \frac{n^2}{m} \log C$	Binary blocking flow	Goldberg & Rao
1994	nm falls $m > n^{1+\varepsilon}$	Combinatorial game	King, Rao & Tarjan
2013	nm falls $m \leq \mathcal{O}(n^{\frac{16}{15}-\varepsilon})$	Binary blocking flow	Orlin

} $\Rightarrow \mathcal{O}(nm)$

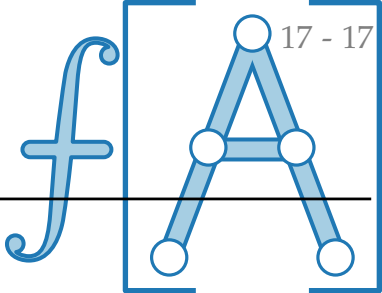
Kurze Geschichte der Berechnung max. Flüsse



<i>Jahr</i>	<i>Laufzeit $\mathcal{O}(\cdot)$</i>	<i>Methode</i>	<i>Autoren</i>
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2\sqrt{m}$	- " -	Cheriyān & Maheshwari
1988	$nm \log \frac{n^2}{m}$ max. Kapazität	- " -	Goldberg & Tarjan
1998	$m \min\{n^{\frac{2}{3}}, \sqrt{m}\} \log \frac{n^2}{m} \log C$	Binary blocking flow	Goldberg & Rao
1994	nm falls $m > n^{1+\varepsilon}$	Combinatorial game	King, Rao & Tarjan
2013	nm falls $m \leq \mathcal{O}(n^{\frac{16}{15}-\varepsilon})$	Binary blocking flow	Orlin
2020	$m^{\frac{4}{3}+o(1)} C^{\frac{1}{3}}$	Interior point method	Kathuria, Liu & Sidford

} $\Rightarrow \mathcal{O}(nm)$

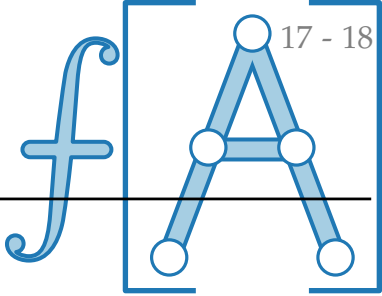
Kurze Geschichte der Berechnung max. Flüsse



Jahr	Laufzeit $\mathcal{O}(\cdot)$	Methode	Autoren
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2 \sqrt{m}$	- " -	Cheriyān & Maheshwari
1988	$nm \log \frac{n^2}{m}$ max. Kapazität	- " -	Goldberg & Tarjan
1998	$m \min\{n^{\frac{2}{3}}, \sqrt{m}\} \log \frac{n^2}{m} \log C$	Binary blocking flow	Goldberg & Rao
1994	nm falls $m > n^{1+\varepsilon}$	Combinatorial game	King, Rao & Tarjan
2013	nm falls $m \leq \mathcal{O}(n^{\frac{16}{15}-\varepsilon})$	Binary blocking flow	Orlin
2020	$m^{\frac{4}{3}+o(1)} C^{\frac{1}{3}}$	Interior point method	Kathuria, Liu & Sidford
2020	$(m + n^{\frac{3}{2}}) \log C \text{ polylog } n$	- " -	Brand et al.

} $\Rightarrow \mathcal{O}(nm)$

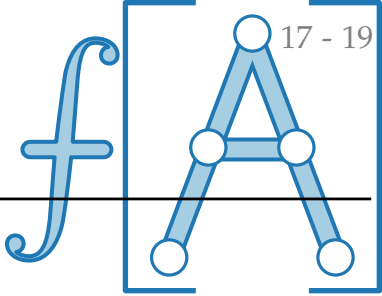
Kurze Geschichte der Berechnung max. Flüsse



Jahr	Laufzeit $\mathcal{O}(\cdot)$	Methode	Autoren
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2 \sqrt{m}$	- " -	Cheriyān & Maheshwari
1988	$nm \log \frac{n^2}{m}$ max. Kapazität	- " -	Goldberg & Tarjan
1998	$m \min\{n^{\frac{2}{3}}, \sqrt{m}\} \log \frac{n^2}{m} \log C$	Binary blocking flow	Goldberg & Rao
1994	nm falls $m > n^{1+\varepsilon}$	Combinatorial game	King, Rao & Tarjan
2013	nm falls $m \leq \mathcal{O}(n^{\frac{16}{15}-\varepsilon})$	Binary blocking flow	Orlin
2020	$m^{\frac{4}{3}+o(1)} C^{\frac{1}{3}}$	Interior point method	Kathuria, Liu & Sidford
2020	$(m + n^{\frac{3}{2}}) \log C \text{ polylog } n$	- " -	Brand et al.
2021	$m^{\frac{3}{2}-\frac{1}{328}} \log C \text{ polylog } n$	- " -	Gao, Liu & Peng

} $\Rightarrow \mathcal{O}(nm)$

Kurze Geschichte der Berechnung max. Flüsse



Jahr	Laufzeit $\mathcal{O}(\cdot)$	Methode	Autoren
1956	$m f^* $	resid. <i>s-t</i> -path	Ford & Fulkerson
1970	nm^2	shortest resid. <i>s-t</i> -path	Edmonds & Karp
1972	n^2m	- " -	Dinic
1978	n^3	- " -	Malhotra, Kumar & Maheshwari
1982	$nm \log n$	- " -	Sleator & Tarjan
1988	n^2m	Push-Relabel	Goldberg & Tarjan
1988	n^3	- " -	Goldberg & Tarjan
1988	$n^2\sqrt{m}$	- " -	Cheriyān & Maheshwari
1988	$nm \log \frac{n^2}{m}$ max. Kapazität	- " -	Goldberg & Tarjan
1998	$m \min\{n^{\frac{2}{3}}, \sqrt{m}\} \log \frac{n^2}{m} \log C$	Binary blocking flow	Goldberg & Rao
1994	nm falls $m > n^{1+\varepsilon}$	Combinatorial game	King, Rao & Tarjan
2013	nm falls $m \leq \mathcal{O}(n^{\frac{16}{15}-\varepsilon})$	Binary blocking flow	Orlin
2020	$m^{\frac{4}{3}+o(1)} C^{\frac{1}{3}}$	Interior point method	Kathuria, Liu & Sidford
2020	$(m + n^{\frac{3}{2}}) \log C \text{ polylog } n$	- " -	Brand et al.
2021	$m^{\frac{3}{2}-\frac{1}{328}} \log C \text{ polylog } n$	- " -	Gao, Liu & Peng
2022	$m^{1+o(1)} \log C$	Dynamic data structures	Chen, Kyng, Liu, Peng, Gutenberg & Sachdeva

} $\Rightarrow \mathcal{O}(nm)$